INTRODUCTION · EXAM STRATEGY - ALL SECTIONS

How This Course Works

What the Professional Data Engineer exam looks like today, how this course was built, the company you will design for, and a route through eleven hours of material.

EXAM Professional Data Engineer MODULES 45 RUNTIME Over 11 hours

admin@sthithapragna.com

The exam rarely asks what a product is. It asks which product fits, and why the others do not.

admin@sthithapragna.com

The exam, as Google describes it today

- **2 hours**, **40 to 50** multiple-choice and multiple-select questions.
- English or Japanese. **Online**, or at a **test centre**.
- Valid for **2 years**. No prerequisites. Three years' experience recommended.
- The **pass mark is not published**. Any figure you read online is a guess.

> CURRENCY CHECK
>
> Checked against Google's certification page on 14 October 2026.

Five sections, and how the course spends its time

SECTION	GUIDE WEIGHT	TIME IN THIS COURSE
1 Designing data processing systems	22%	2 h 05, modules 02 to 09
2 Ingesting and processing the data	25%	2 h 09, modules 10 to 18
3 Storing the data	20%	1 h 52, modules 19 to 26
4 Preparing and using data for analysis	15%	1 h 20, modules 27 to 32
5 Maintaining and automating workloads	18%	1 h 35, modules 33 to 39

THE NUMBERS

Teaching time tracks the official weights to within about one percentage point per section.

Renewing in two years

- Sit the **standard exam** again, as for the first time.
- Or a **renewal exam**: 1 hour, 20 questions, during the renewal window.
- Or **Google Skills** courses and skill badges, in the last year of your certificate.
- A renewal exam lasts **2 years**. A Google Skills renewal lasts **1 year**.

> CURRENCY CHECK
>
> Google Skills is the new name for Google Cloud Skills Boost.

Old names on the exam, new names in the console

- Google's banner: the exam "will soon be updated to reflect recent branding changes".
- The guide still says **Dataproc**, **Cloud Composer**, **BigLake** and **Dataplex**.
- The console now says Managed Spark, Managed Airflow, Lakehouse and Knowledge Catalog.
- This course teaches **both names**, every time. Module 40 lists them all.

> THE PATTERN
>
> Answer with the name the question uses. A renamed product is still right. A retired one is not.

How this course was made

- A bank of practice questions showed **which topics and decisions** the exam tests.
- **None** of those questions is reused. Every scenario and paper is new.
- Every fact was checked against **Google Cloud documentation**, in October 2026.
- Where a page could not be checked, the course says so: **Honest limit**.

> REMEMBER
>
> Topics the bank missed, such as embeddings and Managed Kafka, were added from the current guide.

Meet the company you will design for

- **Larkspur Energy**: a fictional supplier in Bristol, with a German arm in Hamburg.
- **3.1 million** smart meters, sending a reading every half hour.
- Moving off **Oracle billing**, a **40-node Hadoop** cluster, and spreadsheets.
- German customer data must stay **in Germany**. UK work runs in **London**.

> FOR LARKSPUR
>
> Partners: Westgrid, the grid operator, wants demand within five minutes. Hearthly wants a private match.

How every module runs

01
The core idea
One sentence to hold on to

02
Teaching slides
Compares, tables, real code

03
Phrases
Words in a question that pick the answer

04
Worked scenarios
Pause, decide, then hear why

05
In five lines
The module, compressed

REMEMBER

When a scenario says pause now, do pause. Choosing is where the learning happens.

What the callouts mean

CALLOUT	WHAT IT TELLS YOU
The signal	The phrase in a question that decides the answer
The trap	A tempting wrong answer, and why it is wrong
The tiebreaker	How to choose between two options that both work
Currency check	Something renamed, retired or new since older courses
Honest limit	Something the docs did not confirm, or a figure to treat with care

THE PATTERN

Traps and tiebreakers are where most marks are won and lost.

The map, and Section 1: designing systems

MODULE	STARTS AT
01 The Google Cloud data map	0:00:00
02 IAM and organisation policies, 03 Encryption and keys	0:17:19
04 Personal data, 05 Residency and perimeters	0:51:33
06 Reliability and fidelity, 07 Migrating files	1:21:22
08 Migrating databases, 09 Portability and multi-cloud	1:52:49

REMEMBER

Start with module 01. It names every product in the course on one page.

Section 2: ingesting and processing

MODULE	STARTS AT
10 Planning pipelines, 11 Pub/Sub	2:22:35
12 Dataflow and Beam, 13 Windows	2:54:38
14 Watermarks and triggers, 15 Spark and Hadoop	3:21:03
16 Getting data into BigQuery, 17 Low-code and SQL pipelines	3:49:04
18 Deploying pipelines	4:18:46

THE TIEBREAKER

Modules 12 to 14 are the hardest stretch. Watch them in order, and rewatch 14 if late data feels slippery.

admin@sthithapragna.com

Section 3: storing the data

MODULE	STARTS AT
19 Choosing a storage service, 20 Bigtable	4:31:54
21 Relational stores, 22 Cloud Storage	5:01:26
23 BigQuery tables, 24 Modelling the warehouse	5:28:47
25 Lakes and lakehouses, 26 Knowledge Catalog	5:57:14

REMEMBER

Module 19 is the one to rewatch the night before. It decides between every store.

Sections 4 and 5: analysis, and operations

MODULE	STARTS AT
27 Serving BI, 28 Slow queries, 29 Securing BigQuery	6:23:49
30 BigQuery ML, 31 Embeddings and RAG, 32 Sharing	7:06:30
33 Capacity, 34 Right-sizing, 35 Orchestration	7:43:28
36 Monitoring, 37 Operating streaming	8:22:47
38 When pipelines fail, 39 Replication and failover	8:49:05

CURRENCY CHECK

Module 31 covers the newest material in the guide: embeddings, vector search and RAG.

Exam craft and the practice papers

MODULE	STARTS AT
40 What changed: names, retirements, guide vs today	9:18:08
41 Confusion pairs: the one word that separates them	9:29:27
42 How PDE questions are written	9:49:06
43 Practice paper 1, 20 questions	10:01:27
44 Practice paper 2, 45 Practice paper 3	10:27:48

THE NUMBERS

Sixty new questions in all. Paper 3 ends with a way to read your three scores together.

A route through the course

01

Modules 01 to 39

In order, pausing on every scenario

02

Modules 40 to 42

Names, pairs, question craft

03

Paper 1

Score it by section

04

Fix the gap

Rewatch the weakest section

05

Papers 2 and 3

Then read all three scores

THE TIEBREAKER

Short on time? Never skip the scenarios. They are the closest thing to the real exam.

Introduction in five lines

- **2 hours**, 40 to 50 questions, five weighted sections, no published pass mark.
- The exam uses **old names**. This course teaches both, every time.
- Built from what the exam tests, with **no reused questions** and checked docs.
- You design for **Larkspur Energy**, its meters, and its German data.
- Pause on every scenario. **Choosing** is where the learning happens.

> COMING UP
>
> Next: module 01, the Google Cloud data map, and the 2026 names.

MODULE 01 · EXAM STRATEGY - ALL SECTIONS

The Google Cloud Data Map

Every service the exam names, the 2026 renames, and how to read what a question is really asking.

SECTIONS All five MODULE 1 of 45 SCENARIOS 4

admin@sthithapragna.com

Most PDE questions are not about what a service does. They are about which one fits **one sentence in the question**.

One platform, five jobs

01

Ingest

Get data in: event streams, files and database changes.

02

Process

Clean, join and enrich, in batch or as a stream.

03

Store

Pick the store that fits how the data will be read.

04

Serve

Dashboards, sharing, machine learning and AI.

05

Govern and operate

Secure, orchestrate, monitor and recover.

MAP TO THE EXAM

Section 2 covers ingest and process, 3 store, 4 serve, 5 operate. Section 1, design, runs across all five.

Getting data in

SERVICE	CHOOSE IT WHEN
Pub/Sub	Events stream in from many producers and you want serverless, global messaging.
Managed Service for Apache Kafka	You already run Kafka, or need Kafka's API and its connectors.
Storage Transfer Service	Files move from another cloud, a list of URLs, or on-premises file servers.
Transfer Appliance	The data is so large that shipping disks beats the network.
Datastream	Change data capture from Oracle, MySQL, PostgreSQL and more.
Database Migration Service	A whole database moves to Cloud SQL or AlloyDB with minimal downtime.
BigQuery Data Transfer Service	Scheduled loads into BigQuery from SaaS apps and other warehouses.

THE SHORTCUT

Ask what is moving: events, database changes, or files.

Processing it

SERVICE	CHOOSE IT WHEN
Dataflow	Batch or streaming pipelines in Apache Beam. Serverless and autoscaling.
Managed Service for Apache Spark (Dataproc)	Existing Spark or Hadoop code, on clusters or as serverless batches.
Cloud Data Fusion	Visual, low-code pipelines with prebuilt connectors.
Dataform	SQL transformations inside BigQuery, in version control, with tests.
BigQuery SQL	The data is already in BigQuery and SQL can do the job.
Cloud Run functions	Small event-driven steps, such as reacting to a file landing.

THE TIEBREAKER

No existing code usually means Dataflow. Existing Spark or Hadoop code means the Spark service.

Storing it

SERVICE	THE PHRASE THAT GIVES IT AWAY
BigQuery	Analytics with SQL over very large data. The warehouse.
Cloud Storage	Files and objects of any size. The lake and the landing zone.
Bigtable	Huge read and write volumes by key, in single-digit milliseconds.
Spanner	Relational, strongly consistent, and scaling out, even globally.
Cloud SQL	Managed MySQL, PostgreSQL or SQL Server at regional scale.
AlloyDB	PostgreSQL-compatible, with fast analytics on live transactions.
Firestore	Serverless document database for application data.
Memorystore	In-memory cache, answering in under a millisecond.

REMEMBER

Module 19 turns these phrases into a full decision tree. For now, learn the phrase next to each name.

Governing and operating it

SERVICE	WHAT IT IS FOR
IAM and organisation policies	Who can do what, and which settings are allowed at all.
Cloud KMS	Encryption keys you control for data at rest, known as CMEK.
Sensitive Data Protection	Finding, masking and tokenising personal data. Formerly Cloud DLP.
Knowledge Catalog (Dataplex)	Discovery, data quality and lineage across the estate.
Managed Service for Apache Airflow (Composer)	Scheduling and orchestrating multi-step pipelines as DAGs.
Workflows	Lightweight orchestration of API calls and serverless steps.
Cloud Monitoring and Cloud Logging	Metrics, alerts, logs and audit trails.

ON THE EXAM

Sections 1 and 5 live here: least privilege, and knowing before the customer does.

Larkspur Energy, end to end

Datastream
Oracle billing CDC
50 tables

Pub/Sub
meter-readings topic
About 1,700 a second

Spark service
Hadoop history, 1.2 PB
Nightly settlement

Dataflow
Dedupe and validate
Windows, late data

BigQuery warehouse
148.8m readings a day
50 billing tables
UK and Germany
Layers: raw, curated, marts

Bigtable
meter_live table
App reads in ms

Knowledge Catalog
Discovery, quality
Lineage

Dashboards
Data Studio for teams
Looker for the board

BigQuery sharing
Westgrid demand feed
Hearthly clean room

REMEMBER

Do not memorise this now. By the end of the course you will be able to defend every box on it.

Meet Larkspur Energy

- **3.1 million smart meters** in the UK and Germany, each sending a reading every half hour.
- **Signal drops.** A meter buffers its readings for up to 72 hours, then uploads them late.
- **An Oracle billing system** and an eight-year **Hadoop history** to migrate by June 2027.
- **Personal and bank data**, and German customer data that must stay in Germany.
- **Two partners**: a grid operator wanting data within five minutes, and a thermostat company.

> REMEMBER
>
> Larkspur is fictional. Its numbers are fixed for the whole course, so you can check one module against another.

In April 2026 Google renamed much of its data stack. The exam guide **still uses the old names**.

Older name, current name

OLDER NAME	CURRENT NAME	CHANGED
Dataproc	Managed Service for Apache Spark	April 2026
Cloud Composer	Managed Service for Apache Airflow	April 2026
BigLake	Google Cloud Lakehouse	April 2026
Dataplex	Knowledge Catalog	April 2026
Vertex AI	Gemini Enterprise Agent Platform	April 2026
Looker Studio	Data Studio, its original name	April 2026
Cloud DLP	Sensitive Data Protection	Earlier
Analytics Hub	BigQuery sharing	Earlier

ON THE EXAM

Same products, same features. Answer with whichever name the question uses.

What the rename did not change

- **APIs, client libraries and the CLI.** Commands such as gcloud dataproc keep working.
- **IAM role names.** Roles such as roles/dataproc.editor keep their old spelling.
- **The exam answer.** A question saying Dataproc wants the same answer as one saying Managed Spark.
- **The trap:** picking an option because its name sounds newer. A new name is not a new product.

> ON THE EXAM
>
> For Dataproc, BigLake and Dataplex, Google states the API, CLI and IAM names are unchanged.

Retired, or on the way out

PRODUCT	STATUS	USE INSTEAD
Data Catalog	Phased shutdown from 1 June 2026	Knowledge Catalog
Pub/Sub Lite	Turned down 18 March 2026	Pub/Sub, or Managed Kafka
gsutil	Legacy. Leaves the gcloud CLI package after March 2027	gcloud storage
Cloud Functions	Renamed Cloud Run functions, August 2024	Cloud Run functions

ON THE EXAM

If an older question's answer is Data Catalog, the idea still holds. Map it to Knowledge Catalog.

Words that eliminate options

THE QUESTION SAYS	WHAT IT RULES OUT
Least operational overhead	Anything you patch, size or keep running: VMs, long-lived clusters.
Minimal code changes	Rewrites. Keep the engine you already run, such as Spark or Kafka.
Most cost-effective	Capacity that sits idle between spiky or occasional jobs.
Near real time, within seconds	Batch loads and scheduled jobs.
Google-recommended practice	Clever custom code where a managed feature exists.
Without moving the data	Copies and exports. Think external tables, Omni, sharing.
Low-code or no-code	Hand-written Beam or Spark. Think Data Fusion.

ON THE EXAM

Find the qualifier before you read the options. It usually removes two of them.

Two qualifiers that pull opposite ways

WHEN IT SAYS

Least operational overhead

- Serverless first: BigQuery, Dataflow, Pub/Sub
- Serverless Spark rather than clusters
- Managed features over custom code
- Accept a rewrite if it removes servers

WHEN IT SAYS

Minimal changes, fast migration

- Keep the engine you already run
- Spark and Hive jobs on Managed Spark
- Kafka clients on Managed Kafka
- Modernise later, once you are out

THE TIEBREAKER

If a question carries both, the deadline usually wins. Migrate first, modernise second.

How the worked scenarios work

- **Pause** when you hear 'pause and decide', and commit to a letter.
- **Find the deciding sentence** in the situation before you weigh the options.
- **Listen to why the wrong answers are wrong.** That is what transfers to new questions.
- **Watching without answering** feels like learning and teaches almost nothing.

> REMEMBER
>
> Four scenarios close this module. They use only what you have seen so far.

> Westgrid, Larkspur's grid partner, needs total demand per region for each half hour, within five minutes of the half hour closing. Readings land on a Pub/Sub topic at about 1,700 a second, rising sixfold every evening.
>
> The team has no one on call for infrastructure and must choose the option with the least operational overhead.

What should the team build?

A. A Spark Streaming job on a long-running Dataproc cluster that reads the topic

B. A streaming Dataflow pipeline that reads the topic, aggregates by region and half hour, and writes to BigQuery

C. A Cloud Run function per message that updates running totals in Cloud SQL

D. A Kafka Streams application on GKE, fed by Managed Service for Apache Kafka

◆ PAUSE AND DECIDE

What should the team build?

A. Spark Streaming on a long-running Dataproc cluster

Works, but a long-running cluster is yours to size, patch and keep alive through the evening surge.

B. Streaming Dataflow into BigQuery

Serverless, autoscales through the surge, handles time windows natively, and writes straight to the warehouse.

C. A Cloud Run function per message into Cloud SQL

About 150 million calls a day, and Cloud SQL becomes the bottleneck for constant updates.

D. Kafka Streams on GKE with Managed Kafka

Adds a second messaging system and a cluster to run. Kafka fits when you already have Kafka.

THE SIGNAL

'Least operational overhead' plus a streaming workload points to Dataflow almost every time.

> Larkspur's platform team keeps runbooks full of commands such as gcloud dataproc clusters create, and IAM policies that grant roles/dataproc.editor to a pipeline service account.
>
> A new engineer reads that Dataproc is now called Managed Service for Apache Spark, and raises an urgent ticket: every script and policy must be rewritten before the next release.

What is the correct response to the ticket?

A. Move the scripts to the new CLI command group, and keep the existing IAM roles

B. Keep the scripts, but replace the IAM roles with the new Managed Spark roles

C. Migrate to the new product, because Dataproc clusters stop running after the rename

D. Change nothing. The rename did not change the API, CLI or IAM names

◆ PAUSE AND DECIDE

What is the correct response to the ticket?

A Move the scripts to a new CLI command group

There is no new command group to move to. gcloud dataproc is still the CLI.

B Replace the IAM roles with new Managed Spark roles

The roles kept their names. You would be granting roles that do not exist.

C Migrate, because Dataproc clusters stop running

Nothing was migrated or switched off. It is the same service under a new product name.

D Change nothing

Google states the API, client library, CLI and IAM names are unchanged.

THE SIGNAL

Rename questions test whether you treat a new name as a new product. It is not.

> Larkspur's customer app shows each customer their latest meter reading. It must answer in single-digit milliseconds for any of 3.1 million meters, while readings are written at about 1,700 a second, and around 10,000 a second in the evening peak.
>
> A prototype on Cloud SQL slows down every evening.

Which storage service should hold the live readings?

A. Bigtable, with the meter ID leading the row key

B. BigQuery, with the table clustered on the meter ID

C. A larger Cloud SQL for PostgreSQL instance with an index on the meter ID

D. Cloud Storage, with one object per meter overwritten on each reading

◆ PAUSE AND DECIDE

Which storage service should hold the live readings?

A Bigtable, meter ID leading the row key

Built for heavy writes and millisecond reads by key. Leading with the meter ID spreads writes across nodes.

B BigQuery, clustered on meter ID

Superb for scanning history. Not built to fetch one row in a few milliseconds.

C A larger Cloud SQL instance

Buys time, but the evening write surge is exactly where the prototype already struggles.

D One Cloud Storage object per meter

Object reads take tens of milliseconds, and every write replaces a whole object. A lake, not a database.

THE SIGNAL

Single-digit milliseconds, lookups by key, and a heavy write rate: that is the Bigtable sentence.

> Larkspur runs about 300 Spark and Hive jobs on a 40-node Hadoop cluster in its Bristol data centre. The lease ends on 30 June 2027, nine months away.
>
> The team wants to stop managing HDFS and hardware, and must keep code changes to a minimum to hit the date.

Which migration approach fits best?

A. Rewrite the jobs as Dataflow pipelines, then switch off the cluster

B. Translate the jobs to BigQuery SQL and run them with Dataform

C. Copy the data to Cloud Storage and run the existing jobs on Managed Service for Apache Spark

D. Recreate the Hadoop cluster on Compute Engine VMs, with HDFS on persistent disks

◆ PAUSE AND DECIDE

Which migration approach fits best?

A Rewrite as Dataflow pipelines

Perhaps a good long-term home for some jobs. Rewriting 300 in nine months is not a minimal change.

B Translate to BigQuery SQL with Dataform

Same problem. Translating everything is a modernisation project, not a migration.

C Cloud Storage plus Managed Service for Apache Spark

Jobs run largely as they are, Cloud Storage replaces HDFS, and clusters can be created per job.

D Hadoop on Compute Engine VMs

Minimal code changes, but you still manage hardware, just virtual. The question wanted that gone.

THE SIGNAL

'Minimal code changes' plus 'stop managing HDFS' means Cloud Storage plus the Spark service. Modernise later.

Module 01 in five lines

- Five jobs: **ingest, process, store, serve, govern and operate**.
- Ingest by what moves: **events, database changes, or files**.
- **New pipelines** go to Dataflow. **Existing Spark** goes to the Spark service.
- The April 2026 renames changed **names, not products**. Use the question's name.
- **Find the qualifier first.** It usually removes two options.

COMING UP

Next: module 02, IAM and organisation policies for data.

MODULE 02 · SECTION 1 - DESIGNING DATA PROCESSING SYSTEMS

IAM and Organisation Policies for Data

Who can touch the data, which identity a pipeline runs as, and the rules nobody can break.

SECTION 1 of 5 EXAM WEIGHT 22% MODULE 2 of 45

admin@sthithapragna.com

An IAM grant only ever adds access. To **take access away**, you need a different tool.

Grants flow down, never up

01

Organisation

larkspur.energy. A grant here reaches everything.

02

Folder

uk, de and shared. Groups projects by country or team.

03

Project

lk-warehouse-prod. Where billing and most grants live.

04

Dataset or bucket

curated, raw, marts. The usual home for data grants.

05

Table or view

The finest level. Use it when a dataset is too broad.

ON THE EXAM

A role granted at the organisation reaches every table in every project. No lower grant can switch it off.

Basic, predefined, custom

KIND	EXAMPLES	VERDICT
Basic	Owner, Editor, Viewer	Avoid. Google says they grant excessive and uneven access.
Predefined	BigQuery Data Viewer, Dataflow Worker	The default answer. Built by Google, kept up to date.
Custom	Your own list of permissions	Only when no predefined role is narrow enough. You maintain it.

THE DETAIL THEY TEST

Custom roles live at the organisation or a project, never a folder. An option that says otherwise is wrong.

The BigQuery roles worth knowing

ROLE	WHAT IT LETS YOU DO	LOWEST LEVEL
Data Viewer	Read tables and their metadata	Table
Data Editor	Read and change tables, views, routines	Table
Data Owner	Everything above, plus manage access	Table
Metadata Viewer	See schemas and access, not the data	Table
Job User	Run jobs and queries in a project	Project
User	Run jobs, and create datasets	Project
Admin	Everything in the project	Project

THE TIEBREAKER

Data roles say what you can read. Job User says where you can run the query. Most people need one of each.

Reading data is not the same as running a query

DATA VIEWER

What you can read

- Granted where the data lives
- On a dataset, table or view
- Has no permission to run jobs
- Alone, it cannot run a query

JOB USER

Where the query runs

- Granted where the work is billed
- On a project only
- Has no permission to read data
- Alone, it reads nothing

THE PATTERN

Larkspur analysts get Job User on lk-analytics-prod, and Data Viewer on the curated dataset only.

Granting a dataset role in SQL

```
1  -- Read access to one dataset, for a group, in BigQuery's own SQL
2  GRANT `roles/bigquery.dataViewer`
3    ON SCHEMA `lk-warehouse-prod.curated`
4    TO "group:analysts@larkspur.energy";
5
6  -- Job User is project-level, so it is granted with IAM, not SQL
7  gcloud projects add-iam-policy-binding lk-analytics-prod \
8    --member="group:analysts@larkspur.energy" \
9    --role="roles/bigquery.jobUser"
```

3 SCHEMA means dataset. The grant stops at curated, so raw stays private.

4 A group, never a list of people. Joiners and leavers change the group, not the policy.

9 The query runs and is billed in the analytics project, not the warehouse.

THE TAKEAWAY

Grant to groups, at the lowest level that does the job. That sentence answers a lot of questions.

Buckets: one rule for every object

- **Uniform bucket-level access** turns off per-object ACLs. Access comes from IAM only.
- Enforced by default for **organisations created since 3 May 2024**.
- **Public access prevention** stops anyone making the bucket or its objects public.
- **Fine-grained ACLs** on single objects are the legacy model and hard to audit.

> ON THE EXAM
>
> If an option manages access object by object, it is almost always the wrong one.

A pipeline is not a person. It runs as a **service account**, and that account needs its own least privilege.

Three identities behind every Dataflow job

01

The launcher

You or Cloud Build. Needs Service Account User on the worker account.

02

Dataflow service agent

Google-managed. Checks quota and starts the worker VMs.

03

Worker service account

What the code runs as. Reads sources, writes sinks.

ON THE EXAM

By default, workers run as the Compute Engine default service account. Replace it with your own, least privilege.

Keys, or something better

SERVICE ACCOUNT KEY

A copy of the master key

- Long-lived until someone deletes it
- Can leak through code, laptops, tickets
- Audit logs show the account, not who used it
- Blocked by default on newer organisations

KEYLESS OPTIONS

A visitor badge that expires

- Attach the account to the VM, job or function
- Impersonate it, with Token Creator
- Workload Identity Federation for outside systems
- Short-lived tokens, both names in the logs

THE TIEBREAKER

If an option creates a key and stores it somewhere, it is almost always the wrong option.

IAM conditions: when, and on what

- **Time-bound access.** A contractor's role that expires on a set date, with no one remembering to remove it.
- **Resource name.** Access to tables or objects whose names start with a prefix.
- Conditions attach to a **grant**, so you narrow a predefined role rather than inventing a custom one.
- A condition still only narrows a grant. It cannot block a principal who has access another way.

> ON THE EXAM
>
> Temporary access is a conditions question. 'Remember to revoke it' is never the exam's answer.

IAM decides **who** can act. Organisation policies decide **what is allowed at all**, even for owners.

Constraints a data engineer meets

CONSTRAINT	WHAT IT ENFORCES
gcp.resourceLocations	New resources only in allowed locations, e.g. Frankfurt.
iam.allowedPolicyMemberDomains	Grants only to identities in your own domain.
storage.uniformBucketLevelAccess	No per-object ACLs on any bucket.
storage.publicAccessPrevention	Nothing in Cloud Storage can be made public.
compute.vmExternalIpAccess	VMs, including Dataflow workers, get no external IP addresses.
iam.managed.disableServiceAccountKeyCreation	No new service account keys.

WATCH FOR

No external IPs on workers means Private Google Access. Module 10.

Secure by default, since May 2024

- Organisations created **on or after 3 May 2024** start with seven constraints already enforced.
- **Service account keys** cannot be created or uploaded.
- **Default service accounts** no longer receive the Editor role automatically.
- **Domain restricted sharing** and **uniform bucket-level access** are on.
- Older study material often answers with a downloaded key. On a new organisation, that fails.

> ON THE EXAM
>
> If a scenario hits an organisation policy error when creating a key, the fix is keyless, not disabling the policy.

Resource locations: what it does and does not do

- Set it on a **folder**, and every project below inherits it. Larkspur sets Frankfurt on the de folder.
- **Not retroactive.** It applies only to newly created resources. Audit what already exists.
- **Global resources**, and ones without a location choice, are not affected.
- It controls **where resources are created**. Google says it is not a data storage commitment.

> COMING UP
>
> For full residency guarantees, add Assured Workloads and VPC Service Controls. Module 05 builds that design.

IAM or organisation policy?

USE IAM WHEN

It is about a principal

- A person or workload needs access
- Access should be narrowed or expire
- Different teams need different data
- Granted by resource owners

USE AN ORG POLICY WHEN

It is about a setting

- No one may ever choose that setting
- Owners must be bound too
- It must apply across many projects
- Set centrally, inherited down

THE TIEBREAKER

A question about who: IAM. A question about what nobody may configure: organisation policy.

When you really must remove access

- **Remove the grant** where it was made. Usually the right answer, and usually higher up than you think.
- **Deny policies** block chosen permissions, whatever any allow policy says. Checked before allows.
- **Principal access boundaries** limit which resources a principal is eligible to reach at all.
- Adding a narrower role lower down removes nothing. That is the classic distractor.

> ON THE EXAM
>
> If a user still has access after you tightened a dataset, look up the hierarchy for an inherited grant.

> Larkspur's warehouse lives in project lk-warehouse-prod, with datasets raw, curated and marts. Sixty analysts, all in the group analysts@larkspur.energy, run their queries from project lk-analytics-prod.
>
> They need to query the curated dataset. They must not be able to read raw, which still holds unmasked personal data.

Which access design follows least privilege?

A Grant BigQuery Data Viewer to the analysts group on project lk-warehouse-prod

B Grant BigQuery Admin to the analysts group on the curated dataset

C Grant Data Viewer on the curated dataset and Job User on lk-analytics-prod, both to the group

D Grant the Viewer basic role on both projects to each analyst individually

◆ PAUSE AND DECIDE

Which access design follows least privilege?

A Data Viewer on the warehouse project

A project grant flows down to every dataset, including raw. And nothing here lets them run a query.

B BigQuery Admin on curated

Far more than reading: they could change data and access. Still no permission to run jobs.

C Data Viewer on curated, Job User on the analytics project

Read access stops at curated, and queries run and bill where they belong. Granted to a group.

D Viewer basic role, per person

A basic role is too broad, reaching raw too, and sixty individual grants are impossible to keep tidy.

THE SIGNAL

'Must not read raw' rules out any project-level data grant. Reading and running are two separate grants.

> Larkspur's streaming Dataflow job in lk-ingest-prod reads a Pub/Sub subscription and writes to the BigQuery raw dataset. It runs as the Compute Engine default service account, which holds the Editor role. Cloud Build launches the job.
>
> A security review demands least privilege, with no long-lived credentials.

What should the team do?

A. Create a dedicated worker service account with only the roles the job needs, and let Cloud Build act as it

B. Keep the default service account, remove Editor, and grant it Owner on the raw dataset

C. Create a key for a new service account and store it in Cloud Storage for the job to load

D. Grant the Dataflow service agent Pub/Sub Subscriber and BigQuery Data Editor on the project

◆ PAUSE AND DECIDE

What should the team do?

A Dedicated worker account; Cloud Build acts as it

Dataflow Worker, Subscriber on one subscription, data roles on raw only. Cloud Build gets Service Account User.

B Default account, Owner on raw

Other services share the default account, and Owner can change access to raw. Least privilege it is not.

C A key stored in Cloud Storage

A long-lived credential sitting in a bucket. The review ruled this out, and new organisations block it.

D Roles on the Dataflow service agent

The service agent starts workers. Your code runs as the worker account, so these grants miss.

THE SIGNAL

'Runs as the default service account' is a signal. The fix is a dedicated worker account, not a bigger grant.

German customer data must stay in Germany. All of Larkspur Energie's projects sit in the de folder.

The security team needs a guarantee that nobody, project owners included, can create a BigQuery dataset or Cloud Storage bucket outside Frankfurt in any current or future project in that folder.

What should the security team do?

A Remove the Owner role from everyone on the projects in the de folder

B Add an IAM condition to BigQuery Data Editor grants that checks the dataset's location

C Deploy a Cloud Run function that deletes any dataset or bucket created outside europe-west3

D Set the resource locations constraint on the de folder to in:europe-west3-locations

◆ PAUSE AND DECIDE

What should the security team do?

A. Remove Owner from everyone

Other roles can still create datasets and buckets anywhere. Fewer owners is not a location rule.

B. An IAM condition on Data Editor

Narrows one grant for one role. Owners and other roles are untouched, so it is not a guarantee.

C. A function that deletes violations

Reactive. The data has already landed outside Germany before anything is deleted.

D. Resource locations constraint on the folder

Blocks creation outside Frankfurt for everyone, and every future project inherits it.

THE SIGNAL

'Nobody, owners included' plus 'future projects' is organisation policy language, set on the folder.

> Larkspur's Google Cloud organisation was created in 2025. Its deployment pipeline runs in GitHub Actions, outside Google Cloud, and must deploy Dataflow templates to lk-ingest-prod.
>
> An engineer tries to create a service account key for the pipeline and gets an organisation policy error. The team wants no long-lived credentials and no new infrastructure to run.

What should the team do?

A. Ask an administrator to disable the key creation constraint, then store the key as a GitHub secret

B. Use Workload Identity Federation so the pipeline can impersonate a deployment service account

C. Store a senior engineer's own credentials in the pipeline, since that account already has access

D. Run a self-hosted runner on a VM with the deployment service account attached

◆ PAUSE AND DECIDE

What should the team do?

A. Disable the constraint and store a key

Switches off a deliberate safeguard to create the long-lived credential the team said it did not want.

B. Workload Identity Federation for GitHub

GitHub's own identity token is exchanged for short-lived Google credentials. No key, nothing to run.

C. A person's credentials in the pipeline

Ties production to one human, breaks when they leave, and the logs blame them for every deployment.

D. Self-hosted runner on a VM

Keyless, and it would work. But it is new infrastructure to patch and run, which the team ruled out.

THE SIGNAL

An outside system plus 'no keys' is Workload Identity Federation. The policy error is a hint, not a bug.

Module 02 in five lines

- Grants flow **down** and only **add**. A narrower grant lower down removes nothing.
- **Predefined roles, to groups, at the lowest level** that does the job. Never basic roles.
- BigQuery needs two grants: a **data role** where data lives, **Job User** where queries run.
- Workloads get **dedicated service accounts** and **keyless** credentials.
- **Nobody, owners included** means an organisation policy, set on a folder or the organisation.

COMING UP

Next: module 03, encryption and key management.

MODULE 03 · SECTION 1 - DESIGNING DATA PROCESSING SYSTEMS

Encryption and Key Management

Default keys, CMEK, Cloud HSM, Cloud EKM, CSEK and crypto-deletion, and how to tell which one a question wants.

SECTION 1 of 5 EXAM WEIGHT 22% MODULE 3 of 45

admin@sthithapragna.com

Everything is already encrypted at rest. The question is always **who holds the key**.

Envelope encryption: a key in a lockbox

01

Data key

A data encryption key locks each chunk of your data.

02

Key in KMS

A key encryption key wraps the data key. It never leaves the key service.

03

Stored together

Ciphertext sits beside its wrapped data key. Neither is useful alone.

04

To read

The service asks KMS to unwrap the data key, then decrypts.

WHY IT MATTERS

Whoever controls the key encryption key controls the data. That is the lever CMEK hands to you.

Who holds the key: a ladder of control

OPTION	WHERE THE KEY LIVES	CHOOSE IT WHEN
Google default	Google's key service	No stated requirement. The least effort.
CMEK, Cloud KMS	Your key, in Cloud KMS	You must rotate, disable or audit the key yourself.
CMEK, Cloud HSM	Your key, in Google's HSMs	The key must live in certified hardware.
CMEK, Cloud EKM	Outside Google, with a partner	Google must never hold the key material.
CSEK	You send it with each request	Cloud Storage only. You manage everything.
Client-side	Never reaches Google	Google must never see plaintext at all.

THE TIEBREAKER

Climb only as far as the requirement forces you. Every rung adds work and risk.

Default encryption or CMEK?

GOOGLE DEFAULT

Nothing to do

- Encrypted at rest automatically
- Google manages and rotates keys
- No extra cost or setup
- You cannot revoke the key

CMEK

Your key, Google's service

- You set the rotation schedule
- You can disable or destroy it
- Every use of the key is logged
- You must grant and manage access

THE TIEBREAKER

No requirement to control the key means default encryption. 'Control', 'revoke' or 'rotate' means CMEK.

Three things that make CMEK work

- **Location match.** The key ring sits in the same location as the data. A London dataset needs a London key.
- **The service agent** needs Cloud KMS CryptoKey Encrypter/Decrypter on the key. Not you, the service.
- **Defaults.** A BigQuery dataset default key, or a bucket default key, protects new tables and objects.
- **Separate project.** Keys in their own project, so key admins and data admins are different people.

> ON THE EXAM
>
> A CMEK setup that fails with a permission error is nearly always the service agent missing its role.

A CMEK-protected table at Larkspur

```
1  -- Billing data, protected by Larkspur's own key
2  CREATE TABLE `lk-warehouse-prod.curated.bills` (
3    bill_id      STRING,
4    account_id   STRING,
5    amount_gbp   NUMERIC
6  )
7  OPTIONS (
8    kms_key_name =
9      'projects/lk-kms-prod/locations/europe-west2/keyRings/lk-uk/cryptoKeys/billing'
10 );
```

9 The key lives in lk-kms-prod, a separate project for key administration.

9 europe-west2 matches the dataset's London location. A key elsewhere will not work.

BigQuery always protects a new table with the key's primary version.

WORTH KNOWING

The key name never includes a version. BigQuery uses whichever version is primary.

What rotation does, and does not, do

- Rotation creates a **new key version** and makes it primary. New data uses it.
- Existing data is **not re-encrypted**. Older versions stay active to decrypt it.
- To retire an old version, **re-encrypt the data** first. Then you can destroy that version.
- Rotation limits how much data one version protects. It does not rescue a compromised key.

> THE TRAP
>
> If a key is compromised, rotating it is not enough. The old version still decrypts the old data.

Disable, schedule, destroy

STATE	CAN DATA BE READ?	REVERSIBLE?
Enabled	Yes	–
Disabled	No, while disabled	Yes, re-enable it
Scheduled for destruction	No	Yes, 30 days by default
Destroyed	Never again	No. The data is crypto-shredded

THE TIEBREAKER

Disable is the emergency brake. Destroy is the shredder. Questions about suspected misuse want disable first.

Destroy the only key, and the data becomes noise. That is a **deletion strategy**, not an accident.

Cloud HSM or Cloud EKM?

CLOUD HSM

Hardware, run by Google

- Keys in FIPS 140-2 Level 3 HSMs
- Fully managed, no HSM cluster
- Used exactly like any CMEK key
- Google's hardware holds the key

CLOUD EKM

Your key manager, outside Google

- Key material never leaves your partner EKM
- Fortanix, Futurex or Thales
- Every request carries a justification
- If your EKM is down, so is your data

THE TIEBREAKER

'Must be in hardware' is Cloud HSM. 'Keys must never be held by Google' or 'our own HSM' is Cloud EKM.

CSEK, and client-side encryption

- **CSEK is now a Cloud Storage feature.** CSEK for Compute Engine is deprecated. BigQuery never supported it.
- With CSEK you **send the key with every request**. Google does not keep it. Lose it, lose the data.
- **Client-side encryption** encrypts before upload, so Google never sees plaintext.
- The cost: **nothing in Google Cloud can query or process** client-side encrypted data.

> ON THE EXAM
>
> An option offering CSEK for BigQuery is wrong, however attractive it sounds.

One key per customer, with AEAD functions

```sql
-- 1. One keyset per customer, in a tightly restricted dataset
CREATE TABLE secure.customer_keys AS
SELECT customer_id, KEYS.NEW_KEYSET('AEAD_AES_GCM_256') AS keyset
FROM curated.customers;

-- 2. Encrypt personal data with that customer's own keyset
SELECT c.customer_id,
       AEAD.ENCRYPT(k.keyset, c.email, c.customer_id) AS email_enc
FROM curated.customers c JOIN secure.customer_keys k USING (customer_id);

-- 3. Right to erasure: delete one keyset row
DELETE FROM secure.customer_keys WHERE customer_id = 'C-1048812';
```

3 Each customer gets a keyset, generated inside BigQuery. No external tooling.

12 Delete the keyset, and every copy of that customer's ciphertext is unreadable.

HONEST LIMIT

Deleted keysets survive time travel, up to seven days, plus seven days fail-safe. Shorten the keys dataset's window.

Making CMEK compulsory

CONSTRAINT	WHAT IT ENFORCES
`gcp.restrictNonCmekServices`	Listed services, such as storage or BigQuery, refuse new resources without CMEK.
`gcp.restrictCmekCryptoKeyProjects`	CMEK keys may only come from listed projects or folders.
`Both together`	Every new resource uses CMEK, from your approved key project.

REMEMBER

Like all organisation policies here, these apply to new resources. Audit and migrate what already exists.

Autokey: CMEK without the paperwork

- Creates **key rings and keys on demand** when a resource is created, and grants the service agents.
- Keys land in the **same location** as the resource, in a **dedicated key project** for the folder.
- BigQuery gets a **dataset default key**. Buckets, Pub/Sub topics and Dataflow jobs get one key each.
- Pairs naturally with the CMEK organisation policies: compulsory, and effortless.

> ON THE EXAM
>
> 'CMEK everywhere, least operational overhead' is a strong hint toward Autokey.

Encryption in transit, briefly

- Traffic between Google services is **encrypted by default** on Google's network.
- Your clients use **TLS**. For Cloud SQL, the **Auth Proxy** handles it. Module 21.
- On-premises links use **Cloud VPN**, encrypted, or **Interconnect**, private. Module 07.
- Private is not the same as encrypted. Interconnect is private, but add encryption if a rule demands it.

> ON THE EXAM
>
> In-transit questions usually turn on the connection, not the key. Most are answered in later modules.

Phrases that pick the option

THE QUESTION SAYS	THE ANSWER LEANS TO
Control, rotate or revoke our keys	CMEK with Cloud KMS
Keys must be in **hardware**	CMEK with Cloud HSM
Keys **never held by Google**, our own HSM	Cloud EKM
Google must **never see the plaintext**	Client-side encryption
Delete **one customer's** data everywhere	AEAD keysets, crypto-deletion
Every new resource must use CMEK	CMEK organisation policies, perhaps Autokey

ON THE EXAM

No phrase like these at all? Default encryption is already doing the job.

> Larkspur's auditors require that billing data in BigQuery and Cloud Storage be encrypted with keys Larkspur can rotate on its own schedule and revoke immediately, with every key use logged.
>
> Analysts must keep querying the data in BigQuery, and the platform team wants the least operational overhead.

What should Larkspur implement?

A Keep Google default encryption, and document Google's own rotation schedule for the auditors

B Use customer-supplied keys, sent with every BigQuery query and Cloud Storage request

C Encrypt every file client-side with an on-premises key before loading it

D Use CMEK with Cloud KMS keys in europe-west2, granting each service agent Encrypter/Decrypter

◆ PAUSE AND DECIDE

What should Larkspur implement?

A Google default encryption

Encrypted, yes, but Larkspur cannot set the schedule or revoke the key. The audit asks for both.

B Customer-supplied keys

BigQuery does not support CSEK at all, and sending keys with every request is heavy overhead.

C Client-side encryption

Over-climbs the ladder. Analysts could no longer query the data in BigQuery.

D CMEK with Cloud KMS

Larkspur rotates and revokes, KMS logs every use, and BigQuery queries work as normal.

THE SIGNAL

'Rotate on our schedule' plus 'revoke' is CMEK. 'Must still query' rules out anything above it.

> An engineer creates a key called billing in project lk-kms-prod, in europe-west2. She holds the Cloud KMS Admin role. She then tries to create a CMEK-protected table in a europe-west2 dataset in lk-warehouse-prod.
>
> Table creation fails with a permission error on the key.

What is the most likely fix?

A Grant the engineer Cloud KMS CryptoKey Encrypter/Decrypter on the key

B Grant the BigQuery service account for lk-warehouse-prod Encrypter/Decrypter on the key

C Move the key into lk-warehouse-prod, because keys must be in the same project as the data

D Recreate the key ring in the US multi-region, where BigQuery's key service runs

◆ PAUSE AND DECIDE

What is the most likely fix?

A. Grant the engineer the role

She does not encrypt anything herself. BigQuery does, using its own service account.

B. Grant BigQuery's service account the role

BigQuery wraps and unwraps data keys as its own service account, so that account needs the role.

C. Move the key into the data project

Keys in a separate project are recommended, not forbidden. Project is not the problem.

D. Move the key ring to the US

The opposite of the rule. The key must match the dataset's location, and both are europe-west2.

THE SIGNAL

A permission error on the key during CMEK setup: check the service agent first.

> Larkspur must honour right-to-erasure requests. A customer's personal data appears in dozens of BigQuery tables, and in table snapshots kept for audit.
>
> Finding and deleting every row on each request is impractical. The team wants a native BigQuery approach that makes one customer's data unreadable everywhere at once.

What should the team implement?

A. Encrypt personal columns with AEAD, one keyset per customer, and delete the keyset on request

B. Protect the dataset with one CMEK key, and destroy that key when a request arrives

C. Apply dynamic data masking to the personal columns for that customer

D. Rotate the dataset's CMEK key after each erasure request

◆ PAUSE AND DECIDE

What should the team implement?

A AEAD with a keyset per customer

One deleted row shreds that customer's data in every table and snapshot, and no one else's.

B Destroy one dataset CMEK key

Shreds every customer at once. One erasure request would destroy the whole dataset.

C Dynamic data masking

Hides values from some readers. The data is still stored and readable by others, so nothing is erased.

D Rotate the CMEK key

Rotation re-encrypts nothing and deletes nothing. The old version still decrypts everything.

THE SIGNAL

'One customer, everywhere at once' is per-customer keys. A key shared by everyone can only shred everyone.

> A Cloud Storage bucket of meter exports uses a CMEK key, K1, as its default. The security team believes K1 may have been compromised.
>
> They want the existing objects protected by a fresh key, K1 retired, and a guarantee that no new bucket in the organisation can be created without CMEK.

What should the team do?

A. Rotate K1 and keep using the bucket, because rotation replaces the key material

B. Destroy K1 immediately, so that nobody, including any attacker, can use it

C. Re-encrypt into a bucket using a new key, destroy K1, and enforce restrictNonCmekServices

D. Disable K1 and switch the bucket to Google default encryption

◆ PAUSE AND DECIDE

What should the team do?

A. Rotate K1

Existing objects stay under the old version, which still decrypts them. Nothing is re-protected.

B. Destroy K1 immediately

Every object still encrypted with K1 becomes unreadable. That is data loss, not remediation.

C. Re-encrypt under a new key, destroy K1, enforce the policy

Data moves to a fresh key before K1 goes, and the policy covers every future bucket.

D. Disable K1, use default encryption

Removes CMEK when the requirement was more of it, and does nothing for future buckets.

THE SIGNAL

Compromised key: re-encrypt first, destroy second. 'No new resource without CMEK' adds the policy.

Module 03 in five lines

- Everything is encrypted at rest. The question is **who holds the key**.
- **Control, rotate, revoke** means CMEK. Location must match, and the **service agent** needs the role.
- **Rotation re-encrypts nothing.** A compromised key needs re-encryption, then destruction.
- **Hardware** is Cloud HSM. **Never held by Google** is Cloud EKM. CSEK is Cloud Storage only.
- **Destroy a key to delete data.** Per-customer AEAD keysets shred one customer at a time.

COMING UP

Next: module 04, personal data and Sensitive Data Protection.

MODULE 04 · SECTION 1 - DESIGNING DATA PROCESSING SYSTEMS

Personal Data and Sensitive Data Protection

Find personal data, then hide it in a way that keeps the data useful: tokens, hashes, masks and generalisation.

SECTION 1 of 5 EXAM WEIGHT 22% MODULE 4 of 45

admin@sthithapragna.com

Hiding personal data is easy. Hiding it **while keeping the data useful** is what the exam tests.

Find it, hide it, check it

01

Discover

Continuous profiles show which tables and buckets hold sensitive data.

02

Inspect

A deep scan finds every occurrence in one table, file or stream.

03

De-identify

Redact, mask, tokenise or generalise, using a reusable template.

04

Measure risk

Check that what is left cannot single people out.

CURRENCY CHECK

Sensitive Data Protection is the new name for Cloud DLP. The API is still dlp.googleapis.com, and the guide says Cloud DLP.

infoTypes: what Sensitive Data Protection looks for

- **Built-in infoTypes** cover common data: EMAIL_ADDRESS, PHONE_NUMBER, IBAN_CODE and many national IDs.
- **Custom infoTypes** add your own patterns: a regular expression, or a dictionary of words.
- Each finding has a **likelihood**, from very unlikely to very likely, so you can set a threshold.
- Larkspur adds a custom infoType for its **meter IDs**, which no built-in type knows.

> WORTH KNOWING
>
> An inspect template saves the infoTypes and threshold, so every job and pipeline uses the same rules.

Discovery or an inspection job?

DISCOVERY

Where is it, across everything?

- Organisation, folder or project scope
- Continuous: new tables are profiled
- Sensitivity and data risk per table
- Sends profiles to Security Command Center

INSPECTION JOB

Exactly where, in this one?

- One table, bucket or dataset
- Finds every occurrence, row by row
- Can save findings to BigQuery
- Run on demand or on a schedule

THE TIEBREAKER

'Across the organisation, as data is added' is discovery. 'Every occurrence in this dataset' is an inspection job.

The de-identification toolbox

METHOD	WHAT YOU GET	REVERSIBLE?	JOINS STILL WORK?
Redaction	The value is removed	No	No
Replacement	A fixed word, or the infoType name	No	No
Character masking	****1234	No	No
Crypto hash	A one-way token, using a secret key	No	Yes
Deterministic encryption	A token that can be decrypted	Yes	Yes
Format-preserving	A token shaped like the original	Yes	Yes
Bucketing, date shifting	Ranges, or shifted dates	No	Partly

THE TIEBREAKER

Two questions pick the method: must anyone ever get the real value back, and must the column still join?

Same input, same key, same token. That is what **keeps your joins alive**.

Pseudonymised or anonymised?

PSEUDONYMISED

Reversible with a key

- Deterministic or format-preserving tokens
- Records still link to one person
- Still personal data under GDPR
- Protect the key like the data

ANONYMISED

No way back

- Redaction, generalisation, aggregation
- Nobody can be singled out
- No longer personal data, if truly so
- Harder to achieve than it looks

THE TRAP

A keyed hash still links one person's records, so treat hashed data as pseudonymised, not anonymous.

Deterministic or format-preserving encryption?

DETERMINISTIC ENCRYPTION

The default choice

- Google's recommended method
- No limits on the input
- Much faster
- Output does not look like the input

FORMAT-PRESERVING ENCRYPTION

Only when shape matters

- Keeps the length and alphabet
- Passes legacy format checks
- Fewer security guarantees
- Needs long enough inputs

THE TIEBREAKER

Choose format-preserving only when something downstream insists on the original format.

Keys for tokens you may reverse

- **KMS-wrapped key.** Your token key is encrypted by a Cloud KMS key. The recommended choice.
- **Transient key.** Generated per request and thrown away. You can never re-identify.
- **Unwrapped key.** The raw key sits in your configuration. Google says do not.
- Re-identification is a separate permission. Most people get tokens; a few get the key.

> ON THE EXAM
>
> 'Must be able to recover the original value later' rules out transient keys and hashing.

Known columns, or text that might contain anything?

RECORD TRANSFORMATIONS

You know the column

- Transform the email column, always
- No inspection needed
- Fast and predictable
- Best for tables and CSV files

INFOTYPE TRANSFORMATIONS

It could be anywhere

- Inspect the text first
- Transform only what is found
- The rest of the text survives
- Best for notes, chats, documents

THE PATTERN

Larkspur's call-centre notes hide bank details in free text. That is an infoType transformation.

A de-identify template for the email column

```
{
  "deidentifyConfig": {
    "recordTransformations": {
      "fieldTransformations": [{
        "fields": [{"name": "email"}],
        "primitiveTransformation": {
          "cryptoDeterministicConfig": {
            "cryptoKey": {"kmsWrapped": {
              "wrappedKey": "CiQAz7...",
              "cryptoKeyName": "projects/lk-kms-prod/.../cryptoKeys/sdp"}},
            "surrogateInfoType": {"name": "EMAIL_TOKEN"}
        }}}]}}
}
```

3 A record transformation: the email column is known, so nothing is inspected.

7-8 Deterministic tokens with a KMS-wrapped key: joins work, and the key team can reverse them.

THE TAKEAWAY

Save it once as a template. Dataflow, jobs and the console reuse it by name.

De-identify on the way in

01

Restricted landing

Raw files in a locked bucket, CMEK, short retention.

02

Dataflow

Calls Sensitive Data Protection with your de-identify template.

03

BigQuery curated

Only tokens and masked values reach the analysts.

04

Original kept

If the business needs it, the raw copy stays locked away.

WORTH KNOWING

Google offers a Dataflow template for exactly this: Cloud Storage CSV to BigQuery, de-identified. It is Beta.

No names, and still identifiable

- **Quasi-identifiers** such as postcode, birth date and gender can single people out when combined.
- **k-anonymity**: every record matches at least k minus one others on those columns.
- **l-diversity**: each such group also holds at least l different sensitive values.
- Fix a low score by **generalising**: a postcode district instead of a full postcode, age bands instead of birth dates.

> ON THE EXAM
>
> Sensitive Data Protection's risk analysis computes these for a BigQuery table before you share it.

Which tool for which privacy job

THE QUESTION WANTS	REACH FOR
Where is personal data, **across the organisation**?	Discovery profiles
Every occurrence in this dataset	An inspection job
Join on a value nobody may see	Deterministic tokens, or a keyed hash
Recover the original later	Deterministic encryption, KMS-wrapped key
Some readers see real values, others masked	BigQuery dynamic data masking, module 29
Erase **one person** everywhere	AEAD keysets, module 03
Share without **re-identification**	Risk analysis, then generalise

THE TIEBREAKER

If two options both hide the data, pick the one that keeps the use the question cares about.

> Larkspur's analysts must join billing customers to app sign-ups on email address, but must never see a real address.
>
> Occasionally the fraud team needs to recover the real email for a flagged account.

How should the email field be de-identified in both datasets?

A Replace email with a keyed crypto hash in both datasets

B Deterministic encryption with the same KMS-wrapped key in both datasets

C Redact the email field before either dataset is loaded

D Mask every character of the email except the domain name

◆ PAUSE AND DECIDE

How should the email field be de-identified in both datasets?

A **Keyed crypto hash**

Joins would work, but a hash can never be reversed. The fraud team could not recover anything.

B **Deterministic encryption, same wrapped key**

Same email, same token, so joins work. The key team can decrypt one flagged token.

C **Redact the field**

Nothing left to join on, and nothing to recover.

D **Mask all but the domain**

Thousands of customers would share one masked value, so the join silently matches the wrong people.

THE SIGNAL

Join plus recover means a reversible, consistent token. Join alone would allow a hash.

> Larkspur now has about 400 projects. The security team must know which BigQuery tables and Cloud Storage buckets hold personal data, including tables created next month.
>
> Results must appear in Security Command Center, with the least ongoing effort.

What should the security team set up?

A. A scheduled inspection job for each existing table, run by Cloud Scheduler

B. A weekly Dataflow job that samples every table and writes a report

C. A request for each data owner to tag sensitive columns in Knowledge Catalog

D. Organisation-level discovery, publishing profiles to Security Command Center

◆ PAUSE AND DECIDE

What should the security team set up?

A **Scheduled inspection jobs per table**

Covers only tables that exist today, and someone must add a job for every new one.

B **A weekly Dataflow sampler**

Custom code to build and maintain, when a managed feature already does this.

C **Ask owners to tag columns**

Depends on people remembering. Untagged data is exactly the risk.

D **Organisation-level discovery to Security Command Center**

Continuous, covers new tables automatically, and publishes where security already looks.

THE SIGNAL

'Across the organisation', 'including new tables' and 'least effort' is discovery.

> Call-centre agents have pasted customers' IBANs into a free-text notes column in Oracle billing. The data reaches BigQuery's raw dataset through Datastream.
>
> Analysts read the notes in the curated dataset. Bank details must be removed, but the rest of each note must stay readable.

What should the pipeline that builds curated do?

A Inspect notes for IBAN_CODE and replace each finding with its infoType name

B Apply a record transformation that redacts the whole notes column

C Put a column-level policy tag on notes so analysts cannot read the column

D Apply format-preserving encryption to the whole notes column

◆ PAUSE AND DECIDE

What should the pipeline that builds curated do?

A Inspect for IBANs and replace each finding

Only the bank details change, to the word IBAN_CODE. The rest of the note survives.

B Redact the whole column

Removes the bank details, and every useful word with them.

C Column-level policy tag

Hides the whole column from analysts. They lose the notes they need.

D Format-preserving encryption on the column

Encrypts the entire note, so nothing is readable. FPE suits short structured values.

THE SIGNAL

'Free text' plus 'keep the rest readable' is an infoType transformation, never a whole-column one.

A university wants Larkspur's half-hourly consumption by full postcode for a heat-pump study. Names, emails and account numbers are already removed.

Some postcodes cover only two or three homes. Larkspur must be confident no household can be singled out.

What should Larkspur do before sharing?

A Share as it is, since nothing left in the data is a direct identifier

B Encrypt the postcode deterministically and give the university the key

C Run a k-anonymity risk analysis, then generalise postcodes until groups pass

D Replace each postcode with a keyed crypto hash before sharing

◆ PAUSE AND DECIDE

What should Larkspur do before sharing?

A **Share as it is**

A postcode with three homes plus a consumption pattern can point to one household. Quasi-identifiers.

B **Encrypt, and share the key**

With the key, the university simply decrypts the postcode. Nothing is protected.

C **Risk analysis, then generalise**

Measures the real risk, then widens postcodes to districts until no group is too small.

D **Hash each postcode**

A three-home group is still a three-home group, just with a different label. The risk is unchanged.

THE SIGNAL

Removing names is not anonymising. Small groups need generalisation, and risk analysis tells you how far.

Module 04 in five lines

- The exam asks how to hide personal data **and keep it useful**.
- **Discovery** for across the organisation. **Inspection jobs** for every occurrence here.
- **Join and recover**: deterministic encryption, KMS-wrapped key. **Join only**: a keyed hash.
- **Known column**: record transformation. **Free text**: infoType transformation.
- **No names is not anonymous.** Measure risk, then generalise quasi-identifiers.

COMING UP

Next: module 05, residency, perimeters and environments.

MODULE 05 · SECTION 1 - DESIGNING DATA PROCESSING SYSTEMS

Residency, Perimeters and Environments

Where data may live, where it may go, and how to stop a test from breaking production.

SECTION 1 of 5 EXAM WEIGHT 22% MODULE 5 of 45

admin@sthithapragna.com

Location decides where data **rests**. A perimeter decides where it can **go**. Environments decide who can **break** it.

Region, dual-region, multi-region

LOCATION TYPE	EXAMPLE	WHAT IT PROMISES
Region	europe-west2, London	Data stays in that one region.
Dual-region	Two named regions, Cloud Storage	Two specific places, for resilience.
Multi-region	EU, US	Somewhere within a large area, not one country.

THE TRAP

If the rule names a country, a multi-region is not enough. EU keeps data in the EU, not in Germany.

BigQuery's location rules

- Location is set on the **dataset**, and **cannot be changed** after creation.
- A query **runs where its data lives**, and every table in it must share one location.
- A London table and a Frankfurt table **cannot be joined** in one query.
- To move data, **copy the dataset** to the new location, or recreate it there.

> ON THE EXAM
>
> A 'not found in location' error usually means the query mixed locations, not that the table is missing.

Residency or sovereignty?

DATA RESIDENCY

Where the data is

- Stored in a named region or country
- Processed there as well
- Set by locations and policies
- Easy to prove from configuration

DATA SOVEREIGNTY

Who can reach it

- Which laws and people can access it
- Support staff location and vetting
- Who holds the encryption keys
- Needs controls beyond location

THE TIEBREAKER

Residency is solved by location. Sovereignty adds people and keys: Assured Workloads, CMEK, Cloud EKM.

Layers that keep data in place

LAYER	WHAT IT DOES	COVERED IN
Dataset and bucket location	Puts the data in a region	Here
Processing location	Dataflow --region, BigQuery query location	Here, module 12
Resource locations constraint	Blocks creation anywhere else	Module 02
Assured Workloads	Locations, support staff and keys, as a package	Here
CMEK or Cloud EKM in region	Keys that also stay in place	Module 03

REMEMBER

A full residency design stacks these. One layer alone rarely satisfies a regulator.

Assured Workloads

- Applied to a **folder**. Every project inside inherits a **control package**.
- Packages are **enforced through organisation policies** and other features, so they cannot drift.
- Some packages limit **which Google support staff** can access your data, by location and vetting.
- Key options include **CMEK, Cloud EKM** and **Key Access Justifications**, depending on the package.

> ON THE EXAM
>
> 'A regulated workload, with personnel and location controls, least custom effort' points to Assured Workloads.

Assured Workloads packages, renamed June 2025

OLDER NAME	CURRENT NAME
EU Regions	EU Data Boundary
EU Regions and Support	EU Data Boundary and Support
Sovereign Controls for EU	EU Data Boundary with Access Justifications
UK Regions	UK Data Boundary

ON THE EXAM

Google says the functionality did not change. Recognise both names; answer with the question's.

IAM asks who you are. A perimeter asks where the request comes from, and where the data is going.

What VPC Service Controls stops

- **Copying out.** Data cannot be copied to resources outside the perimeter, even by someone allowed to read it.
- **Stolen credentials** used from networks outside the perimeter.
- **Insiders and compromised code** reaching services outside the perimeter.
- It does not replace IAM. Google recommends **both**, for defence in depth.

> HONEST LIMIT
>
> Some metadata, such as bucket names, is not perimeter-protected. Keep IAM tight there too.

Crossing the fence, on purpose

TOOL	WHAT IT DOES
Restricted services	The APIs the perimeter protects: BigQuery, Cloud Storage, Pub/Sub and more.
Access levels	Allow by context: source IP range, device, identity.
Ingress rule	Lets a named outside source in, to named resources.
Egress rule	Lets inside identities reach a named outside resource.
Perimeter bridge	Joins two perimeters both ways. Google prefers ingress and egress rules.
Dry-run mode	Logs violations without blocking anything.

THE TIEBREAKER

Bridges are bidirectional and coarse. For one partner or one flow, an ingress or egress rule is the answer.

Rolling out a perimeter without an outage

01

Dry run

Create the perimeter in dry-run mode. Nothing is blocked.

02

Read the logs

Every would-be violation is logged. Find the legitimate ones.

03

Add rules

Write ingress and egress rules for each legitimate flow.

04

Enforce

Switch to enforced mode once the logs are quiet.

ON THE EXAM

'Without disrupting existing pipelines' is a dry-run question.

IAM or VPC Service Controls?

IAM

Who may read it

- Grants a person or workload access
- Least privilege on datasets and buckets
- Cannot stop a reader copying it out
- Stolen credentials work anywhere

VPC SERVICE CONTROLS

Where it may go

- Fences projects and their services
- Blocks copies to outside resources
- Blocks valid credentials from outside
- Needs rules for legitimate flows

THE TIEBREAKER

'Even users who can read the data must not move it out', or 'exfiltration': VPC Service Controls.

Development and production, kept apart

- **Separate projects** for dev, test and prod, under separate folders with their own policies.
- **Separate service accounts**, so a dev pipeline physically cannot write to prod.
- **Promote through CI/CD**, never by hand. Module 18 builds that pipeline.
- **No real personal data in dev.** Use synthetic or de-identified samples, from module 04.

> THE PATTERN
>
> A project is the strongest cheap boundary you have: separate IAM, billing, quotas and perimeter.

What each level is good for

LEVEL	DRAW A BOUNDARY HERE FOR
Folder	Country, business unit or environment. Org policies and Assured Workloads.
Project	IAM, billing, quotas, VPC Service Controls perimeter membership.
Dataset	Location, default CMEK key, dataset access, time travel window.
Table and column	Row access policies, policy tags, masking. Module 29.

THE TAKEAWAY

Group data that shares the same rules. Larkspur's German data has its own folder, project and datasets.

Phrases that pick the boundary

THE QUESTION SAYS	REACH FOR
Data must stay **in one country**	Regional locations, plus the locations constraint
Regulated, with **support personnel** controls	Assured Workloads
Prevent **exfiltration**, or copying out	VPC Service Controls
One partner needs **access across** a perimeter	An ingress or egress rule
Without disrupting existing pipelines	Dry-run mode first
Dev must **not affect production**	Separate projects and service accounts

THE TIEBREAKER

Match the boundary to the risk: resting, moving, or breaking.

A Larkspur data scientist with read access to the curated dataset copied a customer table into a project outside Larkspur's warehouse projects. It was legitimate access, used to move data somewhere it should never go.

Security wants copying data out of the warehouse projects blocked, even for users who can read it, and even with stolen credentials.

What should security implement?

A Remove BigQuery Data Viewer from every data scientist

B Enforce domain restricted sharing across the organisation

C Put the warehouse projects in a VPC Service Controls perimeter

D Create a log-based alert that fires whenever a BigQuery copy job runs

◆ PAUSE AND DECIDE

What should security implement?

A Remove Data Viewer from everyone

Stops the copying by stopping the work. The scientists legitimately need to read the data.

B Domain restricted sharing

Controls who can be granted roles, not where data can be copied. The copier was already inside the domain.

C A VPC Service Controls perimeter

Copies to resources outside the fence are refused, whatever the reader's permissions, and stolen credentials fail from outside.

D Alert on copy jobs

Detects the copy after the data has left. The requirement was to block it.

THE SIGNAL

'Even users who can read it' and 'even with stolen credentials' is VPC Service Controls language.

> An Irish health-tech start-up is moving patient analytics to Google Cloud. Its regulator requires data to stay in the EU, and Google support staff who could access it to be EU-based.
>
> The start-up has a two-person platform team and wants the least custom configuration.

What should the start-up do?

A Use Assured Workloads with the EU Data Boundary and Support package

B Set the resource locations constraint to EU locations on the workload folder

C Create every BigQuery dataset and bucket in the EU multi-region

D Place all the workload's projects inside a VPC Service Controls perimeter

◆ PAUSE AND DECIDE

What should the start-up do?

A Assured Workloads, EU Data Boundary and Support

One package covers EU locations and EU-based support staff, enforced automatically.

B Resource locations constraint

Handles where resources are created, but says nothing about who at Google can support them.

C EU multi-region everywhere

Right place for the data, but relies on everyone remembering, and ignores the support staff rule.

D A VPC Service Controls perimeter

Stops data leaving through APIs. It does not set locations or control support personnel.

THE SIGNAL

Location plus personnel, least effort: Assured Workloads. Location alone would have been the constraint.

> An analyst joins German accounts in lk-de-warehouse-prod, located in europe-west3, with the tariffs table in lk-warehouse-prod, located in europe-west2. The query fails, saying a dataset was not found in location europe-west3.
>
> German personal data must not leave Germany. Tariffs contain prices only, with no personal data.

What should the team do?

A Rerun the query with its location set to the EU multi-region

B Copy the German accounts table into a London dataset, then join

C Grant the analyst BigQuery Job User on the German project

D Copy the tariffs table into a Frankfurt dataset, then join there

◆ PAUSE AND DECIDE

What should the team do?

A **Run it in the EU multi-region**

EU is a different location again. Neither regional dataset lives there, so the query still fails.

B **Copy German accounts to London**

It would work, and it breaks the rule. German personal data would leave Germany.

C **Grant Job User**

Not a permission problem. The error is about location, and the tables live in two places.

D **Copy tariffs to Frankfurt, join there**

Non-personal data moves to the personal data. The join works and nothing personal leaves Germany.

THE SIGNAL

'Not found in location' means mixed locations. Move the data that is allowed to move.

> Larkspur's pipeline developers test Dataflow jobs against the production BigQuery project. Last week a test deployment overwrote a production table.
>
> The team needs isolation, realistic test data, and no real customer data outside production.

What should the team change?

A. Prefix test datasets with dev_ inside the production project, and agree a naming convention

B. Separate dev and prod projects with their own service accounts, and give dev a de-identified sample

C. Keep one project, and add IAM conditions so developers cannot write to tables starting with prod

D. Copy all production data nightly into a dev project, so that tests are realistic

◆ PAUSE AND DECIDE

What should the team change?

A. A naming convention in prod

A convention is a promise, not a boundary. The next mistake writes to prod just as easily.

B. Separate projects, accounts, de-identified sample

A dev job cannot reach prod at all, and tests stay realistic without real customers.

C. IAM conditions on table names

Fragile: every new table needs the right name, and the pipeline's own account still writes to prod.

D. Copy all production data to dev

Realistic, isolated, and puts every customer's personal data into an environment built for experiments.

THE SIGNAL

Isolation is a project boundary. 'No real customer data' adds de-identification from module 04.

Module 05 in five lines

- Three boundaries: **where data rests**, **where it can go**, and **who can break it**.
- BigQuery location is **fixed per dataset**, and a query cannot **mix locations**.
- **Residency** is location. **Sovereignty** adds people and keys: Assured Workloads.
- **Exfiltration** means VPC Service Controls. Roll it out in **dry-run** first.
- **Dev and prod** are separate projects, with no real personal data in dev.

> COMING UP
>
> Next: module 06, reliability and fidelity: cleaning, validation and ACID.

MODULE 06 · SECTION 1 - DESIGNING DATA PROCESSING SYSTEMS

Reliability and Fidelity

Cleaning, validation, LLM-written SQL, and choosing between consistency and availability.

SECTION 1 of 5 EXAM WEIGHT 22% MODULE 6 of 45

admin@sthithapragna.com

A pipeline that runs is not a pipeline that is right.

admin@sthithapragna.com

Larkspur's plausible wrong answers

QUIRK	WHAT GOES WRONG	WHERE TO CATCH IT
Meters resend batches	Duplicates inflate demand	Deduplicate on meter and time
Firmware 2.1.x unit bug	Values 1,000 times too big	A range rule on kwh
Oracle dates as text	Day and month swapped	Parse explicitly, then check
October clock change	A repeated hour doubles readings	Store and check in UTC
Uploads up to 72 hours late	Yesterday's total keeps changing	Event-time windows, module 14

WHY IT MATTERS

None of these crash a pipeline. Each one produces a believable number.

Layers of validation

01

At the edge

Schema and type checks as data lands. Bad records go aside.

02

In the pipeline

Business rules in Dataflow or SQL. Quarantine, don't drop.

03

In the warehouse

Dataform assertions run with every build.

04

Across the estate

Scheduled data quality scans on any table.

05

Watched

Failures raise alerts, not just log lines.

THE PATTERN

Each layer catches what the one before cannot see. The exam names a tool for most of them.

Quarantine the bad rows, or fail the load?

QUARANTINE

Keep going, keep the evidence

- Good rows load on time
- Bad rows go to a dead-letter table
- Someone can investigate later
- Dataflow side outputs, error tables

FAIL THE LOAD

Nothing lands until it is right

- One bad row stops the whole batch
- max_bad_records set to zero
- Right when partial data is dangerous
- Needs an alert, or it fails silently

THE TIEBREAKER

Bad rows must be kept for analysis: dead-letter. Partial data would mislead: fail and alert.

admin@sthithapragna.com

Who is doing the cleaning?

TOOL	BEST FOR	SIGNAL IN THE QUESTION
Dataflow	Code, streaming, complex logic, dead-letter outputs	Streaming, custom logic, scale
Cloud Data Fusion	Visual pipelines, Wrangler, many connectors	GUI, low-code, many sources
Dataform	SQL transformations and tests inside BigQuery	SQL, version control, ELT
BigQuery data preparation	Gemini-suggested cleaning steps, no code	Analysts, quick, in BigQuery
Dataprep by Trifacta	A partner product, still in older questions	Older material; recognise it

THE TIEBREAKER

Match the tool to the person and the place the data lives, not to the size of the job.

BigQuery data preparation

- **Generally available since March 2025.** Gemini suggests cleaning and enrichment steps.
- Every suggestion shows **the SQL it will run**, so nothing is a black box.
- A **validation step** sends failing rows to an **error table**. With no error table, the run fails.
- **Dataform schedules** the preparation, so it inherits version control and CI/CD.

> ON THE EXAM
>
> Older questions answer 'low-code cleaning' with Dataprep. Newer ones may expect BigQuery data preparation.

Dataform: tests that travel with the code

```
config {
  type: "table",
  schema: "curated",
  assertions: {
    uniqueKeys: [["meter_id", "reading_ts"]],
    nonNull: ["meter_id", "reading_ts", "kwh"],
    rowConditions: ["kwh BETWEEN 0 AND 50"]
  }
}
SELECT meter_id, reading_ts, kwh
FROM ${ref("meter_readings")}
QUALIFY ROW_NUMBER() OVER (
  PARTITION BY meter_id, reading_ts ORDER BY received_ts DESC) = 1
```

5, 7 Duplicates and the 1,000-times unit bug both fail the build.

12-13 Keep only the latest copy of each reading, so resends collapse.

HOW IT WORKS

Each assertion is a query that must return zero rows.

Dataform assertions or data quality scans?

DATAFORM ASSERTIONS

Tests in the pipeline code

- Written by engineers, in SQLX
- Run with every build
- Version controlled with the SQL
- Cover tables Dataform builds

DATA QUALITY SCANS

Checks on any table

- Configured, not coded
- Scheduled, full or incremental
- Rules suggested from data profiles
- Results to BigQuery, alerts, scorecards

THE TIEBREAKER

'In our Dataform pipeline' is assertions. 'Governance team, any table, no code' is a data quality scan.

When Gemini writes the query

- Gemini in BigQuery **generates SQL** from a prompt or comment, and **explains** existing queries.
- Google warns output **can be plausible but incorrect**, and asks you to validate all of it.
- The **same prompt can give different SQL** on different days.
- Treat it as a **draft**: review it, version it, and test it against numbers you already trust.

> THE TRAP
>
> An explanation from the same model is not a test. Reconcile against a known total instead.

Every store makes a promise about consistency. The exam asks **which promise you need**.

What each store promises

STORE	ITS PROMISE	WATCH FOR
Spanner	ACID, externally consistent, across regions	Stricter than serialisable
Cloud SQL, AlloyDB	Full ACID within one primary	Regional scale
BigQuery	Multi-statement ACID, snapshot isolation	100 tables; conflicts cancelled
Bigtable	Atomic on a single row only	No multi-row transactions
Bigtable, replicated	Eventual, with multi-cluster routing	Single-cluster for read-your-writes

ON THE EXAM

'Move money between two rows atomically' rules out Bigtable.

Bigtable routing: consistency or failover?

SINGLE-CLUSTER ROUTING

Consistency first

- All requests go to one cluster
- Read-your-writes consistency
- The only way to use single-row transactions
- Failover is manual

MULTI-CLUSTER ROUTING

Availability first

- Nearest available cluster serves
- Automatic failover
- Eventual consistency
- No single-row transactions

THE TIEBREAKER

Routing is set per app profile. Different workloads on one instance can choose differently.

Designing for things going wrong

- **Assume duplicates.** Most delivery is at least once, so make writes idempotent.
- **Keep the raw input**, so you can reprocess after a bug. Module 38.
- **Store time in UTC.** Local time repeats an hour every October.
- **Place work deliberately**: regions, zones and replicas. Modules 37 and 39 go deeper.

> THE TAKEAWAY
>
> If a rerun would double-count, the design is not fault tolerant, however many replicas it has.

Phrases that pick the answer

THE QUESTION SAYS	REACH FOR
Bad rows must be **kept for analysis**	A dead-letter table or error table
Partial data would **mislead**	Fail the load, and alert
Tests **in our SQL pipeline**	Dataform assertions
Governance team, **any table, no code**	Knowledge Catalog data quality scans
Analysts clean data **without code**	BigQuery data preparation, or Data Fusion
Changes to **several rows, all or nothing**	A transactional store, not Bigtable
Automatic failover for Bigtable	Multi-cluster routing, eventual consistency

THE TIEBREAKER

When an option would silently drop or skip data, it is almost never right.

A partner sends Larkspur a daily CSV file of grid connection changes. A few rows each day are malformed or corrupted.

Analysts need the good rows in BigQuery on time every morning, and the bad rows must be kept so they can be raised with the partner.

How should the team build the load?

A. A BigQuery load job with max_bad_records set to zero

B. A Dataflow batch pipeline with a dead-letter table for failed rows

C. An external table over the files, filtering bad rows out in each query

D. A BigQuery load job with max_bad_records set high enough to always succeed

◆ PAUSE AND DECIDE

How should the team build the load?

A Load job, zero bad records allowed

One bad row fails the whole file, so the good rows miss the morning deadline every day.

B Dataflow with a dead-letter table

Good rows load on time, and every bad row is kept with its error for the partner.

C External table, filter in queries

Every analyst repeats the filtering, and the bad rows are never collected anywhere.

D Load job, high bad-record limit

The load succeeds, but the bad rows are simply discarded. Nothing is kept for the partner.

THE SIGNAL

'On time' rules out failing the load. 'Kept for the partner' rules out skipping. Dead-letter does both.

> Larkspur's data governance team, who do not write code, must check curated.readings_hh every day for nulls, impossible kWh values and duplicates.
>
> They want rules suggested from the data itself, results they can chart, and an alert when a check fails.

What should the governance team use?

A Dataform assertions added to the curated pipeline's SQLX files

B A Cloud Run function that runs validation queries and emails the results

C A validation step in a BigQuery data preparation, with an error table

D A Knowledge Catalog data quality scan, using rules recommended from a data profile

◆ PAUSE AND DECIDE

What should the governance team use?

A **Dataform assertions**

Good tests, but they are code in the engineers' pipeline. The governance team does not write code.

B **A Cloud Run function**

Custom code to build and maintain, with no rule suggestions and no charting.

C **BigQuery data preparation validation**

Validates rows while preparing data. It is not a scheduled check on an existing table.

D **A data quality scan**

No code, rules suggested from profiling, results exported for charts, and failures can raise alerts.

THE SIGNAL

'Do not write code', 'existing table', 'rules suggested' and 'alert': a data quality scan.

> Analysts used Gemini in BigQuery to write the SQL behind a monthly report for the energy regulator. The figures look reasonable.
>
> Compliance wants assurance that generated queries are correct before anything is submitted, without banning the tool.

What should the analytics team do?

A Review it, version it in Dataform, and assert it matches billing totals

B Disable Gemini in BigQuery for the analytics team

C Generate each query three times and use the version that appears most often

D Ask Gemini to explain each query, and accept it when the explanation matches the intent

◆ PAUSE AND DECIDE

What should the analytics team do?

A A reviewed, versioned, tested draft

Review catches logic errors, and an assertion against billing totals proves the numbers.

B Disable Gemini

Compliance asked not to ban it. The problem is validation, not the tool.

C Majority vote of three generations

Three plausible wrong answers can agree. Agreement is not correctness.

D Accept if the explanation matches

The explanation comes from the same model. It can be confidently wrong in the same way.

THE SIGNAL

Generated SQL is validated the way any SQL is: review, version control, and tests against known numbers.

> Larkspur's live-reading Bigtable instance has clusters in two regions. The customer app only reads, and must fail over automatically if a region goes down.
>
> A separate job updates a per-meter counter with an atomic increment, which needs a single-row transaction.

How should routing be configured?

A One app profile with multi-cluster routing, used by both workloads

B One app profile with single-cluster routing, used by both workloads

C A multi-cluster profile for the app, and a single-cluster profile for the counter job

D Move live readings to Spanner, which supports transactions across regions

◆ PAUSE AND DECIDE

How should routing be configured?

A Multi-cluster for both

The app fails over, but single-row transactions cannot be enabled, so the counter job breaks.

B Single-cluster for both

The counter works, but failover is manual, which the app ruled out.

C A profile per workload

Each workload gets the promise it needs from the same instance: failover for one, transactions for the other.

D Migrate to Spanner

A whole migration to solve a routing setting. Bigtable already meets both needs.

THE SIGNAL

When two workloads need different promises, look for a per-workload setting before a new product.

Module 06 in five lines

- A pipeline that runs is not a pipeline that is **right**. Validate in layers.
- **Keep** bad rows in a dead-letter table, or **fail and alert**. Never skip silently.
- **Dataform assertions** for pipeline code. **Data quality scans** for any table, no code.
- **Gemini-written SQL is a draft.** Review it, version it, test it against known numbers.
- **Bigtable is single-row only.** Multi-cluster routing trades consistency for failover.

COMING UP

Next: module 07, migrating files and bulk data.

MODULE 07 · SECTION 1 - DESIGNING DATA PROCESSING SYSTEMS

Migrating Files and Bulk Data

gcloud storage, Storage Transfer Service, Transfer Appliance, and the network underneath them.

SECTION 1 of 5 EXAM WEIGHT 22% MODULE 7 of 45

admin@sthithapragna.com

Every bulk migration starts with one sum: **size divided by bandwidth.**

What Larkspur has to move

DATA	SIZE	THE CATCH
Reading history in HDFS	1.2 PB of ORC, 8 years	2.9 million small files
New readings, until cutover	About 30 GB a day	The history keeps growing
Engineers' manuals	18,000 PDF pages	Small, one-off copy
Westgrid settlement files	Daily, in Amazon S3	Data in another cloud

THE CONSTRAINT

The Bristol lease ends on 30 June 2027, and today's link is 1 Gbps.

A terabyte a day for every 100 Mbps

LINK	PER DAY, FLAT OUT	1.2 PB TAKES
100 Mbps	About 1 TB	About 3 years
1 Gbps, Larkspur today	About 10 TB	Almost 4 months
10 Gbps, Interconnect on order	About 100 TB	About 11 days
Five Transfer Appliances	No link needed	Weeks, plus shipping

HONEST LIMIT

These are best cases. Real links rarely run flat out, and the business still needs its link.

Google's starting point

WHERE THE DATA IS	UNDER 1 TB	OVER 1 TB
On-premises	gcloud storage	Storage Transfer Service or Transfer Appliance
Another Cloud Storage region	gcloud storage	Storage Transfer Service
Another cloud, such as S3 or Azure	Storage Transfer Service	Storage Transfer Service

THE TIEBREAKER

Over the network would take more than about a week? Google suggests Transfer Appliance.

gcloud storage: the command line

- **For small, one-off copies** and syncs: **cp**, **mv** and **rsync**.
- **Checksums are automatic.** MD5 or CRC32C are compared, and a bad copy is deleted.
- **Parallel composite uploads** split files over 150 MiB into up to 32 chunks.
- **Composite objects have no MD5**, and the trick is skipped with retention policies.

> CURRENCY CHECK
>
> gsutil is legacy, and leaves the Google Cloud CLI after March 2027. Answers should say gcloud storage.

gcloud storage in practice

```
# One-off copy of the manuals, then rerun to pick up edits
gcloud storage rsync --recursive /srv/manuals gs://lk-manuals

# Big exports: parallel composite uploads, Standard class only
gcloud config set storage/parallel_composite_upload_enabled True
gcloud storage cp ./exports/*.orc gs://lk-history-land/

# Spot-check: compare the local hash with the object's hash
gcloud storage hash ./exports/part-0001.orc
gcloud storage objects describe gs://lk-history-land/part-0001.orc
```

2 A rerun copies only new or changed files.

5-6 Files over 150 MiB go up in parallel chunks.

9-10 A composite object shows a CRC32C, but no MD5.

HOW IT WORKS

Every copy is already checked automatically. The spot-check is evidence for your records.

Storage Transfer Service

- **Managed jobs**, once or on a schedule. Retries, logging and monitoring built in.
- **Agentless** from Cloud Storage, Amazon S3, Azure Blob and URL lists.
- **Agents** for file systems, **HDFS** and S3-compatible storage. At least three per pool.
- **Incremental**: compares time and size, copies what changed. **Bandwidth** can be capped.

> HOW IT WORKS
>
> Agents run next to the data. Add more agents, and the transfer speeds up.

Agents in Bristol, bucket in London

01

Install agents

Containers on machines that can read the Hadoop cluster.

02

Form a pool

Three or more agents, same access, bandwidth capped.

03

Compare

The service checks both sides and finds what changed.

04

Copy and check

Agents read, checksum and send. A mismatch fails.

05

Land

Objects arrive in a europe-west2 bucket, with logs.

FOR LARKSPUR

Run it nightly for new days, then once more at cutover with writes frozen.

Scheduled or event-driven?

SCHEDULED

List, compare, copy

- Runs once, or on a timetable
- Lists the source on every run
- Can delete extra destination files
- Right for batches and migrations

EVENT-DRIVEN

Listen, then copy

- Listens for change notifications
- S3 via SQS, Azure via queues, Pub/Sub
- No listing, so near real time
- Deletions are not carried across

THE TIEBREAKER

'Within minutes of arrival' points to event-driven. 'Mirror deletions' points to scheduled.

Newer transfer options

- **Google-managed private network** for S3 sources: **no AWS internet egress charges**.
- Instead, a **per-GiB fee** to Google. Capacity is shared, and can slow at peak times.
- **Transfer Appliance online mode** streams to your bucket without shipping the box.
- **Cross-Cloud Interconnect**: a dedicated link to AWS, Azure, Oracle or Alibaba.

> THE SIGNAL
>
> 'Cut the egress bill from AWS' is a signal for the managed private network.

Transfer Appliance: a van full of disks

- **Google ships you a storage server.** Copy over NFS, seal it, and ship it back.
- **AES-256 encrypted**, with your key in Cloud KMS. Wiped to NIST 800-88 afterwards.
- **Up to 300 TB each.** Order several, and fill them in parallel.
- Google's example: **300 TB in under 25 days**, then about **10 business days** to land.

> THE SIGNAL
>
> Use it when the network would take more than about a week, or there is no good network.

2.9 million small files

- **Small files are slow everywhere.** Per-file overhead dominates, whatever the link.
- Transfer Appliance **rehydration slows down** with very small files.
- Storage Transfer Service: keep source folders **under about 1 million files**.
- **Compact before you move.** Fewer, larger ORC files copy faster and query faster.

> THE TAKEAWAY
>
> Fixing small files at the source helps the transfer now, and Spark and BigQuery later.

Four ways into Google Cloud

OPTION	CAPACITY	ENCRYPTED?	PICK IT WHEN
Public internet	Whatever you buy	Only by TLS in apps	Small, quick, cheap
HA VPN	1 to 3 Gbps per tunnel	Yes, IPsec	Modest bandwidth, encrypted
Dedicated Interconnect	10, 100 or 400 Gbps circuits	Not by default	You reach a colocation site
Partner Interconnect	50 Mbps to 50 Gbps	Not by default	A provider reaches Google

REMEMBER

HA VPN gives a 99.99% SLA with the right topology. A single interface gives none.

Private is not encrypted

HA VPN OVER INTERCONNECT

IPsec, end to end

- Layer 3 encryption on the private link
- Traffic never touches the internet
- Covers the provider's network too
- Several tunnels to fill a big link

MACSEC

Encrypts one hop

- Layer 2, IEEE 802.1AE
- Your router to Google's edge
- Partner: the provider's leg is separate
- Google advises adding IPsec too

THE TRAP

Interconnect keeps traffic off the internet. It does not encrypt it.

Validate before you delete the source

- **Checksums in flight.** Storage Transfer Service checks end to end, and fails mismatches.
- **Count and compare** afterwards: files, bytes and checksums, folder by folder.
- **Freeze writes** for the final pass, or a changing file may land half-written.
- **Delete the source last.** The service cannot restore what you removed.

> FOR LARKSPUR
>
> The lease ends on 30 June 2027. Plan the final sync and its checks well before then.

Phrases that pick the answer

THE QUESTION SAYS	REACH FOR
A few hundred GB, one-off, from a server	gcloud storage
Petabytes, slow or shared link	Transfer Appliance
From Amazon S3 or Azure, recurring	Storage Transfer Service
On-premises **NFS or HDFS**, good network	Storage Transfer Service with agents
Within minutes of new objects arriving	Event-driven transfers
Encrypted over Interconnect	HA VPN over Interconnect, or MACsec

THE PATTERN

Size, source and speed decide most of these questions.

> Larkspur must move 1.2 PB of frozen Hive history from Bristol before the lease ends on 30 June 2027. The 1 Gbps internet link also carries the company's daily traffic.
>
> A 10 Gbps Partner Interconnect is on order, but the provider has not given an installation date. New days keep arriving until cutover.

What should the team do?

A Copy the history with gcloud storage rsync from an edge node, running at night

B Use Storage Transfer Service agents over the 1 Gbps link, with a bandwidth cap

C Wait for the Partner Interconnect, then run Storage Transfer Service across it

D Ship Transfer Appliances for the history, then sync new days with agents

◆ PAUSE AND DECIDE

What should the team do?

A gcloud storage rsync at night

A small-job tool for a petabyte. Running only at night, it would take about a year.

B Agents over the 1 Gbps link

The right tool on the wrong link. Capped to protect the business, it runs for many months.

C Wait for the Interconnect

It bets a fixed deadline on an installation date that nobody has given.

D Appliances, then agents

The bulk travels by road in weeks. Agents copy each new day, with a final pass at cutover.

THE SIGNAL

Do the division first. Months over the network, against a fixed deadline, means the van.

> The field engineer assistant needs Larkspur's 18,000 pages of manuals in Cloud Storage. They total under 50 GB, on one file server in Bristol.
>
> It is a one-off copy, with occasional edits afterwards. The team wants the simplest option that checks what it copies.

Which approach fits best?

A Run gcloud storage rsync from the file server, and rerun it after edits

B Deploy a pool of three Storage Transfer Service agents next to the server

C Order a Transfer Appliance, and copy the manuals onto it over NFS

D Use the BigQuery Data Transfer Service to load the files into a dataset

◆ PAUSE AND DECIDE

Which approach fits best?

A gcloud storage rsync

Under a terabyte, one-off, with automatic checksums. A rerun copies only the edits.

B Three transfer agents

It works, but it means containers, a pool and jobs, for a copy that takes minutes.

C A Transfer Appliance

Days of shipping, for data that crosses a 1 Gbps link in about five minutes.

D BigQuery Data Transfer Service

It loads BigQuery tables, and cannot read a file server. The assistant needs the files.

THE SIGNAL

Under a terabyte, from one machine, is gcloud storage. 'Simplest' is a requirement, not a hint.

> Westgrid writes settlement files into an Amazon S3 bucket throughout the day. Larkspur needs each file in Cloud Storage within minutes of it arriving.
>
> The bucket already holds millions of older files, and Westgrid never deletes anything.

How should Larkspur copy the files?

A. A VM running gcloud storage rsync from the S3 bucket every five minutes

B. A Storage Transfer Service job from S3, scheduled to run every hour

C. An event-driven Storage Transfer Service job, fed by S3 notifications

D. BigQuery Data Transfer Service for Amazon S3, loading a settlement table

◆ PAUSE AND DECIDE

How should Larkspur copy the files?

A rsync from a VM, every five minutes

Each run lists millions of files, and the VM is yours to patch and watch.

B A scheduled hourly job

Managed, but an hour is not minutes, and every run lists the whole bucket.

C An event-driven transfer

SQS notifications trigger each copy. No listing, near real time, no deletes to miss.

D BigQuery DTS for Amazon S3

It loads tables on a schedule. Larkspur asked for the files in Cloud Storage.

THE SIGNAL

'Within minutes' plus millions of files to list points to events, not schedules.

> The 10 Gbps Partner Interconnect is finally live, and Larkspur will send new data and the final sync across it.
>
> Security requires all traffic between Bristol and Google Cloud to be encrypted at the network layer, including across the provider's network.

What should the network team do?

A Nothing extra, because Interconnect traffic never crosses the internet

B Run HA VPN tunnels over the Interconnect, enough of them to fill the link

C Replace the Interconnect with HA VPN tunnels over the 1 Gbps internet link

D Enable MACsec on the Interconnect, and rely on it as the only encryption

◆ PAUSE AND DECIDE

What should the network team do?

A. Nothing extra

Private is not encrypted. Off the internet is not the same as protected.

B. HA VPN over Interconnect

IPsec runs end to end, across the provider too. Several tunnels fill the link.

C. HA VPN over the internet

Encrypted, but capped by a 1 Gbps link while the Interconnect sits idle.

D. MACsec alone

It covers the hop to Google's edge. The provider's network is left out.

THE SIGNAL

'Including the provider's network' rules out MACsec alone. IPsec over Interconnect covers every hop.

Module 07 in five lines

- Do the division first: **100 Mbps moves about 1 TB a day**.
- **gcloud storage** for small copies. **Storage Transfer Service** for big, recurring or cross-cloud.
- **Agents** for file systems and HDFS. **Event-driven** for minutes. **Scheduled** to mirror deletes.
- **Transfer Appliance** when the network would take over a week. Compact small files first.
- **Private is not encrypted.** Add IPsec or MACsec, then validate before deleting.

COMING UP

Next: module 08, migrating databases and warehouses.

MODULE 08 · SECTION 1 - DESIGNING DATA PROCESSING SYSTEMS

Migrating Databases and Warehouses

Database Migration Service, Datastream, BigQuery Data Transfer Service, and moving the SQL as well as the data.

SECTION 1 of 5 EXAM WEIGHT 22% MODULE 8 of 45

admin@sthithapragna.com

Moving a database is easy. Moving it **while the business keeps writing to it** is the job.

Larkspur's databases

SYSTEM	TODAY	WHERE IT GOES
Billing	Oracle 19c, 6 TB, 410 tables	50 tables copied into BigQuery
Tariff service	PostgreSQL on a VM	Cloud SQL for PostgreSQL
Settlement history	Hive on the Hadoop cluster	Cloud Storage, then BigQuery
Analyst SQL	Hundreds of Hive scripts	Translated to GoogleSQL

THE QUIRKS

Oracle stores dates as text, and call-centre notes hold pasted bank details.

One-time or continuous?

ONE-TIME

Dump, load, switch

- A point-in-time copy
- Writes stop during the copy
- Downtime lasts the whole copy
- Fine for small or quiet systems

CONTINUOUS

Copy, then follow changes

- A full load first
- Then every change, as it happens
- Cut over once both sides match
- Downtime of minutes, not hours

THE SIGNAL

'Minimal downtime' always means continuous replication, usually called change data capture.

Four ways to move a database

TOOL	MOVES DATA INTO	SIGNAL IN THE QUESTION
Database Migration Service	Cloud SQL or AlloyDB	Lift the database, minimal downtime
Datastream	BigQuery or Cloud Storage	Keep analytics fed from a live database
BigQuery Data Transfer Service	BigQuery	Scheduled loads, warehouses, SaaS
Batch exports	Cloud Storage, then a load job	One-off history, no change feed

THE TIEBREAKER

Ask where the data lands. That alone rules out half the options.

Database Migration Service

- **Same engine**: MySQL, PostgreSQL or SQL Server into Cloud SQL, or PostgreSQL into AlloyDB.
- **Different engine**: Oracle or SQL Server into PostgreSQL, on Cloud SQL or AlloyDB.
- A **conversion workspace** converts the schema, with Gemini helping on the hard parts.
- **Continuous** jobs load, follow changes, then **promote** the replica at cutover.

> THE TRAP
>
> It lands in Cloud SQL or AlloyDB. Not BigQuery, and not Spanner, which has its own migration tool.

Connecting to the source

METHOD	HOW TRAFFIC FLOWS	GOOGLE'S ADVICE
IP allowlist	Over the public internet	Easiest, least secure
Reverse SSH tunnel	Through a bastion VM you run	Short-lived tests, small databases
VPC peering	Private, over VPN or Interconnect	Long-running or high-volume moves
Private Service Connect	Private, through a network attachment	Easiest private option, existing instance

ON THE EXAM

A large migration over weeks, on a private link, points to VPC peering.

Datastream: change data capture as a service

- **Serverless CDC** from MySQL, PostgreSQL, Oracle, SQL Server, MongoDB, Spanner and Salesforce.
- Lands in **BigQuery** or **Cloud Storage**. Other targets go through Dataflow templates.
- **Backfill** copies existing rows first, then changes stream on top.
- Pick **schemas, tables and even columns**. Cleared columns are never copied.

> HOW IT WORKS
>
> Datastream reads the database's change log, so the source barely notices.

Merge or append-only?

MERGE

A live mirror of the source

- Tables look like the source
- Max staleness: freshness against cost
- Needs a primary key
- Right for current-state reporting

APPEND-ONLY

Every change, kept

- One row per change event
- A change type on every row
- Works without a primary key
- Right for history and audit

THE TRADE-OFF

Lower staleness means fresher data and more merge cost. Zero minutes is freshest and dearest.

Fixing Oracle's text dates, once

```sql
-- Billing lands in raw via Datastream. Notes was never streamed.
CREATE OR REPLACE VIEW curated.bills AS
SELECT
  bill_id,
  account_id,
  PARSE_DATE('%d/%m/%Y', period_start) AS period_start,
  amount_gbp,
  TIMESTAMP_MILLIS(datastream_metadata.source_timestamp) AS changed_at
FROM raw.billing_bills;
```

6 Day first, stated explicitly. A malformed date fails loudly.

8 When the change happened in Oracle, from Datastream's metadata.

THE PATTERN

Datastream copies what is there. Cleaning belongs downstream, written once and tested.

Datastream: the fine print

- Oracle: **LogMiner** may replicate some uncommitted changes. The **binary reader** is Preview.
- No primary key? Merges use Oracle's **ROWID**, which might not be unique.
- **Private connectivity** uses VPC peering, reaching on-premises over VPN or Interconnect.
- Merges run as **BigQuery jobs**. Google recommends a slot reservation for predictable cost.

> FOR LARKSPUR
>
> Larkspur's finance director wanted predictable spend. CDC merges belong in the reservation plan.

BigQuery Data Transfer Service

- **Scheduled, managed loads** into BigQuery. No pipelines to write.
- **Warehouses**: Teradata, Amazon Redshift, Snowflake, and Hive metastore tables.
- **Databases and SaaS**: Oracle, MySQL, PostgreSQL, Salesforce, Google Ads and more.
- **Storage**: Cloud Storage, Amazon S3 and Azure Blob, on a schedule.

> THE TIEBREAKER
>
> Data Transfer Service is scheduled batch. Changes within minutes, with history, is Datastream.

The BigQuery Migration Service toolkit

TOOL	WHAT IT DOES
Migration assessment	Inventories the old warehouse, and estimates effort and cost
Batch SQL translator	Translates thousands of scripts at once into GoogleSQL
Interactive SQL translator	One query at a time, side by side
Gemini-enhanced rules	Customise translations in plain language. Preview
Data Validation Tool	Open source. Proves source and target tables match

THE POINT

Moving the data is the easy half. The SQL, and the trust in its numbers, is the hard half.

Hive to BigQuery, for Larkspur

- The files are already in **Cloud Storage**, thanks to module 07.
- **Load** them into native BigQuery tables, or **query in place** as Lakehouse tables.
- Hive scripts go through the **batch SQL translator**. Hive is a supported dialect.
- Spark jobs can stay Spark, on **Managed Service for Apache Spark**.

> THE PATTERN
>
> Migrate in waves by use case, not all at once. Each wave is validated before the next.

Validate before you cut over

- **Counts first**: rows per table, and per partition for big tables.
- **Aggregates next**: sums, minimums and maximums of key columns.
- **Row hashes** where every value matters, such as billing amounts.
- **Run in parallel** for a while, and compare the reports people actually use.

> THE TRAP
>
> Row counts alone miss a swapped day and month. Aggregates and hashes catch it.

Phrases that pick the answer

THE QUESTION SAYS	REACH FOR
Minimal downtime, into Cloud SQL or AlloyDB	Database Migration Service, continuous
Oracle to PostgreSQL	Database Migration Service, conversion workspace
Live database to **BigQuery**, fresh within minutes	Datastream
Keep every change for audit	Datastream, append-only
Teradata, Redshift, Snowflake, scheduled loads	BigQuery Data Transfer Service
Thousands of scripts to convert	Batch SQL translator

THE PATTERN

Destination first, then downtime, then freshness.

> Analysts need 50 of Larkspur's Oracle billing tables in BigQuery, never more than a few minutes behind the source.
>
> Oracle keeps running billing, and its database team will not accept extra query load. Nobody wants to write or run pipeline code.

What should the team use?

A. Nightly Data Pump exports to Cloud Storage, loaded by a scheduled job

B. A Database Migration Service continuous job from Oracle into BigQuery

C. Datastream from Oracle into BigQuery, with only the 50 tables selected

D. A Dataflow JDBC pipeline that queries every table every five minutes

◆ PAUSE AND DECIDE

What should the team use?

A Nightly Data Pump exports

Up to a day behind, not a few minutes.

B Database Migration Service

It lands in Cloud SQL or AlloyDB. BigQuery is not a destination.

C Datastream, 50 tables

Reads the change log, so Oracle feels little load. Minutes fresh, no code.

D A Dataflow JDBC pipeline

Code to own, and repeated full queries that Oracle's team refused.

THE SIGNAL

'A few minutes behind' plus 'no extra load' means log-based CDC. Into BigQuery, that is Datastream.

> Larkspur's tariff service runs PostgreSQL on a VM in Bristol. It must move to Cloud SQL for PostgreSQL with no more than a few minutes of downtime.
>
> The team wants weeks of testing against the copy before cutover. The Partner Interconnect is live.

How should they migrate?

A Take a full dump on a quiet Sunday night, then restore it into Cloud SQL

B A continuous Database Migration Service job over VPC peering, then promote

C Datastream from PostgreSQL straight into the new Cloud SQL instance

D A one-time Database Migration Service job, connected by IP allowlist

◆ PAUSE AND DECIDE

How should they migrate?

A. A Sunday dump and restore

Writes stop for the whole dump and restore. That is hours, not minutes.

B. Continuous, VPC peering

Follows changes for weeks of testing, then cuts over in minutes.

C. Datastream into Cloud SQL

Datastream lands in BigQuery or Cloud Storage, not Cloud SQL.

D. One-time job, IP allowlist

One-time means downtime, and the public internet ignores the Interconnect.

THE SIGNAL

'Weeks of testing' plus 'minutes of downtime' means a continuous job. Long and private means VPC peering.

> A retailer is moving a 300 TB Teradata warehouse to BigQuery. It has about 40,000 SQL scripts, and leadership wants an effort estimate before approving the project.
>
> Finance will only sign off once migrated tables are shown to match the originals.

Which set of tools fits?

A. Assessment, batch translator, Data Transfer Service, Data Validation Tool

B. Gemini in BigQuery to rewrite each script, then Storage Transfer Service

C. Datastream from Teradata into BigQuery, then Dataform assertions

D. CSV exports on a Transfer Appliance, scripts rewritten by hand, counts

◆ PAUSE AND DECIDE

Which set of tools fits?

A The Migration Service toolkit

Estimate first, translate in bulk, load on a schedule, then prove it matches.

B Gemini rewrites, Storage Transfer

Forty thousand scripts one by one, and files that never become tables.

C Datastream, then assertions

Datastream does not read Teradata, and assertions test rules, not matches.

D Appliance, hand rewrites, counts

Months of manual SQL, and row counts alone will not satisfy finance.

THE SIGNAL

Estimate, translate in bulk, move, prove. Each phrase maps to one tool in the Migration Service toolkit.

> Larkspur is setting up the Datastream stream for billing. The Oracle accounts table has a free-text notes column where call-centre agents have pasted IBANs.
>
> Analysts never use the notes. Security says bank details must not reach BigQuery in any form.

What should the team do?

A Stream every column, then hide notes from analysts with a policy tag

B Stream every column, and run a weekly scan to find and delete the IBANs

C Ask the call centre to stop pasting bank details, then stream everything

D Clear the notes column in the stream, so it is never copied to BigQuery

◆ PAUSE AND DECIDE

What should the team do?

A Stream, then a policy tag

Hidden is not absent. The IBANs still sit in BigQuery.

B Stream, then a weekly scan

Bank details live in BigQuery for up to a week at a time.

C Ask the call centre

Good practice, but years of pasted IBANs are already in the column.

D Clear the notes column

The column never leaves Oracle, and analysts lose nothing they use.

THE SIGNAL

'Must not reach BigQuery in any form' rules out every fix that happens after loading.

Module 08 in five lines

- **Minimal downtime** means continuous replication: full load, follow changes, then cut over.
- **Database Migration Service** lands in Cloud SQL or AlloyDB. Never BigQuery.
- **Datastream** feeds BigQuery from a live database. Merge for now, append-only for history.
- **Data Transfer Service** is scheduled batch. The **translators** move the SQL.
- **Prove it matches** with aggregates and hashes. Counts alone miss swapped dates.

COMING UP

Next: module 09, designing for portability and multi-cloud.

MODULE 09 · SECTION 1 - DESIGNING DATA PROCESSING SYSTEMS

Designing for Portability and Multi-cloud

Open file and table formats, Lakehouse, BigQuery Omni, portable code, and residency across clouds.

SECTION 1 of 5 EXAM WEIGHT 22% MODULE 9 of 45

admin@sthithapragna.com

Portability is not refusing to commit. It is **keeping the exit door unlocked**.

Four layers to keep open

LAYER	LOCKS YOU IN	KEEPS IT OPEN
File format	A store's own internal storage	Parquet, Avro, ORC
Table format	Metadata only one engine reads	Apache Iceberg
Catalog	A catalog only one engine queries	An Iceberg REST catalog
Code	APIs tied to one service	Apache Beam or Apache Spark

THE TRAP

Open files are not enough. Without an open table format and catalog, each engine sees only loose files.

Rows or columns?

FORMAT	LAYOUT	BEST FOR
Avro	Row-based, schema inside the file	Fast appends, streaming archives, schema change
Parquet	Columnar, compressed	Analytics that read a few columns of many rows
ORC	Columnar, compressed	Hive estates, like Larkspur's history
CSV and JSON	Text, no enforced types	Interchange with systems that need text

THE TIEBREAKER

Row formats write fast. Column formats read fast. Pick by what happens to the data most often.

Apache Iceberg: the inventory list

- Files are the boxes. **Iceberg is the inventory list** that says which boxes make up the table.
- **Atomic commits**: a writer swaps in a new list, so readers never see half a change.
- **Snapshots** give time travel. **Schema evolution** adds or renames columns safely.
- **Many engines, one table**: Spark, Flink, Trino and BigQuery read the same list.

> ON THE EXAM
>
> 'Several engines must read and write the same tables, with transactions' points to Iceberg.

BigLake is now Lakehouse

- **22 April 2026**: BigLake became **Google Cloud Lakehouse**.
- The BigLake metastore became the **Lakehouse runtime catalog**.
- **API and IAM names did not change**, and many docs pages still say BigLake.
- **BigLake tables for Apache Iceberg in BigQuery** are now **Apache Iceberg managed tables**.

> REMEMBER
>
> The exam guide still says BigLake. Answer with whichever name the question uses.

Lakehouse tables: governed tables over open files

- **Access delegation**: a connection reads the bucket, so users need table access only.
- **Row and column security** and masking, even for Spark through the Storage API.
- Parquet, ORC, Avro, CSV, JSON, **Iceberg** and **Delta Lake**, on Cloud Storage, S3 or Azure.
- **Metadata caching** speeds up tables with many files, like Larkspur's history.

> THE SIGNAL
>
> 'Analysts must not read the bucket directly' points to a Lakehouse table, not a plain external table.

Two kinds of Iceberg table: who writes?

ICEBERG MANAGED TABLES

BigQuery writes

- Data in your bucket, as Parquet
- BigQuery DML, streaming, compaction
- Others read exported metadata
- Never edit the files directly

RUNTIME CATALOG TABLES

Open engines write

- Spark, Flink or Trino write Iceberg
- Iceberg REST catalog endpoint
- BigQuery queries the same tables
- BigQuery writes are Preview

THE TIEBREAKER

Ask who writes the table. BigQuery: managed. Spark or another engine: the runtime catalog.

An Iceberg managed table for Larkspur

```
1  CREATE TABLE curated.readings_iceberg (
2    meter_id STRING, reading_ts TIMESTAMP, kwh NUMERIC)
3  CLUSTER BY meter_id
4  WITH CONNECTION `lk-warehouse-prod.europe-west2.lake`
5  OPTIONS (
6    file_format = 'PARQUET',
7    table_format = 'ICEBERG',
8    storage_uri = 'gs://lk-lake-prod/readings');
9
10 EXPORT TABLE METADATA FROM curated.readings_iceberg;
```

4 The connection reads and writes the bucket, not the users.

6-7 Open files, open table format, in Larkspur's own bucket.

10 Publishes Iceberg metadata for Spark. Run it on a schedule.

HOW IT WORKS

BigQuery owns the writes. The files stay open, and stay in Larkspur's bucket.

BigQuery Omni: take the engine to the data

- The BigQuery engine **runs inside AWS or Azure**, next to the data.
- AWS: N. Virginia, Oregon, Seoul, Sydney, Ireland, **Frankfurt**. Azure: East US 2.
- Queries **Lakehouse tables over S3 or Blob**. No managed tables, no DML, no BigQuery ML.
- Billed on demand or with **Enterprise edition** reservations.

> THE POINT
>
> Omni moves the compute, not the data. The question to ask is where the results go.

Where do Omni results go?

STATEMENT	WHAT CROSSES CLOUDS	LIMIT
`EXPORT DATA`	Nothing. Results stay in S3 or Blob	None crossing clouds
`Cross-cloud join`	The Omni side's result	60 GB per transfer
`CREATE TABLE AS SELECT`	A filtered result, into BigQuery	60 GB per transfer
`LOAD DATA`	Raw files, unfiltered	No limit
`Cross-cloud materialized view`	A continuous, incremental replica	Not for Azure Blob

THE SIGNAL

'Results must stay in AWS' is EXPORT DATA. 'Keep a fresh copy in BigQuery' is a cross-cloud materialized view.

The cross-cloud Lakehouse

- **Preview**: query Iceberg tables through remote catalogs. AWS Glue, Databricks Unity, Snowflake.
- Google calls it **a modern alternative to BigQuery Omni**.
- It **caches data segments inside Google Cloud** to cut egress. A cache is a copy.
- Runs over the internet, or privately over **Cross-Cloud Interconnect**.

> HONEST LIMIT
>
> Too new for most exam questions. Know it exists, and know that caching moves data.

Residency across clouds

- **Where does processing run?** Omni runs in the source cloud's region.
- **Where do results land?** Joins and CTAS copy results into a BigQuery region.
- **Are there caches?** Caches and replicas are copies, and rules usually cover copies.
- **Who pays egress?** The source cloud bills data leaving it.

> THE TAKEAWAY
>
> Trace every copy, not just the original. That is how residency questions are won.

Apache Beam or Apache Spark?

APACHE BEAM

One pipeline, many runners

- Dataflow, Flink, Spark and more
- Batch and streaming in one model
- Best for new pipelines
- Dataflow adds autoscaling

APACHE SPARK

Runs wherever Spark runs

- Managed Service for Apache Spark
- Serverless batches, or clusters
- Best for existing Spark code
- Lift and shift Hadoop jobs

THE TIEBREAKER

Existing Spark code: keep it Spark. A new pipeline that must stay portable: Beam.

Phrases that pick the answer

THE QUESTION SAYS	REACH FOR
Query S3 in place, results stay in AWS	BigQuery Omni with EXPORT DATA
Several engines share tables, with transactions	Apache Iceberg
Spark writes, **BigQuery reads**	Iceberg in the Lakehouse runtime catalog
No bucket access for analysts	Lakehouse table with a connection
Schema changes often, fast appends	Avro
Existing Spark jobs, minimal change	Managed Service for Apache Spark

THE PATTERN

Open formats, one copy, and engines that come to the data.

Larkspur Energie inherited a 40 TB archive of grid sensor data, stored as Parquet in Amazon S3 in Frankfurt. Its agreement says the files must not be copied out of AWS.

Analysts in Hamburg want to query the archive with BigQuery SQL, and keep their results in the same AWS account.

What should the team set up?

A Storage Transfer Service, copying the archive into a europe-west3 bucket

B BigQuery Omni in AWS Frankfurt, with results written out by EXPORT DATA

C The cross-cloud Lakehouse, with caching turned on to cut egress charges

D A Dataflow pipeline that reads the S3 files and streams rows to BigQuery

◆ PAUSE AND DECIDE

What should the team set up?

A Copy with Storage Transfer

It copies the files out of AWS, which the agreement forbids.

B Omni, then EXPORT DATA

The query runs in AWS Frankfurt, and the results stay in S3.

C Cross-cloud Lakehouse

Its cache stores data segments in Google Cloud, and it is Preview.

D A Dataflow pipeline

It streams every row out of AWS, and adds a pipeline to run.

THE SIGNAL

'Must not be copied out' plus 'results stay in AWS' means the engine goes to the data.

> Larkspur archives every raw meter message to Cloud Storage as it arrives, so it can replay history into BigQuery after a bug.
>
> New firmware adds fields every few months, and old files must stay readable without a separate schema file.

Which file format should the archive use?

A CSV, compressed with gzip, with a header row at the top of each file

B Parquet, written in small files as the messages arrive in the archive

C Avro, with the writer's schema stored inside every file it writes

D JSON lines, one message per line, with no schema stored anywhere

◆ PAUSE AND DECIDE

Which file format should the archive use?

A **Compressed CSV**

No types and no schema, and new fields shift the columns.

B **Parquet, small files**

Built for big analytic reads. Record-by-record writes make tiny files.

C **Avro, schema inside**

Row-based for fast appends. The schema travels and evolves with the data.

D **JSON lines**

Readable, but nothing enforces types, and every row repeats field names.

THE SIGNAL

'Appends as it arrives' plus 'schema changes, no separate schema file' is Avro.

> Larkspur's settlement tables are written daily by Spark jobs on Managed Service for Apache Spark. Analysts query the same tables in BigQuery.
>
> The board wants one copy of the data in an open format, which a different engine such as Trino could use later.

How should the tables be stored?

A Native BigQuery tables, with Spark writing through the BigQuery connector

B Iceberg managed tables in BigQuery, with Spark writing files to the bucket

C Parquet files as Hive external tables, with a nightly copy into BigQuery

D Iceberg tables in the Lakehouse runtime catalog, via its REST endpoint

◆ PAUSE AND DECIDE

How should the tables be stored?

A Native BigQuery tables

One copy, but in BigQuery's own storage, so not an open format.

B Managed Iceberg, Spark writes

Editing files outside BigQuery can lose data. BigQuery must write.

C Hive tables, nightly copy

Two copies to keep in step, and plain Parquet has no transactions.

D Runtime catalog Iceberg

Spark commits Iceberg, BigQuery reads it, and Trino can join later.

THE SIGNAL

Spark writes and BigQuery reads: Iceberg through the runtime catalog. One copy, open, shared.

> Larkspur runs about 300 Spark settlement jobs on the Bristol Hadoop cluster. They must move before the lease ends on 30 June 2027.
>
> The team wants minimal code changes, as few clusters to patch as possible, and the freedom to run the jobs elsewhere later.

What is the best plan?

A Keep the jobs as Spark, on Managed Service for Apache Spark

B Rewrite every job in Apache Beam, and run them all on Dataflow

C Rewrite every job as BigQuery SQL, scheduled through Dataform

D Install Spark on self-managed Compute Engine VMs, as it is today

◆ PAUSE AND DECIDE

What is the best plan?

A Keep Spark, managed

Few code changes, serverless batches where possible, and Spark runs anywhere.

B Rewrite in Beam

Portable, but three hundred rewrites against a fixed deadline.

C Rewrite as BigQuery SQL

Three hundred rewrites, and the logic is tied to one warehouse.

D Self-managed Spark on VMs

Few code changes, but every cluster is yours to patch.

THE SIGNAL

Existing Spark code plus a deadline means keep it Spark. Spark is already portable.

Module 09 in five lines

- Keep four layers open: **files, table format, catalog, code**.
- **Avro** for appends and changing schemas. **Parquet** or **ORC** for analytics.
- **Iceberg** shares tables. BigQuery writes: **managed**. Spark writes: **runtime catalog**.
- **Omni** moves the engine, not the data. **EXPORT DATA** keeps results in that cloud.
- **Trace every copy**: results, replicas and caches all count for residency.

COMING UP

Next: module 10, the start of section 2, planning pipelines.

MODULE 10 · SECTION 2 - INGESTING AND PROCESSING THE DATA

Planning Pipelines

Sources, sinks, the right service for the job, and the network the workers run in.

SECTION 2 of 5 EXAM WEIGHT 25% MODULE 10 of 45

admin@sthithapragna.com

Before you pick a service, answer three questions: **how fresh, how complex, and who maintains it.**

Batch, micro-batch or streaming

PATTERN	FRESHNESS	AT LARKSPUR
Batch	Hours, on a schedule	Nightly billing and settlement
Micro-batch	Minutes, small chunks	Regional demand for Westgrid
Streaming	Seconds, record by record	Live readings in the customer app

THE TIEBREAKER

Choose the slowest pattern the business can accept. Streaming costs more to run and to reason about.

ETL or ELT?

ETL

Transform, then load

- Clean in Dataflow, Spark or Data Fusion
- Only finished data lands
- Right when raw data must not land
- Or when logic is not SQL

ELT

Load, then transform

- Land raw data in BigQuery
- Transform with SQL and Dataform
- Raw data kept for reprocessing
- The usual default on BigQuery

THE TIEBREAKER

ELT is the default. ETL wins when data must be masked before it lands, or the logic needs real code.

Who should do the work?

SERVICE	SWEET SPOT	SIGNAL IN THE QUESTION
Dataflow	Batch and streaming in one model, autoscaled	Streaming, late data, complex code
Managed Service for Apache Spark	Existing Spark and Hadoop code	Spark, Hive, lift and shift
Cloud Data Fusion	Visual pipelines, many connectors	Low-code, GUI, many sources
Dataform	SQL transformations inside BigQuery	SQL, ELT, version control, tests
BigQuery SQL	Scheduled and continuous queries	Data already in BigQuery
Cloud Run functions	Small reactions to single events	One file arrives, call one thing

THE TIEBREAKER

Match the service to the people and the code that already exist, not to the size of the data.

A quick decision path

01

Already in BigQuery?

SQL can do it: Dataform or scheduled queries.

02

Spark code exists?

Keep it: Managed Service for Apache Spark.

03

Low-code team?

Visual and connectors: Cloud Data Fusion.

04

Streaming or complex?

Code at scale, late data: Dataflow.

05

One small event?

A few seconds of glue: Cloud Run functions.

HOW TO USE IT

Stop at the first question that says yes. It is a starting point, not a law.

Names and newcomers

- **Cloud Functions** is now **Cloud Run functions**. The original is **1st gen**.
- Event-driven functions run up to **9 minutes**. HTTP functions up to **60 minutes**.
- **Cloud Data Fusion** keeps its name, and runs on short-lived **Spark** clusters it creates.
- **BigQuery continuous queries**: SQL that never stops, writing to Pub/Sub, Bigtable or tables.

> REMEMBER
>
> Older questions say Cloud Functions. Same product family, answer the same way.

Continuous queries or Dataflow?

CONTINUOUS QUERIES

Streaming in SQL

- Data already flowing into BigQuery
- Needs an Enterprise reservation
- Joins and windows still Preview
- Reprocessing can duplicate output

DATAFLOW STREAMING

Streaming in code

- Any source, any sink
- Watermarks and late data
- Exactly-once processing options
- More to build and operate

THE TIEBREAKER

Simple SQL on data already in BigQuery: continuous queries. Late data, state or many sinks: Dataflow.

Larkspur's pipelines, end to end

SOURCE	DOES THE WORK	SINK
Pub/Sub meter-readings	Dataflow streaming	raw.meter_readings, Bigtable meter_live
Oracle billing	Datastream	BigQuery raw
Westgrid files in S3	Storage Transfer Service	Cloud Storage landing bucket
raw and curated tables	Dataform	curated and marts

THE PATTERN

One stream can feed two sinks: history for analysts, the latest value for the app.

admin@sthithapragna.com

Where Dataflow workers live

- Workers are VMs in **your subnet**. Size its IP range for autoscaling.
- **No public IPs**? Then turn on **Private Google Access** to reach Google APIs.
- Need PyPI or another outside site? **Cloud NAT**, or a **custom container**.
- Without Streaming Engine or Shuffle, workers talk on **TCP 12345 and 12346**.

> REMEMBER
>
> Behind VPC Service Controls, workers must use restricted.googleapis.com.

Launching Larkspur's pipeline, privately

```
1  SUBNET="https://www.googleapis.com/compute/v1/projects/lk-network-prod/\
2  regions/europe-west2/subnetworks/lk-dataflow"
3
4  python -m larkspur.readings_pipeline \
5    --runner=DataflowRunner \
6    --project=lk-ingest-prod \
7    --region=europe-west2 \
8    --subnetwork="$SUBNET" \
9    --no_use_public_ips \
10   --service_account_email="$LK_WORKER_SA" \
11   --dataflow_kms_key="$LK_DATAFLOW_KEY" \
12   --enable_streaming_engine
```

1-2 A Shared VPC subnet in the host project needs its full URL.

9 No public IPs, so the subnet needs Private Google Access.

10-11 A dedicated worker identity, and Larkspur's own key.

A BONUS

Streaming Engine also removes the need for worker-to-worker firewall ports.

Two identities, one key

- The **Dataflow service agent** creates workers and checks quota. Leave its roles alone.
- The **worker service account** reads and writes your data. Make it dedicated and minimal.
- **Shared VPC**: grant **Compute Network User** in the host project to the Dataflow service account.
- Data is encrypted in transit and at rest by default. A **CMEK** covers disks and state.

> THE SIGNAL
>
> 'Permission denied on the subnet' is a Network User problem, not a BigQuery role problem.

Phrases that pick the answer

THE QUESTION SAYS	REACH FOR
SQL team, data in BigQuery, version control	Dataform
Existing Spark jobs	Managed Service for Apache Spark
Visual, low-code, many connectors	Cloud Data Fusion
Streaming, late data, exactly-once	Dataflow
Must not land unmasked	ETL: transform before loading
No external IPs on workers	Private Google Access on the subnet

THE PATTERN

Freshness, complexity, and the people who will maintain it.

> Larkspur's analysts own the SQL that turns raw readings into curated tables and marts. Today it lives in saved queries that run in the wrong order on bad days.
>
> They want version control, tests, and steps that run in dependency order. Nobody on the team writes Java or Python.

What should they use?

A. A Dataflow batch pipeline, rewritten in Java, that runs once a night

B. Dataform, with each step as SQLX and assertions checking the results

C. A Cloud Data Fusion pipeline that copies tables between datasets

D. A Cloud Run function that runs the saved queries one after another

◆ PAUSE AND DECIDE

What should they use?

A **Dataflow in Java**

Nobody can maintain Java, and the data is already in BigQuery.

B **Dataform**

SQL they already know, in Git, with tests and dependency order built in.

C **Cloud Data Fusion**

Moves data through Spark clusters for work BigQuery can do itself.

D **A Cloud Run function**

Hand-written ordering, no tests, and a 9-minute limit on events.

THE SIGNAL

SQL team, data in BigQuery, version control and tests: that is Dataform's whole job.

> An organisation policy forbids external IP addresses on every VM. Larkspur's new Dataflow job starts, but its workers cannot reach Pub/Sub or BigQuery.
>
> Security also says nothing in the pipeline may reach the public internet.

What should the team configure?

A. Give the worker service account the BigQuery Admin and Pub/Sub Admin roles

B. Add a firewall rule allowing TCP 12345 and 12346 between the workers

C. Ask for a policy exception, so that the workers can have external IPs

D. Run with no public IPs, and turn on Private Google Access for the subnet

◆ PAUSE AND DECIDE

What should the team configure?

A Admin roles

Roles do not create a network path, and Admin breaks least privilege.

B Worker firewall ports

Those ports are for workers talking to each other, not to Google APIs.

C A policy exception

It reopens the internet path that security has ruled out.

D Private Google Access

Internal-only workers reach Google APIs, and nothing reaches the internet.

THE SIGNAL

'No external IPs' plus 'cannot reach Google APIs' means Private Google Access on the subnet.

> Larkspur's networks live in a Shared VPC host project, lk-network-prod. A Dataflow job in the service project lk-ingest-prod must run in its subnet.
>
> The job fails at launch with a permission error on the subnetwork.

What fixes the launch?

A. Give the Dataflow service account Compute Network User on the host subnet

B. Give the worker service account the Dataflow Admin role in the host project

C. Create a VPC in the service project, and peer it with the host network

D. Give the engineer who launches the job the Owner role on the host project

◆ PAUSE AND DECIDE

What fixes the launch?

A Network User for Dataflow

The documented grant, with the subnet's full URL in the job.

B Dataflow Admin for workers

Wrong identity and wrong role. Workers do not attach the subnet.

C A peered VPC

Sidesteps the shared network that the platform team runs.

D Owner for the engineer

Far too broad, and Dataflow's own account still lacks access.

THE SIGNAL

A permission error on a Shared VPC subnet is a Compute Network User grant in the host project.

> Larkspur Energie receives German customer files each night. Each record holds a national ID number that must be tokenised.
>
> Security says the raw IDs must never be stored in BigQuery, not even in a raw dataset for a few minutes.

How should the pipeline be designed?

A Load the files into a raw dataset, and have Dataform tokenise into curated

B Load the files into BigQuery, and protect the ID column with a policy tag

C Tokenise in a Dataflow pipeline, and write only tokens into BigQuery

D Load with Data Fusion, then apply dynamic masking to the ID column

◆ PAUSE AND DECIDE

How should the pipeline be designed?

A Dataform after loading

Classic ELT, so the raw IDs land in BigQuery first.

B A policy tag

Hidden from most users, but the raw IDs are still stored.

C Tokenise in Dataflow

ETL: the IDs are replaced in flight, so only tokens ever land.

D Dynamic masking

Masking changes what people see, not what is stored.

THE SIGNAL

'Must never be stored, not even raw' rules out every load-then-fix design. That is ETL.

> The customer app shows each meter's latest reading. Readings arrive on Pub/Sub at up to 10,000 a second in the evening, including resent duplicates.
>
> The app must see a new reading in Bigtable within seconds, and the pipeline must scale on its own.

Which service should process the stream?

A. A Dataflow streaming pipeline from Pub/Sub, writing to Bigtable

B. A Cloud Run function per message, writing each one to Bigtable

C. A scheduled query every minute that copies new rows into Bigtable

D. A Spark batch job every five minutes on a long-running cluster

◆ PAUSE AND DECIDE

Which service should process the stream?

A Dataflow streaming

Seconds of latency, autoscaling, and state to drop duplicate resends.

B A function per message

It scales, but each call is alone, with no shared state for duplicates.

C A scheduled query

A minute behind at best, and it needs the data in BigQuery first.

D Spark every five minutes

Minutes, not seconds, and a cluster to keep running.

THE SIGNAL

Seconds, scale on its own, and duplicates to handle: a streaming pipeline with state.

> When a Westgrid settlement file lands in Larkspur's landing bucket, the team wants to post a message to the operations channel and start the load workflow.
>
> Files arrive a few times a day. The work takes a couple of seconds, and nobody wants a server to look after.

What should react to each new file?

A. A Dataflow streaming pipeline that watches the bucket around the clock

B. A Cloud Run function, triggered by the bucket's object finalised event

C. A Cloud Data Fusion pipeline, scheduled to run once every minute

D. A cron job on a small Compute Engine VM that lists the bucket

◆ PAUSE AND DECIDE

What should react to each new file?

A Dataflow, always on

Workers running all day for a few seconds of work.

B A Cloud Run function

Runs only when a file lands, for seconds, with no server to manage.

C Data Fusion every minute

A Spark cluster spun up every minute to post a message.

D Cron on a VM

A server to patch and pay for, and polling instead of events.

THE SIGNAL

One event, a few seconds of work, no server: Cloud Run functions.

Module 10 in five lines

- Ask three questions: **how fresh, how complex, who maintains it**.
- **ELT** by default on BigQuery. **ETL** when raw data must never land, or logic needs code.
- **Dataform** for SQL, **Spark** for Spark, **Data Fusion** for low-code, **Dataflow** for streaming.
- **Cloud Run functions** for one small event. **Continuous queries** for SQL streaming.
- No public IPs needs **Private Google Access**. Shared VPC needs **Network User**.

COMING UP

Next: module 11, Pub/Sub delivery, replay and failure.

MODULE 11 · SECTION 2 - INGESTING AND PROCESSING THE DATA

Pub/Sub: Delivery, Replay and Failure

Subscriptions, ack deadlines, replay, dead letters, exactly-once, ordering, and when Kafka fits better.

SECTION 2 of 5 EXAM WEIGHT 25% MODULE 11 of 45

admin@sthithapragna.com

Pub/Sub delivers every message **at least once**. Your design decides what happens the second time.

admin@sthithapragna.com

From meter to subscriber

01

Publish

The head-end publishes each reading to meter-readings.

02

Store

The topic stores it, checks its schema, and fans out.

03

One copy each

Every subscription gets its own copy of every message.

04

Deliver

A subscriber receives it, and must ack before the deadline.

05

Or again

No ack in time, or a nack, and it is delivered again.

THE PATTERN

Subscriptions are independent. A slow archive never holds up the live pipeline.

Four ways to receive

TYPE	HOW IT WORKS	PICK IT WHEN
Pull	Your client asks for messages	High throughput, Dataflow, exactly-once
Push	Pub/Sub POSTs to your HTTPS endpoint	Serverless receivers, such as Cloud Run
BigQuery	Writes straight into a table	No processing needed before storing
Cloud Storage	Writes batches of files to a bucket	Cheap raw archives

THE SIGNAL

No transformation, straight to a table: a BigQuery subscription, not a Dataflow job.

Sign for it in time

- Default **10 seconds**. Configurable from **10 to 600 seconds**.
- Client libraries **extend the lease** while a message is still being worked on.
- Miss the deadline and the message is **delivered again**, even if you finish later.
- A **negative ack** asks for redelivery straight away. Useful for transient errors.

> THE SIGNAL
>
> Rising duplicates with a slow subscriber usually means missed deadlines, not a chatty publisher.

How long does Pub/Sub remember?

SETTING	DEFAULT	RANGE
Subscription retention	7 days	10 minutes to 31 days
Retain acked messages	Off	Keeps acked messages for replay
Topic retention	Off	10 minutes to 31 days
Snapshot lifetime	Up to 7 days	Less if the backlog is old
Subscription expiry	31 days idle	1 day up, or never

CURRENCY CHECK

Older material says subscriptions keep messages for 7 days at most. The maximum is now 31.

admin@sthithapragna.com

Seek to a time, or to a snapshot?

SEEK TO A TIME

Rewind the clock

- Messages after the time become unacked
- Needs retained messages
- Recover from any surprise
- A future time purges a backlog

SEEK TO A SNAPSHOT

Rewind to a bookmark

- Saved ack state at one moment
- No retention settings needed
- Take one before every deploy
- Lasts up to 7 days

THE TIEBREAKER

Planned risk, like a deployment: snapshot first. Unplanned trouble: seek to a time, if retention is on.

A safe deployment at Larkspur

```
1  # 1. Before deploying new subscriber code, bookmark the ack state
2  gcloud pubsub snapshots create pre-deploy-1009 \
3      --subscription=readings-dataflow
4
5  # 2. The new code has a bug: rewind to the bookmark
6  gcloud pubsub subscriptions seek readings-dataflow \
7      --snapshot=pre-deploy-1009
8
9  # 3. No snapshot? Rewind to a time, if acked messages are retained
10 gcloud pubsub subscriptions seek readings-dataflow \
11     --time=2026-10-09T12:00:00Z
```

`2-3` Seconds to run, and it saves a bad deploy.

`6-7` Everything since the bookmark is delivered again.

`10-11` Works only with retention switched on beforehand.

REMEMBER

Replay means duplicates by design. Idempotent writes make replay safe.

Retries and dead letters

- Retry policy: **immediately** by default, or **exponential backoff** from 10 to 600 seconds.
- A **dead-letter topic** takes a message after **5 to 100** attempts. Default 5.
- The **Pub/Sub service agent** needs Publisher on the dead-letter topic, and Subscriber on the subscription.
- Forwarding is **best effort**, so watch the dead-letter topic with an alert.

> THE PATTERN
>
> A poison message retried forever blocks good work. Back off, then set it aside.

Exactly-once delivery, precisely

- **Pull subscriptions only**, including streaming pull. Not push, not BigQuery, not Storage.
- Subscribers must connect in **the same region**.
- No redelivery after a **successful ack**, and none while a message is out.
- **Publisher retries can still create copies**. Latency is noticeably higher.

> THE TRAP
>
> Exactly-once delivery is not exactly-once data. Still deduplicate on a business key.

Ordering keys

- Messages with the **same key**, published in the **same region**, arrive in order.
- Each key is limited to **1 MBps** of publishing.
- One redelivery **replays every later message** for that key, even acked ones.
- Often you can **sort by event time** instead, and skip ordering entirely.

> THE TIEBREAKER
>
> Ask whether order matters, or only the final answer. Timestamps and idempotent writes are cheaper.

Filters, schemas and transforms

- **Filters** select on **attributes only**, and cannot be changed after creation.
- **Schemas** in Avro or Protocol Buffer. A message that does not match is **not published**.
- **Single message transforms**: a JavaScript function or AI inference, on a topic or subscription.
- Transforms handle **one message at a time**. Aggregation still needs Dataflow.

> THE SIGNAL
>
> 'Reject malformed messages at the door' is a topic schema. 'Mask one field' can be a transform.

Pub/Sub or Managed Service for Apache Kafka?

PUB/SUB

Serverless messaging

- No partitions or clusters to size
- Up to 31 days of retention
- Pay for what you use
- Dead-letter topics built in

MANAGED KAFKA

Kafka, run for you

- Clusters and partitions to plan
- Indefinite retention, event sourcing
- Kafka API: existing apps move as is
- Same platform on-premises too

THE TIEBREAKER

Existing Kafka apps, minimal change: Managed Kafka. Pub/Sub Lite was turned down on 18 March 2026.

admin@sthithapragna.com

Phrases that pick the answer

THE QUESTION SAYS	REACH FOR
Duplicates rise with a slow subscriber	A longer ack deadline
Reprocess after a bad deploy	Seek to a snapshot or a time
No processing, straight to a table	BigQuery subscription
Poison messages keep failing	Backoff and a dead-letter topic
Never process twice	Exactly-once on pull, plus dedupe
Existing Kafka apps	Managed Service for Apache Kafka

THE PATTERN

At least once is the default. Design every subscriber to survive the second delivery.

> A Larkspur subscriber calls a slow tariff service for every reading, taking about 30 seconds per message. Its custom client never extends the ack deadline, which is still the default.
>
> Duplicate readings downstream have risen sharply, but the head-end publishes each reading only once.

What should the team change first?

A Turn on message ordering, so that messages arrive one at a time

B Attach a dead-letter topic, with five delivery attempts

C Raise the ack deadline above the processing time

D Switch the subscription from pull delivery to push delivery

◆ PAUSE AND DECIDE

What should the team change first?

A. Message ordering

Ordering does not stop redelivery. It can make it worse.

B. A dead-letter topic

Good messages would be set aside just for being slow.

C. Longer deadline

30 seconds of work outlasts a 10-second deadline. Fix that, and the copies stop.

D. Push delivery

The endpoint would still be slow, and still miss its deadline.

THE SIGNAL

Slow work plus the default deadline means redelivery. The duplicates are self-inflicted.

> A new version of Larkspur's subscriber went live at 09:00. Until noon it acknowledged every reading but wrote nothing, because of a bug.
>
> Nobody took a snapshot. The subscription retains acknowledged messages for 7 days, and the bug is now fixed.

How should the team recover the three lost hours?

A Ask the head-end team to republish every reading sent since 09:00

B Create a snapshot of the subscription now, then seek to that snapshot

C Create a new subscription, which will receive the last three hours

D Seek the subscription to a time just before the 09:00 deployment

◆ PAUSE AND DECIDE

How should the team recover the three lost hours?

A **Republish from the head-end**

Slow, manual, and it depends on a system that may not keep them.

B **Snapshot now, then seek**

A snapshot taken now bookmarks the present, after the loss.

C **A new subscription**

New subscriptions start from now. They do not see older messages.

D **Seek to a time**

Retained acked messages after 09:00 become unacked, and are delivered again.

THE SIGNAL

No snapshot, but acked messages retained: seek to a time before the problem began.

> Larkspur wants every raw reading copied into raw.meter_readings exactly as published, with no transformation. The topic already has an Avro schema.
>
> The table accepts duplicates, and the team wants the least code and infrastructure possible.

How should the readings reach the table?

A. A Dataflow streaming job that reads the topic and writes each row

B. A push subscription to a Cloud Run service that inserts rows

C. A BigQuery subscription that uses the topic's Avro schema

D. A Cloud Storage subscription, followed by an hourly load job

◆ PAUSE AND DECIDE

How should the readings reach the table?

A **A Dataflow job**

It works, but it is a pipeline to run for no transformation at all.

B **Push to Cloud Run**

A service to write, deploy and scale, just to insert rows.

C **BigQuery subscription**

Straight into the table, mapped by the schema, with no code.

D **Storage, then load jobs**

Two steps, an hour of delay, and a scheduler to manage.

THE SIGNAL

No processing, straight to a table, least code: a BigQuery subscription.

> A few malformed readings make Larkspur's subscriber throw an error every time. Pub/Sub redelivers them immediately, again and again, which wastes capacity during the evening surge.
>
> The team wants bad messages set aside for investigation after a few tries.

What should they configure?

A. Seek the subscription to a future time, to purge the stuck messages

B. Raise the ack deadline to its maximum of 600 seconds

C. Turn on exactly-once delivery for the subscription

D. Add a dead-letter topic, with an exponential backoff retry policy

◆ PAUSE AND DECIDE

What should they configure?

A Seek to the future

It purges every pending message, good ones included.

B A longer ack deadline

The message fails fast. A longer deadline changes nothing.

C Exactly-once delivery

It stops duplicates after acks. These messages are never acked.

D Dead-letter topic

Retries slow down, then bad messages move aside for investigation.

THE SIGNAL

'Keeps failing' plus 'set aside for investigation' is a dead-letter topic. 'Wasting capacity' adds backoff.

> Larkspur's payment service reads card payment events from a pull subscription in europe-west2. A payment must never be applied twice.
>
> Publishers sometimes retry after network errors. Each event carries a unique payment ID.

What is the right design?

A Enable exactly-once delivery, and still dedupe on the payment ID

B Enable exactly-once delivery on a push subscription to Cloud Run

C Rely on default delivery, because Pub/Sub never repeats a message

D Turn on message ordering, because ordering also removes duplicates

◆ PAUSE AND DECIDE

What is the right design?

A Exactly-once, deduped

No redelivery after acks, and the ID catches publisher retries.

B Exactly-once on push

Push subscriptions do not support exactly-once delivery.

C Default delivery

Default delivery is at least once. Duplicates are normal.

D Message ordering

Ordering controls sequence, not copies.

THE SIGNAL

'Never twice' plus 'publishers retry' means exactly-once delivery and a business-key check.

> Larkspur Energie runs Apache Kafka on-premises in Hamburg, with about 40 Kafka Streams applications.
>
> It wants to move them to Google Cloud with minimal code changes, while some stay on-premises for another year.

Where should the applications go?

A Rewrite every application for Pub/Sub, using ordering keys

B Move them to Managed Service for Apache Kafka

C Move them to Pub/Sub Lite, which also uses partitions

D Run self-managed Kafka on Compute Engine virtual machines

◆ PAUSE AND DECIDE

Where should the applications go?

A **Rewrite for Pub/Sub**

Forty rewrites, and two platforms during the transition.

B **Managed Kafka**

The Kafka API, so apps move as they are, and match on-premises.

C **Pub/Sub Lite**

Turned down on 18 March 2026. It is no longer an option.

D **Self-managed Kafka**

Same API, but brokers and upgrades become the team's job.

THE SIGNAL

Existing Kafka apps plus minimal change means Managed Kafka. Pub/Sub Lite is gone.

Module 11 in five lines

- Delivery is **at least once**. Design every subscriber to survive a second delivery.
- Ack deadline **10 to 600 seconds**. Missed deadlines look like duplicates.
- **Snapshot before deploys**. Seek to a time only if messages were retained.
- Backoff plus a **dead-letter topic** for poison messages. Exactly-once is **pull only**.
- Existing Kafka apps go to **Managed Kafka**. Pub/Sub Lite is gone.

COMING UP

Next: module 12, Dataflow and Beam, and how a pipeline really runs.

MODULE 12 · SECTION 2 - INGESTING AND PROCESSING THE DATA

Dataflow and Beam: How a Pipeline Really Runs

PCollections and transforms, joins, fusion, autoscaling, streaming modes, templates and connectors.

SECTION 2 of 5 EXAM WEIGHT 25% MODULE 12 of 45

admin@sthithapragna.com

You write the recipe. Dataflow decides the kitchen, the cooks and the order of the steps.

Six words to know

TERM	WHAT IT IS
Pipeline	The whole recipe, from sources to sinks
PCollection	A data set flowing between steps. Bounded or unbounded
PTransform	A step that turns PCollections into new ones
ParDo and DoFn	Your code, run on each element in parallel
GroupByKey, Combine	Bring elements with the same key together
Runner	What executes the recipe: Dataflow, Flink, Spark

HOW IT WORKS

PCollections are immutable. Every step makes a new one, which is what lets steps run in parallel.

A DoFn with a dead-letter output

```
1  class ParseReading(beam.DoFn):
2      def process(self, msg):
3          try:
4              r = json.loads(msg)
5              if "meter_id" not in r:
6                  raise ValueError("no meter_id")
7              yield r
8          except Exception as e:
9              yield beam.pvalue.TaggedOutput("bad", {"raw": msg, "error": str(e)})
10
11 out = msgs | "Parse" >> beam.ParDo(ParseReading()).with_outputs("bad", main="good")
12 out.good | "Raw" >> beam.io.WriteToBigQuery(RAW_TABLE)
13 out.bad | "Dead letter" >> beam.io.WriteToBigQuery(ERROR_TABLE)
```

9 Bad ones go to a tagged output, with the error, never dropped.

13 The dead-letter table, ready for investigation.

THE PATTERN

One bad message must never crash a streaming pipeline. Catch it, tag it, keep it.

Side input or CoGroupByKey?

SIDE INPUT

Broadcast a small lookup

- Copied to every worker
- Small and slowly changing
- No shuffle of the main data
- Refresh it on a schedule in streaming

COGROUPBYKEY

Join two big collections

- Both sides shuffled by key
- Large on both sides
- Costs a shuffle
- Both inputs need the same key

THE TIEBREAKER

A few thousand rows that rarely change: side input. Millions on both sides: CoGroupByKey.

Fusion: usually a gift, sometimes a trap

- Dataflow **fuses** neighbouring steps, so it never stores the data between them.
- The trap: a step that **fans out**, like one file name becoming millions of rows.
- Fused to a tiny input, the fan-out runs on **only a few workers**.
- Break the fusion with **Redistribute**, called **Reshuffle** in older material.

> REMEMBER
>
> Partial combining before a GroupByKey is automatic. Never hand-roll it.

How Dataflow decides how many workers

- Signals: average **CPU** and the pipeline's **parallelism**.
- **Batch**: on by default, re-checked every 30 seconds. Default cap 2,000 workers.
- **Streaming**: on by default only with **Streaming Engine**.
- Set **maxNumWorkers** for cost control. Make sure quota and the subnet can keep up.

> THE POINT
>
> Autoscaling can only add workers that have something to do. Parallelism is the real limit.

Why won't it scale?

SYMPTOM	LIKELY CAUSE	FIX
Few workers, stuck reading	One large gzip file cannot be split	Many files, or uncompressed input
One worker at 100%, others idle	A hot key in a GroupByKey	Hot-key fan-out, or a better key
Fan-out step crawls	Fused to a tiny input	Redistribute after the fan-out
Stops at a fixed count	maxNumWorkers, quota or IP range	Raise the limit that is binding

THE TRAP

Bigger machines rarely fix a parallelism problem. Find what cannot be split.

Exactly-once or at-least-once?

EXACTLY-ONCE

The default

- No records dropped or duplicated
- Right for counts, sums, money
- Higher cost and latency
- Works with every sink

AT-LEAST-ONCE

Cheaper and faster

- Nothing lost, a few duplicates
- Map-only copies, downstream dedupe
- Needs Streaming Engine
- BigQuery: Storage Write API at-least-once

THE TIEBREAKER

Aggregations need exactly-once. A copy into a table that is deduplicated later can save money with at-least-once.

Four features to recognise

FEATURE	WHAT IT DOES	SIGNAL
Streaming Engine	Moves execution off workers into the service	Smaller workers, smoother scaling
Dataflow Shuffle	Moves batch shuffles off workers	Faster batch scale-down
Dataflow Prime	Vertical autoscaling of worker memory	Out-of-memory errors
FlexRS	Delayed start, mostly preemptible VMs, discount	Batch, not time-critical, cheaper

THE TRAP

FlexRS can wait up to six hours to start. 'Must finish by 06:00' may rule it out.

Classic or Flex templates?

CLASSIC

A frozen graph

- The job graph is saved once
- Parameters need ValueProvider
- The shape cannot change at launch
- Older, still common

FLEX

A container that builds the graph

- Image in Artifact Registry
- Graph built at launch from parameters
- Steps can change by parameter
- Google's recommendation for new work

THE TIEBREAKER

Different steps for different parameters points to Flex. Google-provided templates cover common jobs.

Reading and writing everything

- **PubSubIO**, **BigQueryIO**, **BigtableIO**, **KafkaIO**, text and Avro files on Cloud Storage.
- **JdbcIO** for Cloud SQL and other databases. Partition reads, or one worker does it all.
- **Managed I/O**: Dataflow runs and upgrades the Iceberg, Kafka and BigQuery connectors.
- BigQueryIO prefers the **Storage Write API** for streaming writes.

> THE TRAP
>
> A JDBC read with no partitioning is a single-threaded query, whatever the worker count.

Phrases that pick the answer

THE QUESTION SAYS	REACH FOR
Small lookup, changes rarely	Side input
Fan-out runs on few workers	Redistribute, or Reshuffle
One worker hot in a grouping	Fix the hot key
Map-only, dedupe later, cut cost	At-least-once streaming mode
Out-of-memory errors on workers	Dataflow Prime vertical autoscaling
Steps vary by launch parameter	Flex template

THE PATTERN

Most Dataflow questions are about parallelism, state, or who launches the job.

> A Larkspur batch job reads a small manifest listing 2,000 exported history files, then expands each file name into millions of readings.
>
> maxNumWorkers is 100, but the job sits on two busy workers for hours while the rest stay idle.

What should the team change?

A Raise maxNumWorkers from 100 to 500, to give it more room

B Switch the workers to a larger machine type with more memory

C Add a Redistribute step right after the fan-out

D Run the job in streaming mode instead of batch

◆ PAUSE AND DECIDE

What should the team change?

A. A higher worker cap

The cap was never reached. More room does not create parallel work.

B. Bigger machines

Two bigger workers are still only two workers.

C. Redistribute

Breaks the fusion, so millions of rows spread across every worker.

D. Streaming mode

The data is bounded. The fusion problem would remain.

THE SIGNAL

A tiny input fanning out into millions of rows, on few workers, is fusion. Redistribute breaks it.

> Larkspur's streaming pipeline must add a region name to every reading, from a postcode area lookup of about 3,000 rows.
>
> The lookup changes a few times a year. The stream peaks at 10,000 readings a second.

How should the pipeline do the lookup?

A Load the lookup as a side input, and refresh it on a schedule

B Join the stream to the lookup table with a CoGroupByKey step

C Query Cloud SQL from inside the DoFn, once for every reading

D Write readings to BigQuery, and join them later in a nightly batch

◆ PAUSE AND DECIDE

How should the pipeline do the lookup?

A A refreshed side input

Small, slow-changing data on every worker, with no shuffle.

B CoGroupByKey

Shuffles ten thousand readings a second to join three thousand rows.

C Cloud SQL per reading

Ten thousand queries a second against a database.

D A nightly batch join

The region is needed now, not tomorrow.

THE SIGNAL

Small, rarely changing, needed on every element: a side input.

> A Larkspur pipeline copies readings from Pub/Sub into raw.meter_readings with no aggregation. Duplicates are removed later, when the curated table is built.
>
> The finance director wants the pipeline's cost and latency down.

What should the team change?

A Keep exactly-once mode, and add more workers at peak times

B Switch the job to Dataflow Prime for vertical autoscaling

C Write to BigQuery with nightly file load jobs instead of streaming

D Use at-least-once mode, with the Storage Write API at-least-once

◆ PAUSE AND DECIDE

What should the team change?

A More workers

Adds cost, the opposite of the goal.

B Dataflow Prime

Prime fixes memory problems. Nothing here is short of memory.

C Nightly file loads

Cheaper, perhaps, but a day of latency.

D At-least-once mode

Map-only, deduplicated later: the mode built for this, cheaper and faster.

THE SIGNAL

Map-only copy plus downstream dedupe plus cost pressure points to at-least-once mode.

> Larkspur's operations team launches the readings pipeline for the UK and for Germany, with different parameters. For Germany, the pipeline must add an extra tokenisation step.
>
> Operators have no development tools, and must not need the source code.

How should the pipeline be packaged?

A A classic template, with ValueProvider parameters for each country

B A Flex template whose image builds the graph from its parameters

C The source code in a shared repository, run locally by operators

D Two classic templates, one per country, kept in step by hand

◆ PAUSE AND DECIDE

How should the pipeline be packaged?

A One classic template

Its graph is frozen, so it cannot add a step for Germany.

B A Flex template

The graph is built at launch, so the German step appears when needed.

C Run from source

Operators have no tools, and must not need the code.

D Two classic templates

Two copies to keep in step, and Google recommends Flex for new work.

THE SIGNAL

'Different steps per parameter' plus 'no source code' is a Flex template.

Module 12 in five lines

- Beam is the recipe, Dataflow the kitchen. **Parallelism** is what really scales.
- **Side input** for small lookups. **CoGroupByKey** when both sides are big.
- Fan-out on few workers is **fusion**: Redistribute. One hot worker is a **hot key**.
- **At-least-once** for map-only copies. **Prime** for memory. **FlexRS** for patient batches.
- **Flex templates** build the graph at launch. Partition every JDBC read.

COMING UP

Next: module 13, windows: fixed, sliding, session and global.

MODULE 13 · SECTION 2 - INGESTING AND PROCESSING THE DATA

Windows: Fixed, Sliding, Session, Global

Event time against processing time, the four window types, and picking the right one from the question's wording.

SECTION 2 of 5 EXAM WEIGHT 25% MODULE 13 of 45

admin@sthithapragna.com

A stream never ends, so you can never total it. **Windows cut it into pieces you can finish.**

admin@sthithapragna.com

Event time or processing time?

EVENT TIME

When it happened

- The meter's own timestamp
- reading_ts at Larkspur
- Stable, however late it arrives
- What the business means

PROCESSING TIME

When the pipeline saw it

- The clock on the worker
- Changes with every delay
- Fine for system monitoring
- Wrong for billing and demand

FOR LARKSPUR

Meters can upload up to 72 hours late. Only event time puts a late reading in the right half hour.

admin@sthithapragna.com

Four ways to cut a stream

WINDOW	SHAPE	THINK OF
Fixed, or tumbling	Equal, back to back, no overlap	A bus timetable: every 30 minutes
Sliding, or hopping	A length and a period, overlapping	A moving average on a share ticker
Session	Per key, ends after a quiet gap	A shop visit that ends after 15 quiet minutes
Global	One window for everything	A running total since the very start

REMEMBER

Learn both names for fixed and sliding. Google's docs say tumbling and hopping; Beam says fixed and sliding.

admin@sthithapragna.com

Fixed: one reading, one window

- Equal size, **no overlap**. Every element lands in **exactly one** window.
- Larkspur's **half-hour settlement periods** are fixed windows of 30 minutes.
- Windows line up with the clock, in **UTC**. Use an offset for other alignments.
- Cheapest to run, because nothing is counted twice.

> THE SIGNAL
>
> 'Every hour', 'per half hour', 'each day' all point to fixed windows.

Sliding: the last X, every Y

- Two numbers: the **length** and the **period**, how often a new window starts.
- Windows **overlap**, so each element lands in several of them.
- A 60-minute window every 5 minutes puts each reading in **12 windows**.
- For smooth, rolling numbers: 'usage in the last hour', refreshed often.

> THE SIGNAL
>
> 'The last 30 seconds, every 5 seconds' is a sliding window: length 30, period 5.

Session: until things go quiet

- Defined by a **gap duration**. A pause longer than the gap ends the session.
- **Per key**: each engineer, customer or device gets its own sessions.
- Sessions **vary in length**, and merge when new events close the gap.
- Right for visits, journeys and bursts of activity.

> THE SIGNAL
>
> 'Until there is no activity for 15 minutes' is a session window with a 15-minute gap.

admin@sthithapragna.com

Global: the default, and its catch

- Every PCollection starts in **one global window**. Fine for bounded batch data.
- On a stream, a **GroupByKey** in the global window, with the default trigger, **fails at build time**.
- Fix it with a **non-global window**, or a **trigger** that fires repeatedly.
- Global plus a trigger gives 'since the start, updated every 10 minutes'.

> REMEMBER
>
> Inputs to a CoGroupByKey must share the same windowing, or the pipeline will not build.

Three windows on Larkspur's stream

```
stamped = parsed | "Event time" >> beam.Map(
    lambda r: window.TimestampedValue(r, r["reading_epoch"]))
by_region = stamped | "Key" >> beam.Map(lambda r: (r["region"], r["kwh"]))
half_hours = (by_region
    | "Fixed 30m" >> beam.WindowInto(window.FixedWindows(30 * 60))
    | "Sum" >> beam.CombinePerKey(sum))
last_hour = (by_region
    | "Slide 60m by 5m" >> beam.WindowInto(window.SlidingWindows(60 * 60, 5 * 60))
    | "Sum hour" >> beam.CombinePerKey(sum))
visits = engineer_events | "Gap 15m" >> beam.WindowInto(window.Sessions(15 * 60))
```

`2` Event time from the meter, not the worker's clock.

`5, 8` Half hours counted once; a rolling hour refreshed every 5 minutes.

THE POINT

Same stream, three questions. The window, not the data, decides which question is answered.

admin@sthithapragna.com

Grouping happens per key and per window

- A **GroupByKey** groups by key **inside each window**. One key, two windows, two results.
- Window first, then group. The window decides **when**, the key decides **what**.
- A key that is huge in every window is a **hot key**, every window.
- For per-region totals, the key is the region. For per-meter, the key is the meter.

> THE SIGNAL
>
> 'Per meter, per half hour' means key by meter, then fixed 30-minute windows.

admin@sthithapragna.com

Windows without Beam

- In batch SQL, a fixed window is just **GROUP BY** a truncated or bucketed timestamp.
- **Continuous queries** offer **TUMBLE** for fixed windows. Preview.
- TUMBLE windows use the row's **change timestamp**, up to 24 hours long.
- No sliding or session windows in continuous queries. Those need Dataflow.

> THE TIEBREAKER
>
> 'Sessions' or 'rolling windows' on a live stream still points to Dataflow.

admin@sthithapragna.com

Phrases that pick the window

THE QUESTION SAYS	WINDOW
Every half hour, per day, each hour	Fixed, or tumbling
The last 10 minutes, every minute	Sliding, or hopping
Until 15 minutes of inactivity	Session, per key
Since the start, updated every few minutes	Global, with a repeating trigger
When it happened, late uploads	Event time, not processing time
Per meter, per half hour	Key by meter, then fixed windows

THE PATTERN

Underline the time words in the question. They name the window.

admin@sthithapragna.com

> Westgrid wants Larkspur's total demand for each region, for each half-hour settlement period.
>
> Each reading must count towards exactly one period, the one in which it was taken.

Which windowing should the pipeline use?

A. Sliding windows of 30 minutes, starting every 5 minutes, by region

B. Fixed 30-minute windows on event time, keyed by region

C. Session windows with a 30-minute gap, keyed by region

D. The global window, with a trigger every 30 minutes of processing time

◆ PAUSE AND DECIDE

admin@sthithapragna.com

Which windowing should the pipeline use?

A Sliding windows

Each reading lands in six windows, not exactly one.

B Fixed, event time

One reading, one half hour: the one it was taken in.

C Session windows

Sessions follow activity. Regions never go quiet for 30 minutes.

D Global, processing time

Totals by arrival time, so late readings land in the wrong period.

THE SIGNAL

'Each half hour' plus 'exactly one period' plus 'when it was taken': fixed windows on event time.

admin@sthithapragna.com

> The customer app shows each household its electricity use over the last 60 minutes.
>
> Customers want the number refreshed every 5 minutes, not once an hour.

Which windowing fits this best?

A Fixed 60-minute windows, keyed by meter

B Session windows with a 5-minute gap, keyed by each household's meter

C Fixed 5-minute windows, summed in the app afterwards

D Sliding windows, 60 minutes long, starting every 5 minutes

◆ PAUSE AND DECIDE

admin@sthithapragna.com

Which windowing fits this best?

A Fixed 60-minute windows

Right total, but it updates only once an hour.

B Sessions, 5-minute gap

Meters report every half hour, so every reading is its own session.

C Fixed 5-minute, summed later

Moves the windowing into the app, and every client repeats it.

D Sliding 60 by 5

A rolling hour, recomputed every five minutes, exactly as asked.

THE SIGNAL

'The last 60 minutes, every 5 minutes' is a sliding window: length 60, period 5.

admin@sthithapragna.com

> Field engineers' tablets send diagnostic events while they work at a site. Larkspur wants one summary per site visit, per engineer.
>
> A visit is over when an engineer's tablet has sent nothing for 15 minutes. Visits last anything from 10 minutes to 6 hours.

Which windowing fits this best?

A. Session windows with a 15-minute gap, keyed by engineer

B. Fixed 15-minute windows, keyed by engineer

C. Sliding windows of 6 hours, starting every 15 minutes

D. Fixed 6-hour windows, with every visit trimmed to fit inside one

◆ PAUSE AND DECIDE

admin@sthithapragna.com

Which windowing fits this best?

A Sessions, 15-minute gap

Each engineer's visit ends after 15 quiet minutes, whatever its length.

B Fixed 15-minute windows

Cuts a two-hour visit into eight pieces.

C Sliding 6-hour windows

Overlapping windows, so each visit appears many times.

D Fixed 6-hour windows

Visits cross window edges, and get split or merged by the clock.

THE SIGNAL

'Ends after 15 quiet minutes' plus 'varies in length' plus 'per engineer' is a session window.

admin@sthithapragna.com

> Larkspur wants a running total of energy for each meter, counted from the day it was installed.
>
> The total should be refreshed every 10 minutes, and never reset.

How should the pipeline window the data?

A Fixed 10-minute windows, keyed by meter

B Sliding windows one year long, starting every 10 minutes per meter

C The global window per meter, with a trigger every 10 minutes

D Session windows per meter, with a 10-minute gap

◆ PAUSE AND DECIDE

admin@sthithapragna.com

How should the pipeline window the data?

A Fixed 10-minute windows

Each window holds only its own ten minutes, so the total resets.

B One-year sliding windows

Still forgets anything older than a year, and costs a fortune.

C Global, repeating trigger

One window that never closes, with a result every ten minutes.

D Session windows

A reading every half hour means a new session every time.

THE SIGNAL

'Since installation, never reset, updated every 10 minutes': the global window plus a repeating trigger.

admin@sthithapragna.com

> Larkspur's half-hourly demand totals keep disagreeing with the meters. Readings that arrive late are counted in whichever half hour they reach the pipeline.
>
> Some meters upload up to 72 hours late after losing signal.

What is the root cause, and the fix?

A Processing-time windows: window on the meter's event timestamp instead

B Too few workers: raise the maximum, so that readings are processed sooner

C Fixed windows: switch to sliding windows, to smooth out the totals

D A missing dead-letter output: send late readings to an error table

◆ PAUSE AND DECIDE

What is the root cause, and the fix?

A Use event time

Each reading carries its own time, so late ones rejoin the right half hour.

B More workers

Workers cannot speed up a meter that had no signal for three days.

C Sliding windows

Smoother numbers, still in the wrong half hours.

D Dead-letter late readings

Late readings are valid. Throwing them aside loses real demand.

THE SIGNAL

'Counted when they reach the pipeline' is processing time. The fix is event time.

admin@sthithapragna.com

> A new streaming pipeline reads Pub/Sub and calls GroupByKey to count readings per meter. No windowing or trigger is set.
>
> The job fails before it starts, with an error about the GroupByKey.

What should the developer change?

A Raise the worker count, so the grouping has enough memory

B Apply a window, or a repeating trigger, before the GroupByKey

C Run the same pipeline as a batch job over the subscription

D Replace the GroupByKey with a side input holding the meter counts

◆ PAUSE AND DECIDE

admin@sthithapragna.com

What should the developer change?

A More workers

It failed at construction, before any worker ran.

B Window before grouping

An unbounded GroupByKey needs a window or a trigger, or Beam rejects it.

C Run as batch

A subscription is unbounded, and batch loses the streaming result.

D A side input

Side inputs are for lookups, not for counting the stream itself.

THE SIGNAL

Unbounded data plus GroupByKey plus no window or trigger fails at build time.

admin@sthithapragna.com

Module 13 in five lines

- Window on **event time**, the meter's clock, not the worker's.
- **Fixed** for 'every half hour'. **Sliding** for 'the last X, every Y'.
- **Session** for 'until quiet for N minutes', per key.
- **Global** plus a repeating trigger for 'since the start'.
- An unbounded GroupByKey needs **a window or a trigger**, or it will not build.

COMING UP

Next: module 14, watermarks, late data and triggers.

admin@sthithapragna.com

MODULE 14 · SECTION 2 - INGESTING AND PROCESSING THE DATA

Watermarks, Late Data and Triggers

How long a window waits, what happens to stragglers, and when results are emitted.

SECTION 2 of 5 EXAM WEIGHT 25% MODULE 14 of 45

admin@sthithapragna.com

A watermark is the pipeline's best guess that **everything for this window has arrived**.

Where the watermark comes from

- Sources estimate it. For Pub/Sub, from the **oldest unacknowledged message**.
- By default Pub/Sub watermarks use **publish time**, not when the reading was taken.
- Set a **timestamp attribute**, and the watermark follows **event time** instead.
- An **idle source** holds back the watermark for the whole pipeline.

> FOR LARKSPUR
>
> If the head-end publishes in batches, publish time and reading time can be hours apart.

On time or late?

ON TIME

Before the watermark passes

- Arrives before the window closes
- Included in the main result
- The normal case
- 96% of Larkspur's readings

LATE

After the watermark passes

- Event time is inside a closed window
- Dropped, by default
- Needs allowed lateness to count
- 4% of Larkspur's, after an hour

THE TRAP

By default, late data is silently dropped. Totals look plausible and are quietly wrong.

Keeping the door open

- **Allowed lateness** keeps a window open for a set time after the watermark passes.
- Late elements within it **update the result**, through late firings.
- After the watermark passes the window end **plus** the lateness, the window expires.
- Every open window **holds state**. Longer lateness costs memory and money.

> THE TIEBREAKER
>
> Match allowed lateness to how late data really arrives, and to how much the late part matters.

Four kinds of trigger

TRIGGER	FIRES WHEN	EXAMPLE
Event time	The watermark passes a point	AfterWatermark: the window closes
Processing time	Wall-clock time passes	AfterProcessingTime: every minute
Data-driven	Enough elements arrive	AfterCount: every 1,000 readings
Composite	Rules combine other triggers	Repeatedly, AfterEach, AfterAny, AfterAll

REMEMBER

The default is a single event-time firing when the watermark passes the end of the window.

One window, several results

01

Early panes

Provisional totals every minute, while the half hour runs.

02

On-time pane

When the watermark passes the end of the window.

03

Late panes

Each time a straggler arrives, within the allowed lateness.

04

Expiry

Window end plus lateness: state is dropped.

05

Too late

Anything later is dropped, or caught by a batch fix.

HOW IT WORKS

Each result is called a pane. Downstream systems must expect more than one per window.

Accumulating or discarding panes?

ACCUMULATING

Each pane is the full total so far

- Late pane: the new, complete total
- Downstream overwrites the row
- Right for dashboards and upserts
- Simple to read

DISCARDING

Each pane is only what is new

- Late pane: just the extra readings
- Downstream must add panes up
- Right for append-and-sum sinks
- Overwriting would lose data

THE TIEBREAKER

Overwriting sink: accumulate. Adding sink: discard. Mixing them up corrupts the totals.

Westgrid's half-hour demand

```
demand = (by_region
    | "Half hours" >> beam.WindowInto(
        window.FixedWindows(30 * 60),
        trigger=trigger.AfterWatermark(
            early=trigger.AfterProcessingTime(60),
            late=trigger.AfterCount(1)),
        allowed_lateness=72 * 60 * 60,
        accumulation_mode=trigger.AccumulationMode.ACCUMULATING)
    | "Sum" >> beam.CombinePerKey(sum))
```

4-6 A provisional total each minute, a final at close, an update per straggler.

7-8 Wait 72 hours for late meters. Each pane is the full total.

HOW IT WORKS

The sink upserts by region and half hour, so each accumulating pane replaces the last.

How late is too late?

- Regions are few, so 72 hours of state for **region totals** is cheap.
- Per-meter windows mean **3.1 million keys** held open for 72 hours. Expensive.
- Option: short lateness in streaming, then a **nightly batch recompute** in BigQuery.
- Fast and nearly right now, exactly right by morning.

> THE TIEBREAKER
>
> Only 0.3% of readings are a day late. Paying streaming state for them may not be worth it.

Watermark trouble

- **Data freshness**: how far behind event time the pipeline is. Rising means a lagging watermark.
- **System latency**: how long an item has waited or been processing.
- Common causes: an **idle source**, a **stuck step**, or a sink that will not accept writes.
- Results never appear? Check the watermark before adding workers.

> THE SIGNAL
>
> Windows wait for the watermark. A stuck watermark looks like a pipeline that produces nothing.

Phrases that pick the answer

THE QUESTION SAYS	REACH FOR
Late readings **missing** from totals	Allowed lateness, with late firings
Provisional results before the window closes	Early firings on processing time
Sink **overwrites** each window's row	Accumulating panes
Sink **adds up** every pane	Discarding panes
Windows use **publish time**, not reading time	A timestamp attribute on messages
Data freshness keeps rising	A lagging watermark: idle source or stuck step

THE PATTERN

Watermark says when. Lateness says how long. Trigger says how often. Accumulation says what each result means.

> Larkspur's half-hour region totals use fixed windows on event time, with the default trigger. Readings that arrive two or three hours late never appear in any total.
>
> The totals must eventually include every reading that arrives within three days.

What should the team change?

A Switch to processing-time windows, so late readings land somewhere

B Add more workers, so that the pipeline keeps up with the backlog

C Allow 72 hours of lateness, with a late firing for each straggler

D Use session windows per region, so that windows stay open longer

◆ PAUSE AND DECIDE

What should the team change?

A Processing-time windows

Late readings would land, but in the wrong half hour.

B More workers

The readings are late at the meter. Workers cannot make them earlier.

C 72 hours of lateness

Windows stay open for the stragglers, and each one updates the total.

D Session windows

Regions never go quiet, so sessions would never close.

THE SIGNAL

'Late readings never appear' with the default trigger means zero allowed lateness. Add lateness and late firings.

After adding late firings, Westgrid's dashboard sometimes shows a half-hour total collapse to a tiny number when a straggler arrives.

The sink overwrites the row for each region and half hour with whatever the latest pane says. The pipeline uses discarding mode.

What should the team change?

A Turn off late firings, so that only the on-time pane is written

B Increase the allowed lateness, so stragglers arrive inside it

C Make the sink append every pane as a new row in the table

D Switch the pipeline's accumulation mode to accumulating

◆ PAUSE AND DECIDE

What should the team change?

A **No late firings**

The dashboard stops collapsing, but late readings vanish again.

B **More lateness**

The stragglers are already inside it. That is why panes fire.

C **Append every pane**

Correct only if every reader sums the rows. The dashboard reads one.

D **Accumulating mode**

Each late pane carries the full new total, so overwriting is safe.

THE SIGNAL

An overwriting sink plus discarding panes loses data. Overwrite means accumulate.

> Larkspur's control room wants a provisional demand figure for the current half hour every minute, while it is still running.
>
> Westgrid still needs one settled figure as soon as each half hour closes.

How should the trigger be set?

A Sliding windows of 30 minutes, starting every minute

B Fixed 1-minute windows, summed by the control room's dashboard

C Early firings every minute, inside the watermark trigger

D A data-driven trigger that fires after every 1,000 readings

◆ PAUSE AND DECIDE

How should the trigger be set?

A Sliding windows

Rolling half hours, not the current settlement period.

B 1-minute windows

Pushes the summing onto the dashboard, and loses the settled figure.

C Early firings

Provisional totals every minute, then the on-time figure at close.

D Every 1,000 readings

Fires by volume, so quiet minutes give no update at all.

THE SIGNAL

'Provisional, every minute' plus 'settled at close' means early firings inside AfterWatermark.

> Larkspur's head-end collects readings and publishes them to Pub/Sub in batches, sometimes hours after the meters took them. Each message body holds reading_ts.
>
> The half-hour windows behave as if readings happened when the head-end published them.

What fixes the windows?

A Increase the allowed lateness, so that the batches still count

B Use processing-time triggers, so that the windows close on time

C Publish with ordering keys, so the batches arrive in order

D Put reading_ts in an attribute, and read it as the timestamp

◆ PAUSE AND DECIDE

What fixes the windows?

A More lateness

The readings are not late. They are being stamped with the wrong time.

B Processing-time triggers

Changes when results fire, not which window a reading joins.

C Ordering keys

Order does not change the timestamp each message carries.

D A timestamp attribute

Messages carry event time, so windows and the watermark follow it.

THE SIGNAL

Windows following publish time means the source has no event timestamp. Give it one.

A per-meter streaming pipeline keeps 3.1 million meters' windows open for 72 hours of allowed lateness. Its state, and its bill, keep growing.

Only 0.3% of readings arrive more than a day late. Billing is recalculated from BigQuery every night anyway.

What is the best change?

A Shorten the lateness, letting the nightly batch catch the rest

B Keep 72 hours of lateness, and move to larger worker machines

C Set the allowed lateness to zero, and ignore all late readings

D Move every meter into the global window, with no triggers at all

◆ PAUSE AND DECIDE

What is the best change?

A Short lateness, nightly fix

Streaming covers most late data cheaply; the batch catches the rest.

B Larger machines

Pays more to hold state for a tiny fraction of readings.

C Zero lateness

Drops the 4% that arrive after an hour, every day.

D Global, no triggers

A stream in the global window with no trigger never emits.

THE SIGNAL

Expensive state for a tiny late tail, plus an existing nightly recompute: shorten the lateness.

A Larkspur pipeline reads two Pub/Sub subscriptions: live readings, and a test feed that has been silent for a week.

Its data freshness rises steadily, and its windowed results have stopped appearing, though readings keep arriving.

What should the team do?

A Add more workers, so the pipeline can catch up with the readings

B Remove the silent test feed, or move it into a separate pipeline

C Raise the allowed lateness, so windows wait longer for the data

D Switch the job to at-least-once mode, to reduce its latency

◆ PAUSE AND DECIDE

What should the team do?

A More workers

Workers are not the bottleneck. The watermark is.

B Remove the idle feed

The idle source stops holding the watermark back, and windows close.

C More lateness

Windows would wait even longer for a watermark that is not moving.

D At-least-once mode

Changes duplicates and cost, not a watermark stuck by an idle source.

THE SIGNAL

Rising freshness, no results, and an idle source: the idle source is holding the watermark.

Module 14 in five lines

- The **watermark** guesses when a window is complete. Late data is dropped by default.
- **Allowed lateness** keeps windows open for stragglers, at the cost of state.
- Triggers: **early** panes on processing time, **on-time** at the watermark, **late** per straggler.
- Overwriting sink: **accumulating**. Adding sink: **discarding**.
- Stamp messages with **event time**. An idle source can freeze the watermark.

COMING UP

Next: module 15, Spark and Hadoop on Google Cloud.

MODULE 15 · SECTION 2 - INGESTING AND PROCESSING THE DATA

Spark and Hadoop on Google Cloud

Managed Service for Apache Spark: clusters or serverless, Cloud Storage instead of HDFS, workers, autoscaling and metastores.

SECTION 2 of 5 EXAM WEIGHT 25% MODULE 15 of 45

admin@sthithapragna.com

Stop treating a cluster as a building you own. Treat it as **a taxi you call for one journey**.

On-premises Hadoop, or the Google way?

ON-PREMISES

One big cluster, always on

- Data lives in HDFS on the cluster
- Sized for the busiest night
- Shared by every team
- Idle most of the day, still paid for

ON GOOGLE CLOUD

Storage apart, compute on demand

- Data lives in Cloud Storage
- Clusters sized for each job
- One cluster per job or team
- Deleted when the work is done

THE POINT

Separating storage from compute is what makes short-lived clusters possible.

Dataproc has a new name

- Since April 2026 it is **Managed Service for Apache Spark**.
- It covers the old **Dataproc** clusters and **Serverless for Apache Spark**.
- The **CLI still says gcloud dataproc**, and APIs and roles did not change.
- Clusters start, scale and shut down in **about 90 seconds** on average.

> REMEMBER
>
> The exam guide says Dataproc. Answer with the name the question uses.

Serverless or cluster?

	SERVERLESS	CLUSTER
What you manage	Nothing but the job	Machines, software, sizing
Startup	About 50 seconds	About 120 seconds
You pay for	Job run time only	Cluster up time
Runs	Spark batches and sessions	Spark, Hive, Flink, Trino, Kafka
Control	Custom containers	Machine types, SSH, init actions

THE TIEBREAKER

Spark only, intermittent, no ops team: serverless. Hive or deep control: a cluster.

Cloud Storage instead of HDFS

- Jobs read **gs://** paths directly. The connector is **preinstalled**.
- Data **survives the cluster**, so clusters can be deleted freely.
- Spark, Hive, BigQuery and Dataflow all read the **same files**.
- HDFS ties data to machines. Keep it only for **scratch space**, if at all.

> THE TRAP
>
> Lift-and-shift answers that keep data in on-cluster HDFS are usually the trap.

Call a cluster, use it, delete it

- **Job-scoped clusters**: create, run, delete. Sized for that one job.
- **Scheduled deletion**: after 5 minutes to 14 days idle, at a set time, or at a maximum age.
- Orchestrate with **workflow templates** or **Managed Airflow**.
- Idle cluster? **Delete it**. Recreating is as fast as resizing.

> THE SIGNAL
>
> 'Cluster idle most of the day' points to job-scoped clusters or serverless, not autoscaling down.

Primary or secondary workers?

	PRIMARY WORKERS	SECONDARY WORKERS
Role	Compute, plus HDFS DataNodes	Compute only, no HDFS
Type	Standard VMs	Preemptible by default, or Spot
Cost	Full price	Much cheaper
Risk	Stable	Can be reclaimed at any time
Guidance	The steady core	Keep preemptible under half of all workers

REMEMBER

Spot VMs have no 24-hour limit. Expect transient task failures, and raise retries.

Autoscaling policies, and when not to use them

- Scales **workers only**, never masters, from **YARN pending memory**.
- Set the **graceful decommission timeout** longer than the longest job.
- Not for **on-cluster HDFS**, not for **Spark Structured Streaming**, not for idle clusters.
- **Enhanced Flexibility Mode**: scale only secondary workers, decommission timeout 0s.

> THE TRAP
>
> Streaming Spark plus autoscaling is a trap. Google says they are not compatible.

Init actions and a shared metastore

- **Initialization actions**: scripts from Cloud Storage, run as root on every node.
- They run at creation, with a **10-minute** default timeout. Copy public samples to your own bucket.
- **Custom images** bake the software in, instead of installing it each time.
- Short-lived clusters need a **shared metastore**, or table definitions vanish with them.

THE OPTIONS

Dataproc Metastore is a managed Hive metastore. The Lakehouse runtime catalog also offers a Hive endpoint.

Larkspur's nightly settlement cluster

```
gcloud dataproc clusters create lk-settle-nightly \
    --region=europe-west2 \
    --num-workers=4 --num-secondary-workers=3 \
    --secondary-worker-type=spot \
    --autoscaling-policy=lk-secondary-only \
    --dataproc-metastore=projects/lk-data-prod/locations/europe-west2/services/lk-hms \
    --max-idle=30m

# A small, irregular job: no cluster at all
gcloud dataproc batches submit pyspark gs://lk-jobs-prod/tariff_check.py \
    --region=europe-west2
```

3-4 Spot secondary workers, kept under half the total.

6-7 Tables live in a shared metastore. The cluster deletes itself when idle.

REMEMBER

The command line still says dataproc, even though the product has a new name.

Spark and BigQuery together

- The **BigQuery connector** reads tables through the **Storage Read API**.
- Writes are staged in a **Cloud Storage bucket** during the job.
- Serverless Spark gets BigQuery read costs **included**.
- Spark also runs inside **BigQuery Studio notebooks**, on serverless sessions.

> THE SIGNAL
>
> Spark should read BigQuery through the connector, not by exporting tables to files first.

Lift and shift, then modernise

01
Move the data
History into Cloud Storage, compacted. Module 07.

02
Lift and shift
The same Spark and Hive jobs, on clusters. Few code changes.

03
Make it ephemeral
Job-scoped clusters, shared metastore, Spot workers.

04
Go serverless
Small and irregular Spark jobs as batches.

05
Modernise
Hive SQL into BigQuery, tables into Iceberg.

THE PATTERN

Lift and shift first to meet the deadline. Modernise once nothing is at risk.

Phrases that pick the answer

THE QUESTION SAYS	REACH FOR
Idle most of the day	Job-scoped clusters, or serverless
Data must survive the cluster	Cloud Storage, not HDFS
Cut cost on batch workers	Spot secondary workers, under half
Table definitions **lost** between clusters	A shared metastore
Spark only, no ops team, irregular	Serverless batches
Hive, Trino or Flink alongside Spark	A cluster

THE PATTERN

Data in Cloud Storage, metadata in a metastore, compute only while it is needed.

> After the lift and shift, Larkspur runs a 40-node cluster around the clock. It is busy for about six hours each night and nearly idle the rest of the day.
>
> All data and table definitions already live outside the cluster.

How should the team cut the cost?

A Keep the 40-node cluster running, and buy committed use discounts

B Autoscale the always-on cluster down to two workers in the daytime

C Move the data into HDFS on persistent disks for faster nightly runs

D Create a cluster for each nightly run, and delete it afterwards

◆ PAUSE AND DECIDE

How should the team cut the cost?

A **Committed use discounts**

A discount on eighteen idle hours is still eighteen idle hours.

B **Autoscale down by day**

Google advises deleting idle clusters, not shrinking them.

C **Data into HDFS**

Ties data to machines, which stops the cluster being deleted.

D **A cluster per run**

Pay for six busy hours, and nothing for the idle day.

THE SIGNAL

Idle most of the day, with data and metadata outside: job-scoped clusters.

> Larkspur now creates a fresh cluster for each settlement job. Each new cluster starts with an empty Hive metastore.
>
> Jobs fail because the tables they expect are missing, and analysts cannot find tables created by earlier runs.

What should the team do?

A. Back up each cluster's metastore database, and restore it on the next

B. Attach every cluster to one shared Dataproc Metastore service

C. Add a script to every job that recreates all of the tables first

D. Keep one small permanent cluster that exists only to host the metastore

◆ PAUSE AND DECIDE

What should the team do?

A Back up and restore

Fragile, slow, and two overlapping clusters would clash.

B A shared metastore

Every cluster sees the same tables, and they outlive each one.

C Recreate tables per job

Hundreds of definitions duplicated in every job, and drift.

D A permanent cluster

An always-on server just for metadata, which a managed service replaces.

THE SIGNAL

Short-lived clusters losing table definitions need a shared, managed metastore.

> To save money, a Larkspur cluster now runs 2 primary workers and 10 Spot secondary workers.
>
> Jobs have become slow and unpredictable, with many failed tasks whenever Spot capacity is reclaimed.

What should the team change?

A. Keep Spot workers under half of the total, and turn on EFM

B. Make every worker Spot, so that the whole cluster is cheaper

C. Store shuffle data in HDFS on the secondary workers instead

D. Raise the graceful decommission timeout to a full 24 hours

◆ PAUSE AND DECIDE

What should the team change?

A Under half, plus EFM

Google's guidance: a stable core, and a mode built for preemption.

B All Spot

More reclaimed machines, more failures, and no stable core.

C HDFS on secondaries

Secondary workers store no HDFS data at all.

D A 24-hour timeout

Decommission timeouts do not stop Spot capacity being reclaimed.

THE SIGNAL

More than half preemptible, with failures on reclaim: rebalance, and use Enhanced Flexibility Mode.

> Larkspur's data scientists have written a new PySpark job that scores tariff changes. It runs a few times a week, at unpredictable times.
>
> It uses only Spark, and the team has nobody to manage infrastructure.

Where should the job run?

A. A long-running shared cluster, with an autoscaling policy attached

B. A Spark cluster on GKE, run by the central platform team

C. A serverless batch workload, submitted whenever it is needed

D. Rewrite the job in Apache Beam, and run it on Dataflow

◆ PAUSE AND DECIDE

Where should the job run?

A A shared cluster

Always on for a job that runs a few times a week.

B Spark on GKE

Infrastructure for another team, when nobody has time for it.

C A serverless batch

Spark only, irregular, no infrastructure: pay just for each run.

D Rewrite in Beam

A rewrite of working Spark code for no gain.

THE SIGNAL

New, Spark only, irregular, no ops team: serverless batches.

Module 15 in five lines

- Dataproc is now **Managed Service for Apache Spark**. The CLI still says dataproc.
- Data in **Cloud Storage**, not HDFS, so clusters can be **short-lived**.
- **Serverless** for Spark-only, irregular work. **Clusters** for Hive, control, migrations.
- **Spot secondary workers** under half. No autoscaling for HDFS or Structured Streaming.
- Ephemeral clusters need a **shared metastore**. Delete idle clusters.

COMING UP

Next: module 16, getting data into BigQuery, including CDC.

MODULE 16 · SECTION 2 - INGESTING AND PROCESSING THE DATA

Getting Data into BigQuery

Load jobs, the Storage Write API, subscriptions, change data capture, external tables, and deduplication.

SECTION 2 of 5 EXAM WEIGHT 25% MODULE 16 of 45

admin@sthithapragna.com

Batch is free and patient. Streaming is fast and billed. **External tables do not move the data at all.**

Six ways into BigQuery

METHOD	BEST FOR	COST AND FRESHNESS
Load jobs	Batches of files	Free in the shared pool
Storage Write API	Streaming from code or Dataflow	Seconds; 2 TiB a month free
Pub/Sub subscription	Messages straight to a table	Seconds; no code
Datastream	Database changes	Minutes; managed CDC
Transfers, scheduled queries	SaaS, warehouses, recurring SQL	On a schedule
External tables	Files queried where they sit	No load; slower queries

THE TIEBREAKER

Queried many times a day: load it. Queried rarely: it can stay outside.

Load jobs: the free, patient door

- Avro, Parquet, ORC, CSV and JSON. **Free** in the shared slot pool.
- **Autodetect** infers a schema, but misses headers when every column is text.
- **max_bad_records** defaults to **0**: one bad row fails the load.
- Gzip CSV cannot be read in parallel, and is limited to 4 GB per file. Append or overwrite.

> THE TIP
>
> Self-describing formats like Avro and Parquet carry their own schema, so autodetect is not needed.

Storage Write API or legacy streaming?

STORAGE WRITE API, GRPC

The recommended way

- Exactly-once with committed streams
- Atomic batch commits with pending streams
- USD 0.025 per GiB, 2 TiB a month free
- Google's choice for new projects

LEGACY STREAMING, REST

Formerly insertAll

- Now called Storage Write API (REST)
- insertId: best-effort dedupe, one minute
- USD 0.01 per 200 MiB, 1 KB minimum per row
- Still supported, not recommended

CURRENCY CHECK

'Streaming inserts' in older questions means insertAll. The modern answer is the Storage Write API.

Four kinds of write stream

STREAM	SEMANTICS	USE IT FOR
`Default`	At least once, visible at once	High-throughput streaming, duplicates tolerated
`Committed, with offsets`	Exactly once, visible at once	Streams where every row must land once
`Pending`	Invisible until an atomic commit	Batch loads that must land all or nothing
`Buffered`	Held until flushed	Advanced cases, mostly inside Beam

THE SIGNAL

'No duplicates, written from our own code' means committed streams with offsets.

Streaming without writing the client

- **Pub/Sub BigQuery subscription**: messages straight into a table, no code. At least once.
- **Dataflow BigQueryIO** uses the Storage Write API, when messages need processing first.
- **Continuous queries** stream results out of BigQuery, into Pub/Sub, Bigtable or tables.
- All of them land rows in seconds. They differ in where the logic lives.

> THE TIEBREAKER
>
> No transformation: subscription. Transformation or joins: Dataflow. Logic in SQL on data already in BigQuery: continuous queries.

Three ways to apply changes

- **Datastream** from a database: managed CDC, merge or append-only. Module 08.
- **CDC through the Storage Write API**: tag rows UPSERT or DELETE, into a table with a primary key.
- Primary keys are **not enforced**, and active CDC tables cannot be changed with DML.
- Or land changes **append-only**, then apply them with a scheduled **MERGE**.

> REMEMBER
>
> max_staleness trades freshness for cost. Background apply jobs use a BACKGROUND reservation, or on-demand.

The classic MERGE, with deduplication

```sql
MERGE curated.accounts AS t
USING (
  SELECT * FROM raw.account_changes
  WHERE change_ts > @last_run
  QUALIFY ROW_NUMBER() OVER (
    PARTITION BY account_id ORDER BY change_ts DESC) = 1
) AS s
ON t.account_id = s.account_id
WHEN MATCHED AND s.op = 'D' THEN DELETE
WHEN MATCHED THEN UPDATE SET tariff_id = s.tariff_id, iban_token = s.iban_token
WHEN NOT MATCHED AND s.op != 'D' THEN
  INSERT (account_id, customer_id, tariff_id, iban_token)
  VALUES (s.account_id, s.customer_id, s.tariff_id, s.iban_token)
```

5-6 Keep only the latest change per account, or MERGE meets two matches.

9-10 Deletes remove the row. Updates overwrite it.

THE PATTERN

Rerunning this MERGE gives the same result. Idempotent loads make retries safe.

External tables, or load the data?

QUERY IN PLACE

External and Lakehouse tables

- No load step, files stay put
- Always as fresh as the files
- Slower, with fewer features
- Federated queries reach Cloud SQL, Spanner, AlloyDB

LOAD

Native BigQuery tables

- A load or stream step
- Fresh as of the last load
- Fastest queries, every feature
- Partitioning, clustering, caching

THE TIEBREAKER

'Query occasionally, keep one copy' points to external. 'Dashboards hit it all day' points to loading.

Data Transfer Service and scheduled queries

- **Data Transfer Service**: scheduled loads from SaaS apps, warehouses, S3, Azure and Cloud Storage.
- **Scheduled queries**: one SQL statement on a timetable, writing a destination table.
- Need steps in order, tests and version control? That is **Dataform**.
- Need other services in the chain? That is **Managed Airflow** or Workflows. Module 35.

> THE TIEBREAKER
>
> One query, no dependencies: a scheduled query. A graph of SQL: Dataform.

Deduplicating Larkspur's readings

- Meters **resend** batches, so raw readings contain exact and near duplicates.
- Keep one row per **meter_id** and **reading_ts**, the latest by **received_ts**.
- **SELECT DISTINCT** fails, because each copy has a different received time.
- Neither insertId nor exactly-once delivery covers resends from the source.

> THE PATTERN
>
> Deduplicate on the business key, in a step you can rerun. Never rely on the transport.

Phrases that pick the answer

THE QUESTION SAYS	REACH FOR
Nightly files, cheapest	Load jobs
Exactly once from our own code	Storage Write API, committed stream
Streaming inserts, insertAll	Legacy; prefer the Storage Write API
Upserts and deletes, no MERGE jobs	CDC with UPSERT and DELETE
Queried **all day**, stored as files	Load into native tables
One latest row per key	QUALIFY ROW_NUMBER, then MERGE

THE PATTERN

Freshness decides the door. Query frequency decides whether to load at all.

> Every night, Larkspur's billing system exports about 200 GB of Parquet files to Cloud Storage. Analysts query the results all the next day.
>
> Finance wants the cheapest reliable way to get them into BigQuery by 06:00.

How should the files be ingested?

A. A load job from Cloud Storage into a native table, after each export

B. Legacy streaming inserts, sending every row as soon as it is read

C. A Dataflow pipeline that streams the rows in with the Storage Write API

D. External tables over the files, queried directly by every analyst

◆ PAUSE AND DECIDE

How should the files be ingested?

A A load job

Free in the shared pool, batch-shaped, and the native table is fast all day.

B Legacy streaming

Billed per row, for data that arrives once a night anyway.

C Dataflow streaming

A pipeline and per-GiB charges, for a nightly batch.

D External tables

Free to set up, but slow and costly when queried all day.

THE SIGNAL

Nightly files, cheapest, queried all day: a load job into a native table.

> Larkspur's payment service is written in Java and writes each payment event straight into BigQuery.
>
> After network errors it retries, and finance has found duplicate payments in the table. Every payment must land exactly once.

How should the service write to BigQuery?

A Legacy streaming inserts, with an insertId on every row it sends

B The Storage Write API, using a committed stream with offsets

C The Storage Write API default stream, retrying any failed append

D A load job every minute, from files written by the service

◆ PAUSE AND DECIDE

How should the service write to BigQuery?

A insertId

Best-effort dedupe for about a minute. Google says not to rely on it.

B Committed stream, offsets

A repeated offset is rejected, so a retry cannot land twice.

C Default stream

At least once by design. Retries can still duplicate.

D A load per minute

A file pipeline, with load-job limits, to fix a write problem.

THE SIGNAL

'Exactly once' from your own code: a committed stream with offsets.

A third-party pricing engine at Larkspur emits a change event whenever a tariff is created, changed or withdrawn. It has no database that Larkspur can read.

A BigQuery table must mirror the current tariffs within minutes, and the team does not want to run MERGE jobs.

How should the changes be applied?

A Append every event to a table, and run MERGE every five minutes

B Use legacy streaming inserts, with the tariff ID as the insertId

C Stream rows tagged UPSERT or DELETE into a table with a primary key

D Use Datastream to replicate the pricing engine straight into BigQuery

◆ PAUSE AND DECIDE

How should the changes be applied?

A **Append and MERGE**

Works, but it is exactly the MERGE job the team ruled out.

B **insertId as the key**

insertId only drops duplicates briefly. It never updates a row.

C **UPSERT and DELETE**

BigQuery applies the changes itself, keyed on the primary key.

D **Datastream**

Datastream reads databases. This engine has none to read.

THE SIGNAL

Change events, no database, no MERGE jobs: CDC through the Storage Write API.

> Westgrid's daily settlement files sit in Cloud Storage as CSV, behind an external table. Larkspur's dashboards query them dozens of times an hour.
>
> The queries are slow and expensive, and a file never changes once it has arrived.

What should the team do?

A. Keep the external table, and buy more slots for the dashboards

B. Use federated queries with EXTERNAL_QUERY to read the files

C. Stream every row in again each hour, using the REST streaming API

D. Load each file into a native, partitioned table when it arrives

◆ PAUSE AND DECIDE

What should the team do?

A More slots

Pays more to scan the same CSV files again and again.

B EXTERNAL_QUERY

That reaches Cloud SQL, Spanner and AlloyDB, not files.

C Re-stream every hour

Pays to ingest the same rows repeatedly, and creates duplicates.

D Load once, natively

Files never change, so one free load gives fast, cheap queries.

THE SIGNAL

Queried all day, never changing: load once into native, partitioned tables.

> Larkspur's raw.meter_readings contains resent readings. Copies share meter_id and reading_ts, but each has its own received_ts.
>
> curated.readings_hh must hold exactly one row per meter per half hour, keeping the copy received last.

Which query logic is right?

A QUALIFY ROW_NUMBER, partitioned by meter and time, by received_ts

B SELECT DISTINCT across every column of the raw readings table

C Rely on the insertId set by the pipeline when the rows were written

D GROUP BY meter_id, keeping the maximum kwh value for each meter

◆ PAUSE AND DECIDE

Which query logic is right?

A QUALIFY ROW_NUMBER

One row per business key, and the latest copy wins.

B SELECT DISTINCT

The copies differ in received_ts, so none are removed.

C insertId

The meter sent the reading twice. The transport cannot know that.

D GROUP BY meter_id

Collapses every half hour into one row per meter.

THE SIGNAL

One row per business key, latest copy wins: ROW_NUMBER over the key, ordered by received time.

> Every morning, Larkspur's finance team needs a summary table rebuilt from curated.bills. It is one SQL statement, with no other steps and no dependencies.
>
> The team wants the simplest option with nothing to deploy.

What should run the statement?

A. A Dataflow batch pipeline that rebuilds the table every morning

B. A scheduled query that writes the result to a destination table

C. A Cloud Run function on a cron schedule that runs the bq tool

D. A Datastream stream from curated.bills into the summary table

◆ PAUSE AND DECIDE

What should run the statement?

A. A Dataflow pipeline

A whole pipeline for one SQL statement.

B. A scheduled query

One statement, a timetable, a destination table, nothing to deploy.

C. A Cloud Run function

Code and a scheduler to maintain, for one query.

D. Datastream

Datastream reads operational databases, not BigQuery tables.

THE SIGNAL

One statement, no dependencies, nothing to deploy: a scheduled query.

Module 16 in five lines

- **Load jobs** are free in the shared pool. Use them for batches of files.
- **Storage Write API** for streaming. Committed stream with offsets for exactly once.
- Legacy streaming is now **Storage Write API (REST)**. insertId is best effort only.
- Changes: **Datastream**, **UPSERT and DELETE**, or append then **MERGE**.
- Deduplicate on the **business key** with ROW_NUMBER. Load data that is queried all day.

COMING UP

Next: module 17, low-code, SQL pipelines and AI enrichment.

MODULE 17 · SECTION 2 - INGESTING AND PROCESSING THE DATA

Low-code, SQL Pipelines and AI Enrichment

Cloud Data Fusion, Dataform, BigQuery data preparation, and enriching data with AI in SQL or in Dataflow.

SECTION 2 of 5 EXAM WEIGHT 25% MODULE 17 of 45

admin@sthithapragna.com

Choose the tool by **who will maintain it**, then by where the data already lives.

admin@sthithapragna.com

Six tools for this module

TOOL	WHO IT SUITS	WHAT IT DOES
Cloud Data Fusion	Integration teams, low-code	Visual pipelines across many sources
Dataform	Analysts and engineers who write SQL	SQL pipelines in BigQuery, in Git
BigQuery data preparation	Analysts, no code	Gemini-suggested cleaning steps
Dataprep by Trifacta	Partner product	Visual cleaning, in older questions
BigQuery AI functions	SQL users	Call Gemini models from SQL
Dataflow Enrichment, RunInference	Pipeline engineers	Lookups and model calls in a stream

THE TIEBREAKER

The first two columns decide most questions. The tools overlap; the people do not.

Visual pipelines, run on Spark

- Built on open-source **CDAP**. Pipelines are drawn as a graph of **plugins**.
- Each run gets a **short-lived Spark cluster**, created and deleted for you.
- **Private instances** through Private Service Connect or VPC peering.
- **CMEK** in every edition, set at creation, covering resources plugins create.

> REMEMBER
>
> The CMEK key must be regional, in the instance's region, and applies only to new resources.

Wrangler: see the data, then clean it

- Interactive cleaning on a **sample**, the first **1,000 records**.
- Each step is a **directive**. The ordered list is a **recipe**.
- **Batch pipelines only**. Wrangler does not stream.
- Turn the recipe into a **pipeline**, or the work is not kept.

> THE SIGNAL
>
> Wrangler is Data Fusion's answer to 'analysts must clean data visually before it loads'.

Dataform: software engineering for SQL

- **SQLX** files. **ref()** links them, so Dataform works out the order.
- Code lives in **Git**: Dataform's own, or GitHub, GitLab, Azure DevOps, Bitbucket.
- **Development workspaces** are private copies. **Compilation** turns SQLX into plain SQL.
- Each repository runs as its own **custom service account**.

> THE SIGNAL
>
> 'Version control, code review, dependency order, tests' in a SQL question means Dataform.

An incremental table for Larkspur's readings

```
1  config {
2    type: "incremental",
3    uniqueKey: ["meter_id", "reading_ts"],
4    protected: true,
5  }
6  SELECT meter_id, reading_ts, kwh, received_ts
7  FROM ${ref("meter_readings")}
8  WHERE kwh IS NOT NULL
9  ${when(incremental(),
10   `AND received_ts > (SELECT MAX(received_ts) FROM ${self()})`)}
11 QUALIFY ROW_NUMBER() OVER (
12   PARTITION BY meter_id, reading_ts ORDER BY received_ts DESC) = 1
```

3-4 Resends merge on the key. A full refresh is blocked.

9-10 Later runs read only rows received since the last run.

THE DETAIL

Filtering on received time still catches meters that upload days late.

Release or workflow configuration?

RELEASE CONFIGURATION

How to compile

- Which branch or commit
- Project, prefixes, suffixes, variables
- One per environment: dev, test, prod
- Produces a compilation result

WORKFLOW CONFIGURATION

When to run

- Runs a release's compilation result
- On a schedule, by tags or actions
- Overlapping runs are skipped
- Or run from Airflow, Workflows, Cloud Build

THE TIEBREAKER

Release says what and where. Workflow says when.

Incremental tables, precisely

- **First run**: the whole query runs, and builds the table.
- **Later runs**: the incremental filter is added, so only new rows are read.
- **uniqueKey** merges matching rows instead of appending them.
- Change the query, and you need a **full refresh**. **protected: true** blocks one.

> THE TRAP
>
> Protect tables whose source no longer holds the full history, or a refresh deletes the past.

BigQuery data preparation, and Dataprep

- **BigQuery data preparation**: Gemini suggests steps, and shows the SQL for each.
- Steps include filter, **deduplicate**, join and **validation**, with failures sent to an error table.
- Write modes: full refresh, append, **incremental**, **upsert**. **Dataform** schedules them.
- **Dataprep by Trifacta** is an Alteryx partner product. Recognise it in older questions.

> THE SIGNAL
>
> 'Analysts, no code, data already in BigQuery' points to BigQuery data preparation.

BigQuery AI functions

FUNCTION	WHAT IT DOES
AI.GENERATE	Any prompt, any data, optionally structured output
AI.GENERATE_TABLE	Extract fields into a schema you define
AI.CLASSIFY	Sort text into categories you list
AI.IF and AI.SCORE	Filter or rank rows with a plain-language condition
AI.EMBED	Turn text or images into embeddings
AI.COUNT_TOKENS	Estimate the input size before you pay for it

THE SIGNAL

Data already in BigQuery and a SQL team: enrich it in place, with no pipeline.

Enrich in SQL, or in the pipeline?

BIGQUERY AI FUNCTIONS

In SQL, over tables

- Data already in BigQuery
- Batch over many rows
- Analysts can own it
- Pay per model call

DATAFLOW

In the stream

- Enrichment: lookups in Bigtable, BigQuery, Feature Store
- RunInference: your own model in the workers
- Throttling and retries built in
- GPUs for heavy models

THE TIEBREAKER

Streaming, per-record, sub-second: Dataflow. A monthly batch of rows in BigQuery: SQL.

Enrich carefully

- Model output is **probabilistic**. Validate a sample against known answers.
- **Count tokens** before a big run. A prompt per row adds up fast.
- Store the **model and prompt version** with every result, so you can rerun.
- Send only the columns the model needs. **Personal data** rules still apply.

> THE TAKEAWAY
>
> Treat AI enrichment like any other transformation: tested, versioned and governed.

Phrases that pick the answer

THE QUESTION SAYS	REACH FOR
Low-code, many sources, integration team	Cloud Data Fusion
Clean **visually** before loading	Wrangler in Data Fusion
SQL with **Git, tests, dependency order**	Dataform
Only **new rows** each run, merge on a key	Dataform incremental table, uniqueKey
Classify or extract from text in BigQuery	BigQuery AI functions
Per-record lookup in a stream	Dataflow Enrichment transform

THE PATTERN

People first, then where the data lives, then how fresh the result must be.

> Larkspur's integration team must bring data from 15 business systems into BigQuery. The team builds pipelines visually and does not write code.
>
> Security requires every table and bucket the pipelines create to use Larkspur's own key in europe-west2.

Which approach fits best here?

A. A Dataflow pipeline in Java for each system, with CMEK flags on each job

B. Dataform SQLX files in Git, reading each system through external tables

C. A Cloud Data Fusion instance in europe-west2, with CMEK set at creation

D. A Cloud Run function per system, writing rows with the Storage Write API

◆ PAUSE AND DECIDE

Which approach fits best here?

A **Dataflow in Java**

Fifteen coded pipelines for a team that does not write code.

B **Dataform**

SQL over BigQuery. It cannot connect to fifteen outside systems.

C **Data Fusion with CMEK**

Visual pipelines, many connectors, and the key covers what plugins create.

D **Cloud Run functions**

Fifteen services to code and run, with no visual designer.

THE SIGNAL

Visual, many sources, no code, and CMEK on created resources: Cloud Data Fusion.

> Larkspur's curated readings table is rebuilt in full every night, scanning eight years of raw data. The raw table now expires partitions after 90 days.
>
> The team wants each run to process only new rows, merge resent readings, and never lose older history.

How should the Dataform table be defined?

A As a view, so that it always reflects the current raw table

B As a table that is rebuilt from scratch with each nightly run

C As a scheduled query that appends every raw row each night

D As an incremental table, with a uniqueKey and protected: true

◆ PAUSE AND DECIDE

How should the Dataform table be defined?

A A view

Every query scans raw, and history older than 90 days vanishes.

B A nightly rebuild

Rebuilt from 90 days of raw, it silently deletes older history.

C Append everything

Re-appends rows each night, so duplicates pile up.

D Incremental, protected

New rows only, resends merged, and no full refresh can erase the past.

THE SIGNAL

Only new rows, merge on a key, and a source that forgets: incremental, with uniqueKey and protected.

> Larkspur has about 2 million field-engineer job notes in a BigQuery table. Each month, analysts want every new note sorted into one of six fault categories.
>
> The analysts know SQL. There are no labelled examples and no machine learning team.

How should they classify the notes?

A Use AI.CLASSIFY in BigQuery, listing the six categories in the query

B Train a custom TensorFlow model, and serve it with Dataflow RunInference

C Write regular-expression directives for each category in Wrangler

D Use the Dataflow Enrichment transform with a Bigtable lookup table

◆ PAUSE AND DECIDE

How should they classify the notes?

A AI.CLASSIFY in SQL

The data is in BigQuery, the team writes SQL, and no training is needed.

B A custom model

Needs labelled data and an ML team, and neither exists.

C Wrangler regexes

Brittle patterns for free text, rewritten whenever wording changes.

D Enrichment lookup

A lookup matches keys. It cannot read and judge free text.

THE SIGNAL

Text in BigQuery, a SQL team, no labels: classify in place with AI functions.

> Each streaming reading must be enriched with the customer's current tariff before it reaches the live pricing view. The tariff table has 3.1 million rows.
>
> Tariffs change constantly as customers switch. The stream peaks at 10,000 readings a second.

How should the pipeline enrich each reading?

A Load all 3.1 million tariff rows as a side input, refreshed every hour

B Use the Dataflow Enrichment transform, with a Bigtable lookup per reading

C Call AI.GENERATE from BigQuery to work out the tariff for each reading

D Have a Cloud Run function query Cloud SQL for each individual reading

◆ PAUSE AND DECIDE

How should the pipeline enrich each reading?

A A big side input

Millions of rows on every worker, and up to an hour out of date.

B Enrichment from Bigtable

Fast key lookups, current data, with throttling and retries built in.

C AI.GENERATE

A model guessing a fact that sits in a table. Slow, costly and wrong.

D Cloud SQL per reading

Ten thousand queries a second on a database built for transactions.

THE SIGNAL

Large, constantly changing lookup, per record, in a stream: the Enrichment transform with Bigtable.

Module 17 in five lines

- **Data Fusion** for visual, many-source pipelines. **Wrangler** cleans a sample, batch only.
- **Dataform**: SQLX in Git. Release says what and where, workflow says when.
- **Incremental tables** read new rows; **uniqueKey** merges; **protected** blocks full refresh.
- **BigQuery AI functions** enrich in SQL. **Dataflow Enrichment** looks up per record.
- Validate, version and govern AI output like any other transformation.

COMING UP

Next: module 18, deploying pipelines with CI/CD, templates and environments.

MODULE 18 · SECTION 2 - INGESTING AND PROCESSING THE DATA

Deploying Pipelines

Source control, Cloud Build, Terraform and Infrastructure Manager, templates as artefacts, DAGs, Dataform, and testing.

SECTION 2 of 5 EXAM WEIGHT 25% MODULE 18 of 45

admin@sthithapragna.com

If it is not in Git, it does not exist. Every environment is built from the same commit.

From commit to production

01
Commit
A change is pushed and reviewed in Git.

02
Test
Unit tests on the Direct Runner, DAG checks, Dataform compile.

03
Build once
A Flex template image, tagged with the commit.

04
Preproduction
Integration and end-to-end tests, on real services.

05
Promote
The same artefact goes to production.

THE RULE

Build once, promote the same artefact. Rebuilding for production means production runs untested code.

Everything has a home

WHAT	WHERE IT LIVES	HOW IT SHIPS
Pipeline code	Git	Cloud Build, into a Flex template
Infrastructure and IAM	Terraform in Git	Infrastructure Manager
Dataflow jobs	Artifact Registry and Cloud Storage	flex-template run
Airflow DAGs	Git	Copied to the environment's dags folder
SQL pipelines	Dataform repository in Git	Release and workflow configurations
Secrets	Secret Manager	Read at run time, never committed

THE TRAP

A key or password in Git is a security incident, even in a private repository.

Where the code lives

- **GitHub, GitLab or Bitbucket**, with branches, reviews and protected main branches.
- **Cloud Source Repositories** closed to new customers on **17 June 2024**.
- Google's hosted option now is **Secure Source Manager**, regional and single tenant.
- Dataform and Cloud Build both connect to these Git hosts directly.

> CURRENCY CHECK
>
> Older questions answer 'store code on Google Cloud' with Cloud Source Repositories. Recognise it, but know it is closed to newcomers.

Cloud Build: the conveyor belt

- A **build config** in YAML lists steps. Each step runs in its **own container**.
- **Triggers** start builds on a push or a pull request.
- **Private pools** reach private networks. Artefacts go to **Artifact Registry**.
- Run each trigger as a **dedicated service account**, with only the roles it needs.

> WORTH KNOWING
>
> Cloud Build meets SLSA level 3 for supply-chain security, which matters for regulated firms.

Testing and packaging the readings pipeline

```
substitutions:
  _REPO: europe-west2-docker.pkg.dev/lk-ci-prod/pipelines
  _SPECS: gs://lk-artifacts-prod/templates
steps:
- id: unit-tests
  name: python:3.11
  entrypoint: bash
  args: ["-c", "pip install -r requirements.txt && python -m pytest tests/"]
- id: build-template
  name: gcr.io/google.com/cloudsdktool/cloud-sdk
  args: ["gcloud", "dataflow", "flex-template", "build",
         "${_SPECS}/readings-${SHORT_SHA}.json",
         "--image-gcr-path=${_REPO}/readings:${SHORT_SHA}",
         "--sdk-language=PYTHON", "--flex-template-base-image=PYTHON3"]
```

8 Unit tests first. A failure stops the build.

12-13 Image and spec are tagged with the commit, so every run is traceable.

THE PATTERN

The template built here is the one preproduction tests and production later runs.

Flex templates as release artefacts

- An **image** in Artifact Registry, plus a **spec file** in Cloud Storage.
- Launch with **flex-template run**, passing parameters for each environment.
- Same template in every project. Only the **parameters** differ.
- The launching identity needs **Service Account User** on the worker account.

> THE RULE
>
> Environment differences belong in parameters and Terraform variables, never in a separate build.

Terraform and Infrastructure Manager

- **Terraform** declares projects, datasets, topics, buckets and IAM, in Git.
- **Infrastructure Manager** runs Terraform for you, as Cloud Build jobs.
- It keeps **state** and **revisions**, and shows a **preview** before applying.
- It deploys **infrastructure**, not applications. Pipelines still ship as templates.

> THE SIGNAL
>
> 'Repeatable environments, reviewed changes, see the impact first' points to Terraform with previews.

Deploying DAGs to Managed Airflow

- DAGs are Python files in the environment's bucket, in the **dags** folder.
- Changes reach Airflow in **a few minutes**. Running tasks finish on the old file.
- Deleting a file does **not** remove the DAG's history from the Airflow UI.
- Keep **separate test and production environments**, and test DAGs in CI first.

> THE TRAP
>
> Never edit DAGs directly in the production bucket. One import error can break scheduling.

Deploying SQL pipelines

- **Release configurations** per environment: production compiles from main.
- **Schema suffixes** and **table prefixes** keep development output away from production.
- **Workflow configurations** schedule each environment's runs.
- Compile in CI, so a broken reference fails before anything runs.

> THE POINT
>
> A developer's run must never overwrite curated.readings_hh. Suffixes make that impossible.

Four levels of pipeline test

LEVEL	RUNS ON	PROVES
Unit	Direct Runner, tiny data	Each DoFn and transform is correct
Transform integration	Direct Runner, Create or TestStream	Transforms work together, including late data
System integration	Real services or emulators	Connections, schemas and permissions
End to end	Dataflow, 1% of data, then full	Scale, cost and production-like behaviour

THE TIP

For streaming, test with a second subscription on the same topic, beside production.

Separate projects, same code

SEPARATE PROJECTS

One per environment

- A sandbox per developer
- Preproduction close to production
- Production locked down
- Quotas and IAM kept apart

SAME EVERYTHING ELSE

One Terraform, one artefact

- Same modules, different variables
- Same Flex template, different parameters
- Same Dataform code, different suffixes
- Promote; never rebuild

FOR LARKSPUR

Larkspur's lk-ingest-dev, lk-ingest-preprod and lk-ingest-prod differ only in variables.

Phrases that pick the answer

THE QUESTION SAYS	REACH FOR
Same artefact in every environment	Build one Flex template, promote it
Infrastructure reviewed and repeatable	Terraform with Infrastructure Manager
Test a stream on **real traffic**, safely	A second subscription, in preproduction
DAGs **broke scheduling**	Git, CI tests, automated copy to the bucket
Dev runs **overwrote** production tables	Dataform release configs with suffixes
Store code on Google Cloud, new team	Secure Source Manager, or an external Git host

THE PATTERN

Nothing by hand, everything from Git, one build promoted through every environment.

> Larkspur's team builds the readings pipeline from source in the test project, and again from source when it deploys to production.
>
> Last month the production job behaved differently from the one that passed testing, because a dependency had changed in between.

What should the team change?

A Rebuild for production from the main branch, after every test passes

B Build one Flex template per commit, and promote that same artefact

C Let a senior engineer launch production jobs from their own laptop

D Switch to a classic template, rebuilt separately for each project

◆ PAUSE AND DECIDE

What should the team change?

A **Rebuild from main**

Still a second build, so a dependency can still change between them.

B **One template, promoted**

Production runs exactly the image that preproduction tested.

C **Launch from a laptop**

Unreviewed, unrepeatable, and tied to one person's machine.

D **Classic, rebuilt per project**

Two builds again, and Google recommends Flex for new work.

THE SIGNAL

Tested and production builds differ: build once, and promote the artefact.

> Larkspur wants to test a new version of the streaming readings pipeline against real production traffic for a week.
>
> The production pipeline must not lose or delay any messages during the test.

How should the test be run?

A. Add a second subscription, and run the new version in preproduction

B. Point the new version at the production subscription, beside the old one

C. Rely on Direct Runner unit tests, using a sample of production messages

D. Run the new version on production, then seek the subscription back

◆ PAUSE AND DECIDE

How should the test be run?

A A second subscription

Each subscription gets every message, so production is untouched.

B Share the subscription

Two pipelines would split the messages, and production misses half.

C Unit tests only

Useful, but they never meet real volume, timing or late data.

D Seek back afterwards

Production runs untested code first, and replay creates duplicates.

THE SIGNAL

Real traffic without touching production: a second subscription on the same topic.

> Larkspur's datasets, Pub/Sub topics and IAM bindings were all created by hand in the console. Development and production have quietly drifted apart.
>
> The security team now wants every change reviewed, and wants to see a change's impact before it is applied.

What should the platform team adopt?

A. A wiki page listing the console steps for building each environment

B. Shell scripts of gcloud commands, run by whoever needs a change to be made

C. A fresh copy of the production project, taken each time dev is needed

D. Terraform in Git, applied through Infrastructure Manager with previews

◆ PAUSE AND DECIDE

What should the platform team adopt?

A A wiki of steps

Still manual, so environments keep drifting.

B gcloud scripts

No state, no preview, and no record of what is deployed.

C Copy production

Copies production data and access into a looser environment.

D Terraform and Infra Manager

Reviewed in Git, previewed first, with state and revisions kept.

THE SIGNAL

Drift, reviews and previews: infrastructure as code, applied by a managed service.

> Larkspur's engineers edit Airflow DAGs directly in the production environment's bucket. Last week a DAG with an import error disrupted scheduling for an hour.
>
> The team wants reviews and automatic checks before any DAG reaches production.

What should the team do?

A Restrict bucket edits to two senior engineers, who check every change by eye

B Leave DAGs in the bucket, and watch the Airflow UI for import errors

C Keep DAGs in Git, test them in CI, and let Cloud Build copy them across

D Write DAGs straight into the Airflow metadata database instead

◆ PAUSE AND DECIDE

What should the team do?

A Two careful engineers

Fewer people, same risk, and still no review or tests.

B Watch for errors

Finds the break after it has happened, not before.

C Git, CI, Cloud Build

Reviewed, tested for import errors, then copied by automation.

D The metadata database

DAGs are files in the bucket. That database is not for code.

THE SIGNAL

Broken DAGs in production plus a wish for reviews: Git, CI checks, automated deployment.

Module 18 in five lines

- Everything lives in **Git**. Secrets live in **Secret Manager**.
- **Build once** with Cloud Build, then promote the same Flex template.
- **Terraform** through **Infrastructure Manager**, with previews and state.
- DAGs tested in CI, then copied to the bucket. Dataform suffixes protect production.
- Cloud Source Repositories is **closed to new customers**. Test streams on a second subscription.

COMING UP

Next: section 3 begins, with module 19, choosing a storage service.

MODULE 19 · SECTION 3 - STORING THE DATA

Choosing a Storage Service

The decision by access pattern, and the one phrase in each question that names the store.

SECTION 3 of 5 EXAM WEIGHT 20% MODULE 19 of 45

admin@sthithapragna.com

Choose storage by **how the data will be read**, not by how big it is.

Eight access patterns, eight stores

ACCESS PATTERN	STORE	THE PHRASE THAT DECIDES
Regional transactions	Cloud SQL	MySQL, PostgreSQL or SQL Server, lift and shift
Transactions plus analytics	AlloyDB	PostgreSQL-compatible, analytics on live data
Global transactions	Spanner	Global, strongly consistent, 99.999%
Wide-column, by key	Bigtable	Low-latency reads and writes at huge scale
Documents for apps	Firestore	Mobile, offline sync, flexible documents
Cache	Memorystore	Sub-millisecond, in front of a database
Objects and files	Cloud Storage	Files, images, archives, the data lake
Analytics	BigQuery	Ad hoc SQL over terabytes

THE PATTERN

Find the phrase first. The product name usually follows from it.

Where Larkspur keeps what

DATA	STORE	WHY
Live reading per meter, for the app	Bigtable	Millions of keys, millisecond lookups
Tariff service	Cloud SQL for PostgreSQL	A standard relational app
Hot tariff lookups	Memorystore	Sub-millisecond reads in front of Cloud SQL
Warehouse and history analytics	BigQuery	SQL over billions of readings
Raw files, manuals, archives	Cloud Storage	Cheap, durable objects

THE POINT

One company, five stores. Real systems mix them, and the exam expects you to.

Cloud SQL, AlloyDB or Spanner?

	CLOUD SQL	ALLOYDB	SPANNER
Engines	MySQL, PostgreSQL, SQL Server	PostgreSQL	GoogleSQL or PostgreSQL dialect
Scale	One primary, read pools	One primary, read pools	Horizontal, virtually unlimited
Best SLA	99.99%, Enterprise Plus	Zonal failover built in	Up to 99.999%, multi-region
Analytics	Use a replica or BigQuery	Columnar engine, on live data	Columnar engine, Data Boost
Signal	Lift and shift	PostgreSQL plus analytics	Global, strongly consistent

THE TIEBREAKER

Writes from several regions at once, with no conflicts, is the one job only Spanner does.

Spanner editions, briefly

- **Standard**: regional only, 99.99%. Relational and key-value.
- **Enterprise**: adds Spanner Graph, full-text and vector search, the columnar engine, autoscaling.
- **Enterprise Plus**: dual- and multi-region, up to **99.999%**, geo-partitioning.
- Capacity starts at **100 processing units**. One node is 1,000.

> THE SIGNAL
>
> 'Multi-region' or 'five nines' in a Spanner question means Enterprise Plus.

Bigtable: a giant sorted map

- **Billions of rows**, thousands of columns, petabytes. One row key, sorted.
- Google quotes **low single-digit millisecond** latency, and up to 17K requests a second per node.
- Atomic on **one row only**. Not relational, and not for ad hoc analytics.
- Add clusters to replicate. Multi-region instances reach **99.999%**.

> THE SIGNAL
>
> Time series, IoT, ad tech and fraud scoring are Bigtable's home ground.

Firestore: documents for apps

- A **serverless document database**. No sharding, no maintenance windows.
- Client SDKs give **real-time updates** and **offline sync** for mobile and web.
- ACID multi-document transactions, multi-region, up to **99.999%**.
- **Enterprise edition** adds **MongoDB compatibility**. Datastore apps keep working.

> THE SIGNAL
>
> 'Mobile app, works offline, syncs when back online' points straight to Firestore.

Memorystore: the fast lane

- Managed **Valkey**, **Redis** and **Memcached**, with **sub-millisecond** reads.
- Sits **in front of** a database, holding the answers asked for most.
- Valkey: cluster mode, up to five replicas per shard, across zones.
- A cache, not the **system of record**. Plan for it to be empty.

> THE SIGNAL
>
> 'Sub-millisecond' plus 'the same few items read constantly' means a cache.

Cloud Storage or BigQuery?

CLOUD STORAGE

Objects of any size

- Files, images, PDFs, backups
- The landing zone and the lake
- Classes and lifecycle: module 22
- Read by every engine

BIGQUERY

Analytics at any scale

- Ad hoc SQL over petabytes
- Serverless, columnar storage
- Not for millisecond lookups
- Not for heavy row-by-row updates

THE TRAP

BigQuery answers questions about data. It is not the database behind an app's every click.

Six choices that look right and are not

THE TEMPTING CHOICE	WHY IT FAILS
BigQuery behind a mobile app	Built for scans, not millisecond lookups
Bigtable for joins and transactions	One row atomic, no relational joins
Cloud SQL for writes in three regions	One primary, in one region
Spanner for a small regional app	Works, but costs more than it needs to
Memorystore as the only copy	A cache can be emptied
Firestore for warehouse analytics	Built for app reads, not big scans

THE PATTERN

Most wrong answers use a real product for the wrong access pattern.

> The customer app shows each of 3.1 million meters' latest reading. Writes arrive at up to 10,000 a second in the evening.
>
> Every app screen reads one meter by its ID, and must respond in a few milliseconds. A Cloud SQL prototype could not keep up.

Which store should hold the live readings?

A Cloud SQL for PostgreSQL, on a larger machine with an index

B A BigQuery table, queried by the app for each screen it shows

C Bigtable, with a row for each meter, keyed by its meter ID

D Memorystore as the only store, with no database behind it

◆ PAUSE AND DECIDE

Which store should hold the live readings?

A. A bigger Cloud SQL
One primary still takes every write. The prototype already hit that wall.

B. BigQuery per screen
Built for scans, not millisecond lookups for every screen.

C. Bigtable by meter
Key lookups in milliseconds, and writes that scale across nodes.

D. Memorystore alone
Fast, but a cache is not the system of record.

THE SIGNAL

Millions of keys, heavy writes, single-key reads in milliseconds: Bigtable.

> A ticketing company sells seats for concerts in Europe, America and Asia from servers in all three regions at once.
>
> A seat must never be sold twice, even during a regional outage, and the business wants five nines of availability.

Which database fits this best?

A Spanner, multi-region, in the Enterprise Plus edition

B Cloud SQL, with read replicas in each of the three regions

C Bigtable, with a cluster in each region and multi-cluster routing

D AlloyDB, with a read pool of nodes added in each region

◆ PAUSE AND DECIDE

Which database fits this best?

A Spanner Enterprise Plus

Writes everywhere, strongly consistent, up to 99.999%.

B Cloud SQL replicas

Replicas only read. Every sale still goes to one region.

C Bigtable multi-cluster

Eventually consistent, so two regions could sell one seat.

D AlloyDB read pools

Read pools only read. Writes still go to one primary.

THE SIGNAL

Writes in several regions, never conflicting, five nines: Spanner Enterprise Plus, multi-region.

> Larkspur's tariff team wants to keep using PostgreSQL. It also wants pricing analysts to run heavy analytical queries on live tariff and contract data.
>
> Those queries must not slow down transactions, and the team does not want a separate copy in BigQuery.

Which service fits best?

A Cloud SQL for PostgreSQL, with one large read replica for analysts

B Spanner, using its PostgreSQL dialect for the tariff application

C A nightly export of the tariff tables into BigQuery for analysts

D AlloyDB for PostgreSQL, with its columnar engine turned on

◆ PAUSE AND DECIDE

Which service fits best?

A Cloud SQL replica

Offloads the reads, but row storage makes heavy analytics slow.

B Spanner PostgreSQL

A bigger move than needed, for a regional workload.

C Nightly BigQuery export

A separate copy, a day old, which the team ruled out.

D AlloyDB columnar

PostgreSQL, with fast analytics on live data and no second copy.

THE SIGNAL

'PostgreSQL' plus 'analytics on live data' plus 'no second copy': AlloyDB.

> Larkspur's pricing engine reads the same few hundred active tariffs about 5,000 times a second. Cloud SQL's CPU is close to its limit.
>
> Each lookup should take well under a millisecond. Tariffs change only a few times a day.

What should the team add?

A. A larger Cloud SQL machine, sized for the peak read load

B. A copy of the tariff table in Bigtable, for the pricing engine

C. Memorystore for Valkey, as a cache in front of Cloud SQL

D. BigQuery BI Engine, holding the tariffs in memory for queries

◆ PAUSE AND DECIDE

What should the team add?

A. A larger machine

Pays to read the same rows thousands of times, still not sub-millisecond.

B. A Bigtable copy

Milliseconds, not sub-millisecond, and a second store to sync.

C. Memorystore cache

Sub-millisecond reads of hot data, and Cloud SQL's load falls.

D. BI Engine

Accelerates BigQuery dashboards, not an application's lookups.

THE SIGNAL

Same few items, read constantly, sub-millisecond: a cache in front of the database.

> Larkspur's field engineers need a tablet app for job sheets. Many meters are in basements with no signal.
>
> Engineers must keep working offline, and their changes must sync automatically when signal returns. Job sheets vary in shape.

Which store should back the app?

A Cloud SQL, reached through an API that the tablets call

B Firestore, with its client SDKs and offline support enabled

C Bigtable, with a row for each job sheet, keyed by job ID

D Cloud Storage, with a separate JSON file saved for each job sheet

◆ PAUSE AND DECIDE

Which store should back the app?

A Cloud SQL via an API

No signal, no API. The app stops working in the basement.

B Firestore with offline

Works offline, syncs on reconnect, and fits varied documents.

C Bigtable

Server-side only, with no offline client sync.

D JSON files

No offline sync, no queries, and conflicting edits to manage.

THE SIGNAL

Offline mobile work with automatic sync and flexible documents: Firestore.

> Larkspur wants to keep 18,000 pages of engineers' manuals as PDFs, plus site photos, for the field assistant and for archive.
>
> Some files are hundreds of megabytes. They are written once and read occasionally.

Where should the files live?

A In a BigQuery table, with each file stored in a BYTES column

B In Bigtable, with each file split across the cells of one row

C In Firestore, with one document for each manual or photo

D In a Cloud Storage bucket, as objects, read where they sit

◆ PAUSE AND DECIDE

Where should the files live?

A BigQuery BYTES

Analytics storage, not a file store for large binaries.

B Bigtable cells

Cells are for small values. Large files fight the design.

C Firestore documents

Documents are limited to 1 MiB, far below these files.

D Cloud Storage objects

Built for files of any size, cheap, and readable by every engine.

THE SIGNAL

Files of any size, written once, read occasionally: Cloud Storage.

> Analysts want to explore eight years of Larkspur's readings, about 1.2 PB, with ad hoc SQL.
>
> Queries join readings to customers and tariffs, and scan billions of rows. Nobody wants to manage servers.

Which store fits this best?

A Bigtable, using its SQL support for the analysts' questions

B Cloud SQL for PostgreSQL, on its largest available machine

C BigQuery, with the readings in partitioned, clustered tables

D Spanner Enterprise, with its columnar engine switched on for analysts

◆ PAUSE AND DECIDE

Which store fits this best?

A Bigtable SQL

Great for key lookups, but ad hoc joins at this scale are not its job.

B Largest Cloud SQL

A single machine cannot scan a petabyte quickly.

C BigQuery

Serverless, columnar, built for ad hoc joins over petabytes.

D Spanner columnar

Analytics on operational data, not an eight-year warehouse.

THE SIGNAL

Ad hoc SQL, joins, petabytes, no servers: BigQuery.

> A payroll application runs on MySQL 8.0, with a 200 GB database and modest load in one region.
>
> The company wants to move it to Google Cloud with no code changes, and a 99.99% availability SLA that includes maintenance.

Which service fits this best?

A Spanner, after rewriting the application's queries for GoogleSQL

B AlloyDB, migrated with Database Migration Service

C Bigtable, with each payroll record stored as a row

D Cloud SQL for MySQL, in the Enterprise Plus edition

◆ PAUSE AND DECIDE

Which service fits this best?

A Spanner

A rewrite, which the company ruled out, for a modest regional load.

B AlloyDB

PostgreSQL only. A MySQL app would need changes.

C Bigtable

Not relational, so payroll queries would need rewriting.

D Cloud SQL Enterprise Plus

MySQL as is, and 99.99% including maintenance.

THE SIGNAL

MySQL, no code changes, regional, 99.99% with maintenance: Cloud SQL Enterprise Plus.

Module 19 in five lines

- Choose by **access pattern**: by key, transaction, document, cache, file or scan.
- **Cloud SQL** lift and shift. **AlloyDB** PostgreSQL plus analytics. **Spanner** global writes.
- **Bigtable** for huge key-value scale in milliseconds. **Firestore** for app documents and offline sync.
- **Memorystore** is a cache. **Cloud Storage** holds files. **BigQuery** answers analytics.
- Most wrong answers use a real product for the **wrong access pattern**.

COMING UP

Next: module 20, Bigtable row keys and performance.

MODULE 20 · SECTION 3 - STORING THE DATA

Bigtable: Row Keys and Performance

Row key design, hotspots, tall and wide tables, garbage collection, storage, scaling and the newest features.

SECTION 3 of 5 EXAM WEIGHT 20% MODULE 20 of 45

admin@sthithapragna.com

A row key is a phone book. Everyone called Smith on one page is a hotspot.

One sorted key, split across nodes

- Rows are **sorted by row key**, as bytes, and split into ranges called tablets.
- Each node serves some tablets. Bigtable **rebalances** them as load changes.
- Fast reads: **one key**, a **prefix**, or a **key range**.
- There is **one index**, the row key. Anything else means scanning.

> THE RULE
>
> Design the key around the questions you will ask. You cannot add another index later.

Larkspur's two row keys

THE PROTOTYPE

reading_ts#meter_id

- Every new reading has the newest time
- So every write lands at the end
- One tablet, one node, red hot
- Other nodes sit idle

THE FIX

meter_id#reverse_ts

- 3.1 million different prefixes
- Writes spread across every node
- Each meter's newest reading first
- One prefix scan per meter

THE PATTERN

Time first makes a hotspot. Identity first, then time, spreads the load and keeps a meter's rows together.

Row key patterns to know

PATTERN	WHAT IT DOES	WATCH OUT
Identity first	Spreads writes across the key space	Pick a high-cardinality field
Reverse timestamp	Newest rows sort first	Never at the very start
Field promotion	Moves a queried field into the key	Order fields by how you filter
Salting	A small prefix bucket spreads hot writes	Reads must fan out to every bucket
Hashing the key	Even spread	Loses order; Google advises readable keys
Reverse domain	com.company.product groups by site	Only helps with many domains

THE TIEBREAKER

Readable, high-cardinality prefixes beat clever tricks. Salt only when one value is unavoidably hot.

Building Larkspur's row key

```python
MAX_LONG = 2**63 - 1

def row_key(meter_id: str, reading_ts_ms: int) -> bytes:
    reverse_ts = MAX_LONG - reading_ts_ms
    return f"{meter_id}#{reverse_ts:019d}".encode()

# Bad:  2026-10-09T17:30Z#MTR00012345   every write lands at the end
# Good: MTR00012345#9223370245287775807  spread by meter, newest first

# Latest reading: scan the prefix MTR00012345#, limit 1 row
```

4 Subtracting from the maximum long makes newer times sort first.

5 Padded to a fixed width, so the bytes sort correctly.

REMEMBER

Keep each row key well under the 4 KB limit. Short keys are cheaper to store and compare.

Tall or wide?

TALL

One row per reading

- Key: meter#reverse_ts
- Small rows, simple writes
- Garbage collect by age
- Google's default advice for time series

WIDE

One row per meter per day

- Key: meter#date, 48 columns
- Fewer rows, one read per day
- Row-level atomic updates
- Keep rows under 100 MB

THE NUMBERS

Limits: key 4 KB, cell 10 MB, row ideally under 100 MB and never over 256 MB, about 100 column families.

Column families and garbage collection

- Each **column family** has its own **garbage collection** policy.
- Rules: **max versions**, **max age**, combined as union or intersection.
- **No default TTL**. gcloud and cbt keep unlimited versions unless you say otherwise.
- Deletion happens at compaction, **up to a week** later. Filter reads to match.

> FOR LARKSPUR
>
> Put data with different lifetimes in different families: raw for 90 days, summaries kept for years.

Key Visualizer: see the hotspot

- A **heatmap**: time across, row-key ranges down. **Bright** means hot.
- A bright **horizontal band** is a hot key range.
- A steady **diagonal** marks sequential keys, such as timestamps first.
- Needs at least **1 GB per cluster**. New tables take days to show data.

> THE SIGNAL
>
> 'Some nodes busy, others idle' plus a bright band in Key Visualizer means fix the row key, not the node count.

SSD or HDD?

SSD

The default for almost everything

- 5 TB per node
- Low, predictable latency
- Serving and real-time reads
- Usually the cheapest overall

HDD

Only for patient batch data

- 16 TB per node
- 5% of SSD's read rows per second
- Scans and writes, no users waiting
- Can need more nodes than it saves

THE TRAP

Storage type is permanent. To change it, back up the table and restore it into a new instance.

Nodes and autoscaling

- Each node serves roughly **17,000 requests a second**, by Google's figure.
- Autoscaling targets **CPU**, from 10% to 80%, and **storage**, 50% by default.
- Scales up **immediately**, down slowly. Rebalancing takes minutes.
- Latency-sensitive apps: keep storage utilisation **under 60%**.

> THE TRAP
>
> More nodes help only when load is spread. A hotspot stays a hotspot on a bigger cluster.

Editions and newer features

- **Editions, since April 2026**: Enterprise, and Enterprise Plus. Older instances are Enterprise.
- **Enterprise Plus**: an in-memory tier for sub-millisecond reads (Preview), bigger tiered storage.
- **Data Boost**: separate compute for batch and analytical reads, so serving is not disturbed.
- **Continuous materialized views**: SQL results kept up to date, even with a different key.

> THE SIGNAL
>
> 'Analytics on Bigtable without hurting the app' points to Data Boost.

Phrases that pick the answer

THE QUESTION SAYS	REACH FOR
Some nodes busy, others idle	Redesign the row key
Key starts with a timestamp	Identity first, reverse timestamp after
Newest first for each device	Reverse timestamp in the key
Old cells still returned	Garbage collection rule, plus a read filter
Batch only, no users, huge volume	HDD, in a new instance
Analytics must not hurt serving	Data Boost

THE PATTERN

The row key decides performance. Nodes, disks and editions only tune it.

> Larkspur's readings table uses the row key reading_ts#meter_id. During the evening surge, one node runs at full CPU while the others are nearly idle.
>
> Key Visualizer shows one bright band at the end of the key range.

What should the team change?

A Double the node count, so that the surge has more capacity

B Move the table to HDD storage, which holds more data per node

C Hash the entire row key, so that every write lands at random

D Lead the key with the meter ID, then add a reversed timestamp

◆ PAUSE AND DECIDE

What should the team change?

A **Double the nodes**

The hot range still lives on one node. The others stay idle.

B **Move to HDD**

Slower reads, and no change to where writes land.

C **Hash the whole key**

Spreads writes, but per-meter range reads become impossible.

D **Meter first, then time**

Writes spread across meters, and each meter's newest row is first.

THE SIGNAL

One node hot, a bright band at the end, timestamp first: redesign the key, identity first.

> Larkspur keeps raw readings in a Bigtable column family created with cbt. Policy says raw cells must be kept for 90 days only.
>
> The table keeps growing, and some queries still return cells older than 90 days.

What should the team do?

A Set a max-age rule of 90 days on the family, and filter reads to match

B Write a nightly job that scans the whole table and deletes the old rows

C Set a max-versions rule of one cell on the column family

D Set a 90-day time to live on the table, instead of the family

◆ PAUSE AND DECIDE

What should the team do?

A Max age, plus a filter

Garbage collection removes old cells; the filter hides them meanwhile.

B A nightly delete job

A full scan every night, doing what garbage collection does free.

C Max versions of one

Keeps the newest cell per column, not 90 days of readings.

D A table-level TTL

Garbage collection rules are set per column family.

THE SIGNAL

Keep for N days in Bigtable: a max-age rule per family, plus a matching read filter.

> A 400 TB Bigtable instance on SSD holds years of grid sensor data. It is read only by a daily batch job that scans it for reports.
>
> No application or person waits on it, and finance wants the storage bill cut.

What should the team do?

A. Switch the existing instance's storage type from SSD to HDD

B. Create an HDD instance, and restore the table there from a backup

C. Keep SSD, and add nodes so that the daily scans finish faster

D. Move the sensor data into Memorystore, to make reads cheaper

◆ PAUSE AND DECIDE

What should the team do?

A Switch in place

Storage type is permanent. It cannot be changed on an instance.

B New HDD instance, restore

Batch-only scans suit HDD, and backup and restore is the move.

C More SSD nodes

Raises the bill, which is the opposite of the goal.

D Memorystore

Memory is the most expensive place to keep 400 TB.

THE SIGNAL

Batch-only, latency-insensitive, huge: HDD. But storage type is permanent, so restore into a new instance.

> Larkspur's analysts run large daily scans of the live readings table for reports. While they run, the customer app's read latency spikes.
>
> The app must stay fast, and the analysts still need their scans every day.

What should the team do?

A Raise the autoscaling CPU target, so that the existing nodes work harder

B Switch the instance to HDD, so that big scans are cheaper

C Run the analysts' scans through Data Boost, apart from serving

D Copy the whole table into Memorystore, just for the analysts

◆ PAUSE AND DECIDE

What should the team do?

A Higher CPU target

Fewer nodes, hotter nodes, and worse latency for the app.

B Switch to HDD

Slower reads for the app, and storage type is permanent.

C Data Boost

Separate compute for the scans, so serving nodes are untouched.

D Memorystore copy

An expensive copy of a huge table, to fix a scheduling clash.

THE SIGNAL

Analytics must not hurt serving: Data Boost, or a separate cluster and app profile.

Module 20 in five lines

- One index, the **row key**. Design it around the questions you will ask.
- Never lead with a timestamp. **Identity first**, then a **reverse timestamp**.
- Garbage collection is **per family**, with no default TTL. Filter reads to match.
- **SSD** by default. **HDD** only for batch scans, and the choice is permanent.
- Hotspots need a new key, not more nodes. **Data Boost** isolates analytics.

COMING UP

Next: module 21, relational and operational stores.

MODULE 21 · SECTION 3 - STORING THE DATA

Relational and Operational Stores

Cloud SQL availability, replicas and connections; Spanner keys, interleaving, configurations and transactions; Memorystore tiers.

SECTION 3 of 5 EXAM WEIGHT 20% MODULE 21 of 45

admin@sthithapragna.com

High availability keeps you up. Replicas spread the reads. **Only some stores spread the writes.**

High availability or read replica?

HIGH AVAILABILITY

Stay up when a zone fails

- Primary and standby in two zones
- Synchronous, so no committed data lost
- Failover in about 60 seconds
- Standby does not serve reads

READ REPLICA

Spread the reads

- Asynchronous copy, slightly behind
- Read-only, offloads reports
- Does not fail over by itself
- Ten or fewer direct replicas advised

THE TIEBREAKER

'Survive a zone failure automatically' is HA. 'Reports slow the primary' is a read replica.

Surviving a regional outage

- A **cross-region read replica** keeps a copy in another region.
- In a disaster you **promote** it, and it becomes a standalone primary.
- Replication is asynchronous, so the last moments of writes may be lost.
- **Enterprise Plus** adds advanced disaster recovery and a **write endpoint**.

> REMEMBER
>
> HA covers a zone. A cross-region replica covers a region. Many designs need both.

Connecting safely

- Google recommends **private IP**: private services access, or Private Service Connect across VPCs.
- On public IP, use the **Cloud SQL Auth Proxy** or a **Language Connector**.
- **IAM database authentication**: short-lived tokens instead of passwords.
- Direct private connections are fastest, but you **manage SSL** yourself.

> THE NUMBERS
>
> Instances hold up to 64 TB on dedicated cores. Past that, the workload is outgrowing Cloud SQL.

Larkspur's tariff database

```
gcloud sql instances create lk-tariffs-prod \
    --database-version=POSTGRES_16 \
    --edition=ENTERPRISE_PLUS --tier=db-perf-optimized-N-8 \
    --region=europe-west2 --availability-type=REGIONAL \
    --network=projects/lk-network-prod/global/networks/lk-vpc \
    --no-assign-ip \
    --database-flags=cloudsql.iam_authentication=on

# A cross-region replica for disaster recovery
gcloud sql instances create lk-tariffs-dr \
    --master-instance-name=lk-tariffs-prod --region=europe-west1
```

4 Regional means high availability: a standby in a second zone.

6-7 Private IP only, and IAM logins instead of passwords.

CHECK RESIDENCY

Tariffs carry no residency rule, so a replica outside the UK is allowed. German personal data could not do this.

Spanner keys: spread the writes

- Never lead a busy table's key with a value that **only goes up**, or only down.
- Use **UUID version 4**. Version 1 hides a timestamp at the front.
- For numbers, a **bit-reversed sequence**. Or swap the key order, identity first.
- Indexes are tables too, so the **same rule** applies to indexed columns.

> THE TRAP
>
> Reversing the sort order does not help. A descending sequence is still monotonic.

Keys and interleaving, if billing moved to Spanner

```
 1  CREATE TABLE Accounts (
 2    AccountId  STRING(36) NOT NULL,   -- UUID v4, never a sequence
 3    CustomerId STRING(36) NOT NULL,
 4    TariffId   STRING(20),
 5  ) PRIMARY KEY (AccountId);
 6
 7  CREATE TABLE Bills (
 8    AccountId  STRING(36) NOT NULL,
 9    BillDate   DATE NOT NULL,
10    AmountGbp  NUMERIC,
11  ) PRIMARY KEY (AccountId, BillDate DESC),
12    INTERLEAVE IN PARENT Accounts ON DELETE CASCADE;
```

2 A random key spreads new accounts across every split.

11-12 Bills live beside their account, newest first.

THE PAYOFF

Reading an account and its recent bills becomes one local read, not a join across servers.

Spanner configurations

CONFIGURATION	REPLICAS	AVAILABILITY
Regional	3 read-write, in three zones	99.99%
Dual-region	6, across two regions in one country	99.999%
Multi-region	Two read-write regions, a witness, optional read-only	99.999%

THE TRADE-OFF

Writes need a quorum, so multi-region writes cost a little latency. Reads do not need a quorum.

Read-write or read-only transactions?

READ-WRITE

For changes

- Takes locks
- May abort, so retry
- Keep it short
- Externally consistent

READ-ONLY

For reading

- No locks, blocks nobody
- A consistent snapshot
- Can read slightly in the past
- Lower latency, fewer aborts

THE RULE

Use read-only for reads. For bulk updates, Partitioned DML; for writes that need no atomicity, batch writes.

Memorystore tiers and failover

- **Redis Basic**: one node, no replica, no failover. A pure cache.
- **Redis Standard**: a replica in another zone, with **automatic failover**.
- **Read replicas** scale reads. Redis Cluster and Valkey cluster mode shard data.
- Tiers cannot be converted. Move with **export and import**.

> THE SIGNAL
>
> 'The cache must survive a node or zone failure' rules out Basic.

Phrases that pick the answer

THE QUESTION SAYS	REACH FOR
Survive a **zone** failure automatically	Cloud SQL high availability
Reports slow the primary	A read replica
Survive a **region** failure	Cross-region replica, promote in a disaster
Connect **without passwords**	Auth Proxy or connector, IAM database auth
Spanner writes on **one split**	UUID v4 or bit-reversed keys
Dashboard reads that **must not block**	Spanner read-only transactions

THE PATTERN

Up, read, write: three different dials. Find which one the question is turning.

> Larkspur's tariff database runs on a single-zone Cloud SQL instance in europe-west2.
>
> If that zone fails, the database must come back automatically within minutes, without losing any committed change.

What should the team configure?

A. High availability, as a regional instance with a standby

B. A read replica placed in a different zone of the same region

C. Nightly exports of the database to a Cloud Storage bucket

D. A cross-region replica in europe-west1, promoted by hand

◆ PAUSE AND DECIDE

What should the team configure?

A High availability

Synchronous standby in another zone, automatic failover in about a minute.

B A read replica

Asynchronous, and it does not fail over by itself.

C Nightly exports

Up to a day of lost changes, and a slow manual restore.

D A cross-region replica

Manual promotion, asynchronous, and a region is not the risk here.

THE SIGNAL

Zone failure, automatic, no committed data lost: Cloud SQL high availability.

> Larkspur's pricing analysts run heavy reports against the tariff database every morning, and the application slows down while they run.
>
> The reports can use data that is a few seconds old.

What should the team do?

A Turn on high availability, and send the reports to the standby

B Create a read replica, and point the reporting tool at it

C Double the primary's machine size before every morning run

D Cache every report's result in Memorystore for the analysts

◆ PAUSE AND DECIDE

What should the team do?

A Reports on the standby

The HA standby does not serve reads.

B A read replica

Reports run on the copy, and the primary serves the app.

C A bigger primary

Pays for a larger machine to keep two workloads fighting.

D Cache the reports

Heavy, varied reports change; a cache does not run them.

THE SIGNAL

Reads slowing the primary, a little staleness allowed: a read replica.

> A Spanner table of meter events uses EventId as its primary key, taken from a sequence that increases by one each time.
>
> Write latency climbs at peak, and monitoring shows most writes landing on a single split.

What should the team change?

A Declare the key in descending order, so new rows sort first

B Add nodes to the instance, so that the hot split has more capacity

C Key the table with UUID v4 values, or a bit-reversed sequence

D Add a secondary index on EventId, to spread the reads

◆ PAUSE AND DECIDE

What should the team change?

A **Descending order**

Still monotonic, so writes still pile onto one end.

B **More nodes**

One hot split stays on one server, however many there are.

C **UUID v4 or bit-reversed**

Random-looking keys spread writes across every split.

D **An index on EventId**

Indexes are tables, so it hotspots too, and reads were not the issue.

THE SIGNAL

Writes on one split from a sequence key: make the key non-monotonic.

> A Larkspur service on GKE must connect to a Cloud SQL instance that currently has only a public IP address.
>
> Security wants encrypted connections, no long-lived database passwords, and no certificates to manage by hand.

How should the service connect?

A Add 0.0.0.0/0 to authorized networks, and use a strong password

B Connect directly, with hand-managed SSL certificates and a shared password

C Copy the needed tables into BigQuery, and query them from there

D Use the Cloud SQL Auth Proxy, with IAM database authentication

◆ PAUSE AND DECIDE

How should the service connect?

A **Open to the internet**

Exposes the database to everyone, with a long-lived password.

B **Hand-managed SSL**

Certificates and a shared password, both of which security ruled out.

C **Copy into BigQuery**

Changes the architecture, and the service needs the live database.

D **Auth Proxy and IAM**

Encrypted, identity-checked, with short-lived tokens refreshed for you.

THE SIGNAL

Public IP, no passwords, no certificates to manage: Auth Proxy or a connector, with IAM auth.

> A dashboard reads across several Spanner tables every few seconds. Its numbers must be consistent with each other, and data up to 10 seconds old is fine.
>
> Its queries must never block or slow down the application's writes.

Which approach fits best here?

A Read-only transactions, reading at a timestamp a few seconds old

B Read-write transactions, so that every single read is fully locked

C A nightly copy of the tables in Cloud SQL, for the dashboard

D Partitioned DML statements, run by the dashboard each time

◆ PAUSE AND DECIDE

Which approach fits best here?

A **Read-only, slightly stale**
No locks, a consistent snapshot, and lower latency.

B **Read-write**
Locks and aborts, exactly what must not touch the writers.

C **A nightly Cloud SQL copy**
A day stale, when ten seconds was the limit.

D **Partitioned DML**
A tool for bulk changes, not for reading.

THE SIGNAL

Consistent reads that must not block writers: read-only transactions, optionally stale.

> Larkspur caches tariffs in a Memorystore for Redis Basic instance. When its node failed last month, every request fell through to Cloud SQL at once, and the database buckled.
>
> The cache must now survive a node or zone failure automatically.

What should the team do?

A. Resize the Basic tier instance to a much larger memory size

B. Move to Standard tier, with a replica in another zone

C. Store the cache's contents in Cloud SQL as a backup copy

D. Raise the cache entry TTLs, so that items last for days

◆ PAUSE AND DECIDE

What should the team do?

A. A larger Basic instance
Still one node, with no replica and no failover.

B. Standard tier
A replica in another zone, and automatic failover.

C. Back up into Cloud SQL
Puts the load back on the database it was protecting.

D. Longer TTLs
Staler data, and the next failure still empties everything.

THE SIGNAL

A cache that must survive a node or zone failure: Standard tier, not Basic.

Module 21 in five lines

- Cloud SQL **HA** survives a zone. **Read replicas** spread reads, and do not fail over.
- A **cross-region replica**, promoted in a disaster, covers a region.
- Private IP, the **Auth Proxy** or connectors, and **IAM database auth**.
- Spanner: no monotonic keys, **interleave** children, read-only transactions for reads.
- Memorystore **Basic** has no failover. **Standard** does.

COMING UP

Next: module 22, Cloud Storage classes, lifecycle and retention.

MODULE 22 · SECTION 3 - STORING THE DATA

Cloud Storage: Classes, Lifecycle and Retention

Storage classes and minimum durations, Autoclass, lifecycle rules, versioning and soft delete, retention and Bucket Lock, locations and turbo replication.

SECTION 3 of 5 EXAM WEIGHT 20% MODULE 22 of 45

admin@sthithapragna.com

The class sets the price of reading.
Retention decides whether deleting is allowed at all.

Four classes, one API

CLASS	MINIMUM DURATION	RETRIEVAL FEE	READ ABOUT
Standard	None	None	Often
Nearline	30 days	Yes	Once a month
Coldline	90 days	Yes	Once a quarter
Archive	365 days	Yes	Under once a year

THE TRAP

Every class reads in milliseconds, with the same eleven nines of durability. Archive is not tape.

What a minimum duration really means

- Delete a Coldline object at day 10, and you pay for **all 90 days**.
- Early deletion charges apply to soft-deleted objects too, except in Standard.
- A lifecycle class change counts the **time already spent** in the earlier class.
- The bucket's **default class** only applies to new objects.

> REMEMBER
>
> Short-lived data belongs in Standard, however cheap the colder price looks.

Autoclass or lifecycle rules?

AUTOCLASS

Follows actual reads

- Everything starts in Standard
- 30 days unread: Nearline
- Archive as terminal: Coldline, then Archive
- A read moves it back to Standard
- No retrieval or early deletion fees

LIFECYCLE RULES

Follows age and dates

- You write the conditions
- Only moves to colder classes
- Can also delete objects
- Cheapest when the pattern is known
- Cannot set classes with Autoclass on

THE TIEBREAKER

Nobody knows how the data will be read: Autoclass. The pattern is fixed by age: lifecycle rules.

Larkspur's history bucket

```
{"lifecycle": {"rule": [
  {"action": {"type": "SetStorageClass", "storageClass": "COLDLINE"},
   "condition": {"age": 90, "matchesStorageClass": ["STANDARD"]}},
  {"action": {"type": "SetStorageClass", "storageClass": "ARCHIVE"},
   "condition": {"age": 365, "matchesStorageClass": ["COLDLINE"]}},
  {"action": {"type": "Delete"}, "condition": {"age": 2557}},
  {"action": {"type": "Delete"},
   "condition": {"isLive": false, "daysSinceNoncurrentTime": 30}}
]}}

gcloud storage buckets update gs://lk-history-prod --lifecycle-file=rules.json
```

2-5 Settlement re-runs read the last quarter, then the files go cold.

6 About seven years, then delete. The retention policy still has the last word.

HONEST LIMIT

Rules can take up to 24 hours to take effect, and actions run asynchronously after that.

How rules combine

- Conditions in one rule are joined with **AND**. Separate rules act as **OR**.
- If two rules match at once, **Delete** beats a class change.
- A class change is a Class A operation, with **no retrieval fee**.
- Transitions only go **colder**. Nothing moves an object back to Standard.

> THE PATTERN
>
> Delete waits while an object has a hold, or is still inside its retention period.

Object Versioning or soft delete?

OBJECT VERSIONING

Keeps older versions

- Overwrite or delete makes a noncurrent version
- Kept until you remove it
- Lifecycle rules tidy old versions
- Every version is billed

SOFT DELETE

An undo window

- On by default for 7 days
- Set from 7 to 90 days, or 0 to turn off
- Restores deleted objects and buckets
- Billed while it is kept

THE TIEBREAKER

'Recover last week's accidental delete' is soft delete. 'Get back the version before the bad job' is versioning.

Retention policies and Bucket Lock

- A **retention policy** blocks delete and overwrite until each object is old enough.
- **Locking** it is irreversible. Lengthen it later, but never shorten or remove it.
- **Event-based holds** start the clock when the hold is released, such as a contract end.
- **Object Retention Lock** sets retention object by object, not for the whole bucket.

> THE TRAP
>
> A locked bucket cannot be deleted, and the project gets a lien. Test the policy unlocked first.

Where the bucket lives

LOCATION TYPE	WHAT YOU GET	REPLICATION
`Region`	Copies across zones in one region	Within the region
`Dual-region`	Two regions that you choose	Default RPO 12 hours
`Dual-region, turbo`	The same two regions	RPO 15 minutes
`Multi-region`	Regions Google chooses, far apart	Default RPO 12 hours

THE SIGNAL

Turbo replication exists only on dual-region buckets. A multi-region cannot turn it on.

Sharing and encrypting

- A **signed URL** lets anyone holding it read or upload, for up to **7 days**.
- Prefer **uniform bucket-level access**, so IAM alone decides who gets in.
- A **default CMEK** key encrypts new objects. Its key ring sits in the bucket's location.
- Changing the default key does not re-encrypt old objects. **Rewrite** them.

> FOR LARKSPUR
>
> German customer files go in europe-west3, with their key ring in europe-west3 too.

Newer than most question banks

- **Rapid Cache**, formerly Anywhere Cache: an SSD read cache in a zone, for hot analytics and AI reads.
- **Rapid** storage class, in zonal **Rapid Buckets**, for the lowest latency.
- **Hierarchical namespace**: real folders, atomic folder rename, up to 8x initial QPS.
- Namespace is chosen at creation, and rules out versioning, holds and Bucket Lock.

> CURRENCY CHECK
>
> Exam questions may still say Anywhere Cache. It is the same product under its new name.

Phrases that pick the answer

THE QUESTION SAYS	REACH FOR
Access pattern is **unknown**	Autoclass
Data cools on a **fixed schedule**	Lifecycle rules
Nobody may delete it, **even admins**	Retention policy, then Bucket Lock
Undo an **accidental delete**	Soft delete
Survive a region, **15 minutes** of loss	Dual-region with turbo replication
Share one file, **no Google account**	Signed URL

THE PATTERN

Cheap to keep, safe to keep, quick to recover: three different questions.

> Larkspur's data scientists keep training extracts in one bucket. Some are read daily for weeks, others never again.
>
> Nobody can predict which. They want the lowest storage cost, with no rules to maintain and no retrieval charges.

What should the team configure?

A Set Nearline as the bucket's default storage class

B A lifecycle rule moving every object to Coldline at 30 days

C Turn on Autoclass for the bucket

D Set Archive as the default, and copy files to Standard on request

◆ PAUSE AND DECIDE

What should the team configure?

A Nearline default

Retrieval fees on every daily read, and a 30-day minimum.

B Coldline at 30 days

A fixed rule for an unknown pattern, and retrieval fees on reads.

C Autoclass

Follows real reads, and charges no retrieval or early deletion fees.

D Archive plus copies

A 365-day minimum, retrieval fees, and copies to manage by hand.

THE SIGNAL

Unpredictable reads, no rules, no retrieval charges: Autoclass.

> Larkspur's settlement files must be kept unaltered for seven years.
>
> The auditor wants proof that nobody, including project owners, can delete them or shorten that period.

What should the team do?

A. Turn on Object Versioning, with a rule deleting versions after seven years

B. An IAM deny policy on deletes, for everyone except a break-glass group

C. Set soft delete to its longest window, so deletions can be undone

D. Add a seven-year retention policy to the bucket, then lock it

◆ PAUSE AND DECIDE

What should the team do?

A Versioning

Old versions can still be deleted by anyone with permission.

B IAM deny policy

An administrator can change the policy, so it proves nothing.

C Long soft delete

Ninety days at most, and the setting can be changed.

D Locked retention policy

Irreversible: no delete, no overwrite, no shortening.

THE SIGNAL

Nobody, including owners, can delete or shorten it: a locked retention policy.

> Larkspur's settlement output bucket must survive the loss of a whole region.
>
> No more than 15 minutes of newly written files may be at risk. The data may be held in either of two chosen regions.

Which location should the bucket use?

A. A multi-region bucket, with the default replication settings

B. A dual-region bucket, with turbo replication turned on

C. A regional bucket, plus a nightly copy to a second region

D. A regional bucket, with soft delete kept for the full ninety days

◆ PAUSE AND DECIDE

Which location should the bucket use?

A Multi-region

Default replication allows up to twelve hours of loss, and no turbo.

B Dual-region with turbo

A fifteen-minute recovery point, backed by an SLA.

C Nightly copy

Up to a day of new files at risk.

D Soft delete

Protects against deletes, not against losing a region.

THE SIGNAL

Lose a region, fifteen minutes at most: dual-region with turbo replication.

> The nightly settlement run writes staging extracts that a job deletes after three days.
>
> To save money, an engineer put them in a Coldline bucket. The storage bill went up, not down.

What should the team do?

A. Keep the extracts in Standard, in a bucket with soft delete off

B. Move the extracts to Archive, which has the lowest price at rest

C. Turn on Object Versioning, so that each deletion is recorded

D. Add a lifecycle rule moving the extracts to Nearline after a day

◆ PAUSE AND DECIDE

What should the team do?

A Standard, no soft delete

No minimum duration, and deleted extracts stop costing at once.

B Archive

A 365-day minimum, so each three-day file costs a year.

C Versioning

Keeps even more copies, and changes nothing about the minimum.

D Nearline after a day

Lifecycle cannot move objects to Nearline from Coldline.

THE SIGNAL

Short-lived files: Standard, and no soft delete on temporary data.

Module 22 in five lines

- Nearline **30**, Coldline **90**, Archive **365** days. All read in milliseconds.
- **Autoclass** for unknown reads. **Lifecycle rules** for a known schedule.
- **Soft delete** undoes deletes. **Versioning** keeps old versions.
- A **locked retention policy** stops anyone deleting or shortening.
- Turbo replication: **dual-region only**, a 15-minute RPO.

COMING UP

Next: module 23, BigQuery tables: partitioning, clustering and storage.

MODULE 23 · SECTION 3 - STORING THE DATA

BigQuery Tables: Partitioning, Clustering and Storage

Partitioning types and pruning, require partition filter, clustering, active and long-term storage, logical and physical billing, time travel, snapshots and clones.

SECTION 3 of 5 EXAM WEIGHT 20% MODULE 23 of 45

admin@sthithapragna.com

Partitioning picks the drawers. **Clustering sorts what is inside them.**

Three ways to partition

TYPE	PARTITIONS ON	GRANULARITY
`Time-unit column`	A DATE, TIMESTAMP or DATETIME column	Hour, day, month or year
`Ingestion time`	When the row arrived, in _PARTITIONTIME	Hour, day, month or year
`Integer range`	An INTEGER column: start, end, interval	One partition per interval

THE TIEBREAKER

Daily is the default. Hourly suits huge volumes over under six months. Monthly suits small daily volumes.

Larkspur's raw readings table

```
1  CREATE TABLE raw.meter_readings (
2    meter_id STRING NOT NULL, reading_ts TIMESTAMP NOT NULL, kwh NUMERIC,
3    register STRING, received_ts TIMESTAMP, fw_version STRING)
4  PARTITION BY DATE(reading_ts)
5  CLUSTER BY meter_id
6  OPTIONS (require_partition_filter = TRUE,
7           partition_expiration_days = 400);
```

4 One partition per reading day, in UTC. Late uploads land in the right day.

6-7 No date filter, no query. Partitions older than about 13 months are dropped.

FOR LARKSPUR

The seven-year copy lives in Cloud Storage, from module 22. BigQuery keeps the working set.

When does a filter prune?

- A **constant** on the partitioning column prunes, including BETWEEN.
- Supported functions such as **DATE_TRUNC** are fine, with constant arguments.
- A **subquery**, or arithmetic on the column, scans every partition.
- **OR** with another column also turns pruning off.

> THE TRAP
>
> 'WHERE day = (SELECT MAX(day) FROM log)' reads the whole table. Look the date up first.

Guard rails and limits

- **Require partition filter**: a query with no usable filter fails, even through a view.
- **Partition expiration** drops whole partitions once they pass a set age.
- Up to **10,000 partitions** per table. Older question banks say 4,000.
- Aim for **10 GB or more** per partition. Tiny partitions only add metadata.

> FOR LARKSPUR
>
> Larkspur adds about 30 GB of readings a day, so daily partitions are the right size.

Which one does the question need?

PARTITIONING

Coarse and predictable

- One column only
- Cost is known before the query runs
- Expire or overwrite one partition
- Long-term pricing per partition

CLUSTERING

Fine-grained

- Up to four columns, order matters
- Suits high-cardinality columns
- Skips blocks inside each partition
- Estimate is only an upper bound

THE TIEBREAKER

Need a firm cost before the query runs, or to drop data by date: partition. Many filter columns: cluster.

Getting clustering right

- Put the column filtered **most often first**. Order decides the sort.
- Worth it for tables, or partitions, over about **64 MB**.
- **Automatic reclustering** is free, and uses no query capacity.
- Changing the columns only sorts **new** data. Old data needs an UPDATE.

> REMEMBER
>
> Clustering on meter_id, then register, helps a filter on meter_id. A filter on register alone gains little.

Active and long-term storage

- Untouched for **90 consecutive days**: the price drops automatically, by up to about half.
- Judged **per partition**, with no change in speed or durability.
- Loads, DML, appends and **CREATE OR REPLACE** restart the clock.
- Querying, exporting or copying the table out does **not**.

> THE PATTERN
>
> Partition by date, and only the days still changing pay the active price.

Logical or physical billing?

LOGICAL

The default

- Billed on uncompressed bytes
- Time travel and fail-safe included
- Simple and predictable

PHYSICAL

Compressed bytes

- Billed on bytes after compression
- Time travel and fail-safe billed extra
- Higher rate per byte, fewer bytes

THE TIEBREAKER

Data that compresses well and rarely changes: physical. Heavy updates and deletes can eat the saving.

Time travel and fail-safe

- **Time travel**: query or restore any state from the last **7 days**.
- Set the window from **2 to 7 days**, on a dataset or project.
- Restore a deleted table with **bq cp** and a time decorator.
- **Fail-safe** keeps 7 more days. Fixed, and only Customer Care can recover it.

> THE TRAP
>
> Under logical billing, a shorter window saves nothing. Under physical billing, it can.

Undoing a bad night

```
1  -- What did the bills table hold an hour ago?
2  SELECT COUNT(*) FROM curated.bills
3    FOR SYSTEM_TIME AS OF TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 1 HOUR)
4  WHERE period_start >= '2026-09-01';
5
6  -- Restore a table deleted by mistake, as it was two hours ago
7  bq cp lk-warehouse-prod:curated.bills@-7200000 \
8        lk-warehouse-prod:curated.bills_restored
```

3 Read the table as it stood one hour ago.

7 A relative decorator, in milliseconds: two hours back.

HONEST LIMIT

Time travel restores data, not metadata such as descriptions or labels.

Snapshot, clone or copy?

OPTION	WRITABLE?	YOU PAY FOR
Table snapshot	No, read-only	Only data that has since changed
Table clone	Yes	Only data that differs from the base
Table copy	Yes	A full second copy

THE TIEBREAKER

Before a risky change: a snapshot. A sandbox on production data: a clone.

Phrases that pick the answer

THE QUESTION SAYS	REACH FOR
Queries filter by **date**, scan everything	Partition on that date
Filter on **many** high-cardinality columns	Cluster, in filter order
Stop **accidental full scans**	Require partition filter
Old data is **still billed as active**	Stop rewriting old partitions
Writable test copy, little extra cost	Table clone
Data **compresses well**, rarely changes	Physical storage billing

THE PATTERN

Bytes read, bytes kept, bytes recovered: find which one the question is about.

> curated.readings_hh holds 40 TB in one unpartitioned table.
>
> Nearly every query filters on the reading date and a few meter_id values, yet every query scans the whole table.

Which table design fits best?

A Partition by ingestion time, so each day's load lands in its own partition

B Partition by the date of reading_ts, and cluster by meter_id

C Integer range partitioning on a hash of meter_id, one per meter

D Cluster by reading_ts and meter_id, and leave the table unpartitioned

◆ PAUSE AND DECIDE

Which table design fits best?

A **Ingestion time**

Late uploads land on the arrival day, not the reading day queries use.

B **Reading date plus meter**

Partitions match the date filter. Clustering skips other meters.

C **Range on a meter hash**

Millions of meters, against a limit of 10,000 partitions.

D **Clustering only**

Helps, but no firm cost estimate, no expiry and no per-day pricing.

THE SIGNAL

Filter by date, then by an ID: partition on the date, cluster on the ID.

> Analysts often run SELECT * on raw.meter_readings while exploring, and forget the date filter.
>
> Each slip scans the whole table. The finance director wants those accidental full scans stopped.

What should the team do?

A Ask analysts to add LIMIT 1000 to every exploratory query

B Give the analysts a view that selects every column of the table

C Cluster the table on every column that the analysts filter on

D Set require_partition_filter on the table

◆ PAUSE AND DECIDE

What should the team do?

A LIMIT 1000

LIMIT trims the output, not the bytes scanned.

B A view over the table

Still scans every partition, just through another name.

C Cluster on everything

Only four columns, and no help when nothing is filtered.

D Require partition filter

Queries without a usable date filter fail, before scanning.

THE SIGNAL

Stop accidental full scans: require partition filter.

> A daily report filters raw.meter_readings with WHERE DATE(reading_ts) = (SELECT MAX(load_date) FROM ops.load_log).
>
> The table is partitioned by DATE(reading_ts), yet the report scans every partition.

What should the team change?

A Find the date first, then filter on it as a fixed value

B Add clustering on reading_ts, so blocks are skipped instead

C Change the table to hourly partitions on reading_ts

D Turn on require_partition_filter for the readings table

◆ PAUSE AND DECIDE

What should the team change?

A Look up first

A constant filter lets BigQuery prune before it plans the scan.

B Cluster on reading time

Treats the symptom, and the cost stays unpredictable.

C Hourly partitions

Twenty four times more partitions, and the subquery still blocks pruning.

D Require a filter

The report would just be refused, because a subquery cannot prune.

THE SIGNAL

A subquery in the partition filter stops pruning. Make the date a constant.

A nightly job deduplicates raw.meter_readings by rebuilding it with CREATE OR REPLACE TABLE.

Resends and late uploads only ever touch the last three days, yet every partition is still billed as active storage.

What change lets older partitions reach long-term pricing?

A Switch the dataset over to the physical storage billing model

B Shorten the dataset's time travel window to the two-day minimum

C Deduplicate with a MERGE on the last three days only

D Export every partition older than ninety days to Cloud Storage

◆ PAUSE AND DECIDE

What change lets older partitions reach long-term pricing?

A **Physical billing**

Changes how bytes are counted. Rewritten partitions are still active.

B **Two-day time travel**

Saves nothing under logical billing, and the rewrite continues.

C **Merge on recent days**

Only three partitions change, and the rest go long-term after 90 days.

D **Export old data**

Exporting does not reset the clock, but the nightly rewrite still does.

THE SIGNAL

Old data billed as active: stop rewriting partitions that have not changed.

> Engineers must test a risky change to curated.bills, rewriting rows freely using real production data.
>
> Production must not be touched, and the test should not pay for a full second copy of the table.

What should the team create?

A. A table snapshot of curated.bills, and run the test against it

B. A copy of curated.bills in a development dataset, every morning

C. A table clone of curated.bills in a development dataset

D. Time travel queries against curated.bills, as it stood earlier

◆ PAUSE AND DECIDE

What should the team create?

A Snapshot

Read-only, so the test cannot rewrite rows.

B Daily copy

Writable, but every copy is billed in full.

C Clone

Writable, and billed only for the rows the test changes.

D Time travel

Read-only views of the past, and the live table is still at risk.

THE SIGNAL

Writable, production data, no full copy: a table clone.

> The raw dataset holds readings that compress about eight to one. Tables are append-only, and rows are almost never updated or deleted.
>
> Finance wants a lower BigQuery storage bill, without changing any queries.

What should the team do?

A Shorten time travel to two days, and keep logical billing

B Move the older data into Cloud Storage, behind external tables

C Recluster the tables on more columns, so they compress better

D Switch the dataset to the physical storage billing model

◆ PAUSE AND DECIDE

What should the team do?

A Shorter time travel

Included in logical billing, so the bill does not move.

B External tables

Changes queries and performance, against the brief.

C More clustering

Four columns at most, and not a billing lever.

D Physical billing

Bills the compressed bytes: about an eighth as many.

THE SIGNAL

Compresses well, rarely changes: physical storage billing.

Module 23 in five lines

- Partition on the date queries **filter by**. Cluster on up to four columns.
- Subqueries and OR **stop pruning**. Require partition filter stops full scans.
- **90 days** untouched makes a partition long-term. Rewrites restart the clock.
- **Physical billing** suits data that compresses well and rarely changes.
- Time travel **2 to 7 days**, then 7 of fail-safe. Snapshot reads, clone writes.

COMING UP

Next: module 24, modelling the warehouse.

MODULE 24 · SECTION 3 - STORING THE DATA

Modelling the Warehouse

Star schema or one big table, nested and repeated fields, layers of modelling, slowly changing dimensions, append-only history, and keys BigQuery does not enforce.

SECTION 3 of 5 EXAM WEIGHT 20% MODULE 24 of 45

admin@sthithapragna.com

Storage is cheap. Shuffles are not. **Model for the questions people ask.**

Star schema or one big table?

STAR SCHEMA

Facts and dimensions

- Narrow facts, keys to dimensions
- Change a dimension in one place
- Familiar to analysts and BI tools
- Joins happen at query time

DENORMALISED

One wide or nested table

- Everything a query needs, together
- No joins, so no shuffles
- Values repeat on many rows
- A change can touch many rows

THE TIEBREAKER

BigQuery runs both well. Ask which query runs most often, and which change happens most often.

Keep children inside the parent

- A **STRUCT** groups related fields. An **ARRAY** holds the many side of a one-to-many.
- Children sit beside their parent, so there is **no join and no shuffle**.
- Best when the data is **hierarchical** and always read together.
- Avoid for **high-cardinality** children. Nesting stops at 15 levels.

> THE NUMBERS
>
> In Google's own example, nesting cut a query from about 25 seconds to 10, and read less data.

Bills with their lines inside

```
CREATE TABLE curated.bills_nested (
  bill_id      STRING NOT NULL,
  account_id   STRING NOT NULL,
  period_start DATE,
  amount_gbp   NUMERIC,
  lines ARRAY<STRUCT<
    line_type  STRING,     -- energy, standing charge, VAT
    kwh        NUMERIC,
    amount_gbp NUMERIC>>)
PARTITION BY DATE_TRUNC(period_start, MONTH)
CLUSTER BY account_id;
```

6-9 Three to eight lines per bill, stored inside the bill row.

10 Monthly partitions: a few million bills a month is too little for daily.

FOR LARKSPUR

Two point two million accounts make a few million bills a month, so monthly partitions stay well sized.

Reading the lines, with no join

```sql
-- Standing charges per bill this year, read straight from the bill rows
SELECT b.bill_id, SUM(l.amount_gbp) AS standing_gbp
FROM curated.bills_nested AS b, UNNEST(b.lines) AS l
WHERE b.period_start >= '2026-01-01'
  AND l.line_type = 'standing charge'
GROUP BY b.bill_id;
```

3 UNNEST turns the array into rows, inside each bill. Nothing is shuffled.

REMEMBER

A query that ignores the lines never reads them. Columnar storage skips unused fields.

One warehouse, three layers

01

Raw

As the source sent it. Append-only, duplicates allowed.

02

Curated

Cleaned and deduplicated. Star or nested, by subject.

03

Marts

Wide tables for one audience, such as Westgrid.

04

Views

Stable names on top, while tables underneath change.

THE PATTERN

The degree of normalisation changes by layer. There is no single right answer for the whole warehouse.

Move first, then improve

- Google advises moving the schema **unchanged** in the first iteration.
- Upstream pipelines and existing reports keep working, and tools can automate it.
- Then evolve in stages: nested fields, **partitioning** and **clustering**.
- **Facade views** keep old names working while the tables change.

> THE TRAP
>
> Keeping the old model forever keeps its technical debt. Lift first, but plan the next step.

When a dimension changes

APPROACH	WHAT HAPPENS	HISTORY KEPT
Type 1	Overwrite the old value	None
Type 2	Add a row with valid-from and valid-to	Full, row by row
Type 3	Keep a previous-value column	One step back
Daily snapshot	Copy the whole dimension each day	By date

THE SIGNAL

'What was it at the time?' needs Type 2 or snapshots. 'Fix a typo' is Type 1.

Which tariff was this bill on?

```sql
-- Join each bill to the tariff that was live when it started
SELECT b.bill_id, b.period_start, h.tariff_id
FROM curated.bills AS b
JOIN curated.account_tariff_history AS h
  ON  h.account_id = b.account_id
  AND b.period_start >= h.valid_from
  AND b.period_start <  h.valid_to;   -- open rows use DATE '9999-12-31'
```

6-7 A point-in-time join: the version whose dates cover the bill.

REMEMBER

Use a far-future date for the current row, not NULL. Then one condition covers every row.

Append changes, then resolve them

- BigQuery is happiest **appending**. Many small updates are slow and costly.
- Datastream's **append-only** mode keeps every change as a new row.
- A view picks the **latest version** of each key, and drops deletes.
- Merge mode keeps a current copy instead, and needs a **source primary key**.

THE PAYOFF

Appending keeps the full history for free, and leaves old partitions untouched.

The current state of every account

```
1 CREATE OR REPLACE VIEW curated.accounts_current AS
2 SELECT * EXCEPT (rn) FROM (
3   SELECT *, ROW_NUMBER() OVER (PARTITION BY account_id
4               ORDER BY datastream_metadata.source_timestamp DESC,
5                        datastream_metadata.change_sequence_number DESC) AS rn
6   FROM raw.accounts_changes
7   WHERE datastream_metadata.change_type != 'UPDATE-DELETE')
8 WHERE rn = 1 AND datastream_metadata.change_type != 'DELETE';
```

3-5 Number each account's changes, newest first.

7-8 Skip before-images, keep the newest, and drop deleted accounts.

HONEST LIMIT

Check how your source's sequence numbers sort before trusting them as a tiebreaker.

Keys that BigQuery does not enforce

- Primary and foreign keys are declared **NOT ENFORCED**. Nothing is checked.
- They feed the optimiser: **join elimination** and better join order.
- If the data breaks them, queries can return **wrong results**.
- Deduplicate in the pipeline. The key describes the data, it does not police it.

> THE TRAP
>
> Declaring a primary key does not stop duplicates. The exam likes to test this.

Phrases that pick the answer

THE QUESTION SAYS	REACH FOR
Children **always read with** the parent	Nested and repeated fields
Joins **shuffle** and run slowly	Denormalise, or nest the child
What was it **at the time**?	Type 2, valid-from and valid-to
Keep **every change** from the source	Append-only, then a latest view
Migrate without breaking reports	Move as-is, then evolve behind views
Expect the key to **reject duplicates**	Not enforced: dedupe in the pipeline

THE PATTERN

Shape follows the question: what is read together, and what must be remembered.

> The most common billing query joins curated.bills to curated.bill_lines, billions of rows, and spends most of its time shuffling.
>
> Lines are always read together with their bill, and each bill has three to eight lines.

What redesign fits best?

A Cluster both tables on bill_id, and keep the join as it is

B Nest the lines inside each bill, as a repeated STRUCT

C Normalise further, moving line types into a table of their own

D Move the bill lines into Bigtable, with bill_id as the row key

◆ PAUSE AND DECIDE

What redesign fits best?

A Cluster and keep the join

Helps a little, but the join still shuffles billions of rows.

B Nest the lines

No join, no shuffle: lines sit inside their bill.

C Normalise further

Adds another join to a query already slowed by joins.

D Bigtable

An operational store, not for analytical SQL.

THE SIGNAL

Children always read with the parent, few per parent: nest them.

> The complaints team must show which tariff an account was on when each bill was issued, going back years.
>
> Accounts change tariff about once a year. Today, the accounts table simply overwrites tariff_id.

What should the team change?

A. A Type 2 tariff history, with valid-from and valid-to dates

B. Keep overwriting, and use time travel to read older tariff values

C. Add a previous_tariff_id column to the accounts table

D. Copy tariff_id onto every half-hourly reading as it is loaded

◆ PAUSE AND DECIDE

What should the team change?

A Type 2 history

Every version kept, joined by date to each bill.

B Time travel

Seven days at most, not years.

C Previous-value column

Type 3 looks back one change only.

D Copy onto readings

Wrong table, billions of copies, and no history for the past.

THE SIGNAL

What was it at the time, years back: a Type 2 dimension.

> An engineer declared PRIMARY KEY (account_id) NOT ENFORCED on curated.accounts, expecting it to block duplicates from the CDC feed.
>
> A week later, duplicate accounts appear, and some join results are wrong.

What should the team do?

A. Recreate the key as ENFORCED, so that BigQuery rejects duplicates

B. Deduplicate upstream, as the key only guides the optimiser

C. Drop the key, because primary keys slow down BigQuery joins

D. Cluster the table on account_id, so duplicate rows merge on write

◆ PAUSE AND DECIDE

What should the team do?

A Enforced key

BigQuery does not offer enforced primary keys.

B Deduplicate upstream

The key is a promise to the optimiser. The pipeline must keep it.

C Drop the key

Keys speed up joins when the data honours them.

D Cluster on the key

Clustering sorts rows. It never merges them.

THE SIGNAL

Keys are not enforced: deduplicate in the pipeline, or results can be wrong.

> Larkspur must move 50 Oracle billing tables to BigQuery before the data-centre lease ends.
>
> Existing reports must keep working, and the team wants a nested model in the long run.

What approach should the team take?

A. Move the schema as it is, then evolve it behind views

B. Redesign every table as nested before any data is moved

C. Flatten all fifty tables into one wide table during the move

D. Keep Oracle for reports, and copy only aggregates to BigQuery

◆ PAUSE AND DECIDE

What approach should the team take?

A Move, then evolve

Reports keep working. Views hide each later change.

B Redesign first

Breaks reports and pipelines, and risks the deadline.

C One wide table

Huge repetition, and every report must be rewritten.

D Keep Oracle

The data centre is closing, so Oracle cannot stay.

THE SIGNAL

A deadline plus reports that must keep working: move as-is, then evolve behind views.

Module 24 in five lines

- Storage is cheap, shuffles are not. Model for the **commonest query**.
- **Nest** children that are always read with their parent, and few per parent.
- **Type 2** rows with valid-from and valid-to answer 'what was it then?'.
- **Append** every change, and resolve the current state with a view.
- Keys are **not enforced**. Move schemas as-is first, then evolve.

COMING UP

Next: module 25, lakes and lakehouses.

MODULE 25 · SECTION 3 - STORING THE DATA

Lakes and Lakehouses

Lake zones in Cloud Storage, file formats, Lakehouse tables with metadata caching and fine-grained security, Iceberg, object tables and ObjectRef, monitoring the lake.

SECTION 3 of 5 EXAM WEIGHT 20% MODULE 25 of 45

admin@sthithapragna.com

A lake is just files until something governs them. **Lakehouse tables give files the rules of a warehouse.**

Load it, or leave it in the lake?

LOAD INTO BIGQUERY

Native tables

- Fastest queries and every feature
- Streaming, DML, time travel
- BigQuery manages the storage
- Other engines read through APIs

LEAVE IN THE LAKE

Open files

- One copy shared by Spark and BigQuery
- Open formats, no lock-in
- You manage files and layout
- Lakehouse tables add governance

THE TIEBREAKER

Only BigQuery uses it: load it. Spark and BigQuery share it: keep one open copy in the lake.

Larkspur's lake, zone by zone

01

Landing

Files as delivered. Short-lived, soft delete off.

02

Raw

Immutable history. Retention, colder classes.

03

Curated

Clean Parquet or Iceberg. Partitioned, compacted.

04

Consumption

Lakehouse tables and marts that people query.

THE PATTERN

One bucket per zone, so permissions, lifecycle and retention are set zone by zone.

Formats, file sizes and folders

- **Parquet** or **ORC** for analysis: columnar, compressed, read only the columns asked for.
- **Avro** for loading and exchange. CSV and JSON belong in landing only.
- Millions of **small files** slow everything. Compact them into larger files.
- Hive layout: **key=value** folders, such as run_date=2026-10-09, in a fixed order.

FOR LARKSPUR

Google notes that listing millions of files can take several minutes. Larkspur's history had 2.9 million.

External table or Lakehouse table?

STANDARD EXTERNAL TABLE

For a quick look

- Users need access to the bucket too
- No row or column security
- Lists files at query time
- Fine for ad hoc discovery

LAKEHOUSE TABLE

For governed access

- A connection's service account reads the files
- Users need access to the table only
- Row, column and masking policies
- Metadata caching speeds planning

THE TIEBREAKER

Governance needed: a Lakehouse table. A quick ad hoc look with no governance: an external table.

Settlement runs, as a Lakehouse table

```
1  CREATE EXTERNAL TABLE curated_lake.settlement_runs
2  WITH PARTITION COLUMNS (run_date DATE)
3  WITH CONNECTION `lk-warehouse-prod.europe-west2.lk-lake-conn`
4  OPTIONS (
5    format = 'PARQUET',
6    uris = ['gs://lk-lake-prod/settlement/*'],
7    hive_partition_uri_prefix = 'gs://lk-lake-prod/settlement',
8    require_hive_partition_filter = TRUE,
9    max_staleness = INTERVAL 4 HOUR, metadata_cache_mode = 'MANUAL');
10
11 CALL BQ.REFRESH_EXTERNAL_METADATA_CACHE('curated_lake.settlement_runs');
```

3 The connection makes this a Lakehouse table. Its service account reads the files.

9-11 Cache the file list, and refresh it as soon as the nightly job finishes.

FOR LARKSPUR

The Spark job finishes by 02:00, so a manual refresh fits better than a timer.

Caching what the lake looks like

- Caches file names, partitions and row counts, plus **statistics** for Parquet.
- Staleness from **30 minutes to 7 days**. Older than that, queries list files again.
- **Automatic** refresh every 30 to 60 minutes, or **manual** on your schedule.
- Refreshes use query capacity. Give them a **BACKGROUND** reservation.

> THE TIEBREAKER
>
> Files land at a known time: manual, after the job. Files land at random: automatic.

Row and column rules on files

- **Row access policies** and **policy tags** work on Lakehouse tables.
- **Dynamic masking** works on Lakehouse tables over Cloud Storage.
- Spark, Trino and Presto read through the **Storage Read API**, which enforces them.
- Anyone with direct **bucket access** can still read every file.

> THE TRAP
>
> Granting analysts read access on the bucket lets them walk round every policy. Grant the table only.

Iceberg in the lakehouse, briefly

- **Iceberg managed tables**: BigQuery writes, with DML and streaming, into your bucket.
- Spark-written Iceberg registers in the **Lakehouse runtime catalog**.
- Iceberg tracks files in its own metadata, so no folder listing is needed.
- Module 09 covered who writes which kind. The rule still holds.

> REMEMBER
>
> Needs updates and streaming on one open copy: an Iceberg managed table.

Object tables or ObjectRef?

OBJECT TABLE

A table of files

- One row per object: uri, type, size
- Read-only, through a connection
- Feeds AI functions and Document AI
- Row policies decide which files

OBJECTREF

A file inside a row

- A STRUCT column in a normal table
- Links a record to its file
- Mixes structured and unstructured
- Build it from an object table's ref

FOR LARKSPUR

The eighteen thousand pages of manuals become an object table, ready for RAG in module 31.

Monitoring and cost

- Find cache refresh jobs in **INFORMATION_SCHEMA.JOBS**, and check cache use per query.
- **Storage Insights datasets** put bucket and object metadata into BigQuery, daily.
- On-demand queries over files bill **bytes read**. Parquet and partition filters cut it.
- **Lifecycle rules** per zone keep landing clean and raw cold.

> HONEST LIMIT
>
> Storage Insights needs a Storage Intelligence subscription. Check before you rely on it.

Phrases that pick the answer

THE QUESTION SAYS	REACH FOR
Spark and BigQuery share **one copy**	Open files in the lake, Lakehouse tables
Row security on files, **no bucket access**	Lakehouse table with a connection
Queries spend minutes **listing files**	Metadata caching, and compaction
Files land at a **known time**	Manual cache refresh after the job
Analyse **PDFs or images** from SQL	Object table and AI functions
Updates and streaming on open files	Iceberg managed table

THE PATTERN

Lake questions turn on three words: share, govern, or speed up.

> Westgrid analysts query Larkspur's settlement Parquet files through a standard external table, and hold read access on the bucket.
>
> They must now see only rows for their own grid region, without a second copy of the data.

What should the team do?

A Keep the external table, and give Westgrid a view filtered to its region

B Copy Westgrid's rows into a separate bucket for them every night

C Put the bucket inside a VPC Service Controls perimeter

D A Lakehouse table with a row access policy, and remove bucket access

◆ PAUSE AND DECIDE

What should the team do?

A A filtered view

Bucket access still lets them read every file directly.

B A nightly copy

A second copy, which the brief rules out.

C A perimeter

Controls where requests come from, not which rows are seen.

D Lakehouse table plus policy

The connection reads files. Rows are filtered per user.

THE SIGNAL

Row-level rules on files, and no second copy: a Lakehouse table, with bucket access removed.

> A Lakehouse table covers two million Parquet files. Queries spend minutes before reading any data.
>
> A Spark job rewrites the files once a night, finishing by 02:00, and the files do not change during the day.

What should the team do?

A. Enable automatic metadata caching, with a 30-minute staleness

B. Convert the files to CSV, which BigQuery can list more quickly

C. Enable manual metadata caching, refreshed right after the job

D. Buy more slots, so that listing the files finishes faster

◆ PAUSE AND DECIDE

What should the team do?

A Automatic, 30 minutes

Refreshes all day for files that never change, and can still go stale.

B CSV

Larger files, no column pruning, and just as many to list.

C Manual, after the job

One refresh a night, exactly when the files change.

D More slots

Listing files is not the part that more slots fix.

THE SIGNAL

Minutes spent listing, files that change at a known time: manual metadata caching.

> Larkspur keeps 18,000 pages of engineering manuals as PDFs in gs://lk-manuals.
>
> Data scientists want to extract the text with Document AI from SQL, without copying the files anywhere.

What should the team set up?

A. Load each PDF into a BYTES column of a standard BigQuery table

B. A Dataflow job that copies each PDF into a Bigtable row

C. An external table over the bucket, with its format set to PDF

D. An object table over the bucket, then ML.PROCESS_DOCUMENT

◆ PAUSE AND DECIDE

What should the team set up?

A BYTES column

Copies every file, which the brief rules out.

B Bigtable

Another copy, in a store with no SQL route to Document AI.

C External table as PDF

PDF is not an external table format.

D Object table

Files stay put. SQL sends them to Document AI.

THE SIGNAL

Unstructured files, analysed from SQL, without copying: an object table.

> The settlement team's Spark jobs and the BigQuery analysts both use the curated settlement data.
>
> It must exist once, in an open format, and BigQuery must be able to update rows with DML and stream new rows in.

Which storage choice fits best?

A Native BigQuery tables, with Parquet copies exported for Spark

B Standard external tables over Parquet files that Spark writes

C An Iceberg managed table, written by BigQuery, read by Spark

D Keep the Hive tables on Hadoop, and query them through Omni

◆ PAUSE AND DECIDE

Which storage choice fits best?

A Native plus exports

Two copies, and the exports are always behind.

B External tables

Read-only from BigQuery: no DML, no streaming.

C Iceberg managed table

One open copy, with DML and streaming from BigQuery.

D Hadoop and Omni

Omni reaches other clouds, not a closing data centre.

THE SIGNAL

One open copy that BigQuery updates and streams into: an Iceberg managed table.

Module 25 in five lines

- Zones in **separate buckets**: landing, raw, curated, consumption.
- **Parquet** in Hive folders, and compact the small files.
- **Lakehouse tables** read through a connection, with row and column rules.
- **Metadata caching**: manual after a known job, automatic otherwise.
- **Object tables** for files, **Iceberg managed tables** for updates.

COMING UP

Next: module 26, Knowledge Catalog and federated governance.

MODULE 26 · SECTION 3 - STORING THE DATA

Knowledge Catalog and Federated Governance

The renames, the metadata model, glossary, discovery, profiling and quality, lineage, data products, lakes and zones, and data mesh with federated governance.

SECTION 3 of 5 EXAM WEIGHT 20% MODULE 26 of 45

admin@sthithapragna.com

Find it, trace it, trust it. Then let each team own it, under shared rules.

One product, several names

- Dataplex became Dataplex Universal Catalog, now **Knowledge Catalog**, since 10 April 2026.
- The API, the CLI and IAM names are **unchanged**. It is still gcloud dataplex.
- **Data Catalog** is shut down, in phases from 1 June 2026. Knowledge Catalog replaces it.
- Exam questions may say Dataplex or Data Catalog. Read both as Knowledge Catalog.

REMEMBER

Existing Dataplex lakes, scans and settings keep working. Nothing needed migrating.

One catalog, six jobs

CAPABILITY	THE QUESTION IT ANSWERS
Catalog and search	What data do we have, and where is it?
Aspects and glossary	What does it mean, and who owns it?
Discovery	Which new files can become tables?
Profiling and quality	What does it look like, and is it right?
Lineage	Where did it come from, and what depends on it?
Data products	What is packaged, promised and ready to use?

THE PATTERN

Technical metadata is collected automatically. Business meaning is what people must add.

Entries, aspect types and aspects

- Every table, bucket or model has an **entry**, created automatically for Google Cloud sources.
- An **aspect type** is a template. An **aspect** is that template filled in on an entry or a column.
- **Required** aspects come with the entry type. Optional ones are added by people.
- System aspects include **overview**, **contacts** and **schema**. Scores land as aspects too.

> CURRENCY CHECK
>
> Gemini in BigQuery does not read custom aspects. Put meaning in descriptions, the overview aspect or glossary terms.

Aspect or glossary term?

ASPECT

Structured facts

- Fields from a template
- Owner, domain, retention class
- Same shape on every entry
- Searchable and filterable

GLOSSARY TERM

Shared meaning

- A business definition in words
- Such as 'active account'
- Linked to columns in many tables
- Synonyms and related terms

THE TIEBREAKER

'Who owns this, and how long do we keep it?' is an aspect. 'What does active mean here?' is a glossary term.

Turning new files into tables

- A **discovery scan** groups files in a bucket into tables, with schemas and partitions.
- It publishes a **BigQuery dataset** named after the bucket.
- With a **connection ID**: Lakehouse tables. Without one: plain external tables.
- Not for **Iceberg** or **Delta Lake** tables. Images and PDFs become object tables.

> THE TRAP
>
> Set the label to user_managed when you own a schema, or the next scan may change it.

Profiling and quality, in the catalog

- **Profiling scans** measure nulls, distinct values, ranges and top values.
- Profiles **recommend quality rules**, so nobody starts from a blank page.
- **Quality scans** publish a scorecard on the entry, and log results for alerts.
- Sensitive Data Protection results can land as aspects too, as in module 04.

> REMEMBER
>
> Module 06 compared quality scans with Dataform assertions. Scans watch data at rest. Assertions guard a pipeline.

Where it came from, and what depends on it

- Turn on the **Data Lineage API** per project, and supported services report by themselves.
- BigQuery copy, load and query jobs, within **24 hours**. Also Composer, Dataflow, Vertex AI.
- **Column-level** lineage for top-level BigQuery columns. Custom tools send **OpenLineage**.
- Lineage is kept for **30 days**, and it is billed.

> THE TRAP
>
> Thirty days of lineage is not a seven-year audit trail. Keep long-term evidence somewhere else.

Impact analysis before a change

01

Oracle billing

Datastream captures every change.

02

raw.accounts_changes

Append-only history, from module 24.

03

curated.accounts_current

The latest-state view.

04

Marts and dashboards

Everything that would break.

FOR LARKSPUR

Before renaming a column, open its lineage graph. Every arrow out is a consumer to warn.

Packaging data for others

- A **data product** packages assets for reuse: discoverable, trusted and accessible.
- It bundles documentation, aspects, and **access groups** mapped to Google Groups.
- A **contract** sets promises, such as refresh schedule and quality standards.
- Consumers, including **AI agents**, find it in search and request access from the owner.

FOR LARKSPUR

Westgrid's regional demand data becomes a data product, with a five-minute freshness promise in its contract.

Lakes, zones and assets

ENTITY	WHAT IT IS	FOR LARKSPUR
Lake	A domain, in one region	Metering, in europe-west2
Raw zone	Data in any format, from outside	Head-end files, as delivered
Curated zone	Structured, required formats, Hive layout	Clean Parquet readings
Asset	A bucket or a BigQuery dataset	gs://lk-lake-prod

HONEST LIMIT

Newer docs lead with the catalog and data products, but exam questions still ask about lakes and zones.

Who owns what, in a federated model?

EACH DOMAIN OWNS

Data as a product

- Its pipelines and its tables
- Publishing data products
- Quality against the contract
- Approving access requests

THE CENTRAL TEAM SETS

Shared rules

- Aspect types and the glossary
- Policies, locations and tags
- Quality thresholds to meet
- The self-serve platform itself

THE PATTERN

Domains own the data. The centre owns the standards. Neither owns both.

Phrases that pick the answer

THE QUESTION SAYS	REACH FOR
One definition of a business term	Glossary term, linked to columns
Owner and retention on **every table**	A required aspect type
New files, **no hand-written schemas**	A discovery scan, with a connection
What breaks if this column changes?	Column-level lineage
Domains **own**, standards stay **shared**	Data mesh, federated governance
A question mentions **Data Catalog**	Read it as Knowledge Catalog

THE PATTERN

Find, trace, trust, own: match the question to one of the four.

> Three Larkspur teams calculate 'active customer' in three different ways, and analysts cannot tell which tables use which.
>
> The data office wants one agreed definition, linked to the relevant columns across datasets, and findable in search.

What should the team use?

A. A glossary term in Knowledge Catalog, linked to those columns

B. A shared spreadsheet of definitions, linked from each dataset description

C. A label called active_customer, added to every table that uses it

D. One view per dataset that computes active customers the agreed way

◆ PAUSE AND DECIDE

What should the team use?

A Glossary term

One definition, linked to columns, and found in search.

B Spreadsheet

Not linked to columns, and not part of catalog search.

C Labels

A key and a value, with no definition behind them.

D A view per dataset

Computes a number, but does not define or link the term.

THE SIGNAL

One agreed business definition, linked to columns: a glossary term.

> Finance plans to change the meaning of amount_gbp in curated.bills, and needs to know which tables and dashboards depend on that column.
>
> Pipelines are BigQuery SQL run by Composer, and the Data Lineage API is enabled.

What should the team do?

A Search INFORMATION_SCHEMA.JOBS by hand for queries that mention the column

B Ask every team to list what depends on the column in a shared sheet

C Turn on Data Access audit logs, and search them for the column name

D Open column-level lineage for the column in Knowledge Catalog

◆ PAUSE AND DECIDE

What should the team do?

A Search job history

Slow, partial, and easy to miss a query.

B Ask every team

Relies on memory, and goes out of date at once.

C Audit logs

Shows who read data, not what was built from it.

D Column-level lineage

Draws every downstream table that uses that column.

THE SIGNAL

What depends on this column: column-level lineage.

> Larkspur now has four domain teams: metering, billing, customer and grid. One central team builds every pipeline, and has become a bottleneck.
>
> Leadership wants domains to own their data, while keeping consistent standards for metadata, quality and access.

Which operating model fits best?

A Keep one central team, and hire more engineers into it

B Domains publish data products, under standards set centrally

C Give each domain its own organisation, with its own rules

D Copy every domain's data into one central project each night

◆ PAUSE AND DECIDE

Which operating model fits best?

A Grow the central team

Still one bottleneck, just a larger one.

B Federated governance

Domains own the data. The centre owns the standards.

C Separate organisations

Ownership without shared standards.

D Central copy

Duplicates data, and leaves ownership unclear.

THE SIGNAL

Domains own, standards stay shared: a data mesh with federated governance.

> Westgrid drops Parquet and CSV files into gs://lk-landing-prod every day, and new folders appear often.
>
> Analysts want each new folder queryable in BigQuery as a governed table, without copying files or writing schemas by hand.

What should the team set up?

A Write a CREATE EXTERNAL TABLE statement for each new folder

B A Cloud Run function that loads each new file into a native table

C A repeating discovery scan, configured with a connection

D A one-off Storage Transfer Service job into a BigQuery dataset

◆ PAUSE AND DECIDE

What should the team set up?

A Hand-written DDL

Exactly the schema writing the brief rules out.

B Load with a function

Copies every file into BigQuery storage.

C Discovery scan

Finds new folders, infers schemas, publishes Lakehouse tables.

D One-off transfer

Moves files once, and does not create tables.

THE SIGNAL

New files, no hand-written schemas, governed and in place: a discovery scan with a connection.

Module 26 in five lines

- Dataplex and Data Catalog questions now mean **Knowledge Catalog**.
- **Aspects** hold structured facts. **Glossary terms** hold shared meaning.
- **Discovery** turns files into tables. Profiles and quality scans build trust.
- **Lineage** shows impact, column by column, for 30 days.
- Data mesh: domains own **data products**, the centre owns the **standards**.

COMING UP

Next: module 27, the start of section four: serving BI, tools and precomputation.

MODULE 27 · SECTION 4 - PREPARING AND USING DATA FOR ANALYSIS

Serving BI: Tools and Precomputation

Looker, Data Studio, Connected Sheets and third-party BI; the query cache, BI Engine, materialized views, scheduled queries and aggregate tables.

SECTION 4 of 5 EXAM WEIGHT 15% MODULE 27 of 45

admin@sthithapragna.com

Every dashboard click is a query. **Decide what to compute once, and what to compute on demand.**

Four ways to look at BigQuery

TOOL	BEST FOR	WHY
Looker	Governed reporting, such as the board	Metrics defined once, in a semantic model
Data Studio	Quick team dashboards	Self-service, fast to build, Pro for teams
Connected Sheets	Spreadsheet users, no SQL	Pivot billions of rows inside Sheets
Third-party BI	Tableau, Power BI and others	JDBC, ODBC or native connectors

CURRENCY CHECK

Looker Studio is called Data Studio again, since April 2026. Data Studio Pro is the paid tier.

Looker or Data Studio?

LOOKER

One governed definition

- Metrics defined once in LookML
- Version-controlled with Git
- Scheduled and embedded delivery
- Aggregate awareness uses summary tables

DATA STUDIO

Fast self-service

- Build a report in minutes
- Many data sources, little setup
- Pro adds team workspaces
- Each report holds its own logic

THE TIEBREAKER

One agreed definition of revenue for the board: Looker. A quick dashboard for a team: Data Studio.

BigQuery inside a spreadsheet

- Pivot tables, formulas and charts run over the **whole table**, as BigQuery queries.
- The sheet previews **500 rows**. Pivot results can reach **200,000 rows**.
- Extracts stop at **500,000 rows**. Over 50,000 rows, the cap is 5 million cells.
- Refreshes can be **scheduled**, and queries bill to the project you chose.

> THE TRAP
>
> Pulling millions of raw rows into Sheets fails. Summarise in BigQuery, and bring back the answer.

The ladder of precomputation

01

Query cache

Free. The same query again, within about a day.

02

BI Engine

In memory. Sub-second dashboard clicks.

03

Materialized views

BigQuery keeps aggregates fresh for you.

04

Scheduled tables

Any SQL, rebuilt on a timetable.

THE PATTERN

Climb only as far as you need. Each rung costs more effort, and covers more cases.

Free, but easy to miss

- Identical queries reuse results for **about 24 hours**, at no charge.
- Per user and project. Sharing across users needs **Enterprise** or Enterprise Plus.
- Missed when tables change, after streaming, or with a **destination table**.
- Missed with **CURRENT_TIMESTAMP**, row-level security, or even changed whitespace.

> THE TRAP
>
> A dashboard filter on CURRENT_DATE never hits the cache. Every click pays again.

Dashboards from memory

- A **reservation** of memory, per region, that caches the data dashboards use.
- Works for Looker, Data Studio, Tableau, Power BI, and any **JDBC or ODBC** tool.
- Choose **preferred tables**. Joins suit a fact table with up to four small dimensions.
- Not for external or Lakehouse tables, row-level security, JSON, or results over 1 GiB.

> THE PAYOFF
>
> No dashboard changes are needed. Queries are accelerated where they run.

Incremental or non-incremental?

INCREMENTAL

The default

- Aggregates such as SUM, COUNT, AVG, MIN, MAX
- Refresh reads only what changed
- Smart tuning rewrites matching queries
- Results reflect the latest base data

NON-INCREMENTAL

Allowed on request

- Most SQL, set with an option
- Every refresh recomputes everything
- Needs a max_staleness setting
- No automatic query rewrite

THE TIEBREAKER

Simple aggregates over a growing table: incremental. Complex SQL that can be a little stale: non-incremental.

Daily kWh for the customer app

```
1 CREATE MATERIALIZED VIEW marts.meter_daily_kwh
2 PARTITION BY DATE(reading_day)
3 CLUSTER BY meter_id
4 OPTIONS (enable_refresh = TRUE, refresh_interval_minutes = 60)
5 AS
6 SELECT TIMESTAMP_TRUNC(reading_ts, DAY) AS reading_day, meter_id,
7        SUM(kwh) AS kwh_total, COUNT(*) AS readings
8 FROM curated.readings_hh
9 GROUP BY reading_day, meter_id;
```

2 Partitioned to line up with the base table's reading-day partitions.

6-7 Simple aggregates, so the view refreshes incrementally.

REMEMBER

Queries still see readings that arrived after the last refresh. The view adds the changes on the fly.

Freshness, refresh and limits

- By default, results reflect the **latest base data**. Changes are read on the fly.
- **max_staleness**, from 30 minutes to 3 days, trades freshness for cost.
- Refresh starts within **5 minutes** of a change, no more than every **30 minutes** by default.
- Up to **100** per base table. No nesting. The query cannot be changed later.

> THE TRAP
>
> Questions that say a materialized view is always hours behind are usually wrong. Staleness is opt-in.

Materialized view or scheduled table?

MATERIALIZED VIEW

BigQuery maintains it

- Restricted SQL, mostly aggregates
- Incremental, always current
- Smart tuning finds it for you
- Nothing to schedule

SCHEDULED QUERY TABLE

You maintain it

- Any SQL, window functions, many joins
- As fresh as the last run
- Queries must name the table
- Dataform when steps depend on each other

THE TIEBREAKER

The SQL fits a materialized view: use one. Window functions or complex joins: a scheduled table.

Phrases that pick the answer

THE QUESTION SAYS	REACH FOR
One definition of a metric for every report	Looker and LookML
Spreadsheet users, **billions of rows**	Connected Sheets
Sub-second dashboards, **no changes**	BI Engine reservation
Simple aggregates, **always current**	Incremental materialized view
Window functions, **changes nightly**	Scheduled query to a table
Same query, **cache never used**	Look for non-deterministic functions

THE PATTERN

Match the audience to the tool, then the query to the rung.

> A board dashboard ranks regions by revenue, using window functions over three large joined tables.
>
> The data changes once a night, when billing closes, yet every dashboard view scans 2 TB.

What should the team do?

A. Add all three tables to a BI Engine reservation as preferred tables

B. Create an incremental materialized view of the whole dashboard query

C. A scheduled query after billing closes, writing a results table

D. Turn off the query cache, so that the ranking is always fully fresh

◆ PAUSE AND DECIDE

What should the team do?

A **BI Engine**

Window functions over large joined tables are outside its limits.

B **Incremental view**

Window functions are not supported incrementally.

C **Scheduled table**

Runs once a night, as the data changes. Clicks read a tiny table.

D **No cache**

Makes every click cost more, and the data only changes nightly.

THE SIGNAL

Complex SQL over data that changes nightly: a scheduled query writing a small table.

> The customer app shows each meter's daily kWh, thousands of times an hour, from the huge half-hourly readings table.
>
> Readings that arrived minutes ago must appear, and analysts run the same daily totals against the base table by hand.

What should the team build?

A. An incremental materialized view, with smart tuning for analysts

B. A scheduled query every five minutes, overwriting a daily totals table

C. A logical view over the readings table that computes the daily totals

D. A nightly export of daily totals to Cloud Storage, behind an external table

◆ PAUSE AND DECIDE

What should the team build?

A Incremental view

Current to the minute, and analysts' queries are rewritten to use it.

B Five-minute schedule

Full rebuilds all day, and analysts still scan the base table.

C Logical view

Saves nothing. Every request recomputes from the base table.

D Nightly export

A day out of date, and slower to query.

THE SIGNAL

Simple aggregates that must stay current, queried many ways: an incremental materialized view.

> Larkspur's finance team works entirely in spreadsheets, and wants to pivot two billion readings by month and tariff without writing SQL.
>
> Their CSV exports fail beyond a million rows.

What should the team give them?

A. Exported CSV files in Cloud Storage, opened in Sheets one by one

B. Connected Sheets, with a pivot table over the BigQuery table

C. A Data Studio report with a table listing every single row

D. A Looker model, with a LookML developer assigned to each analyst

◆ PAUSE AND DECIDE

What should the team give them?

A CSV exports

The same row limits that already fail.

B Connected Sheets

The pivot runs in BigQuery over all rows. Sheets shows the result.

C Data Studio table

A dashboard, not a spreadsheet, and no row-level pivots.

D Looker per analyst

Heavy and costly for people who want a spreadsheet.

THE SIGNAL

Spreadsheet users, billions of rows, no SQL: Connected Sheets.

> Team dashboards in Data Studio sit on a 50 GB star schema: one fact table and three small dimension tables.
>
> Each click takes four to six seconds. Users want sub-second response, without anyone rebuilding the dashboards.

What should the team do?

A Rewrite each dashboard query as a scheduled query to its own table

B Move the four tables into Cloud SQL, which answers small reads faster

C Extend the dataset's time travel window to the full seven days

D A BI Engine reservation, with the star schema as preferred tables

◆ PAUSE AND DECIDE

What should the team do?

A **Scheduled tables**
Means rebuilding every dashboard, which the brief rules out.

B **Cloud SQL**
A transactional database, and every dashboard must be repointed.

C **Time travel**
Recovery, not speed.

D **BI Engine**
In-memory, sub-second, and no dashboard changes.

THE SIGNAL

Sub-second dashboards on a star schema, with no rebuilds: BI Engine.

Module 27 in five lines

- **Looker** for governed metrics, **Data Studio** for quick dashboards.
- **Connected Sheets** pivots billions of rows for spreadsheet users.
- The **cache** is free for a day, but misses on any change or CURRENT_DATE.
- **BI Engine** for sub-second dashboards, with no changes to them.
- **Materialized views** for current aggregates. **Scheduled tables** for anything else.

COMING UP

Next: module 28, fixing slow and expensive queries.

MODULE 28 · SECTION 4 - PREPARING AND USING DATA FOR ANALYSIS

Fixing Slow and Expensive Queries

Finding the costly queries, reading the execution graph, bytes and guard rails, joins and skew, approximate aggregation, search indexes, and what BigQuery tunes for you.

SECTION 4 of 5 EXAM WEIGHT 15% MODULE 28 of 45

admin@sthithapragna.com

A slow query is rarely a mystery. It reads too much, moves too much, or does the same work twice.

From a big bill to a fix

01

Find it

INFORMATION_SCHEMA.JOBS, sorted by cost.

02

Read the graph

Which stage burns the slot time?

03

Cut the bytes

Columns, partition filters, guard rails.

04

Fix the shape

Joins, skew, repeated work.

05

Precompute

The ladder from module 27.

THE PATTERN

Measure before you tune. The most expensive query is often not the one people complain about.

The twenty costliest queries this week

```sql
SELECT user_email, job_id,
       total_bytes_billed / POW(1024, 4) AS tib_billed,
       total_slot_ms / 1000 / 3600      AS slot_hours,
       LEFT(query, 80)                  AS query_start
FROM `region-europe-west2`.INFORMATION_SCHEMA.JOBS
WHERE creation_time >= TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 7 DAY)
  AND job_type = 'QUERY' AND state = 'DONE'
ORDER BY total_bytes_billed DESC
LIMIT 20;
```

2-3 Bytes billed drive on-demand cost. Slot hours drive capacity cost.

5 The view is regional. Name the region, or the query finds nothing.

REMEMBER

Swap JOBS for JOBS_BY_ORGANIZATION to see every project, if you hold the right role.

Reading the execution graph

- A query runs as **stages**, linked by **shuffle**. Darker nodes used more slot time.
- Each stage shows **wait, read, compute and write**, as average and maximum.
- A maximum far above the average means **skew**: a few workers did most of the work.
- Shuffle **spilled to disk** means too much data is moving between stages.

> HONEST LIMIT
>
> Dry runs and cached results have no execution graph. You need a real run to diagnose.

What the stage is telling you

YOU SEE	IT USUALLY MEANS	TRY
High **wait**	Not enough slots right now	Reservation, or another time
High **read**	Too many bytes in	Fewer columns, real partition filters
Compute **max far above avg**	Skewed key	Split or filter the hot key
Shuffle **spilled**	Huge join or grouping	Filter and aggregate first
High **write**	Huge output	Aggregate before returning

THE PATTERN

Fix the darkest stage first. Tuning a cheap stage changes nothing.

Bytes read are bytes billed

- **SELECT *** reads every column. Storage is columnar, so name only the columns you need.
- **LIMIT** does not cut the bytes read on a table that is not clustered.
- **Table preview** and bq head are free. Use them to look at data.
- Partition filters must really **prune**: no subqueries, as module 23 showed.

> FOR LARKSPUR
>
> Larkspur's analysts love SELECT star with LIMIT 10. It still reads every column of every row.

Guard rails before the bill

- A **dry run** is free, and estimates bytes. The estimate is an upper bound.
- **Maximum bytes billed** fails a query before it runs, at no charge.
- **Custom quotas** cap bytes per day, per user or per project.
- **Require partition filter** refuses queries with no usable date filter.

> THE TIEBREAKER
>
> Stop one runaway query: maximum bytes billed. Cap a user's whole day: a custom quota.

Joins: do this, avoid that

DO

Join less data

- Filter and aggregate before the join
- Largest table first, then smaller
- Small tables broadcast to every slot
- INT64 keys beat STRING keys

AVOID

Joins that explode

- Self-joins: use window functions
- Joins that output more rows than in
- ORDER BY in the middle of a query
- A WITH clause referenced many times

THE TRAP

A WITH clause may be evaluated each time it is referenced. Reused results belong in a temporary table.

A self-join, rewritten

```sql
-- Before: a self-join to find the previous half hour
SELECT a.meter_id, a.reading_ts, a.kwh - b.kwh AS change_kwh
FROM curated.readings_hh AS a JOIN curated.readings_hh AS b
  ON  b.meter_id = a.meter_id
  AND b.reading_ts = TIMESTAMP_SUB(a.reading_ts, INTERVAL 30 MINUTE)
WHERE DATE(a.reading_ts) = '2026-10-09';

-- After: one pass, with a window function
SELECT meter_id, reading_ts,
       kwh - LAG(kwh) OVER (PARTITION BY meter_id ORDER BY reading_ts) AS change_kwh
FROM curated.readings_hh
WHERE DATE(reading_ts) = '2026-10-09';
```

3 The table joined to itself: read twice, with a large shuffle.

10 LAG reads the previous row in the same pass. No join at all.

HONEST LIMIT

LAG takes the previous row, even if a reading is missing. Check for gaps if that matters.

Skew: one key, one overworked slot

- Rows with the same key go to the same worker. One huge key means **one slow worker**.
- Common culprits: **NULL**, placeholder values, or one giant customer.
- Filter placeholders out **before** the join, or handle them separately.
- Pre-aggregate the hot key, or spread it with an extra key part.

> FOR LARKSPUR
>
> Rejected readings carry meter_id 'UNKNOWN'. Joining them to meter details puts a third of the rows on one worker.

When close enough is cheaper

- **APPROX_COUNT_DISTINCT** uses HyperLogLog++, with far less memory and a small error.
- **APPROX_QUANTILES** and **APPROX_TOP_COUNT** do the same for percentiles and top lists.
- **HLL_COUNT** sketches can be stored daily, then **merged** into weeks and months.
- Bills and settlement need exact answers. Use COUNT DISTINCT there.

> THE TRAP
>
> Distinct counts cannot be added up across days. Sketches can be merged. That is their whole point.

Finding a needle in a haystack

- A **search index** speeds up point lookups in big STRING and JSON columns.
- Used by **SEARCH**, and also by =, IN and LIKE on indexed columns.
- BigQuery builds and maintains it. Index **storage** is billed.
- Not for fuzzy matching, views, external tables or time travel queries.

> THE TIEBREAKER
>
> A few rows from billions: a search index. A range of rows on a sort key: clustering.

What BigQuery tunes for you

- **History-based optimizations** learn from earlier runs of similar queries.
- Examples: semi-join reduction, join commutation, parallelism adjustment.
- Applied only with high confidence, and **revoked** if they do not help.
- See them in the jobs view, under optimization details.

> REMEMBER
>
> Automatic tuning helps, but it never fixes SELECT star or a missing partition filter.

Phrases that pick the answer

THE QUESTION SAYS	REACH FOR
Which queries cost the **most**?	INFORMATION_SCHEMA.JOBS, by bytes or slots
LIMIT did not lower the bill	Fewer columns, or table preview
Fail before scanning **too much**	Maximum bytes billed
Compute **max far above average**	Skew: fix the hot key
Distinct users, **day to month**	HLL_COUNT sketches, merged
One value among billions of rows	Search index and SEARCH

THE PATTERN

Too much read, too much moved, or work done twice: name which, then fix it.

> Each morning an analyst runs SELECT * with LIMIT 100 on a 30 TB unpartitioned history table, just to look at some rows.
>
> Every run is billed for the whole table.

What should the analyst do instead?

A Add ORDER BY to the query, so that LIMIT can stop reading early

B Use the free table preview, or bq head, to look at the rows

C Dry run the query first each morning, then run it for real

D Raise the project's custom quota, so the query keeps running

◆ PAUSE AND DECIDE

What should the analyst do instead?

A ORDER BY

Sorting needs every row first, so it reads even more.

B Table preview

Free, and touches no quota.

C Dry run first

Shows the cost, then pays it anyway.

D Raise the quota

Makes the waste easier, not smaller.

THE SIGNAL

Just looking at rows: preview, which is free. LIMIT does not cut bytes.

> A join of rejected readings to meter details runs for 40 minutes. In the execution graph, the join stage's maximum compute is 40 times its average.
>
> A third of the rejected readings carry the placeholder meter_id 'UNKNOWN'.

What should the team change?

A Handle placeholder rows separately, and join only real meter IDs

B Buy more slots, so that the join stage has more workers to use

C Add LIMIT to the join, so that fewer rows reach the slow stage

D Convert meter_id to a longer string, so the keys spread out better

◆ PAUSE AND DECIDE

What should the team change?

A Handle the hot key

Removes the skew, so the work spreads evenly.

B More slots

The hot key still lands on one worker.

C LIMIT

Changes the answer, and not the skew.

D Longer strings

The same key still hashes to one place.

THE SIGNAL

Max far above average, one dominant key: skew. Deal with the hot key.

> Finance wants any single analyst query that would scan more than 1 TB to fail before it runs, at no cost.
>
> Normal smaller queries must carry on as usual.

What should the team use?

A. A custom daily query quota of 1 TB for each analyst

B. Require partition filter on every table in the warehouse

C. Email analysts a reminder to dry run large queries first

D. Maximum bytes billed of 1 TB on the analysts' queries

◆ PAUSE AND DECIDE

What should the team use?

A **Daily quota**
Caps the whole day, not one query.

B **Require partition filter**
Blocks unfiltered queries, not large filtered ones.

C **Reminder emails**
Relies on memory, and enforces nothing.

D **Maximum bytes billed**
Each query over the limit fails first, at no charge.

THE SIGNAL

Stop one query before it runs, at no cost: maximum bytes billed.

> A dashboard shows distinct app users per day and per month, using COUNT(DISTINCT) over billions of events. It is slow.
>
> About 1% error is acceptable, and monthly figures should not rescan every raw event.

What should the team build?

A HLL_COUNT sketches stored per day, then merged for each month

B COUNT(DISTINCT) per day, with the daily counts added up for the month

C APPROX_COUNT_DISTINCT over the raw events for the whole month

D A Connected Sheet that counts the distinct users with a formula

◆ PAUSE AND DECIDE

What should the team build?

A Daily sketches, merged

Approximate, cheap, and months merge from days.

B Add daily counts

Counts a user once for every day they appear.

C Approximate over raw

Faster, but it rescans the whole month.

D Connected Sheets

Still runs the same heavy query underneath.

THE SIGNAL

Approximate distinct counts that roll up: HLL_COUNT sketches.

> Field support searches a 20 TB table of device logs, stored as JSON, for one device serial number, several times an hour.
>
> Each search scans the whole table, and returns a handful of rows.

What should the team add?

A Clustering on the time each log line was ingested

B Integer range partitioning on the device serial number

C A search index on the log column, queried with SEARCH

D A materialized view listing every serial number in the table

◆ PAUSE AND DECIDE

What should the team add?

A Cluster on ingest time

Helps time ranges, not one serial.

B Partition on serial

Serials are strings, and far beyond partition limits.

C Search index

Built for point lookups in big text and JSON.

D Materialized view

Still scans the logs to find the matching rows.

THE SIGNAL

A handful of rows from billions, in text or JSON: a search index.

> A long report query defines a costly WITH clause that aggregates 10 TB, then references it four times.
>
> The execution graph shows the same aggregation running four times.

What should the team change?

A Move the WITH clause to the end of the query text instead

B Write the aggregate to a temporary table once, then reuse it

C Add LIMIT inside the WITH clause, so it returns fewer rows

D Turn the WITH clause into a logical view, and query the view

◆ PAUSE AND DECIDE

What should the team change?

A **Move the clause**
Its position changes nothing about evaluation.

B **Temporary table**
Computed once, then read cheaply four times.

C **LIMIT inside**
Changes the results, and still reads ten terabytes.

D **Logical view**
A view is just saved SQL, evaluated each time too.

THE SIGNAL

The same work done several times: materialise it once.

Module 28 in five lines

- Find the cost in **INFORMATION_SCHEMA.JOBS**, then read the execution graph.
- **SELECT *** and LIMIT still read everything. Preview is free.
- **Maximum bytes billed** stops one query. Custom quotas cap a day.
- Max far above average means **skew**. Window functions beat self-joins.
- **Sketches** for distinct counts that roll up, **search indexes** for needles.

COMING UP

Next: module 29, securing analytic data in BigQuery.

MODULE 29 · SECTION 4 - PREPARING AND USING DATA FOR ANALYSIS

Securing Analytic Data in BigQuery

Authorised views, datasets and routines; row access policies; column-level security with policy tags; dynamic data masking; IAM conditions; and where Sensitive Data Protection fits.

SECTION 4 of 5 EXAM WEIGHT 15% MODULE 29 of 45

admin@sthithapragna.com

Share the answer, not the table. Then choose which rows and columns each person sees.

Six controls, six questions

CONTROL	THE QUESTION IT ANSWERS
IAM on datasets and tables	Who can open this at all?
Authorised views, datasets, routines	Can they see results without the source?
Row access policies	Which rows may this group see?
Policy tags	Which columns may they read?
Data masking	What does a protected column show them?
IAM conditions	Only until when, or only on which resources?

THE PATTERN

Most questions name one of these six questions. Find it, and the feature follows.

Share a view, not its source

- The view lives in a **separate dataset**, and is authorised to read the source dataset.
- Users need Data Viewer on the **view's dataset**, and Job User on a project.
- Source and view datasets must be in the **same location**.
- Up to **2,500** authorised resources per dataset. Materialized views can be authorised too.

> THE TRAP
>
> Granting users access to the source dataset as well defeats the whole point.

One view at a time, or a whole dataset?

AUTHORISED VIEW

One by one

- Authorise each view separately
- Fine for a handful of views
- Each new view needs its own grant

AUTHORISED DATASET

All views in it

- Every current and future view is authorised
- Recommended for new and multi-tenant designs
- Creating views there needs extra rights

THE TIEBREAKER

Many views, with more on the way: an authorised dataset. A single shared view: an authorised view.

Functions that read on your behalf

- Table functions, UDFs and stored procedures can be **authorised** on a dataset.
- Callers need permission to **run** the routine, not to read its tables.
- A **table function** with a parameter returns just the slice asked for.
- Authorised **stored procedures** get DDL and DML access. Trust callers fully.

> HONEST LIMIT
>
> CREATE OR REPLACE on an authorised routine removes its authorisation. Authorise it again.

One account's bills, for the app

```
1 CREATE OR REPLACE TABLE FUNCTION app_api.account_bills(acct STRING)
2 AS
3 SELECT bill_id, period_start, amount_gbp
4 FROM curated.bills
5 WHERE account_id = acct;
6
7 -- Then authorise app_api.account_bills on the curated dataset,
8 -- and grant the app's service account access to app_api only.
```

1 The function lives in its own dataset, app_api.

7-8 The app can call it, but cannot query curated.bills directly.

FOR LARKSPUR

The function returns three columns. Nothing else about the account ever leaves.

Row access policies

- **CREATE ROW ACCESS POLICY** names a table, who it grants, and a filter.
- **SESSION_USER()** in the filter shows each person only their own rows.
- Once a table has a policy, anyone in **no** grantee list sees no rows.
- Previews, BI Engine acceleration and partition pruning by the filter do not apply.

> THE TRAP
>
> Users who should still see everything need their own policy, with a TRUE filter.

Protecting vulnerable customers

```
1  CREATE ROW ACCESS POLICY all_but_psr ON curated.customers
2  GRANT TO ('group:billing-analysts@larkspur.energy')
3  FILTER USING (psr_flag = FALSE);
4
5  CREATE ROW ACCESS POLICY psr_team ON curated.customers
6  GRANT TO ('group:psr-team@larkspur.energy')
7  FILTER USING (TRUE);
```

3 Billing analysts never see customers on the priority services register.

7 The PSR team sees every row. Without this policy, they would see none.

REMEMBER

A person in both groups sees the union of the two filters, here every row.

Policy tags on sensitive columns

- A **taxonomy** holds policy tags, up to five levels deep. One tag per column.
- Turn on **enforcement**, then grant **Fine-Grained Reader** to those who may read.
- Without it, **SELECT *** fails. SELECT * EXCEPT the tagged columns still works.
- Newer options: **data governance tags** in Preview, and data policies set on a column.

> THE TRAP
>
> Taxonomies are regional. The taxonomy must sit in the same location as the table.

What a masked column can show

RULE	SHOWS	JOINABLE?
Nullify or default value	NULL, or 0 and empty strings	No
SHA-256 hash	The same hash every time	Yes, across queries
Random hash	A hash salted per query	Only within one query
Email mask	The domain, not the name	No
First four or last four	Four characters, then XXXXX	No
Date year mask or custom	The year only, or your own routine	Custom, if deterministic

THE TRAP

SHA-256 is deterministic, so it can be guessed from known values. Random hash is safer.

Fine-Grained Reader or Masked Reader?

FINE-GRAINED READER

Sees real values

- Granted on the policy tag
- For the few who truly need it
- Such as the fraud investigators' lead

MASKED READER

Sees masked values

- Granted on the data policy
- Existing queries keep working
- Neither role means no access at all

REMEMBER

When several masking rules apply, a fixed order picks one. A custom routine ranks highest, nullify lowest.

Access with conditions attached

- **request.time**: access that expires, such as an auditor until month end.
- **resource.name**: only datasets or tables with a given name or prefix.
- **Resource tags** on datasets, tables and views, such as environment equals prod.
- Not on authorised view, dataset or routine grants, and not for rows or columns.

> THE TRAP
>
> A loose prefix condition can grant far more than intended. Write conditions narrowly.

From sensitive data to safe sharing

01

Find

Sensitive Data Protection discovery profiles every table.

02

Classify

Policy tags on the sensitive columns.

03

Protect

Masking rules, row access policies.

04

Share

Authorised views, datasets and routines.

THE PATTERN

Discovery tells you where personal data is. The BigQuery controls decide who sees it.

Phrases that pick the answer

THE QUESTION SAYS	REACH FOR
Results, but **no access to the source**	Authorised view or dataset
Many views, **more coming**	Authorised dataset
One account's data, **by parameter**	Authorised table function
Each group sees **its own rows**	Row access policy
Hide a column, but **still join on it**	SHA-256 masking rule
Access that **expires** on a date	IAM condition on request.time

THE PATTERN

Rows, columns, values, or time: name the dimension, then pick the feature.

> Westgrid's analysts need always-current regional demand totals computed from curated.readings_hh.
>
> They must never see meter-level rows, and must not be granted any access to the curated dataset.

What should the team set up?

A Data Viewer on the curated dataset, plus a row access policy

B A nightly copy of the regional totals into a separate table

C Data Viewer on the curated dataset, with an IAM condition on one table

D An authorised view that aggregates by region, in its own dataset

◆ PAUSE AND DECIDE

What should the team set up?

A Viewer plus row policy

Access to curated, and meter rows still visible.

B Nightly copy

Not always current, and one more pipeline.

C Viewer with a condition

Still grants access to curated, at meter level.

D Authorised view

Aggregates only, current, and no access to curated.

THE SIGNAL

Results without access to the source: an authorised view.

> Larkspur's fraud team must not see customers' email addresses. But they must compare them with hashed emails saved from earlier investigations.
>
> Their queries run on different days, against different tables.

Which masking rule should their data policy use?

A Nullify, so that the email column always comes back as NULL

B SHA-256 hash, so one email gives the same value every time

C Email mask, so that only the domain of each address is shown

D Random hash, so that each query uses its own fresh salt

◆ PAUSE AND DECIDE

Which masking rule should their data policy use?

A **Nullify**
Nothing left to compare.

B **SHA-256 hash**
Deterministic, so values match across queries and days.

C **Email mask**
Domains match across many different people.

D **Random hash**
Salt changes per query, so saved hashes never match.

THE SIGNAL

Hidden, but matchable across queries: a SHA-256 masking rule.

> The team added a row access policy on curated.customers so billing analysts see only rows where psr_flag is FALSE.
>
> Next morning, the data science team, who used to see every row, see no rows at all.

What should the team do?

A Grant the data science team Fine-Grained Reader on the table

B Drop the policy, and give billing analysts a filtered view instead

C Add a row access policy for data science, with a TRUE filter

D Turn on enforcement on the taxonomy that holds the policy tags

◆ PAUSE AND DECIDE

What should the team do?

A **Fine-Grained Reader**
A column role, needed only for subquery row policies.

B **Drop the policy**
Throws away the protection for billing analysts.

C **A TRUE policy**
Data science is granted every row again.

D **Enforce the taxonomy**
Column security, unrelated to rows.

THE SIGNAL

Users outside every row policy see nothing: grant them a TRUE filter.

> The customer app's service account must fetch one account's bills at a time, by account ID.
>
> It must never be able to query curated.bills directly, or read columns beyond bill ID, period and amount.

What should the team build?

A An authorised table function that takes the account ID

B Data Viewer on curated, with a row policy using SESSION_USER()

C An authorised view that selects every bill for every account

D A nightly export of each account's bills to Cloud Storage

◆ PAUSE AND DECIDE

What should the team build?

A Authorised table function

One account per call, three columns, no table access.

B Row policy on SESSION_USER

One service account is a single user, so it sees all.

C Authorised view of all

Can be read in bulk, all accounts at once.

D Nightly export

Stale, and millions of files to secure.

THE SIGNAL

Results by parameter, without source access: an authorised table function.

Module 29 in five lines

- **Authorised views**, datasets and routines share results, not sources.
- **Row access policies** filter rows. Anyone outside every policy sees none.
- **Policy tags** restrict columns. Fine-Grained Reader sees real values.
- **Masking**: SHA-256 to join across queries, random hash to stay safer.
- **IAM conditions** for time, names and tags, never for rows or columns.

COMING UP

Next: module 30, preparing data for ML with BigQuery ML.

MODULE 30 · SECTION 4 - PREPARING AND USING DATA FOR ANALYSIS

Preparing Data for ML: BigQuery ML and Features

What the PDE still asks about ML, model types at a glance, feature preprocessing and TRANSFORM, splits and leakage, evaluating and predicting, exporting and registering models.

SECTION 4 of 5 EXAM WEIGHT 15% MODULE 30 of 45

admin@sthithapragna.com

On the PDE, ML is a data job. **Build honest features, and keep them identical in training and serving.**

What the exam still asks about ML

- Since late 2023, ML theory belongs to the **Machine Learning Engineer** exam.
- The PDE keeps **preparing data**: BigQuery ML, features, embeddings and RAG.
- Vertex AI is now **Gemini Enterprise Agent Platform**, since April 2026.
- Option names have not moved: the registry is still **'VERTEX_AI'** in SQL.

> HONEST LIMIT
>
> Older banks ask about dropout and overfitting. Those questions come from the previous exam.

BigQuery ML at a glance

YOU WANT TO	REACH FOR
Predict a **number**	Linear regression, boosted tree
Predict a **category**	Logistic regression, boosted tree, random forest, DNN, AutoML
Find **groups** with no labels	K-means
Recommend products or tariffs	Matrix factorization
Forecast a series	ARIMA_PLUS, or TimesFM with AI.FORECAST
Use a model built **elsewhere**	Import TensorFlow, ONNX, XGBoost, or a remote model

THE PAYOFF

BigQuery ML brings the model to the data. Nothing leaves the warehouse to train.

ARIMA_PLUS or TimesFM?

ARIMA_PLUS

Train your own

- One model per series, trained in SQL
- Holidays, seasonality, tuning
- XREG adds outside drivers, such as weather
- ML.FORECAST to predict

TIMESFM

Pretrained, no training

- A foundation model for time series
- AI.FORECAST, with no model to manage
- Accuracy comparable to ARIMA
- Marked Preview on the overview page

THE TIEBREAKER

No model to train or manage: TimesFM. Need tuning, holidays or extra drivers: ARIMA_PLUS.

Feature preprocessing in BigQuery ML

- BigQuery ML does basic preprocessing **automatically**, such as encoding strings.
- Scale: **ML.STANDARD_SCALER**, ML.MIN_MAX_SCALER. Bucket: **ML.QUANTILE_BUCKETIZE**.
- Encode: **ML.ONE_HOT_ENCODER**, ML.LABEL_ENCODER. Fill gaps: **ML.IMPUTER**.
- Combine: **ML.FEATURE_CROSS**. Inspect: **ML.FEATURE_INFO** shows feature statistics.

FOR LARKSPUR

Bucket tenure, cross tariff with region: two lines of SQL, no separate pipeline.

Preprocess in SELECT, or in TRANSFORM?

IN THE SELECT

Training only

- Applied when the model trains
- You must repeat it at prediction
- Not saved when the model is exported
- Two copies drift apart: skew

IN TRANSFORM

Stored in the model

- Saved inside the model itself
- ML.EVALUATE and ML.PREDICT reapply it
- Exported as a transform SavedModel
- One definition, everywhere

THE SIGNAL

'Predictions drift from evaluation results' after hand-copied preprocessing: move it into TRANSFORM.

Larkspur's churn model, with TRANSFORM

```
1  CREATE OR REPLACE MODEL ml.churn_model
2  TRANSFORM (
3    ML.STANDARD_SCALER(avg_daily_kwh) OVER () AS avg_daily_kwh_scaled,
4    ML.QUANTILE_BUCKETIZE(tenure_months, 5) OVER () AS tenure_bucket,
5    ML.FEATURE_CROSS(STRUCT(tariff_type, region)) AS tariff_region,
6    churned
7  )
8  OPTIONS (model_type = 'BOOSTED_TREE_CLASSIFIER',
9           input_label_cols = ['churned'],
10          model_registry = 'VERTEX_AI')
11 AS SELECT * FROM ml.churn_features WHERE snapshot_date < '2026-07-01';
```

3-5 Scale, bucket and cross, stored inside the model.

10 Registered in the Model Registry, ready to deploy.

REMEMBER

Training stops at July. Later snapshots are kept back to evaluate on unseen months.

Honest evaluation

- **AUTO_SPLIT** is the default. RANDOM, CUSTOM, SEQ and NO_SPLIT are the others.
- Time-ordered data: **SEQ** on a date column, or split by date yourself.
- **Leakage**: a feature known only after the outcome, such as a cancellation request.
- Leaky features give superb evaluation scores, then poor results in production.

> THE TRAP
>
> A random split across years of snapshots lets the model peek at the future.

Scoring the model in SQL

```sql
-- How good is it, on months it has not seen?
SELECT * FROM ML.EVALUATE(MODEL ml.churn_model,
  (SELECT * FROM ml.churn_features WHERE snapshot_date >= '2026-07-01'));

-- Score this month's accounts
SELECT account_id, predicted_churned, predicted_churned_probs
FROM ML.PREDICT(MODEL ml.churn_model,
  (SELECT * FROM ml.churn_features WHERE snapshot_date = '2026-10-01'));
```

2 Precision, recall and ROC AUC for a classifier.

7 The TRANSFORM steps are reapplied automatically.

FOR LARKSPUR

ML.EXPLAIN_PREDICT adds which features drove each prediction. Agents can see why.

Export the model, or register it?

EXPORT MODEL

Files in Cloud Storage

- TensorFlow SavedModel, or XGBoost Booster
- TRANSFORM steps go with it
- SELECT-time features do not
- No export with ARRAY or TIMESTAMP features

MODEL REGISTRY

Deploy for online use

- model_registry = 'VERTEX_AI'
- Versions, evaluations, explanations
- Deploy to an endpoint, no serving container
- Remote models cannot be registered

THE TIEBREAKER

Batch scores in the warehouse: ML.PREDICT. Low-latency scores for an app: register and deploy an endpoint.

Features beyond the warehouse

- A **feature store** serves features that live in BigQuery tables to online models.
- BigQuery stays the source of truth. Online serving reads the latest values fast.
- **Point-in-time** training data uses each feature's value as it was then.
- Dataflow's Enrichment transform can look features up mid-stream, from module 17.

> THE PATTERN
>
> The same feature table feeds training and serving. That is another defence against skew.

Phrases that pick the answer

THE QUESTION SAYS	REACH FOR
Predictions **drift** from evaluation	TRANSFORM clause
Superb scores, **poor in production**	Leakage: drop the feature, split by time
Forecast, **no model to manage**	TimesFM with AI.FORECAST
Forecast with **holidays and drivers**	ARIMA_PLUS or ARIMA_PLUS_XREG
Low-latency scores for an app	Model Registry, deploy to an endpoint
Score **millions of rows** nightly	ML.PREDICT in a scheduled query

THE PATTERN

Honest features, identical everywhere: most ML questions test one of the two.

> Data scientists scale and bucket features in a SELECT before CREATE MODEL. The app team rewrote the same steps in Java for predictions.
>
> Live predictions now differ noticeably from the evaluation results.

What should the team change?

A Move the preprocessing into the model's TRANSFORM clause

B Retrain the model every day, so that it adapts more quickly

C Train on several more years of history, to make it more robust

D Send the app team the features as a nightly CSV extract instead

◆ PAUSE AND DECIDE

What should the team change?

A **TRANSFORM**
One definition, stored in the model, reapplied everywhere.

B **Retrain daily**
The two preprocessing copies still differ.

C **More history**
More data does not fix mismatched features.

D **Nightly CSV**
Stale, and still a second copy of the logic.

THE SIGNAL

Two copies of preprocessing drifting apart: put it in TRANSFORM.

> A churn model scores an AUC of 0.99 in evaluation, but performs poorly in production.
>
> It trained on three years of monthly snapshots with a random split, and uses a feature called days_since_cancellation_request.

What should the team do first?

A Run more hyperparameter tuning trials on the same training data

B Switch the model type to a DNN, which can learn richer patterns

C Drop features known only after the outcome, and split by time

D Keep AUTO_SPLIT, and simply train the model on more snapshots

◆ PAUSE AND DECIDE

What should the team do first?

A **More tuning**
Tunes harder on a leaky evaluation.

B **A DNN**
A stronger model exploits the leak even better.

C **Fix the leak**
Honest features and a time split give a believable score.

D **More snapshots**
The leak and the random split remain.

THE SIGNAL

Superb evaluation, poor production: leakage. Remove it, and split by time.

> Westgrid wants next week's half-hourly demand forecast for each of 14 grid regions.
>
> Larkspur has no ML engineers, and wants no model to train, tune or manage.

What should the team use?

A. A DNN regressor in BigQuery ML, retrained every week

B. Export the history to Spark, and build a model in MLlib

C. An ARIMA_PLUS model for each region, tuned for holidays

D. TimesFM through AI.FORECAST, over the regional history

◆ PAUSE AND DECIDE

What should the team use?

A DNN

A model to train and manage every week.

B Spark MLlib

Moves the data, and needs ML engineering.

C ARIMA_PLUS

Good forecasts, but models to train and tune.

D TimesFM

Pretrained. One function call, no model to manage.

THE SIGNAL

Forecast with no model to train or manage: TimesFM with AI.FORECAST.

> During a phone call, a call-centre agent's screen must show the customer's churn risk in under 100 milliseconds.
>
> The model was trained in BigQuery ML, and the features are computed at call time.

How should the model be served?

A Run an ML.PREDICT query against BigQuery for every call

B Register the model, then deploy it to an endpoint

C Score every account nightly, and query that table at call time

D Retrain as a logistic regression, so BigQuery answers faster

◆ PAUSE AND DECIDE

How should the model be served?

A A query per call

BigQuery jobs take far longer than 100 ms.

B Registry and endpoint

Online prediction, built for low latency.

C Nightly scores

Features are computed at call time, so last night is stale.

D Simpler model

Still a BigQuery job for every call.

THE SIGNAL

Low-latency predictions for an app: register the model and deploy an endpoint.

Module 30 in five lines

- The PDE tests **data for ML**. Theory moved to the ML Engineer exam in 2023.
- Preprocess in **TRANSFORM**, so training and prediction can never drift.
- Watch for **leakage**, and split time-ordered data by time.
- **TimesFM** for forecasts with nothing to manage, ARIMA_PLUS for tuning.
- **ML.PREDICT** for batch, **registry and endpoint** for low latency.

COMING UP

Next: module 31, embeddings, RAG and LLM-assisted SQL.

MODULE 31 · SECTION 4 - PREPARING AND USING DATA FOR ANALYSIS

Embeddings, RAG and LLM-Assisted SQL

Parsing and chunking documents, embeddings, vector indexes and VECTOR_SEARCH, retrieval-augmented generation in BigQuery, Gemini in BigQuery, and reviewing generated SQL.

SECTION 4 of 5 EXAM WEIGHT 15% MODULE 31 of 45

admin@sthithapragna.com

A language model answers well only from the right pages. **Finding those pages is a data job.**

Retrieval-augmented generation in BigQuery

01

Object table

The PDFs stay in Cloud Storage, from module 25.

02

Parse and chunk

Document AI's layout parser, via ML.PROCESS_DOCUMENT.

03

Embed

AI.GENERATE_EMBEDDING turns each chunk into a vector.

04

Retrieve

VECTOR_SEARCH finds the closest chunks.

05

Generate

AI.GENERATE answers, using only those chunks.

THE PAYOFF

Every step is SQL, and the files never leave Cloud Storage to be read.

Cutting documents into useful pieces

- The **layout parser** understands headings, lists and tables, not just raw text.
- Each **chunk** should hold one idea, small enough to match a question precisely.
- Keep the **headings above** each chunk, so 'step 3' still says which procedure.
- Store the source **file and page** with every chunk, so answers can cite them.

> THE TRAP
>
> One vector for a whole 40-page manual matches everything a little, and nothing well.

Meaning as a list of numbers

- An **embedding** places text in a space where similar meanings sit close together.
- **AI.GENERATE_EMBEDDING** embeds a whole table. **AI.EMBED** embeds one value.
- Both call an embedding model through a **remote model** and a connection.
- Embed questions with the **same model** as the documents, or distances mean nothing.

> CURRENCY CHECK
>
> Older material says ML.GENERATE_EMBEDDING. Current docs lead with the AI functions.

Embedding and indexing the manuals

```
1  CREATE OR REPLACE MODEL rag.embedder
2    REMOTE WITH CONNECTION `europe-west2.lk-ai-conn`
3    OPTIONS (endpoint = 'text-embedding-005');
4
5  CREATE OR REPLACE TABLE rag.manual_chunks_emb AS
6  SELECT * FROM AI.GENERATE_EMBEDDING(MODEL rag.embedder,
7    (SELECT chunk_id, uri, content FROM rag.manual_chunks));
8
9  CREATE VECTOR INDEX manual_idx ON rag.manual_chunks_emb(embedding)
10   STORING (uri)
11   OPTIONS (index_type = 'IVF', distance_type = 'COSINE');
```

6-7 Embeds every chunk. The text column is named content.

9-10 An IVF index on the vectors, storing the file for filters.

HONEST LIMIT

Endpoint names change as models are replaced. Check the current model list before you build.

Embed in batches, or autonomously?

AI.GENERATE_EMBEDDING

You run it

- A query builds an embedding table
- You rerun it when sources change
- Search with VECTOR_SEARCH
- Easy to forget a refresh

AUTONOMOUS GENERATION

BigQuery keeps it in sync

- A generated column, using AI.EMBED
- Inserts and updates are embedded for you
- Search simply with AI.SEARCH
- One such column per table

THE TIEBREAKER

Sources change often, and nobody should have to remember: autonomous embedding generation.

Exact or approximate?

- Without an index, **VECTOR_SEARCH** checks every row: exact, but slow on big tables.
- **CREATE VECTOR INDEX** gives approximate search: faster and cheaper, recall below 100%.
- **IVF** for small batches of queries. **TreeAH** for hundreds at once, up to 200 million rows.
- Tables under **10 MB** are not indexed, and Standard edition cannot use indexes.

REMEMBER

New rows are searched exactly until the index catches up, so nothing is missed.

Filters, recall and monitoring

- **STORING** columns let a filter run before the search, such as one meter model.
- Filters on other columns run afterwards, and can return fewer than **top_k**.
- Raise **top_k**, or **fraction_lists_to_search**, to win back recall.
- **INFORMATION_SCHEMA.VECTOR_INDEXES** shows coverage and status.

> THE TRAP
>
> Stored columns are ignored on tables with row-level security. Plan filters around that.

Answering an engineer's question

```sql
WITH hits AS (
  SELECT base.uri, base.content
  FROM VECTOR_SEARCH(TABLE rag.manual_chunks_emb, 'embedding',
    (SELECT embedding FROM AI.GENERATE_EMBEDDING(MODEL rag.embedder,
      (SELECT 'How do I reset a meter after error E42?' AS content))),
    top_k => 5, distance_type => 'COSINE'))
SELECT AI.GENERATE(
  CONCAT('Answer only from these extracts, and cite the file. ',
         STRING_AGG(CONCAT(uri, ': ', content), ' | ')),
  connection_id => 'europe-west2.lk-ai-conn').result AS answer
FROM hits;
```

3-6 Embed the question with the same model, then take the five closest chunks.

8 Ground the answer in the extracts, with citations.

THE PATTERN

If the answer is wrong, look at the five chunks first. Retrieval fails more often than generation.

Gemini as a SQL assistant

- **SQL generation** from plain language, completion as you type, and explanations.
- **Data canvas**: find tables, join, query and chart, all in natural language.
- **Data insights** suggests queries from table metadata. **Data preparation** suggests fixes.
- The **Data Engineering Agent** builds and troubleshoots pipelines from prompts.

> FOR LARKSPUR
>
> Good descriptions and glossary terms, from module 26, make every one of these work better.

Plausible is not the same as right

- Google warns that output can seem **plausible but be factually incorrect**.
- Read the **joins and filters**. Check the grain, the dates and the partition filter.
- **Dry run** it for bytes, then compare with a figure you already trust.
- Gemini does not train on your data, but lacks some BigQuery **compliance** offerings.

> THE TRAP
>
> Asking Gemini to explain its own query is not a review. A person must check it.

Phrases that pick the answer

THE QUESTION SAYS	REACH FOR
Answers **vague**, whole documents embedded	Layout parser, chunk with headings
Semantic search, **too slow** at scale	Vector index, accept approximate recall
Embeddings go **stale** as sources change	Autonomous embedding generation
Search by **meaning**, not keywords	VECTOR_SEARCH or AI.SEARCH
Exact keyword or ID **lookup**	Search index, from module 28
Generated SQL looks **plausible**	Review, dry run, compare to a known figure

THE TIEBREAKER

Meaning needs vectors. Exact strings need a search index. Do not mix them up.

> The field engineer assistant gives vague answers about meter procedures.
>
> The pipeline stores one embedding for each manual, and some manuals run to 40 pages.

What should the team change first?

A Switch to a larger embedding model with more dimensions

B Raise top_k to 50, so that more manuals reach the prompt

C Parse with the layout parser, and embed each headed chunk

D Lower the generation model's temperature, for steadier answers

◆ PAUSE AND DECIDE

What should the team change first?

A Larger embedding model

Still one vector per forty pages.

B top_k of 50

Fifty whole manuals in the prompt, still unfocused.

C Chunk, then embed

Each vector holds one idea, so matches become precise.

D Lower temperature

Steadier wording, built on the same wrong extracts.

THE SIGNAL

Vague answers from whole-document vectors: chunk first.

> Engineers' questions search 30 million chunk embeddings, one question at a time.
>
> Each search scans every row, and is too slow and too costly. Slightly approximate results are acceptable.

What should the team add?

A An IVF vector index on the embedding column

B A search index on the chunk text column, for SEARCH

C Clustering of the table on the embedding column

D AI.SIMILARITY comparisons against every row in the table

◆ PAUSE AND DECIDE

What should the team add?

A IVF vector index

Approximate nearest neighbours, fast for single queries.

B Search index

Exact keyword lookups, not meaning.

C Cluster on embeddings

Arrays of numbers cannot be cluster keys.

D AI.SIMILARITY everywhere

Meant for a few comparisons, not millions.

THE SIGNAL

Semantic search too slow, approximate is fine: a vector index.

> Engineering updates manuals every week, and someone is meant to rerun the embedding query afterwards.
>
> They often forget, so new procedures never show up in search results.

What should the team do?

A Add a weekly calendar reminder to rerun the embedding query

B A generated embedding column using AI.EMBED, kept in sync

C Rebuild the vector index every day, on a scheduled query

D Raise fraction_lists_to_search, so the index searches wider

◆ PAUSE AND DECIDE

What should the team do?

A Calendar reminder

Still depends on someone remembering.

B Autonomous embeddings

New and changed chunks are embedded automatically.

C Rebuild the index

Indexes vectors that were never regenerated.

D Search wider

Cannot find chunks that were never embedded.

THE SIGNAL

Embeddings that must track changing sources: autonomous generation.

> An analyst used Gemini in BigQuery to write a quarterly revenue query for the board pack.
>
> The figures look plausible. The board pack goes out tomorrow.

What should happen before the figures are used?

A Publish them, since Google checks the SQL that Gemini writes

B Ask Gemini to explain the query, and accept its explanation

C Turn off the query cache, so the figures are fully fresh

D Check joins and filters, dry run, compare to a known total

◆ PAUSE AND DECIDE

What should happen before the figures are used?

A Publish now

Google warns output can be plausible but wrong.

B Gemini explains itself

A model marking its own work is not a review.

C Cache off

Fresher numbers, the same possible logic error.

D Human review

Checks logic, cost, and the answer itself.

THE SIGNAL

Generated SQL for decisions: a person reviews it and checks the result.

Module 31 in five lines

- RAG in SQL: object table, parse and **chunk**, embed, search, generate.
- Chunks keep their **headings**. Questions use the **same** embedding model.
- **Vector indexes** trade a little recall for speed. IVF for single queries.
- **Autonomous embeddings** keep vectors in step with changing sources.
- Gemini writes SQL fast. A **person** still reviews it before it matters.

COMING UP

Next: module 32, sharing data with BigQuery sharing and clean rooms.

MODULE 32 · SECTION 4 - PREPARING AND USING DATA FOR ANALYSIS

Sharing Data: BigQuery Sharing and Clean Rooms

Copies, authorised views and listings; exchanges, listings and linked datasets; who pays; egress controls; data clean rooms and analysis rules.

SECTION 4 of 5 EXAM WEIGHT 15% MODULE 32 of 45

admin@sthithapragna.com

Share the live data, not a copy. Then decide what the other side may do with it.

Four ways to share, compared

OPTION	BEST WHEN	COPIES DATA?
Export or copy	A one-off file for a system outside Google Cloud	Yes, and it goes stale
Authorised view	Teams inside your own organisation	No
BigQuery sharing listing	Other organisations, discoverable and live	No: a linked dataset
Data clean room	Joining data with a partner, neither sees rows	No

THE TIEBREAKER

Another organisation, live data, no copies: a listing. Combining both sides' data privately: a clean room.

Exchanges, listings and linked datasets

- An **exchange** groups listings. Private by default, or public to every Google Cloud user.
- A **listing** points at a dataset or a Pub/Sub topic, with docs and sample queries.
- Subscribing creates a **linked dataset**: read-only, live, and nothing is copied.
- Analytics Hub is now called **BigQuery sharing**. Roles still say Analytics Hub.

> THE NUMBERS
>
> One shared dataset can feed up to 1,000 linked datasets, each in a subscriber's own project.

What can be shared

- Datasets: tables, views, authorised views, materialized views, routines and ML models.
- **Pub/Sub topics**, for live streaming data, as linked subscriptions.
- Row-level security and masking work, but subscribers need **Enterprise** or Enterprise Plus.
- Public listings can be **sold** through Cloud Marketplace.

> THE TRAP
>
> Shared view definitions show in query plans. Never put keys or secrets in them.

Who pays for what?

PUBLISHER

Pays to keep it

- Storage of the shared data
- Pub/Sub publishing, for topics
- No charge for exchanges or listings

SUBSCRIBER

Pays to use it

- Every query on the linked dataset
- On-demand bytes, or their own slots
- Pub/Sub subscribing, for topics

THE SIGNAL

The partner's queries are billed to the partner. That is often the deciding phrase.

Egress controls and limits

- **Egress controls** on a listing stop copy, clone, export, snapshot and transfers.
- They can also block **CREATE TABLE AS SELECT** and writing results to tables.
- No materialized views or snapshots over linked tables. Huge joins may fail.
- The listing's dataset must be in the **same region** as the exchange.

> REMEMBER
>
> The publisher keeps the encryption key. Disable it, and every subscriber loses access too.

Sharing regional demand with Westgrid

01
Publish
marts.region_demand_hh, in europe-west2.

02
Exchange
Private exchange lk-partners.

03
Listing
Egress restricted, docs, sample queries.

04
Subscribe
Westgrid gets a live linked dataset.

05
Query
In Westgrid's project, on Westgrid's bill.

THE PAYOFF

Updates land the moment Larkspur writes them, so the five-minute target holds.

Joining data nobody may see

- Built on sharing. An **owner** runs it, **contributors** add data, **subscribers** query.
- Contributors set **analysis rules** on views, so raw rows never come back.
- **Query templates**, built from table functions, allow only approved queries.
- Egress of raw data is blocked automatically. Results can be blocked too.

> HONEST LIMIT
>
> Google says rules are not guaranteed to block every extraction attempt. Combine them with templates.

Four analysis rules

RULE	WHAT IT ALLOWS	QUERY SHAPE
Aggregation threshold	Totals only, over at least N people per row	SELECT WITH AGGREGATION_THRESHOLD
Differential privacy	Noisy totals, within a privacy budget	SELECT WITH DIFFERENTIAL_PRIVACY
List overlap	Only rows that match after a join	Join on the allowed columns
Join restriction	Joins only on chosen columns, or none	JOIN_ALL, JOIN_ANY, JOIN_BLOCKED

THE TRAP

When the privacy budget is spent, the view cannot be queried until it is updated or re-created.

Larkspur's side of the Hearthly clean room

```
1  CREATE OR REPLACE VIEW clean.lk_households
2  OPTIONS (privacy_policy = '''{"aggregation_threshold_policy":
3    {"threshold": 50, "privacy_unit_column": "household_key"}}''')
4  AS SELECT household_key, region, tariff_type, avg_daily_kwh
5  FROM shared.hearthly_ready;
6
7  -- What Hearthly may run: totals only, 50 households or more per row
8  SELECT WITH AGGREGATION_THRESHOLD region, tariff_type, COUNT(*) AS homes
9  FROM clean.lk_households
10 GROUP BY region, tariff_type;
```

2-3 Every result row must cover at least 50 distinct households.

8 Queries must aggregate. Rows under the threshold are dropped.

REMEMBER

The household key can never be selected directly, only counted.

Phrases that pick the answer

THE QUESTION SAYS	REACH FOR
Another **organisation**, live, no copies	BigQuery sharing listing
Partner pays **their own queries**	Listing, linked dataset in their project
They must **not export** or copy it	Egress controls on the listing
Combine lists, **neither sees the other's**	Data clean room
Totals only, never single people	Aggregation threshold rule
Queries fail after **many runs** on a DP view	Privacy budget used up

THE PATTERN

Who is it for, who pays, and what may they do with it: three questions, three features.

> Westgrid wants Larkspur's live half-hourly demand data inside its own Google Cloud organisation.
>
> Larkspur must not pay for Westgrid's queries, and Westgrid must not be able to copy or export the data.

What should Larkspur set up?

A Export the table as CSV files to a shared bucket every hour

B A private listing with egress restricted, which Westgrid subscribes to

C An authorised view, with Westgrid's users running jobs in Larkspur's project

D A nightly DTS copy of the table into a dataset in Westgrid's project

◆ PAUSE AND DECIDE

What should Larkspur set up?

A Hourly CSV export

Copies that are stale and easily passed on.

B Private listing

Live, billed to Westgrid, and exports are blocked.

C Authorised view

Queries run in Larkspur's project, on Larkspur's bill.

D Nightly copy

A day out of date, and the copy is theirs to export.

THE SIGNAL

Another organisation, live, partner pays, no exports: a listing with egress controls.

> Larkspur and Hearthly want to know how many customers they share, by region.
>
> Neither company may see the other's customer list, or any individual customer's details.

What should the two companies use?

A Swap lists of hashed email addresses by SFTP, then compare them

B A linked dataset of Larkspur's customers, shared with Hearthly

C An authorised view that joins both tables, shared with both sides

D A data clean room, with list overlap or threshold rules

◆ PAUSE AND DECIDE

What should the two companies use?

A **Swap hashed lists**
Each side holds the other's list, and hashes can be guessed.

B **Linked dataset**
Hearthly sees Larkspur's whole customer list.

C **Authorised view**
Someone must hold both tables, and rows come back.

D **Clean room**
Matching happens inside, and only totals come out.

THE SIGNAL

Combine two parties' data, neither sees rows: a data clean room.

> Westgrid subscribes to a Larkspur listing whose tables carry row access policies. Its queries on the linked dataset fail.
>
> Westgrid runs BigQuery on the Standard edition.

What is the most likely fix?

A Westgrid moves to Enterprise or Enterprise Plus edition

B Larkspur grants Westgrid Fine-Grained Reader on the tables

C Larkspur removes the egress controls from the listing

D Larkspur moves the exchange into the US multi-region

◆ PAUSE AND DECIDE

What is the most likely fix?

A **Higher edition**
Required to query listings with row-level security.

B **Fine-Grained Reader**
A column role. It does not fix the edition.

C **Remove egress controls**
Controls exports, not whether queries run.

D **Move the exchange**
Must stay with the dataset, in Europe.

THE SIGNAL

Shared tables with row-level security: subscribers need Enterprise or Enterprise Plus.

> Hearthly's analysts ran hundreds of queries against a differential privacy view in the clean room.
>
> Now every query against that view fails, although nothing else has changed.

What has most likely happened?

A The linked dataset expired after a fixed number of days

B An egress control has started blocking all of their queries

C The privacy budget is spent, so the view must be updated

D The aggregation threshold was raised above the data size

◆ PAUSE AND DECIDE

What has most likely happened?

A Linked dataset expired

Linked datasets do not expire on their own.

B Egress control

Blocks exports, not ordinary aggregate queries.

C Budget spent

Each query uses epsilon. At zero, the view is locked.

D Threshold raised

Nothing changed, and this is a different rule.

THE SIGNAL

Many queries, then a differential privacy view stops working: the budget is spent.

Module 32 in five lines

- Share live data, not copies. **Authorised views** inside, **listings** across organisations.
- A listing gives a read-only **linked dataset**. The subscriber pays for queries.
- **Egress controls** stop copying and exporting. Row security needs Enterprise.
- **Clean rooms** combine partners' data, and only aggregates come out.
- Rules: **aggregation threshold**, differential privacy, list overlap, join restriction.

COMING UP

Next: module 33, the start of section five: BigQuery capacity and pricing.

MODULE 33 · SECTION 5 - MAINTAINING AND AUTOMATING DATA WORKLOADS

BigQuery Capacity: On-Demand, Editions and Reservations

On-demand or capacity, the three editions, baseline and autoscaling slots, commitments, assignments, idle slots, priority and queues, and cost guard rails.

SECTION 5 of 5 EXAM WEIGHT 18% MODULE 33 of 45

admin@sthithapragna.com

On-demand bills the bytes you read. **Capacity bills the slots you hold. Choose the bill you can predict.**

On-demand or capacity?

ON-DEMAND

Pay per byte

- Billed per TiB processed, first TiB a month free
- About 2,000 slots per project, with bursts
- No commitments, nothing to manage
- Custom quotas cap the bytes

CAPACITY

Pay per slot

- Slots billed per second, one-minute minimum
- Reservations, autoscaling, commitments
- Spend capped by reservation size
- Pick an edition: Standard, Enterprise, Plus

THE TIEBREAKER

Small, spiky use: on-demand. Steady heavy use, or a bill that must not surprise: capacity.

Three editions, compared

EDITION	CAPACITY	ADDS
Standard	Autoscaling only, up to 1,600 slots, no commitments	The basics: no BigQuery ML, no BI Engine, no indexes
Enterprise	Baseline plus autoscaling, with commitments	BigQuery ML, BI Engine, search and vector indexes, continuous queries
Enterprise Plus	As Enterprise	Managed disaster recovery, Assured Workloads compliance

HONEST LIMIT

Google warns that features move between editions. Choose for the workload, then check the current table.

Baseline and autoscaling

- A **slot** is a unit of compute: CPU, memory and I/O together.
- **Baseline** slots are always yours, and always billed, used or not.
- **Autoscaling** adds slots in steps of 50, up to the reservation's maximum.
- Scaled slots are kept for **60 seconds** or more, and billed whether used or not.

> CURRENCY CHECK
>
> Fluid scaling, which you opt into, removes the one-minute minimum for bursty work.

Committing for a discount

- **One-year** or **three-year** commitments, bought in steps of 50 slots.
- They cover **baseline** slots only. Autoscaled slots pay the standard rate.
- They cannot be cancelled early. Set renewal to NONE to stop them renewing.
- Billed to the **administration project**. Unused committed slots become idle slots.

> THE TRAP
>
> Standard edition has no commitments. A question that pairs them is a trap.

Larkspur's reservations, in SQL

```
1 CREATE RESERVATION `lk-admin-prod.region-europe-west2.etl`
2 OPTIONS (edition = 'ENTERPRISE', slot_capacity = 300, autoscale_max_slots = 700);
3
4 CREATE ASSIGNMENT `lk-admin-prod.region-europe-west2.etl.ingest_queries`
5 OPTIONS (assignee = 'projects/lk-ingest-prod', job_type = 'QUERY');
6
7 CREATE RESERVATION `lk-admin-prod.region-europe-west2.bi`
8 OPTIONS (edition = 'ENTERPRISE', slot_capacity = 0, autoscale_max_slots = 400);
```

2 300 baseline slots for the pipelines, autoscaling to 700 at most.

8 No baseline for analysts: pay only when they query, capped at 400.

FOR LARKSPUR

The maximum is the spending ceiling. That is what the finance director asked for.

Who uses which reservation

- Assign a reservation to an **organisation**, **folder** or **project**. The most specific wins.
- Job types: **QUERY**, **PIPELINE**, **BACKGROUND**, ML_EXTERNAL, **CONTINUOUS**.
- No assignment means **on-demand**. An assignment to **None** forces on-demand.
- Loads are free on a shared pool. Assigning PIPELINE moves them onto your paid slots.

> THE TIEBREAKER
>
> Opting one project out of an organisation-wide reservation: assign it to None.

Idle slots and fairness

- Reservations borrow **idle slots** from each other, within one admin project and edition.
- Borrowed slots go back within milliseconds when the owner needs them.
- **ignore_idle_slots** stops a reservation borrowing. It can still lend.
- **Fair scheduling** splits a reservation's slots between the projects using it.

> HONEST LIMIT
>
> Never plan a deadline on idle slots. They can be taken back at any moment.

Interactive, batch and queues

- BigQuery sets **concurrency** dynamically. Extra queries wait in a queue.
- Queues hold **1,000 interactive** and **20,000 batch** queries, per project per region.
- Queued queries time out after 6 hours, or 24 hours for batch, by default.
- **Batch priority**: a separate cap, and interactive queries go first.

> THE TIEBREAKER
>
> Reports that can finish any time before morning: batch priority.

Capping cost on-demand

- **QueryUsagePerDay** caps a project. Default: 200 TiB a day.
- **QueryUsagePerUserPerDay** caps each user in a project. Default: unlimited.
- Checked before a query runs, approximate, and reset at **midnight Pacific**.
- **Maximum bytes billed** stops one query, from module 28. Quotas cap a day.

> REMEMBER
>
> Custom quotas apply to on-demand only. On capacity, the reservation's maximum is the cap.

Larkspur's capacity, end to end

01

lk-admin-prod

Holds the commitments and reservations.

02

etl reservation

Baseline 300, up to 700. Pipelines.

03

bi reservation

No baseline, up to 400. Analysts.

04

Sandboxes

Assigned to None. On-demand with quotas.

THE PATTERN

A committed baseline for steady work, a ceiling on everything else: a bill that can be forecast.

Phrases that pick the answer

THE QUESTION SAYS	REACH FOR
Spend must be **predictable**	Reservation, with a maximum size
Steady load, **cheapest** long term	Commitment covering the baseline
Needs **BigQuery ML** or **BI Engine** on capacity	Enterprise edition
Managed DR, **FedRAMP** or ITAR	Enterprise Plus
One project **out of** the org reservation	Assign it to None
Can finish **any time overnight**	Batch priority

THE PATTERN

What must be predictable, what must be fast, and what can wait: size the bill around those.

> After one on-demand month cost GBP 40,000, the finance director wants a BigQuery bill that cannot exceed a set monthly figure.
>
> Pipelines run steadily all day. Analyst queries come in bursts.

What should the team set up?

A. Custom per-user quotas on every project, staying on-demand

B. On-demand, with maximum bytes billed set on every query

C. Enterprise reservations: a committed baseline, autoscaling capped

D. Standard edition, with autoscaling and no maximum set on any reservation

◆ PAUSE AND DECIDE

What should the team set up?

A Per-user quotas

Cap bytes, but the total still moves, and work stops at the cap.

B Maximum bytes billed

Caps each query, not the month.

C Committed baseline, capped

Steady work is discounted. Bursts stop at the ceiling.

D Standard, uncapped

No commitments, and no ceiling either.

THE SIGNAL

A bill that cannot exceed a figure: reservations with a maximum, and a committed baseline.

> Larkspur is moving BigQuery to capacity pricing. The team needs BigQuery ML, BI Engine and vector indexes for the field engineer assistant.
>
> It has no need for managed disaster recovery or FedRAMP-style compliance controls.

Which edition fits best?

A Enterprise edition, baseline plus autoscaling

B Standard edition, which only autoscales

C Enterprise Plus, with every feature switched on

D Standard edition, with a three-year commitment

◆ PAUSE AND DECIDE

Which edition fits best?

A Enterprise

BigQuery ML, BI Engine and indexes, without paying for Plus.

B Standard

No BigQuery ML, no BI Engine, no indexes.

C Enterprise Plus

Works, but pays for DR and compliance nobody needs.

D Standard with commitment

Standard offers no commitments at all.

THE SIGNAL

BigQuery ML, BI Engine and indexes, without DR or compliance needs: Enterprise.

> An organisation-wide reservation covers every Larkspur project. Analysts' experiments in sandbox projects now take slots the ETL pipelines need.
>
> Sandbox work should not touch the reservation at all, but must stay cheap.

What should the team do?

A. Delete the organisation-wide assignment, for every project

B. Buy more committed slots, so that there is room for both

C. Set ignore_idle_slots on the ETL reservation

D. Assign the sandbox projects to None, and add custom quotas

◆ PAUSE AND DECIDE

What should the team do?

A Delete the assignment

Pushes every project, pipelines included, to on-demand.

B Buy more slots

Pays more, and sandboxes still share the reservation.

C ignore_idle_slots

Stops borrowing, not sandboxes using the same reservation.

D Assign to None

Sandboxes go on-demand, and quotas keep them cheap.

THE SIGNAL

One set of projects out of an org reservation: assign them to None.

> Hundreds of report queries start at 01:00 each night. Some fail with queue errors, and the rest slow early interactive work.
>
> The reports only need to be ready by 07:00.

What should the team change?

A Raise the reservation's autoscaling maximum for the night

B Run the report queries with batch priority

C Spread the reports across more projects, to avoid the queue limit

D Set the queue timeout to -1, so that waiting queries fail quickly

◆ PAUSE AND DECIDE

What should the team change?

A Raise the maximum

Pays more for work that can simply wait.

B Batch priority

A queue of 20,000, and interactive work goes first.

C More projects

Dodges limits, and complicates billing and access.

D Timeout of -1

Turns queuing off, so even more queries fail.

THE SIGNAL

Work that can wait until morning: batch priority.

> A reservation has no baseline and autoscales to 2,000 slots. Hundreds of short, bursty queries run each hour.
>
> Monitoring shows about 300 slots in use on average, yet the bill matches far more.

What best explains the bill?

A Scaled slots bill for a minute or more, whether used or not

B Idle slots borrowed from other reservations are billed twice

C Commitments are charged again for every autoscaled slot

D Short queries cannot use cached results under capacity pricing

◆ PAUSE AND DECIDE

What best explains the bill?

A Scaled, not used

Each burst scales up, and pays for a minute or more.

B Idle slots billed twice

Borrowed idle slots are not billed to the borrower again.

C Commitments on autoscaling

Commitments cover baseline only.

D No cache

Cache rules do not change with pricing model.

THE SIGNAL

Bill above usage on bursty autoscaling: scaled slots, minute minimum. Try fluid scaling or a baseline.

> Larkspur bought a three-year commitment of 500 slots. Every reservation has a baseline of zero, autoscales to 800, and has ignore_idle_slots set to true.
>
> The bill shows the commitment, plus full-price autoscaling on top.

What should the team change?

A Cancel the commitment, and rely on autoscaling slots alone

B Set reservation baselines to use the 500 committed slots

C Raise every reservation's autoscaling maximum to 1,300

D Move every reservation down to the Standard edition

◆ PAUSE AND DECIDE

What should the team change?

A Cancel

Commitments cannot be cancelled early.

B Baselines on the commitment

Committed slots finally do the work, at the discount.

C Raise the maximum

More full-price autoscaling.

D Move to Standard

No commitments, and features are lost.

THE SIGNAL

Paying a commitment and autoscaling together: the commitment must cover baselines.

Module 33 in five lines

- On-demand bills **bytes**. Capacity bills **slots**, and its maximum caps the bill.
- **Standard** is basic, **Enterprise** adds ML and BI Engine, **Plus** adds DR and compliance.
- Autoscaling bills **scaled** slots. Commitments cover **baseline** only.
- Assignments by job type. **None** opts a project out. Idle slots are shared.
- **Batch priority** for work that waits. Custom quotas cap on-demand days.

COMING UP

Next: module 34, right-sizing clusters and pipelines.

MODULE 34 · SECTION 5 - MAINTAINING AND AUTOMATING DATA WORKLOADS

Right-Sizing Clusters and Pipelines

Matching cost to the business need: Spark clusters and Spot VMs, Dataflow levers, Managed Airflow environments, storage levers, and seeing where the money goes.

SECTION 5 of 5 EXAM WEIGHT 18% MODULE 34 of 45

admin@sthithapragna.com

Pay for the deadline, not for the peak. **Idle capacity is the most expensive kind.**

What does the deadline really need?

THE BUSINESS NEED	COST LEVER
Done **by morning**, start time flexible	FlexRS, Spot workers, batch priority
Runs **once a night**	A job cluster or serverless, deleted after
Steady all year	Committed use discounts
Duplicates are tolerable	At-least-once streaming
Bursty, unpredictable	Autoscaling with a sensible maximum

THE PATTERN

Read the deadline in the question first. It usually rules out half the options.

A cluster that lives, or a cluster per job?

PERSISTENT CLUSTER

Always on

- Paid every hour, busy or idle
- Shared by many teams and jobs
- Data in HDFS ties you to it
- Suits steady, shared, interactive use

JOB-SCOPED CLUSTER

Create, run, delete

- Paid only while the job runs
- Sized for one job's needs
- Data in Cloud Storage, outlives it
- Or serverless Spark: no cluster at all

REMEMBER

Data in Cloud Storage is what makes deletion safe. Module 15 showed why.

Cheaper workers, with care

- **Spot VMs** as secondary workers cost far less, and can be reclaimed at any time.
- Secondary workers hold no HDFS. Keep preemptible ones under **half** of all workers.
- **Autoscaling** adds and removes workers, but never shrinks an idle cluster to zero.
- **Scheduled deletion**: delete after idling, at a set time, or at a maximum age.

> THE TRAP
>
> Spot workers suit retryable batch work. Never put a hard deadline on them alone.

Larkspur's nightly settlement, right-sized

```
gcloud dataproc clusters create lk-settle-nightly \
    --region=europe-west2 --num-workers=4 \
    --num-secondary-workers=6 --secondary-worker-type=spot \
    --autoscaling-policy=settle-policy \
    --max-idle=15m --max-age=4h \
    --labels=team=settlement,env=prod,workload=nightly
```

3 Six of ten workers are Spot: cheap, and retryable.

5-6 Deleted when idle or too old. Labels show up in billing.

FOR LARKSPUR

Four standard workers carry the deadline. The Spot workers only make it cheaper and faster.

Dataflow cost levers

- **FlexRS** for batch that can wait: a discounted, delayed start, within six hours.
- **At-least-once** streaming, when the sink can absorb duplicates.
- The smallest **machine type** that works, and a cap on **max workers**.
- Run in the **same region** as sources, sinks and the temp bucket.

> THE TRAP
>
> Many tiny batch jobs each pay to start workers. Fewer, larger runs cost less.

Seeing a job's cost

- The **Cost** page in the Dataflow monitoring UI shows each job's estimated cost.
- **Alerts** fire when a job's cost passes a threshold you set.
- **Pilot runs** on a slice of data estimate cost before the full run.
- **Committed use discounts** suit a predictable streaming minimum for a year or more.

> REMEMBER
>
> Data sampling, not logging every element, is the cheap way to debug a pipeline.

Right-sizing an Airflow environment

- Pick the **size preset** from DAG count and peak concurrent tasks.
- **Workers autoscale** between a minimum and a maximum. Keep the minimum low.
- Start with **two schedulers**. In Gen 3, DAG processors do the parsing.
- No pause button: schedulers, web server and database bill while it **exists**.

> THE TIEBREAKER
>
> Airflow should orchestrate, not compute. Heavy work belongs in BigQuery, Dataflow or Spark.

Storage levers, from section three

LEVER	SAVES ON	MODULE
Lifecycle rules and Autoclass	Cloud Storage classes	22
Soft delete off for temporary data	Deleted bytes still billed	22
Partition expiration, no rewrites	BigQuery active storage	23
Physical storage billing	Compressible, rarely changed data	23
Compact small files	Listing time and operations	25

THE PATTERN

Storage costs grow quietly every month. Set the levers once, when the data first lands.

Budgets or billing export?

BUDGETS AND ALERTS

Warn early

- On actual or forecast spend
- Default alerts at 50, 90 and 100%
- Email, or Pub/Sub for automation
- Do not cap spending by themselves

BILLING EXPORT

Explain the bill

- Cost data in a BigQuery dataset
- Labels, projects and services
- Detailed export adds each resource
- Enable early: not fully backfilled

THE TRAP

'Stop spending automatically' needs a Pub/Sub action, or a hard cap such as a reservation maximum.

Phrases that pick the answer

THE QUESTION SAYS	REACH FOR
Cluster **idle** most of the day	Job-scoped cluster, or scheduled deletion
Batch can **start any time** tonight	FlexRS, or Spot secondary workers
Streaming, **duplicates acceptable**	At-least-once mode
Airflow bill high, **little runs** on it	Smaller preset, low worker minimum
Which **team** spends what?	Labels plus billing export
Warn before the month overruns	Budget with forecast alerts

THE PATTERN

Idle, flexible, tolerant or invisible: each word points at a different lever.

> Larkspur's 20-node Spark cluster runs all day, but the settlement jobs on it take three hours each night.
>
> The input and output data live in Cloud Storage, and the cluster sits idle the rest of the day.

What should the team change?

A. Add an autoscaling policy, so the idle cluster shrinks to zero

B. Switch every worker on the cluster to Spot VMs

C. Create a job cluster each night, with scheduled deletion

D. Buy committed use discounts for all twenty nodes

◆ PAUSE AND DECIDE

What should the team change?

A Autoscaling to zero

Autoscaling never shrinks a cluster to nothing.

B All Spot

Cheaper per hour, still idle 21 hours a day, and fragile.

C Nightly job cluster

Pays for three hours, not twenty four.

D Commit to 20 nodes

Locks in paying for the idle hours.

THE SIGNAL

Idle most of the day, data in Cloud Storage: create per job, delete after.

> A nightly Dataflow batch job rebuilds the tariff analysis. It takes 40 minutes, starts at 22:00, and is only needed by 06:00.
>
> Finance wants the job to cost less, with no change to the code.

What should the team do?

A. Switch the job to at-least-once mode to halve the cost

B. Raise the maximum number of workers, so it finishes sooner

C. Move the job to a region with spare capacity at night

D. Run it with FlexRS, using the cost-optimised goal

◆ PAUSE AND DECIDE

What should the team do?

A At-least-once

A streaming mode, not for batch jobs.

B More workers

Faster, but not cheaper.

C Another region

Moves data across regions, adding cost and risk.

D FlexRS

A discounted rate, starting within six hours.

THE SIGNAL

Batch that can wait hours, cheaper with no code change: FlexRS.

> Larkspur's Managed Airflow environment uses the Large preset with a minimum of 10 workers.
>
> It runs 40 DAGs that mostly trigger BigQuery jobs, and rarely more than a dozen tasks at once.

What should the team change?

A Delete the environment every night, then recreate it

B Move to a smaller preset, and lower the minimum worker count

C Add a third scheduler, so that the tasks finish more quickly

D Run the BigQuery transformations on Airflow workers instead

◆ PAUSE AND DECIDE

What should the team change?

A Delete nightly

Breaks schedules and history, and adds risk.

B Smaller preset, low minimum

Fits forty DAGs, and idle workers stop billing.

C Third scheduler

More fixed cost, and scheduling is not the bottleneck.

D Compute on workers

Moves work to Airflow, raising cost.

THE SIGNAL

An oversized Airflow environment: smaller preset, low worker minimum.

> The finance director wants to know what the settlement team spends each month across Spark, Dataflow and BigQuery,
>
> and to be warned before the month's spend runs over its forecast.

What should the team set up?

A Team labels, billing export, and a budget with forecast alerts

B A separate billing account for each team, checked by hand monthly

C Read the Cost page in the Dataflow monitoring UI once a week

D A single budget on the whole billing account, and nothing else

◆ PAUSE AND DECIDE

What should the team set up?

A Labels, export, budget
Cost by team in SQL, plus early warning.

B Billing account per team
Heavy to run, and still no early warning.

C Dataflow Cost page
Covers Dataflow only, and warns nobody.

D Budget alone
Warns about the total, but cannot split it by team.

THE SIGNAL

Spend by team, plus early warning: labels, billing export and a budget.

Module 34 in five lines

- Pay for the **deadline**, not the peak. Idle capacity costs most.
- Spark: **job clusters** or serverless, Spot secondaries, scheduled deletion.
- Dataflow: **FlexRS** for batch that waits, at-least-once, capped workers.
- Airflow: right **preset**, low worker minimum, no heavy compute on workers.
- **Labels** plus billing export explain the bill. Budgets warn, but do not cap.

COMING UP

Next: module 35, orchestration with Managed Airflow, Workflows and scheduling.

MODULE 35 · SECTION 5 - MAINTAINING AND AUTOMATING DATA WORKLOADS

Orchestration: Managed Airflow, Workflows and Scheduling

Choosing between Managed Airflow, Workflows and Cloud Scheduler; DAG design, retries and idempotency; sensors and deferrable operators; event-driven DAGs; generations; worker evictions.

SECTION 5 of 5 EXAM WEIGHT 18% MODULE 35 of 45

admin@sthithapragna.com

An orchestrator decides what runs, and when. The heavy lifting happens somewhere else.

admin@sthithapragna.com

Three ways to run things in order

TOOL	BEST FOR	WRITTEN IN
Managed Airflow	Data pipelines with dependencies, retries, backfills	Python DAGs
Workflows	Chaining APIs and services, low latency, serverless	YAML or JSON
Cloud Scheduler	One job on a timetable, such as a nightly trigger	A cron schedule

THE TIEBREAKER

Many dependent data steps, with reruns: Airflow. A few API calls, idle most of the time: Workflows.

admin@sthithapragna.com

Managed Airflow or Workflows?

MANAGED AIRFLOW

Data pipelines

- 150+ Google Cloud operators
- Backfills and re-runs built in
- Latency in seconds between tasks
- Provisioned: pays while it exists

WORKFLOWS

Service orchestration

- Connectors for Google APIs, any HTTP call
- Callbacks, and waits of up to a year
- Latency in milliseconds
- Serverless: nothing to pay when idle

REMEMBER

Google's own guide: Airflow for ETL and ELT, Workflows for chaining services.

Tasks that are safe to rerun

- Each task does **one step**, and is **idempotent**: running it twice gives the same result.
- Write by partition: **overwrite** the run's date, never append blindly.
- Set **retries** and a retry delay, so one blip does not fail the whole run.
- Use a **fixed start date**. A dynamic one changes every time the DAG is parsed.

> THE TRAP
>
> Retries, backfills and recovery all assume idempotent tasks. Without it, each rerun duplicates data.

Larkspur's nightly settlement DAG

```
1 with DAG('settlement_nightly', start_date=datetime(2026, 10, 1),
2          schedule='0 1 * * *', catchup=False,
3          default_args={'retries': 2, 'retry_delay': timedelta(minutes=5)}) as dag:
4     wait = GCSObjectExistenceSensor(task_id='wait_for_westgrid', deferrable=True,
5         bucket='lk-landing-prod', object='westgrid/{{ ds }}/settlement.csv')
6     settle = DataprocSubmitJobOperator(task_id='run_settlement', job=SETTLE_JOB)
7     marts = BigQueryInsertJobOperator(task_id='rebuild_marts', configuration=MARTS_SQL)
8     wait >> settle >> marts
```

2-3 Daily at 01:00, no automatic catch-up, two retries per task.

4 A deferrable sensor waits in the triggerer, not on a worker.

FOR LARKSPUR

The {{ ds }} date makes each run read and write only its own day, so reruns are safe.

A sensor, or a deferrable operator?

CLASSIC SENSOR

Holds a worker slot

- Polls on a schedule
- Occupies a worker the whole time
- Hundreds of waits fill the workers
- Fine for short waits

DEFERRABLE

Waits in the triggerer

- Hands the wait to the triggerer
- Frees the worker slot while waiting
- Survives environment updates
- Needs at least one triggerer

THE TIEBREAKER

Many long waits on files or jobs: deferrable operators. They are cheaper and more robust.

Catchup, backfill and failure alerts

- With **catchup** on, a new DAG runs every missed interval since its start date.
- Set catchup off, then **backfill** on purpose, for exactly the dates you need.
- An **on_failure_callback** can post to a channel or open an incident.
- Airflow 3, in Gen 3, adds backfills from the UI and **asset-based** scheduling.

> THE TRAP
>
> A start date a year ago, with catchup on, launches a year of runs on the first deploy.

Starting a DAG when a file lands

01

File lands

Westgrid writes to the landing bucket.

02

Eventarc

Fires on the object finalize event.

03

Cloud Run function

Calls the Airflow REST API.

04

DAG runs

schedule=None, with the event as its config.

HONEST LIMIT

With Private IP or VPC Service Controls, use the Pub/Sub approach instead of the function.

admin@sthithapragna.com

Keeping workers healthy

- The commonest cause of **worker pod eviction** is a task's memory spiking.
- Add worker memory, or lower **worker_concurrency** so each runs fewer tasks.
- Push heavy work out: **KubernetesPodOperator**, or BigQuery, Dataflow, Spark.
- Keep DAG files light. **Top-level code** runs on every parse, and slows everything.

> REMEMBER
>
> An evicted worker interrupts every task running on it. Idempotent, retriable tasks survive that.

Managed Airflow generations

GENERATION	INFRASTRUCTURE	AIRFLOW
Gen 3	Hidden: no cluster in your project	2 and 3, DAG processors scale
Gen 2	Autopilot GKE in your project	2 only
Legacy Gen 1	Standard GKE, scaled by hand	End of life, 15 September 2026

CURRENCY CHECK

Composer 1, 2 and 3 in older questions are these generations. The product is now Managed Airflow.

admin@sthithapragna.com

Phrases that pick the answer

THE QUESTION SAYS	REACH FOR
Many dependent steps, **backfill** needed	Managed Airflow
Chain a few **APIs**, idle most of the day	Workflows
Run one thing **every night**	Cloud Scheduler
Hundreds of sensors **fill the workers**	Deferrable operators
Start when a **file lands**	Eventarc, function, Airflow REST API
Reruns create **duplicates**	Idempotent tasks, overwrite by partition

THE PATTERN

Orchestrate, wait cheaply, and make every step safe to repeat.

admin@sthithapragna.com

> When a customer changes tariff, Larkspur must call the billing API, then the CRM API, then send a confirmation email.
>
> It happens a few hundred times a day, must start within seconds, and nothing else needs orchestrating.

What should the team use?

A. A new Managed Airflow environment, with one DAG run per change

B. Cloud Scheduler, calling each API every minute in turn

C. A Dataflow streaming job that makes the three calls in order

D. Workflows, started per change, with retries on each call

◆ PAUSE AND DECIDE

What should the team use?

A Managed Airflow

Seconds of latency, and paid for even when idle.

B Cloud Scheduler

A timetable, not a reaction to each change.

C Dataflow

A data pipeline engine for a handful of API calls.

D Workflows

Serverless, milliseconds to start, built-in retries.

THE SIGNAL

A few API calls, started per event, idle otherwise: Workflows.

> The settlement DAG appends each night's results to curated.settlement. After a retry last week, one day's results were loaded twice.
>
> Finance needs reruns of any day to be safe.

What should the team change?

A Turn off retries, so that a failed task is never run twice

B Add a deduplication query that runs once at the end of each month

C Make each run overwrite its own day's partition, not append

D Raise worker memory, so that the task never fails mid-load

◆ PAUSE AND DECIDE

What should the team change?

A No retries

Every blip now fails the run instead.

B Monthly dedupe

Wrong figures for up to a month.

C Overwrite by partition

Idempotent: one run or five, the same result.

D More memory

Failures have many causes. Reruns still duplicate.

THE SIGNAL

Reruns duplicate data: make the task idempotent, overwrite by partition.

> The landing DAGs use classic sensors to wait for 300 partner files, some for hours.
>
> Workers are full of waiting sensors, and real tasks sit queued behind them.

What should the team change?

A Raise the worker maximum, so that there is room for every sensor

B Switch the sensors to deferrable mode, with a triggerer running

C Lower the sensors' poke interval, so they finish checking sooner

D Move every partner file check into one large Python task

◆ PAUSE AND DECIDE

What should the team change?

A More workers

Pays for workers that only wait.

B Deferrable sensors

Waiting moves to the triggerer, workers are freed.

C Faster polling

More load, the same occupied slots.

D One big task

Not idempotent, and one failure blocks every file.

THE SIGNAL

Many long waits filling the workers: deferrable operators.

> A new DAG with start_date one year ago and catchup=True was deployed this morning.
>
> It immediately launched 365 runs, overloading the warehouse. Only the last 7 days actually need processing.

What should the team do?

A Set catchup=False, then backfill only the last seven days

B Move the start date to tomorrow, and skip the missing week

C Keep catchup on, but limit the DAG to one active run

D Delete the DAG, and run the seven days by hand in BigQuery

◆ PAUSE AND DECIDE

What should the team do?

A **Catchup off, backfill seven**
Exactly the runs needed, deliberately.

B **Start tomorrow**
The week that needs processing never runs.

C **One active run**
Still a year of runs, just in a slow line.

D **Run by hand**
Loses retries, history and the DAG itself.

THE SIGNAL

Too many historical runs: catchup off, then a deliberate backfill.

Module 35 in five lines

- **Airflow** for data pipelines, **Workflows** for chaining services, **Scheduler** for one timer.
- Tasks are **idempotent**: one step each, overwrite by partition, with retries.
- **Deferrable** operators wait in the triggerer, freeing workers.
- Catchup **off**, then backfill on purpose. Fixed start dates.
- Heavy work out of Airflow. **Gen 3** is current, Gen 1 is end of life.

COMING UP

Next: module 36, monitoring and troubleshooting.

MODULE 36 · SECTION 5 - MAINTAINING AND AUTOMATING DATA WORKLOADS

Monitoring and Troubleshooting

Cloud Monitoring and alerting, Cloud Logging and sinks, audit logs, BigQuery's monitoring charts and jobs views, and reading quota, resource and access errors.

SECTION 5 of 5 EXAM WEIGHT 18% MODULE 36 of 45

admin@sthithapragna.com

Metrics tell you something is wrong. **Logs tell you what. Audit logs tell you who.**

Where to look for what

QUESTION	LOOK IN
Is something wrong right now?	**Cloud Monitoring** metrics and alerts
What exactly failed, and why?	**Cloud Logging**, and job error messages
Who changed or read this?	**Cloud Audit Logs**
Which BigQuery jobs cost or fail most?	**INFORMATION_SCHEMA.JOBS** and admin charts
What does it cost?	**Billing export**, from module 34

THE PATTERN

Each question has a home. Questions that look in the wrong place are designed as traps.

Alerting policies

- **Metric threshold**: fire when a metric crosses a line for a set window.
- **Metric absence**: fire when a metric stops arriving, such as a silent pipeline.
- **Log-based** policies fire when a particular message appears in the logs.
- Notify through **channels**: email, chat, paging tools, Pub/Sub, webhooks.

> REMEMBER
>
> Put the runbook link in the policy's documentation field, so the alert says what to do.

Metrics worth an alert at Larkspur

SERVICE	METRIC	WHY
Pub/Sub	Oldest unacked message age	Subscribers falling behind
Dataflow	Data freshness, system lag	Streaming job stuck or slow
BigQuery	Slot usage near the reservation maximum	Queries queuing
Bigtable	CPU utilisation of the hottest node	Hotspot or undersized cluster
Managed Airflow	Failed task count, DAG run duration	Nightly run at risk

THE TIEBREAKER

Alert on symptoms users feel, such as freshness, before causes such as CPU.

Routing logs with sinks

- The **Log Router** sends each entry through sinks, each with an inclusion filter.
- Destinations: log buckets, **BigQuery**, Cloud Storage, **Pub/Sub**, another project.
- **Aggregated sinks** collect a whole organisation or folder in one place.
- **Exclusion filters** drop noisy logs. Sinks never route logs from before they existed.

> THE TIEBREAKER
>
> SQL over logs: route to BigQuery, or use Log Analytics with a linked dataset. Third-party tools: Pub/Sub.

Every BigQuery audit log, into one dataset

```
gcloud logging sinks create lk-bq-audit \
  bigquery.googleapis.com/projects/lk-sec-prod/datasets/audit_logs \
  --organization=123456789012 --include-children \
  --log-filter='protoPayload.serviceName="bigquery.googleapis.com"' \
  --use-partitioned-tables
```

3 An aggregated sink: every project in the organisation.

4-5 BigQuery audit entries only, into partitioned tables.

THE TRAP

The sink's service account needs write access on the dataset, or nothing arrives.

Four kinds of audit log

TYPE	RECORDS	ON BY DEFAULT?
Admin Activity	Changes to resources and settings	Always, cannot be disabled
Data Access	Reads of data and metadata	Off, except for BigQuery
System Event	Changes made by Google systems	Always, cannot be disabled
Policy Denied	Access blocked by a security policy	Yes, can be excluded

THE TRAP

Reading Data Access logs needs the Private Logs Viewer role. Logs Viewer alone sees nothing there.

Who deleted the bills table?

```sql
SELECT timestamp,
       protopayload_auditlog.authenticationInfo.principalEmail AS who,
       protopayload_auditlog.methodName AS action
FROM `lk-sec-prod.audit_logs.cloudaudit_googleapis_com_activity`
WHERE protopayload_auditlog.resourceName LIKE '%tables/bills'
  AND protopayload_auditlog.methodName LIKE '%Delete%'
  AND DATE(timestamp) >= '2026-10-01'
ORDER BY timestamp DESC;
```

2 The principal who made the call.

4 Admin Activity entries land in the activity table.

7 A date filter prunes the partitioned log table.

REMEMBER

Recovery is time travel, from module 23. The audit log only tells you who, and when.

BigQuery's own monitoring

- **Resource utilisation** charts: slots, concurrency, failed jobs, bytes, shuffle, 30 days back.
- The **operational health** dashboard: top active queries and insights, for reservations.
- Both are built on **INFORMATION_SCHEMA**: JOBS, JOBS_TIMELINE and others.
- Failed jobs group by **error type**, so the commonest error stands out.

> HONEST LIMIT
>
> The operational health dashboard does not cover on-demand workloads. Use the utilisation charts there.

BigQuery errors, decoded

ERROR	MEANS	DO
accessDenied 403	No permission on the resource	Grant access. Retrying will not help
rateLimitExceeded 403	Too many requests too quickly	Back off, then retry in seconds
quotaExceeded 403	A longer quota is used up	Read which one. Wait, or redesign
resourcesExceeded 400	One query needs too much	Reshape it: ORDER BY, skew, size
notFound 404	Wrong name, or wrong location	Check names and the job location
backendError 5xx	Temporary service problem	Retry with exponential backoff

THE TRAP

Three different 403s, three different fixes. Read the reason, not just the code.

Troubleshooting, in order

01
Alert
A symptom metric fires, with a runbook link.

02
Scope
Dashboards: what, where, since when?

03
Logs
The error reason and the failing step.

04
Changes
Audit logs: did anyone change something?

05
Fix and learn
Fix, then add the missing alert or test.

THE PATTERN

Most incidents follow a change. Check the audit log before guessing.

Phrases that pick the answer

THE QUESTION SAYS	REACH FOR
Know when a pipeline goes **silent**	Metric-absence alerting policy
Who deleted or changed it?	Admin Activity audit logs
Who **read** the customer table?	Data Access logs, Private Logs Viewer
Logs from **every project**, queried in SQL	Aggregated sink to BigQuery
Send logs to a **third-party** tool	Sink to Pub/Sub
rateLimitExceeded on bursts	Exponential backoff and retry

THE PATTERN

Something wrong, what, or who: metrics, logs, or audit logs.

> Last Tuesday, Larkspur's head-end stopped publishing meter readings for four hours, and nobody noticed until morning.
>
> There were no errors, because nothing was running at all.

What alert would have caught it?

A. A log-based alert on error entries from the publisher

B. A threshold alert on the subscription's backlog growing

C. A metric-absence alert on the topic's published messages

D. A weekly report of message counts, emailed to the whole team

◆ PAUSE AND DECIDE

What alert would have caught it?

A Log-based errors

Nothing was running, so no errors were logged.

B Backlog growing

No messages arrive, so the backlog never grows.

C Metric absence

Fires when published messages stop arriving.

D Weekly report

Finds out days later.

THE SIGNAL

A silent failure, with no errors: a metric-absence condition.

> Someone deleted a curated BigQuery table overnight. Data Access logging was never changed from its defaults.
>
> The security team must find out who did it.

Where should they look?

A The BigQuery job history of every analyst in the project

B Cloud Monitoring's chart of the dataset's table count

C The project's billing export, filtered to BigQuery

D Admin Activity audit logs for the table's delete call

◆ PAUSE AND DECIDE

Where should they look?

A **Job history**
Deletes may not run as query jobs, and it is slow to search.

B **Table count chart**
Shows that it happened, not who did it.

C **Billing export**
Costs, not actions or identities.

D **Admin Activity logs**
Always written, and record the principal.

THE SIGNAL

Who changed or deleted a resource: Admin Activity audit logs.

> Larkspur's security team wants every BigQuery audit log entry from all 60 projects in one place, queryable with SQL,
>
> including projects created in future.

What should the team set up?

A An aggregated sink at the organisation, to a BigQuery dataset

B A sink in each of the 60 projects, each to a BigQuery dataset

C A sink at the organisation, routing all the logs to Pub/Sub

D Export each project's logs to CSV in Cloud Storage every night

◆ PAUSE AND DECIDE

What should the team set up?

A Aggregated sink to BigQuery
One sink, every project, new ones included, in SQL.

B Sixty sinks
Works today, misses new projects tomorrow.

C Pub/Sub
For streaming to tools, not SQL by itself.

D Nightly CSV
Manual, late, and not queryable directly.

THE SIGNAL

Every project, including new ones, in SQL: an aggregated sink to BigQuery.

> A loader script streams readings into BigQuery in short, heavy bursts.
>
> Some calls fail with rateLimitExceeded, and others succeed a second later. The script gives up on the first error.

What should the team change first?

A Request a much higher project quota from Google Cloud support

B Retry failed calls with exponential backoff and jitter

C Switch the dataset to a different region with spare capacity

D Grant the script's service account the BigQuery Admin role

◆ PAUSE AND DECIDE

What should the team change first?

A Raise the quota

A short-term rate limit, not a used-up quota.

B Backoff and retry

Exactly what the reason asks for.

C Another region

Moves data, and the bursts continue.

D BigQuery Admin

Permissions are not the problem.

THE SIGNAL

rateLimitExceeded on bursts: slow down, and retry with backoff.

> A scheduled query, in a project whose datasets are in europe-west2, fails with notFound for a table that clearly exists.
>
> The job was configured to run in the EU multi-region.

What should the team change?

A Recreate the table in the EU multi-region, then rerun the query

B Grant the scheduled query's account Data Viewer on the dataset

C Run the job in europe-west2, where the datasets live

D Raise the project's query quota, then retry the job later

◆ PAUSE AND DECIDE

What should the team change?

A **Copy to EU**
A second copy, and a residency question.

B **Grant Data Viewer**
Missing access would say accessDenied.

C **Match the location**
Jobs must run where the data lives.

D **More quota**
Not a quota problem.

THE SIGNAL

notFound for a table that exists: check the job's location.

> Auditors ask which people read curated.customers last month. Data Access logs are on, as BigQuery's default.
>
> An analyst with the Logs Viewer role searches, but sees no Data Access entries at all.

What should the team do?

A. Enable Data Access logs for BigQuery, then search again next month

B. Search the Admin Activity logs instead, which record every read

C. Use INFORMATION_SCHEMA.TABLE_STORAGE to list the table's readers

D. Grant the analyst Private Logs Viewer, then search the entries

◆ PAUSE AND DECIDE

What should the team do?

A **Enable logs**
They are already on for BigQuery.

B **Admin Activity**
Records changes, not reads.

C **TABLE_STORAGE**
Storage sizes, not readers.

D **Private Logs Viewer**
The role that can read Data Access entries.

THE SIGNAL

Data Access logs exist but are invisible: the reader needs Private Logs Viewer.

Module 36 in five lines

- Metrics say **something** is wrong, logs say **what**, audit logs say **who**.
- Alert on **symptoms**. Metric absence catches pipelines that go silent.
- **Aggregated sinks** to BigQuery for SQL, to Pub/Sub for outside tools.
- Admin Activity always on. **Data Access** needs Private Logs Viewer.
- Read the error **reason**: rate limit, quota, resources, access, location.

COMING UP

Next: module 37, operating streaming pipelines.

MODULE 37 · SECTION 5 - MAINTAINING AND AUTOMATING DATA WORKLOADS

Operating Streaming Pipelines: Update, Drain, Diagnose

Updating a running job, the compatibility check, drain or cancel, snapshots, replaying from Pub/Sub, the three streaming signals, and finding a stuck step.

SECTION 5 of 5 EXAM WEIGHT 18% MODULE 37 of 45

admin@sthithapragna.com

A streaming job carries state. Every change is a question: what happens to the data in flight?

admin@sthithapragna.com

Four ways to change a running job

ACTION	IN-FLIGHT DATA	USE WHEN
Update with --update	Kept, with state	New code, compatible graph
Drain	Finished, windows closed early	Stopping, or an incompatible change
Cancel	Buffered data may be lost	Speed matters more than data
Snapshot, then new job	Saved, restored into the new job	Rollback, or a safe restart

THE TIEBREAKER

Losing nothing and keeping state is an update. Losing nothing but restarting is a drain.

Replacing a running job

- Launch with **--update**, the **same job name** and the same region.
- A **compatibility check** runs first. If it fails, the old job keeps running.
- State and in-flight records **carry over** to the replacement.
- Just changing the worker range? An **in-flight update**, with no new job.

> REMEMBER
>
> Batch jobs cannot be updated. Fix the code and run the job again.

What breaks the compatibility check

- **Renamed or removed** transforms, unless mapped with --transformNameMapping.
- **Adding or removing side inputs**, or changing a **coder**.
- Removing **stateful** steps, or moving the job to another **zone**.
- Small window changes are allowed. Schemas may **add** fields, never rename them.

> CURRENCY CHECK
>
> Test first: the graph_validate_only option checks compatibility without touching the running job.

Updating the readings pipeline

```
1 gcloud dataflow flex-template run meter-readings-prod \
2   --template-file-gcs-location=gs://lk-artifacts-prod/templates/readings-v42.json \
3   --region=europe-west2 --update \
4   --transform-name-mappings=ParseReading=ParseAndValidate
5
6 # Only the worker range changes? No new job needed:
7 gcloud dataflow jobs update-options JOB_ID --region=europe-west2 \
8   --min-num-workers=5 --max-num-workers=40
```

1-3 Same job name and region, with --update.

4 The step was renamed, so map old to new.

7 An in-flight update, for the evening peak.

HONEST LIMIT

If the check fails, the old job is untouched. Read the error, fix the graph, try again.

Drain or cancel?

DRAIN

Finish, then stop

- Stops reading new data
- Processes what is buffered
- Closes windows early, fires triggers
- Can take a long time

CANCEL

Stop now

- Halts at once
- Buffered data may be lost
- Releases resources quickly
- Cannot be restarted, only cloned

THE TRAP

Draining does not fix a stuck pipeline. A stuck job needs a code fix by update, or a cancel.

Saving state before a risky change

- A **Dataflow snapshot** saves a streaming job's state, optionally with its Pub/Sub sources.
- Start a new job from it with **--createFromSnapshot**, on Streaming Engine, same region.
- Kept for **7 days** by default, up to 30. Pub/Sub snapshots last at most 7 days.
- Sinks are not snapshotted. Writes since the snapshot may be written again.

> THE PATTERN
>
> Snapshot before every risky update. It is the rollback plan.

Replaying from Pub/Sub

- **Seek to a time**: messages after it become unacknowledged again, and are redelivered.
- Needs **retained acknowledged messages** on the subscription, or topic retention.
- **Seek to a snapshot**: return to a saved point, on any subscription to the topic.
- Replays deliver messages **again**. Downstream must deduplicate or overwrite.

> FOR LARKSPUR
>
> Take a Pub/Sub snapshot before deploying new subscriber code. Seek back if it misbehaves.

Three signals of a streaming job

SIGNAL	MEASURES	RISING MEANS
System lag	How long items wait or process, at most	A step is slow or stuck
Data freshness	How far output trails event time	Results are going stale
Backlog	Unread input, in bytes or seconds	Workers cannot keep up

THE TIEBREAKER

Backlog with low CPU: a stuck step or a hot key. Backlog with high CPU: too few workers.

Finding the stuck step

- Streaming retries a failing bundle **forever**, so one bad record can stall the job.
- Batch retries a bundle **four times**, then fails the whole job.
- '**Processing stuck** in step' means slow user code or an external call.
- Catch exceptions in the DoFn, and send bad records to a **dead-letter** output.

> REMEMBER
>
> The Diagnostics tab and worker logs, filtered to errors, point to the failing step.

Phrases that pick the answer

THE QUESTION SAYS	REACH FOR
New code, **keep state**, no data loss	Update with --update
Update failed the **compatibility check**	Drain, then launch the new job
A step was **renamed**	--transformNameMapping
Roll back a bad deployment	Dataflow snapshot, or Pub/Sub seek
Job **stalls** on one bad record	Catch it, dead-letter output
Change only the **worker range**	In-flight update-options

THE PATTERN

Keep the state, finish the data, or stop now: decide before you touch the job.

> The readings pipeline holds an hour of deduplication state. New code renames one step and adds a field to the output.
>
> The team must deploy it without losing state or dropping readings.

How should they deploy?

A Cancel the job, then start the new code as a fresh job

B Update with --update and --transformNameMapping for the step

C Drain the job, then start the new code as a fresh job

D Run the new code beside the old job, writing to the same table

◆ PAUSE AND DECIDE

How should they deploy?

A Cancel and restart

Buffered data and all state are lost.

B Update with mapping

State carries over. The mapping passes the check.

C Drain and restart

No data lost, but the deduplication state is.

D Run both

Every reading written twice.

THE SIGNAL

New code, keep state, a renamed step: update with a name mapping.

> An update to the readings pipeline fails the compatibility check, because the new code removes a side input.
>
> The old job is still running. The new code must go live with no readings lost.

What should the team do?

A Force cancel the old job, then rerun the update command

B Cancel the old job, then launch the new code immediately

C Retry the same update with more workers, so the check passes

D Drain the old job, then launch the new code as a new job

◆ PAUSE AND DECIDE

What should the team do?

A Force cancel

Loses data, may leak VMs, and the check still fails.

B Cancel

Buffered readings are lost.

C More workers

Compatibility has nothing to do with workers.

D Drain, then launch

Buffered readings finish, then the new job starts.

THE SIGNAL

Incompatible change, no data lost: drain, then start the new job.

> The readings pipeline's backlog has grown for an hour and system lag keeps rising, but worker CPU is low.
>
> Worker logs repeat 'Processing stuck in step EnrichTariff' for several minutes at a time.

What is the most likely cause?

A. A slow or hanging external call inside the EnrichTariff step

B. Too few workers for the evening peak, so the job needs more of them

C. Pub/Sub has run out of retention, and is deleting messages

D. BigQuery streaming quota at the sink has been exceeded

◆ PAUSE AND DECIDE

What is the most likely cause?

A Slow external call

Low CPU, stuck in one step: waiting on something.

B Too few workers

High CPU would show that, not low.

C Retention

Messages are waiting, not being deleted.

D Sink quota

Errors would name the write step, not enrichment.

THE SIGNAL

Backlog with low CPU, stuck in one step: slow user code or an external call.

> A deployment of new code to the meter-alerts subscriber mis-flagged every reading for two hours, and acknowledged them all.
>
> The subscription retains acknowledged messages. The team has fixed the code, and needs those two hours reprocessed.

What should the team do?

A Create a new subscription, which receives every older message

B Ask the head-end to republish two hours of readings

C Seek the subscription to the time of the bad deployment

D Drain the subscriber, so the acknowledged messages are kept

◆ PAUSE AND DECIDE

What should the team do?

A New subscription

Only receives messages published after it exists.

B Republish

Slow, and the source may not hold them.

C Seek to the time

Messages since then become unacknowledged again.

D Drain

Finishes buffered work. It does not undo acks.

THE SIGNAL

Reprocess acknowledged messages: seek the subscription back in time.

Module 37 in five lines

- **Update** keeps state. Same name, --update, and a mapping for renamed steps.
- Check fails? **Drain**, then start the new job. Cancel may lose data.
- **Snapshot** before risky changes. Pub/Sub **seek** replays acknowledged messages.
- Lag, freshness, backlog: low CPU means **stuck**, high CPU means **too few workers**.
- Streaming retries **forever**. Dead-letter bad records, never let them stall.

COMING UP

Next: module 38, when pipelines fail: restarts, bad data and reprocessing.

MODULE 38 · SECTION 5 - MAINTAINING AND AUTOMATING DATA WORKLOADS

When Pipelines Fail: Restarts, Bad Data, Reprocessing

Idempotent loads, dead-letter topics and tables, quarantining bad records, time travel and snapshots, backfills, and where workers run.

SECTION 5 of 5 EXAM WEIGHT 18% MODULE 38 of 45

admin@sthithapragna.com

Assume every job will run twice. Design it so the second run is harmless.

admin@sthithapragna.com

Five failures, five remedies

WHAT WENT WRONG	REMEDY
Job died **halfway** and reran	Idempotent writes: overwrite or MERGE
A few **bad records**	Dead-letter output, quarantine table
Bad data **already written**	Time travel, or a table snapshot
Data **missing** for some days	Backfill those intervals
A **zone** went down	Regional placement, not one zone

THE PATTERN

Most questions name the failure in one phrase. Match it to the remedy.

Making a rerun harmless

- **Overwrite the partition** for the run date, never append to the table.
- **MERGE** on a business key, so a repeated row updates instead of duplicating.
- Write files to **fixed paths** per run date, so a rerun replaces them.
- **WRITE_APPEND** plus a retry is how duplicates are born.

> THE TRAP
>
> Turning off retries is not idempotency. It just makes failures permanent.

Two safe ways to load the settlement run

```
# 1. Replace only the run date's partition
bq load --replace --source_format=PARQUET \
  'curated.settlement_daily$20261009' \
  'gs://lk-lake-prod/settlement/run_date=2026-10-09/*.parquet'

-- 2. Or upsert on the business key
MERGE curated.settlement_daily t
USING staging.settlement_20261009 s
ON t.run_date = s.run_date AND t.mpan = s.mpan
WHEN MATCHED THEN UPDATE SET kwh = s.kwh, amount_gbp = s.amount_gbp
WHEN NOT MATCHED THEN INSERT ROW
```

2-3 Replace plus a partition decorator: one day only.

9 Match on the key, so repeats update.

THE PAYOFF

Run either one twice. The table ends up exactly the same.

At-least-once or exactly-once?

AT LEAST ONCE

Repeats possible

- Pub/Sub delivery, by design
- STORAGE_API_AT_LEAST_ONCE writes
- At-least-once streaming mode
- Lower latency and cost

EXACTLY ONCE

Repeats removed

- Dataflow dedupes on message ID
- STORAGE_WRITE_API writes
- Exactly-once streaming mode
- Or dedupe yourself on a key

THE TIEBREAKER

A publisher that retries makes a new message ID. Dedupe on an ID attribute, or a business key.

Pub/Sub dead-letter topics

- After **5 to 100** delivery attempts, default 5, a message goes to the dead-letter topic.
- The Pub/Sub **service agent** needs Publisher on that topic and Subscriber on the subscription.
- Pair it with an **exponential backoff** retry policy: 10 to 600 seconds by default.
- The attempt count is **approximate**, and forwarding is best effort.

> THE TRAP
>
> Dataflow reading Pub/Sub? Do not use dead-letter topics or backoff. Dead-letter inside the pipeline.

The dead-letter pattern inside Dataflow

01
Read
From Pub/Sub, retry immediately

›

02
Try to parse
Catch the exception in the DoFn

›

03
Good records
Main output, on to BigQuery

›

04
Bad records
Tagged output, raw payload and error

›

05
Dead-letter table
Fix the code, then replay

REMEMBER

Rows BigQuery rejects come back from getFailedStorageApiInserts. Send them to the same table.

admin@sthithapragna.com

Bad records in batch loads

- **max_bad_records** defaults to 0. One bad row fails the whole load job.
- Raising it lets the job pass, but the bad rows are **silently lost**.
- Better: load raw into **staging**, validate in SQL, move failures to **quarantine**.
- Keep the reason and the source file, then **alert** on the quarantine count.

> THE TIEBREAKER
>
> Lost rows and quarantined rows look the same on a dashboard. Only one can be repaired.

Time travel: undoing a bad change

- Query any table **as it was**, up to 7 days back by default, with FOR SYSTEM_TIME AS OF.
- The window is set per **dataset** or project, from **2 to 7 days**.
- A **deleted** table comes back with bq cp and an @ timestamp in milliseconds.
- **Fail-safe** keeps 7 more days, but only Customer Care can recover it.

> CURRENCY CHECK
>
> Shortening the window saves money only on physical storage billing. On logical billing it is included.

Restoring the tariff history

```sql
-- A bad MERGE ran at 14:00 yesterday. Rebuild from 13:59.
CREATE OR REPLACE TABLE curated.account_tariff_history_restored AS
SELECT * FROM curated.account_tariff_history
  FOR SYSTEM_TIME AS OF TIMESTAMP '2026-10-09 13:59:00 UTC';

# A table was dropped by mistake? Copy it back from a moment before.
bq cp 'curated.bills@1791385140000' curated.bills_restored
```

4 The table as it stood at 13:59.

7 Epoch milliseconds, inside the window.

THE PATTERN

Restore to a new table first. Check it, then swap it in.

Time travel or a table snapshot?

TIME TRAVEL

Automatic, short

- Always on, nothing to set up
- Any moment in the last 2 to 7 days
- Gone once the window passes
- Covers mistakes you notice quickly

TABLE SNAPSHOT

Deliberate, long

- Read-only copy at one moment
- Kept until it expires, or forever
- Billed only for changed data
- Same region as the base table

THE TRAP

Snapshot before risky changes and at month end. It is not a disaster copy: same region only.

admin@sthithapragna.com

Backfilling the days you missed

- Each run works on its **data interval**, from the logical date, not the clock.
- **Backfill**: Trigger, then Backfill in the UI, or airflow backfill create.
- **Clear** failed tasks to rerun them. Catchup is off by default.
- A streaming gap? Run the **same Beam code** in batch over the archived files.

> REMEMBER
>
> A backfill is only safe if every task is idempotent. It reruns days that may half exist.

Backfilling five nights of settlement

```
# Rerun 1 to 5 October, two nights at a time
airflow backfill create --dag-id settlement_nightly \
  --from-date 2026-10-01 --to-date 2026-10-05 \
  --reprocess-behavior failed --max-active-runs 2

# Or rerun only what failed in one night's run
airflow tasks clear settlement_nightly --only-failed \
  --start-date 2026-10-03 --end-date 2026-10-03
```

3 The intervals to rerun.

4 Only failed runs, two at once.

7 Clearing makes the scheduler run it again.

FOR LARKSPUR

Limit active runs, so the backfill does not starve tonight's real run of slots.

Zones, regions and Dataflow workers

- Set **--region**. With Streaming Engine or Dataflow Shuffle, workers use **every zone**.
- Lose a zone, and workers are **recreated in other zones**. The job carries on.
- **--worker_zone** pins one zone. The docs call it uncommon, and not recommended.
- ZONE_RESOURCE_POOL_EXHAUSTED: that zone has **no capacity**. Let Dataflow choose.

> REMEMBER
>
> Spark clusters are zonal. Keep data in Cloud Storage, so a new cluster can start in another zone.

Phrases that pick the answer

THE QUESTION SAYS	REACH FOR
Retry created **duplicate** rows	Overwrite the partition, or MERGE
Bad messages **stall** the Dataflow job	Catch them, dead-letter table
Push subscriber **keeps failing** on a message	Pub/Sub dead-letter topic
Bad update **yesterday**	Time travel, FOR SYSTEM_TIME AS OF
Keep a copy of month end **for a year**	Table snapshot with expiration
Survive **losing a zone**	--region, no worker zone

THE PATTERN

Repeatable writes, a home for bad records, a way back in time: build all three before you need them.

> An Airflow task loads each night's settlement files into curated.settlement_daily, partitioned by run_date, with WRITE_APPEND.
>
> Last night it failed halfway, Airflow retried it, and that night's totals are now doubled for some meters.

How should the team stop this happening again?

A Turn off Airflow retries, so the task can only ever run once

B Add DISTINCT to every report that reads the settlement table

C Overwrite the run date's partition with WRITE_TRUNCATE

D Raise max_bad_records, so the load finishes on its first attempt

◆ PAUSE AND DECIDE

How should the team stop this happening again?

A **No retries**
Every transient failure now needs a person.

B **DISTINCT in reports**
Hides the problem, and costs every query.

C **Overwrite the partition**
A rerun replaces the day. Retries become safe.

D **Raise bad records**
Drops rows silently. Duplicates remain.

THE SIGNAL

Duplicates after a retry: make the write idempotent, not the retry rarer.

> After a firmware update, some meters send malformed JSON. The Dataflow parse step throws on them, and the readings job is stalling.
>
> Good readings must keep flowing, and the bad messages must be kept so they can be repaired.

What should the team do?

A Catch the parse error, and write bad messages to a dead-letter table

B Drain the job, then restart it with more workers to clear the backlog

C Filter malformed messages out in the parse step, and discard them

D Add a Pub/Sub dead-letter topic to the subscription, with five attempts

◆ PAUSE AND DECIDE

What should the team do?

A **Dead-letter table**
Good data flows, bad data is kept for repair.

B **Drain, more workers**
The same messages fail again on the new job.

C **Filter and discard**
Flowing again, but the data is lost.

D **Dead-letter topic**
Not for Dataflow, which acks before parsing.

THE SIGNAL

Dataflow stalls on bad records: dead-letter inside the pipeline.

> A faulty MERGE ran against curated.account_tariff_history at 14:00 yesterday, overwriting thousands of tariff codes.
>
> The last table snapshot is six days old. The table must be put back as it was, losing nothing since.

What should the team do?

A. Ask Customer Care to recover the table from fail-safe storage

B. Restore last week's table snapshot, then rerun six days of loads

C. Re-ingest the whole account history from the change stream

D. Query it FOR SYSTEM_TIME AS OF 13:59 into a new table

◆ PAUSE AND DECIDE

What should the team do?

A Fail-safe

Only after the time travel window has passed.

B Old snapshot

Six days of reloading, and room for new errors.

C Re-ingest

Slow, and the stream may not hold it all.

D Time travel

Yesterday is well inside the window.

THE SIGNAL

A mistake noticed within days: time travel first.

admin@sthithapragna.com

> The readings Dataflow job was launched with --worker_zone=europe-west2-a years ago. A zonal outage stopped it for three hours.
>
> The team wants future jobs to survive losing a zone, with the least ongoing effort.

What should the team do?

A. Run a duplicate copy of the job in europe-west2-b at all times

B. Set only --region, on Streaming Engine, with no worker zone

C. Move the job to a larger machine type in the same zone

D. Take a Dataflow snapshot every hour, so a restart elsewhere is quick

◆ PAUSE AND DECIDE

What should the team do?

A Duplicate job

Twice the cost, and twice the writes to reconcile.

B Region only

Workers spread across zones, and move on failure.

C Bigger machines

Still one zone. The outage stops it again.

D Hourly snapshots

Faster recovery, but still an outage and manual work.

THE SIGNAL

Survive a zone loss: name the region, not a zone.

Module 38 in five lines

- Assume every job runs **twice**. Overwrite the partition, or MERGE on a key.
- Dataflow? Dead-letter **inside the pipeline**. Pub/Sub dead-letter topics are for other subscribers.
- Batch loads: **quarantine** bad rows. Raising max_bad_records loses them.
- **Time travel** for recent mistakes. **Snapshots** for long-lived copies.
- **Backfill** by data interval. Set the **region**, never pin one zone.

COMING UP

Next: module 39, replication, failover and regional outages.

MODULE 39 · SECTION 5 - MAINTAINING AND AUTOMATING DATA WORKLOADS

Replication, Failover and Regional Outages

RPO and RTO, Cloud SQL HA and cross-region replicas, Spanner configurations, Bigtable replication, dual-region buckets, BigQuery managed disaster recovery, and Memorystore.

SECTION 5 of 5 EXAM WEIGHT 18% MODULE 39 of 45

admin@sthithapragna.com

Two numbers decide every answer. How much data can you lose, and how long can you be down?

RPO and RTO

RPO

Recovery point objective

- How much data you can lose
- Measured back from the failure
- Set by how replication works
- Synchronous can reach zero

RTO

Recovery time objective

- How long you can be down
- Measured forward from the failure
- Set by how failover works
- Automatic beats a manual runbook

REMEMBER

High availability survives a zone, usually automatically. Disaster recovery survives a region, often by hand.

What already survives losing a zone

SURVIVES A ZONE BY DEFAULT	NEEDS YOU TO ADD IT
BigQuery: two zones, always	**Cloud SQL**: turn on HA
Cloud Storage regional bucket	**Bigtable**: add a second cluster
Spanner regional: three zones	**Memorystore**: Standard tier
Dataflow with regional placement	**Spark** clusters: one zone each

THE TRAP

Nothing on this slide survives losing a whole region. That always takes a deliberate choice.

High availability: the zone layer

- A **standby** in a second zone. A write commits only once **both zones** have it.
- Failover is **automatic**, typically about **60 seconds**, on the same IP address.
- Costs **twice** a standalone instance: CPU, memory and storage.
- Same region only. HA does **nothing** for a regional outage.

> THE NUMBERS
>
> HA gives an RPO of zero and an RTO of about a minute, for a zone.

Cross-region replicas: the region layer

- A read replica in **another region**, kept up to date **asynchronously**.
- In an outage, the last transactions may **not have arrived**. RPO is not zero.
- **Promote** it by hand. It becomes a standalone, writable primary.
- **Enterprise Plus** advanced DR: a designated DR replica, failover and switchover back.

> THE PATTERN
>
> Before promoting, check replication lag. Afterwards, turn on HA and recreate the other replicas.

Larkspur's tariff database: the region runbook

```
# London is down. The DR replica is in Belgium, created in module 21.
gcloud sql instances describe lk-tariffs-dr --format='value(state)'

# 1. Promote it: now a standalone, writable primary
gcloud sql instances promote-replica lk-tariffs-dr

# 2. Give the new primary its own zone protection
gcloud sql instances patch lk-tariffs-dr --availability-type=REGIONAL

# 3. Point the tariff service at lk-tariffs-dr, then rebuild replicas
```

5 Manual and deliberate. There is no automatic regional failover here.

8 Regional means HA, in europe-west1 now.

FOR LARKSPUR

Rehearse this runbook. An untested DR plan has an unknown RTO.

Three ways to place a Spanner instance

CONFIGURATION	WHERE	AVAILABILITY
Regional	Three zones, one region	99.99%
Dual-region	Two regions, one country	99.999%, zero RPO
Multi-region	Two read-write regions, plus a witness	99.999%

CURRENCY CHECK

Dual-region and multi-region need the Enterprise Plus edition. Writes cost a little more latency.

Choosing the configuration

- **eur5**: London leads, Belgium second, Netherlands as witness.
- **eur6**: Netherlands leads, Frankfurt second, Zurich as witness.
- **dual-region-germany1**: Berlin and Frankfurt. Both copies stay in Germany.
- No dual-region config pairs London with another UK region.

> THE TRAP
>
> Residency first, then availability. A multi-region config can cross the very border you must keep.

Bigtable replication

- Add clusters in other zones or regions, up to **8 regions** per instance.
- Replication is **eventually consistent**, typically seconds to minutes behind.
- **App profiles** route traffic: single-cluster or multi-cluster routing.
- Conflicting writes to one cell: the **last write wins**.

> THE TIEBREAKER
>
> Need read-your-writes or strong consistency? Single-cluster routing. Fail over to another cluster only when needed.

Two app profiles on meter_live

LIVE-APP

Multi-cluster routing

- The customer app's reads and writes
- Nearest available cluster
- Fails over automatically
- Eventual consistency accepted

ANALYTICS

Single-cluster routing

- Data science scans
- Pinned to the second cluster
- Never slows the customer app
- Reads a copy seconds behind

THE PAYOFF

One instance, two clusters, two profiles: availability and workload isolation from the same design.

Buckets that survive a region

- **Regional**: at least two zones. Survives a zone, not a region.
- **Dual-region**: two regions you name. **Multi-region**: Google picks, 100 miles apart.
- Default replication: 99.9% within 1 hour, all within **12 hours**.
- **Turbo replication**: every object within **15 minutes**. Dual-region only.

> THE NUMBERS
>
> Durability is eleven nines everywhere. The location type changes availability and RPO, not durability.

BigQuery across regions

- Every dataset keeps copies in **two zones** of its location, automatically.
- A **multi-region** location like EU does **not** add cross-region redundancy.
- **Dataset replication**: a read-only replica elsewhere. Promotion needs the primary online.
- **Managed disaster recovery**: compute and storage fail over. Enterprise Plus only.

> THE TRAP
>
> The docs say dataset replication is not a plan for losing a whole region. Managed disaster recovery is.

Managed disaster recovery for the warehouse

```
1  -- 1. An Enterprise Plus failover reservation, standby in Belgium
2  CREATE RESERVATION `lk-admin-prod.region-europe-west2.warehouse_dr`
3  OPTIONS (slot_capacity = 200, edition = ENTERPRISE_PLUS,
4           secondary_location = 'europe-west1');
5
6  -- 2. Attach each dataset to it
7  ALTER SCHEMA curated SET OPTIONS (
8    failover_reservation = 'lk-admin-prod.warehouse_dr');
9
10 -- 3. In an outage, run in the secondary region
11 ALTER RESERVATION `lk-admin-prod.region-europe-west1.warehouse_dr`
12 SET OPTIONS (is_primary = TRUE, failover_mode = 'HARD');
```

3-4 Enterprise Plus, with a secondary location.

8 Each dataset is attached.

12 Hard does not wait. Soft waits, but needs both regions up.

THE NUMBERS

Capacity is ready five minutes after failover starts. The RPO goal is fifteen minutes.

Memorystore failover

- **Basic tier**: one node, no replica. A failure empties the cache.
- **Standard tier**: a primary and a replica, with **automatic failover**.
- About **30 seconds** on average. Clients reconnect to the same IP.
- Replication is **asynchronous**, so acknowledged writes may be lost.

> REMEMBER
>
> A cache can always be rebuilt from the database. Never make it the only copy of anything.

Phrases that pick the answer

THE QUESTION SAYS	REACH FOR
Cloud SQL, survive a **zone**	High availability, regional instance
Cloud SQL, survive a **region**	Cross-region replica, then promote
Zero RPO across regions, relational	Spanner dual-region or multi-region
Bigtable, isolate **batch scans**	Second cluster, single-cluster profile
Bucket, every object within **15 minutes**	Dual-region with turbo replication
BigQuery, query and write after a **region loss**	Managed DR, Enterprise Plus

THE PATTERN

Synchronous means zero loss and more cost. Asynchronous means a small loss and a manual step.

> The tariff database runs on Cloud SQL for PostgreSQL in London, with high availability turned on.
>
> The board wants it back in another region within an hour if London is lost. A few minutes of lost writes is acceptable, at the lowest cost.

What should the team add?

A Nothing, because HA already fails over to a standby automatically

B A cross-region read replica, promoted by hand in an outage

C A move of the whole database to a Spanner multi-region instance

D A nightly export of the database to a bucket in another region

◆ PAUSE AND DECIDE

What should the team add?

A Nothing more

The standby is in the same region.

B Cross-region replica

Minutes of loss, recovery well inside an hour.

C Spanner multi-region

Zero loss, but a migration and far more cost.

D Nightly export

Up to a day of lost writes.

THE SIGNAL

Survive a region, minutes of loss allowed: a cross-region replica.

> The customer app reads and writes meter_live in a single-cluster Bigtable instance in London.
>
> Data science scans of the same table slow the app at peak, and the app must also survive a zone failure.

What should the team do?

A Add nodes to the existing cluster, and run the scans overnight

B Copy meter_live into BigQuery every hour for the data scientists

C Add a second cluster, and give both workloads one multi-cluster profile

D Add a second cluster: scans pinned to it, the app on multi-cluster

◆ PAUSE AND DECIDE

What should the team do?

A More nodes

Still one zone, and scans still compete.

B Hourly copy

Helps the scans, but not the zone failure.

C One shared profile

Scans can land on the app's cluster.

D Two profiles

Isolation for scans, failover for the app.

THE SIGNAL

Isolate workloads and survive a zone: two clusters, two app profiles.

> Larkspur is building a billing ledger for German customers. It needs strong consistency and no lost transactions, even if a region is lost.
>
> Under the data protection agreement, every copy of the data must stay in Germany.

Which design fits these needs?

A Cloud SQL with HA in europe-west3, plus a replica in europe-west1

B Spanner multi-region eur6, which has Frankfurt as a read-write region

C Spanner dual-region-germany1, across Berlin and Frankfurt

D Bigtable with clusters in Frankfurt and Berlin, on multi-cluster routing

◆ PAUSE AND DECIDE

Which design fits these needs?

A Cloud SQL plus replica

The replica leaves Germany, and lags behind.

B Spanner eur6

Copies in the Netherlands and Zurich.

C Spanner dual-region

Zero RPO, and both regions are German.

D Bigtable

Eventually consistent, and no transactions.

THE SIGNAL

Zero loss across regions, one country: Spanner dual-region.

> Larkspur's warehouse runs on BigQuery Enterprise edition in London. A new rule says that if London is lost, analysts must query, and pipelines must write, from Belgium within minutes.
>
> Losing up to about fifteen minutes of changes is acceptable.

What should the team set up?

A Managed DR on an Enterprise Plus failover reservation

B A dataset replica in europe-west1, promoted during the outage

C Datasets moved to the EU multi-region, which spans many regions

D A scheduled nightly copy of every dataset into europe-west1

◆ PAUSE AND DECIDE

What should the team set up?

A **Managed DR**
Compute and storage fail over in minutes.

B **Dataset replica**
Promotion needs London online.

C **EU multi-region**
No cross-region redundancy added.

D **Nightly copy**
A day of loss, and no compute ready.

THE SIGNAL

Query and write after a region loss: managed DR, Enterprise Plus.

Module 39 in five lines

- **RPO** is data you can lose. **RTO** is time you can be down. Read both first.
- Cloud SQL: **HA** for a zone, a **cross-region replica** promoted by hand for a region.
- Spanner dual-region or multi-region: **zero RPO**. Check where the regions are.
- Bigtable: clusters plus **app profiles**. Buckets: dual-region, **turbo** for 15 minutes.
- BigQuery multi-region is not DR. **Managed DR** needs Enterprise Plus.

COMING UP

Next: module 40, every rename and retirement, with the exam-safe answer.

MODULE 40 · EXAM STRATEGY - ALL SECTIONS

What Changed: Names, Retirements, Guide vs Today

Every 2026 rename with its date, what kept its old name underneath, the products that were retired, and how to answer when the exam and the console disagree.

SECTIONS All five MODULE 40 of 45 SCENARIOS 4

admin@sthithapragna.com

The guide was written before the renames. **Know both names, and answer with the one the question uses.**

admin@sthithapragna.com

What the official pages say, on 10 October 2026

- Banner: the exam "will soon be updated to reflect recent branding changes".
- The guide still says **Dataproc**, **Cloud Composer**, **BigLake**, **Dataplex**, **Analytics Hub**, **Cloud DLP**.
- Weights unchanged: **22, 25, 20, 15 and 18** percent across the five sections.
- 2 hours, **40 to 50** questions. The pass mark is not published.

> CURRENCY CHECK
>
> Expect old names on the exam. Expect new names in the console and the docs.

The April 2026 renames

THE EXAM GUIDE SAYS	NOW CALLED	SINCE
Dataplex	Knowledge Catalog	10 Apr 2026
Looker Studio	Data Studio, again	16 Apr 2026
Dataproc	Managed Service for Apache Spark	22 Apr 2026
BigLake	Google Cloud Lakehouse	22 Apr 2026
Vertex AI	Gemini Enterprise Agent Platform	22 Apr 2026
Cloud Composer	Managed Service for Apache Airflow	April 2026

REMEMBER

Six names in two weeks. The products, prices and behaviour did not change.

Older and smaller renames

OLDER NAME	NOW CALLED
Analytics Hub	BigQuery sharing. The guide uses both.
Cloud DLP	Sensitive Data Protection
Cloud Functions	Cloud Run functions, since August 2024
BigLake metastore	Lakehouse runtime catalog
Anywhere Cache	Rapid Cache, for Cloud Storage
Cloud Skills Boost	Google Skills, used for renewals

THE PATTERN

If an option names either version of a pair, treat them as the same product.

What kept the old name underneath

- **CLIs**: still gcloud dataproc, gcloud composer, gcloud dataplex.
- **APIs, client libraries and IAM roles**: unchanged, such as roles/dataproc.editor.
- **SQL values**: a registered model still says MODEL_REGISTRY = 'VERTEX_AI'.
- An option using an old command name is **not** outdated, and not wrong.

> THE TIEBREAKER
>
> The rename is a label on the box. Everything inside the box kept its name.

What was retired, and what replaced it

RETIRED	WHEN	USE INSTEAD
Pub/Sub Lite	Turned down 18 Mar 2026	Pub/Sub, or Managed Kafka
Data Catalog	Shut down from 1 Jun 2026	Knowledge Catalog
Managed Airflow Gen 1	End of life 15 Sep 2026	Gen 3, or Gen 2
gsutil	Leaves the gcloud CLI after Mar 2027	gcloud storage
CSEK for Compute Engine	Deprecated	CMEK in Cloud KMS

THE TRAP

A renamed product can still be the answer. A retired one never is, for a new design.

SQL that moved to a new prefix

- Current docs teach **AI.GENERATE_TEXT**, **AI.GENERATE_EMBEDDING** and **AI.EMBED**.
- Older material says **ML.GENERATE_TEXT** and **ML.GENERATE_EMBEDDING**.
- **VECTOR_SEARCH**, CREATE VECTOR INDEX and ML.FORECAST keep their names.
- TimesFM forecasting, with **AI.FORECAST**, needs no training at all.

> REMEMBER
>
> Either prefix on the exam means the same idea: a remote model called from SQL.

Recent additions worth recognising

- BigQuery **managed disaster recovery** and **continuous queries**.
- **Autonomous embeddings**: a column that embeds itself, with AI.EMBED.
- **Managed Service for Apache Kafka**, and Pub/Sub **single message transforms**.
- June 2026: Dataflow adds **automated stop-and-replace** streaming updates.

> THE SIGNAL
>
> The guide predates these. If one appears, it is usually the newest, simplest answer.

GA or Preview?

GENERALLY AVAILABLE

Safe exam answers

- Covered by the SLA
- Supported for production
- What exam answers assume
- Most of this course

PREVIEW

Know them, rarely pick them

- No SLA, may change
- Bigtable subscriptions from Pub/Sub
- AI.SEARCH, Lakehouse caching
- Data governance tags

THE TIEBREAKER

When an option only works through a Preview feature, prefer the GA route if one exists.

Phrases that pick the answer

THE QUESTION SAYS	TREAT IT AS
Dataproc, or a Dataproc cluster	Managed Service for Apache Spark
Cloud Composer, a Composer environment	Managed Airflow
Dataplex, lakes, zones, aspects	Knowledge Catalog
BigLake tables, Iceberg	Lakehouse
Pub/Sub Lite for a new system	Retired. Choose another option.
Data Catalog tag templates	Retired. Knowledge Catalog aspects.

THE PATTERN

Translate the name, then answer the question as if nothing changed.

admin@sthithapragna.com

> Larkspur's governance team wants every BigQuery table and lake bucket searchable in one place, with business owner and sensitivity recorded on each.
>
> The design must still be supported in 2027.

Which service should hold this metadata?

A Data Catalog, with a tag template for owner and sensitivity

B Knowledge Catalog, with an aspect type for owner and sensitivity

C Sensitive Data Protection, storing results in its inspection jobs

D BigQuery sharing, publishing every dataset as a private listing

◆ PAUSE AND DECIDE

Which service should hold this metadata?

A Data Catalog

Shut down from June 2026.

B Knowledge Catalog

Formerly Dataplex. Aspects carry the fields.

C Sensitive Data Protection

Finds sensitive data. It is not a catalogue.

D BigQuery sharing

Shares data. It does not catalogue it.

THE SIGNAL

Catalogue and tag, still supported: Knowledge Catalog, formerly Dataplex.

> An acquired business streams usage events from on-premises Kafka producers. The team wants a Google-managed service with the fewest code changes.
>
> An old design document proposes Pub/Sub Lite to keep costs down.

What should the team use?

A Pub/Sub Lite zonal topics, as the old design document proposes

B Pub/Sub standard topics, after rewriting every producer's client

C Managed Service for Apache Kafka, keeping the Kafka producers

D Self-managed Kafka brokers on Compute Engine virtual machines

◆ PAUSE AND DECIDE

What should the team use?

A Pub/Sub Lite

Turned down in March 2026.

B Pub/Sub

Works, but every producer must be rewritten.

C Managed Kafka

Kafka clients unchanged, Google runs the brokers.

D Self-managed Kafka

Not Google-managed. The team runs the brokers.

THE SIGNAL

Kafka clients, managed, fewest changes: Managed Service for Apache Kafka.

> The settlement DAGs still run in an old Managed Airflow environment, created years ago as Composer 1, now listed as Legacy Gen 1.
>
> It is October 2026. The team asks what to do with it.

What should the team do?

A Leave it alone, because Legacy Gen 1 is supported until 2028

B Rewrite every DAG as a Workflows definition in YAML instead

C Move the DAGs into Cloud Run jobs triggered by Cloud Scheduler

D Migrate the DAGs to a new Managed Airflow Gen 3 environment

◆ PAUSE AND DECIDE

What should the team do?

A Leave it

Gen 1 reached end of life in September 2026.

B Workflows

A full rewrite, and a poor fit for data DAGs.

C Cloud Run jobs

Loses retries, backfills and dependencies.

D Migrate to Gen 3

Same Airflow DAGs, a supported platform.

THE SIGNAL

Past end of life: migrate within the same product, to Gen 3.

> A nightly script copies settlement files with gsutil. The team now wants soft delete and managed folders on the bucket.
>
> The script must keep working after the next Google Cloud CLI upgrades.

What should the team change?

A Rewrite the script with gcloud storage commands

B Keep gsutil, and pin an old version of the CLI package

C Replace the script with a Storage Transfer Service job

D Replace the copy with bq load into a BigQuery staging table

◆ PAUSE AND DECIDE

What should the team change?

A **gcloud storage**
Current, supports the new features, stays in the CLI.

B **Pin old gsutil**
Minimally maintained, and lacks those features.

C **Storage Transfer**
Possible, but more than the task needs.

D **bq load**
Loads tables. It does not copy files.

THE SIGNAL

A legacy tool leaving the CLI: move to its named successor.

Module 40 in five lines

- The exam uses **old names**. The console uses new ones. Learn both.
- April 2026: **Dataproc, Composer, BigLake, Dataplex, Vertex AI, Looker Studio** renamed.
- CLIs, APIs, IAM roles and SQL values kept the **old names**.
- **Retired**: Pub/Sub Lite, Data Catalog, Airflow Gen 1. Never pick them for new work.
- Prefer **GA** over Preview. Translate the name, then answer.

> COMING UP
>
> Next: module 41, confusion pairs, side by side.

MODULE 41 · EXAM STRATEGY - ALL SECTIONS

Confusion Pairs: The One Word That Separates Them

Eighteen pairs and trios the exam sets against each other, from Bigtable and BigQuery to system lag and data freshness, each with the word that decides it.

SECTIONS All five MODULE 41 of 45 SCENARIOS 8

admin@sthithapragna.com

Most wrong answers are a good product in the wrong place. **Find the word in the question that rules one out.**

admin@sthithapragna.com

Bigtable or BigQuery?

BIGTABLE

Serve by key, fast

- Single-digit millisecond reads by row key
- Huge write throughput, time series
- No SQL joins across tables
- Sized in nodes

BIGQUERY

Analyse with SQL

- Scans and aggregates billions of rows
- Ad hoc SQL, joins, BI tools
- Seconds, not milliseconds
- Serverless, slots

THE WORD

Milliseconds and a key: Bigtable. Analysts and SQL: BigQuery.

Spanner, Cloud SQL or AlloyDB?

PICK	WHEN THE QUESTION SAYS
Cloud SQL	Lift a MySQL, PostgreSQL or SQL Server app. One region, up to 64 TB.
AlloyDB	PostgreSQL, and analytics on live transactions, via the columnar engine.
Spanner	Global or multi-region writes, horizontal scale, 99.999%.
Firestore	Documents for mobile and web apps, with offline sync.

THE WORD

Global and consistent: Spanner. PostgreSQL plus analytics: AlloyDB. Familiar and modest: Cloud SQL.

admin@sthithapragna.com

Dataflow, Spark, Data Fusion or Dataform?

PICK	WHEN THE QUESTION SAYS
Dataflow	New pipelines, streaming and batch in one model, serverless.
Managed Spark	Existing Spark or Hadoop jobs, moved with few changes.
Data Fusion	Visual, drag-and-drop pipelines, many connectors, little code.
Dataform	SQL transformations inside BigQuery, in Git, with tests.

THE WORD

Existing Spark: Managed Spark. Streaming from scratch: Dataflow. Already in BigQuery: Dataform.

admin@sthithapragna.com

Airflow, Workflows or Cloud Scheduler?

PICK	WHEN THE QUESTION SAYS
Managed Airflow	Many data tasks, dependencies, retries, backfills, Python DAGs.
Workflows	A short sequence of API or HTTP calls, serverless, YAML.
Cloud Scheduler	One job, on a timer. A cron line, nothing more.

THE WORD

Backfill or a DAG: Airflow. Chained API calls: Workflows. Just a timer: Scheduler.

admin@sthithapragna.com

Pub/Sub or Managed Kafka?

PUB/SUB

Google-native, serverless

- No partitions or brokers to size
- Global, scales automatically
- Push, pull and export subscriptions
- At-least-once, seek to replay

MANAGED KAFKA

Kafka, run for you

- Existing Kafka clients unchanged
- Partitions and offsets you control
- Kafka Connect included
- Sized in vCPU and memory

THE WORD

Kafka clients or Kafka skills already in place: Managed Kafka. Otherwise, Pub/Sub.

Five ways in, five different jobs

PICK	WHEN THE QUESTION SAYS
Database Migration Service	Move a database to Cloud SQL or AlloyDB, then cut over.
Datastream	Keep streaming changes from a database into BigQuery.
BigQuery Data Transfer Service	Scheduled loads from SaaS apps or other warehouses.
Storage Transfer Service	Files from other clouds or on-premises, over the network.
Transfer Appliance	Hundreds of terabytes, and the network would take months.

THE WORD

Migrate and cut over: DMS. Keep replicating for analytics: Datastream.

admin@sthithapragna.com

Who holds the key?

OPTION	WHO CONTROLS THE KEY
Default encryption	Google creates and rotates it. Nothing to do.
CMEK	You, in Cloud KMS. Rotate, disable, audit. HSM optional.
CSEK	You send it with each request. Storage and Compute only.
Cloud EKM	An external key manager. Google never holds the material.

THE WORD

Key outside Google entirely: EKM. Control and audit inside Google: CMEK. BigQuery never takes CSEK.

admin@sthithapragna.com

Two kinds of masking

SENSITIVE DATA PROTECTION

Change the data

- Inspect and classify, then transform
- Redact, tokenise, or shift dates
- The stored copy is de-identified
- Before data lands, or for extracts

BIGQUERY DATA MASKING

Change what each reader sees

- Data policies on policy tags
- Raw for some readers, masked for others
- Stored data unchanged
- Hash, nullify, last four, and more

THE WORD

Nobody may ever see raw values: de-identify with SDP. Some may, some may not: dynamic masking.

Materialized view, BI Engine or the cache?

PICK	HELPS WHEN
Materialized view	The same aggregation over a big, changing table. Stays fresh.
BI Engine	Interactive dashboards, sub-second, on modest tables, in memory.
Results cache	The identical query again, on unchanged data. Free, about 24 hours.

THE TIEBREAKER

Data keeps arriving: the cache misses, the materialized view does not.

admin@sthithapragna.com

Partitioning or clustering?

PARTITIONING

Coarse slices

- One column: date, timestamp or integer
- Prunes whole partitions, cost known up front
- Expiry and filters per partition
- Up to 10,000 partitions

CLUSTERING

Fine sorting

- Up to four columns, order matters
- Prunes blocks within each partition
- Good for high-cardinality filters
- Cost known only after running

THE PATTERN

Filter on date and on customer? Partition by date, cluster by customer. Usually both.

admin@sthithapragna.com

Update, drain or cancel?

ACTION	IN-FLIGHT DATA	REACH FOR IT WHEN
Update	Kept, with state	New code, compatible graph
Drain	Finished, windows closed early	Stopping, or an incompatible change
Cancel	Buffered data may be lost	Stopping now matters more than data

THE WORD

Keep state: update. Lose nothing but restart: drain. Neither matters: cancel.

admin@sthithapragna.com

Tumbling, hopping or session windows?

WINDOW	SHAPE	EXAMPLE QUESTION
Tumbling, fixed	Back to back, no overlap	Total kWh per half hour
Hopping, sliding	Overlapping, with a period	Five-minute average, every minute
Session	Per key, closed by a gap	One app visit, ended by 30 minutes idle

THE WORD

Per interval, rolling, or idle: those three words pick the window.

admin@sthithapragna.com

Time travel, snapshot or replica?

PICK	PROTECTS AGAINST	LASTS
Time travel	A recent mistake: bad update, dropped table	2 to 7 days, automatic
Table snapshot	Needing a point-in-time copy for longer	Until it expires
Replica or managed DR	Losing the region	Continuous

THE WORD

Mistake this week: time travel. Audit copy for a year: snapshot. Region lost: managed DR.

admin@sthithapragna.com

Authorised view or sharing listing?

AUTHORISED VIEW

Inside your organisation

- A view in another dataset, authorised on the source
- Readers need no access to the base tables
- Filter rows and columns in SQL
- Shared by IAM on the view's dataset

BIGQUERY SHARING

Across organisations

- Publish a listing in an exchange
- Subscribers get a linked, read-only dataset
- No copies, always current
- Subscribers pay for their own queries

THE WORD

Another team: authorised view. Another company, at scale: a sharing listing.

admin@sthithapragna.com

VPC Service Controls or IAM?

IAM

Who may do what

- Grants to identities
- Least privilege, roles, conditions
- A stolen key still works
- Answers: may this person read it?

VPC SERVICE CONTROLS

Where data may go

- A perimeter around projects
- Blocks copying data outside it
- Stops a stolen key working from outside
- Answers: may it leave this boundary?

THE WORD

Exfiltration, even by authorised users: a perimeter. Who can access: IAM. Most designs need both.

admin@sthithapragna.com

System lag or data freshness?

SYSTEM LAG

Is a step stuck?

- Longest time an item has waited or processed
- Measured in processing time
- Rises: slow code, a hot key, an external call
- Points to the pipeline

DATA FRESHNESS

Are results stale?

- How far output trails event time
- Includes late data from sources
- Rises: input late, or pipeline behind
- Points to what users see

THE PATTERN

Alert users on freshness. Debug the pipeline with lag.

> The customer app shows each meter's latest reading. It needs answers in about ten milliseconds, at tens of thousands of reads a second.
>
> Every reading is already loaded into BigQuery for analysts.

Where should the app read from?

A The BigQuery readings table, accelerated by a BI Engine reservation

B The BigQuery readings table, clustered on meter_id for pruning

C A Bigtable table keyed on meter ID, written by the pipeline

D Cloud SQL read replicas, loaded each hour from BigQuery

◆ PAUSE AND DECIDE

Where should the app read from?

A BI Engine

For dashboards, not ten-millisecond lookups.

B Clustered BigQuery

Faster scans, still seconds, not milliseconds.

C Bigtable by key

Single-digit milliseconds at huge read rates.

D Cloud SQL replicas

An hour stale, and a strain at that rate.

THE SIGNAL

Milliseconds by key at high volume: Bigtable, not BigQuery.

> The tariff service runs on PostgreSQL in London. The pricing team wants heavy analytical queries on live transactional data, with no ETL.
>
> The service must stay PostgreSQL-compatible, in one region.

Which database fits best?

A AlloyDB for PostgreSQL, using its columnar engine

B Spanner, in a multi-region configuration for scale

C Cloud SQL, with hourly exports into BigQuery for analysis

D Bigtable, with the transactions stored as wide rows

◆ PAUSE AND DECIDE

Which database fits best?

A **AlloyDB**
PostgreSQL, with analytics on live data.

B **Spanner**
Global scale nobody asked for.

C **Cloud SQL plus exports**
That is ETL, and an hour stale.

D **Bigtable**
Not relational, not PostgreSQL.

THE SIGNAL

PostgreSQL plus analytics on live data: AlloyDB.

> Analysts transform raw tables into marts entirely inside BigQuery, with SQL scripts run by hand.
>
> They want version control, dependency order, and automatic data tests, still in SQL.

What should they adopt?

A Data Fusion pipelines, built in the visual designer

B Dataflow pipelines, written in Java with Apache Beam

C Managed Spark jobs, rewriting the scripts in Spark SQL

D Dataform, with SQLX files in Git and assertions

◆ PAUSE AND DECIDE

What should they adopt?

A **Data Fusion**
Visual, and moves data rather than staying in SQL.

B **Dataflow**
Code, not SQL, and data leaves BigQuery.

C **Managed Spark**
A cluster and a rewrite for SQL that already works.

D **Dataform**
SQL in Git, dependencies, assertions as tests.

THE SIGNAL

SQL already in BigQuery, needing Git and tests: Dataform.

> Once a night, a cache in the tariff service must be refreshed by calling one HTTP endpoint.
>
> Nothing depends on it, and nothing else runs with it. The team wants the simplest managed option.

What should they use?

A. A Managed Airflow DAG with one HTTP operator task

B. A Cloud Scheduler job that calls the endpoint

C. A Workflows definition, started by an Eventarc trigger

D. A Dataflow batch job that makes the HTTP request

◆ PAUSE AND DECIDE

What should they use?

A Airflow DAG

An entire environment for one call.

B Cloud Scheduler

One call, on a timer. Nothing more needed.

C Workflows

Built for chains of calls, and still needs a timer.

D Dataflow

A data pipeline for no data at all.

THE SIGNAL

One call on a timer: Cloud Scheduler.

> An on-premises Oracle billing database must stay in service. Analysts want its changes in BigQuery within minutes, continuously.
>
> Nobody plans to move the application itself.

Which service should carry the changes?

A Database Migration Service, then a cutover to AlloyDB

B Datastream, streaming change data into BigQuery

C BigQuery Data Transfer Service, on a nightly schedule

D Storage Transfer Service, copying nightly export files

◆ PAUSE AND DECIDE

Which service should carry the changes?

A **DMS**
For moving the database, which nobody wants.

B **Datastream**
Continuous change capture into BigQuery.

C **DTS**
Scheduled, not continuous, and not for Oracle CDC.

D **Storage Transfer**
Moves files, a day at a time.

THE SIGNAL

Keep replicating changes for analytics: Datastream.

> A regulator requires that encryption keys for German customer data in BigQuery are never held by Google.
>
> The keys must live in a third-party key manager that Larkspur controls, and every use must be justified.

Which option meets the rule?

A Google default encryption, with keys rotated automatically

B CMEK in Cloud KMS, using the HSM protection level

C CSEK sent with every query by the client

D Cloud EKM, with keys in a partner external key manager

◆ PAUSE AND DECIDE

Which option meets the rule?

A Default

Google holds and manages the keys.

B CMEK with HSM

Your key, but held in Google's HSMs.

C CSEK

Not supported by BigQuery.

D Cloud EKM

Key material never leaves the external manager.

THE SIGNAL

Keys never held by Google: Cloud EKM.

> Dozens of reports compute the same daily totals from a four-terabyte readings table. New rows arrive every few minutes.
>
> The reports must include the newest rows, and the team wants each report to scan far less data.

What should the team build?

A A materialized view holding the daily totals

B Nothing, and rely on the query results cache

C A BI Engine reservation in front of the table

D A scheduled query rebuilding a totals table each night

◆ PAUSE AND DECIDE

admin@sthithapragna.com

What should the team build?

A **Materialized view**
Precomputed, refreshed, and current.

B **Results cache**
Misses, because the table keeps changing.

C **BI Engine**
Speeds dashboards, but the table is huge.

D **Nightly totals**
Small scans, but up to a day stale.

THE SIGNAL

The same aggregate over a changing table: a materialized view.

> Each month end, auditors want a frozen copy of the billing tables as they stood, kept for thirteen months.
>
> The tables change only a little each month, and storage cost should stay low.

What should the team use?

A Time travel, with the window set to its maximum

B A dataset replica in a second region

C A snapshot each month, expiring after 13 months

D A full table copy each month into an archive dataset

◆ PAUSE AND DECIDE

What should the team use?

A Time travel

Seven days at most, not thirteen months.

B Replica

Always current, never frozen.

C Monthly snapshots

Frozen, read-only, billed only for changes.

D Full copies

Frozen, but pays for every byte again.

THE SIGNAL

A frozen copy for months, cheaply: table snapshots with expiry.

Module 41 in five lines

- Milliseconds by key: **Bigtable**. SQL analysis: **BigQuery**. Live analytics in PostgreSQL: **AlloyDB**.
- Existing Spark: **Managed Spark**. SQL in BigQuery: **Dataform**. One timer: **Scheduler**.
- Migrate: **DMS**. Keep replicating: **Datastream**. Key outside Google: **EKM**.
- Changing table, repeated aggregate: **materialized view**. Frozen copy: **snapshot**.
- Find the **one word** that rules a product out, then choose.

> COMING UP
>
> Next: module 42, how PDE questions are written.

MODULE 42 · EXAM STRATEGY - ALL SECTIONS

How PDE Questions Are Written

The anatomy of a question, the qualifier words and what they eliminate, Google-recommended practice, numbered step options, choose-two technique, and pacing.

SECTIONS All five MODULE 42 of 45 SCENARIOS 4

admin@sthithapragna.com

Every question is a design brief with one constraint hidden in it. **Find the constraint, and most options fall away.**

Four parts of every question

PART	WHAT IT HOLDS	WHAT TO DO
Setting	The company, its systems, the volumes	Note the facts that are fixed
Constraint	The qualifier: cheapest, fastest, least effort	Underline it, mentally
Ask	The last sentence: what should they do?	Read it first
Options	One right, one nearly right, two wrong	Strike, then compare two

THE PATTERN

The nearly right option works. It just breaks the constraint. That is the one to beat.

What each qualifier eliminates

THE QUESTION SAYS	STRIKE OUT
Least operational overhead	VMs, clusters you size, custom code you maintain
Most cost-effective	Premium tiers, always-on capacity, extra copies
Minimise latency, near real time	Nightly batches, hourly schedules, manual steps
Without changing code	Rewrites into another framework or language
Sensitive data, comply	Raw copies, broad roles, data leaving the boundary
Minimal data loss	Nightly backups, asynchronous-only designs

THE PAYOFF

One qualifier can strike two options before you think about the products at all.

How the questions have shifted

QUALIFIER IN THE STEM	OLDER HALF	NEWER HALF
Security, compliance, personal data	17	31
Reliability, RPO, outages	7	15
Managed, least operational effort	6	14
Google-recommended practice	3	9
Choose two or three	18	3

HONEST LIMIT

Counts from the practice bank used to plan this course, not from the live exam. The direction is what matters.

What Google-recommended means

- **Managed** over self-managed. **Serverless** over clusters you size.
- A **built-in feature** over custom code that does the same job.
- **Predefined roles** on **groups**, at the narrowest level that works.
- Never a **retired** product, a shared key, or a basic role like Editor.

> THE TIEBREAKER
>
> When two options both work, the recommended one is usually the one with less to look after.

Five steps through any question

01
Read the ask
The last sentence first

02
Find the qualifier
Cheapest, fastest, least effort

03
Strike the breakers
Any option that breaks a fixed fact

04
Compare the two left
On the qualifier alone

05
Pick the simplest
That still meets everything

REMEMBER

If you are comparing four options at the end, you skipped a step.

Nearly right or right?

NEARLY RIGHT

Works, but breaks one thing

- Meets the goal, costs too much
- Or needs a team to run it
- Or loses data the question protects
- Often the most impressive option

RIGHT

Meets every constraint

- Meets the goal and the qualifier
- Respects every fixed fact
- Usually managed, often simpler
- Sometimes the least exciting option

THE TRAP

The most powerful product is wrong when nobody asked for its power.

Options written as numbered steps

- Each option is a **sequence**. One bad step sinks the whole option.
- Options often share most steps. **Compare only the steps that differ.**
- **Order matters**: snapshot before an update, test before production.
- Watch for a step that is **unnecessary**, or a product that is retired.

> THE PATTERN
>
> Read the steps that differ first. Often the question is decided by a single line.

Choose two

- The question tells you **how many** to select. Pick exactly that many.
- Judge each option **on its own**: true or false, against the setting.
- The right pair usually **complements**: one part each, such as storage and access.
- Two options that are **alternatives** to each other? At most one is right.

> REMEMBER
>
> Choose-two is rarer now, but it still appears. Treat it as several true-or-false questions.

Pacing and the clock

- 2 hours for **40 to 50** questions: about **2.5 minutes** each.
- First pass: answer what you can, and **flag** the slow ones.
- Second pass: return to flagged questions with the time left.
- A blank scores nothing. **Answer every question**, even a guess.

> THE NUMBERS
>
> A question taking five minutes is costing you two others. Flag it and move on.

What the exam is not

- **No case studies** in the current guide. Old Flowlogistic and MJTelco material is outdated.
- **No labs**. Every question is multiple choice or multiple select.
- **No published pass mark**. Figures quoted online are guesses.
- **Old product names** are likely, as module 40 showed.

> CURRENCY CHECK
>
> Prepare for the exam Google describes today, not the one older courses describe.

Setting phrases that carry a hint

THE SETTING SAYS	IT USUALLY POINTS TO
A **small team**, no operations staff	Serverless and managed services
Existing Spark, Kafka or SQL code	Keep it: Managed Spark, Managed Kafka, BigQuery
Data **must stay** in a country	Location choice, perimeters, residency checks
Analysts who know SQL	BigQuery, Dataform, BigQuery ML
Within minutes of the event	Streaming, not a schedule
Once a day is enough	Batch loads, which are free

THE PATTERN

The setting is never decoration. Every fact in it is there to rule something out.

> Larkspur's three-person analytics team owns twelve existing PySpark jobs that run for about an hour each night.
>
> Nobody on the team wants to manage infrastructure. The jobs must run with the least operational overhead.

How should the jobs run?

A Spark on Compute Engine VMs, built with an instance template

B Spark on a GKE cluster, using the open-source Spark operator

C A Managed Spark cluster, kept running and sized for the peak

D Managed Spark serverless batches, with no cluster to size

◆ PAUSE AND DECIDE

How should the jobs run?

A Compute Engine

Every VM is the team's to patch and run.

B GKE operator

A Kubernetes cluster to run as well.

C Always-on cluster

Managed, but sized, and idle 23 hours a day.

D Serverless batches

Submit the job. Nothing to size or keep running.

THE SIGNAL

Existing Spark, least overhead, short nightly runs: serverless batches.

> The readings pipeline must get new code that renames one step. Larkspur wants a way back if the new code misbehaves.
>
> No readings may be lost, and the deduplication state must be kept.

Which sequence should the team follow?

A. 1. Cancel the job. 2. Launch the new code. 3. Seek Pub/Sub back to the cancel time.

B. 1. Take a Dataflow snapshot. 2. Update with a name mapping. 3. Roll back from it if needed.

C. 1. Update with a name mapping. 2. Take a Dataflow snapshot. 3. If needed, start from the snapshot.

D. 1. Drain the job. 2. Take a Dataflow snapshot. 3. Launch the new code from that snapshot.

◆ PAUSE AND DECIDE

Which sequence should the team follow?

A **Cancel first**
State is lost, and replays create duplicates.

B **Snapshot, then update**
Rollback point first, state kept by the update.

C **Update, then snapshot**
Too late: the rollback point comes after the change.

D **Drain first**
Draining discards the deduplication state.

THE SIGNAL

Numbered steps: order decides. The rollback point must come first.

> A new team of six analysts needs to query the curated dataset in BigQuery. They must not change data, and must not see other datasets.
>
> The security team asks for the Google-recommended approach.

How should access be granted?

A Data Viewer on the dataset to a group, plus Job User on the project

B The basic Viewer role on the whole project, to each analyst individually

C A shared service account key, with Data Viewer granted on the dataset

D A custom role copied from BigQuery Admin, granted on the project

◆ PAUSE AND DECIDE

How should access be granted?

A **Group, dataset, predefined**
Least privilege, at the narrowest level.

B **Project Viewer**
Sees every dataset, and granted per person.

C **Shared key**
A shared secret. Never recommended.

D **Copied Admin role**
Far too broad, and at project level.

THE SIGNAL

Google-recommended access: predefined roles, a group, the narrowest scope.

> Meter readings land in Cloud Storage as files every thirty minutes. Finance needs yesterday's totals in BigQuery by 07:00 each morning.
>
> The requirement is the lowest cost. Nothing needs to be fresher than that.

How should the data reach BigQuery?

A. A streaming Dataflow job, writing through the Storage Write API

B. BigQuery continuous queries, running on a dedicated reservation

C. A nightly scheduled load job from Cloud Storage into BigQuery

D. Datastream, replicating the files into BigQuery as they arrive

◆ PAUSE AND DECIDE

How should the data reach BigQuery?

A Streaming Dataflow

Fresh, but workers run all day, every day.

B Continuous queries

Real-time capacity nobody asked for.

C Nightly load job

Free on the shared pool, done well before 07:00.

D Datastream

Replicates databases, not files.

THE SIGNAL

Once a day is enough, lowest cost: a batch load job.

Module 42 in five lines

- Read the **ask** first. Find the **qualifier**. It decides the answer.
- Strike options that break a **fixed fact**, then compare the two left.
- Google-recommended: **managed**, built-in, predefined roles on groups.
- Numbered steps: compare **only what differs**. Choose two: judge each alone.
- About **2.5 minutes** a question. Flag, return, and answer everything.

COMING UP

Next: module 43, practice paper one. Twenty new questions, mixed sections.

MODULE 43 · EXAM STRATEGY - ALL SECTIONS

Practice Paper 1

Twenty new questions across all five sections, in exam proportions, each followed by a worked answer. Pause, choose, then check.

SECTIONS All five MODULE 43 of 45 QUESTIONS 20

admin@sthithapragna.com

How to use this paper

- Each question pauses for **nine seconds**. Pause the video if you need longer.
- The real exam allows about **2.5 minutes** per question. Practise reading the ask first.
- Write down your **letter** before each answer is revealed.
- Sections are **mixed**, as on the exam. The score slide maps each question to its section.

> REMEMBER
>
> Score yourself honestly, every time. A lucky guess you cannot explain counts as a miss.

> The readings pipeline sums kWh in 30-minute event-time windows. After an outage, some meters send readings up to two hours late.
>
> Late readings must still update the totals, without holding the first results back.

What should the team configure?

A Processing-time windows, so late readings land in the current window

B Allowed lateness of two hours, with a late trigger per late reading

C A global window with a single trigger once a day at midnight

D Drop late readings, and correct the totals in a nightly batch job

◆ PAUSE AND DECIDE

What should the team configure?

A Processing time

Readings land in the wrong half hour.

B Allowed lateness

On-time result first, then updates as late data lands.

C Global window

Every result held back for a day.

D Nightly correction

Two pipelines, and totals wrong until night.

THE SIGNAL

Late data must still count, without delay: allowed lateness plus late firings.

> Larkspur is creating new BigQuery datasets for German customer data. The data protection agreement says the data must be stored only in Germany.
>
> The team also wants to stop anyone creating German resources elsewhere by mistake.

What should the team do?

A Create the datasets in the EU multi-region, which covers Germany

B Create them in europe-west3, and rely on a naming convention

C Use VPC Service Controls to keep the data inside Germany

D europe-west3, plus a resource locations policy on the de folder

◆ PAUSE AND DECIDE

What should the team do?

A EU multi-region

Spans many countries, not just Germany.

B Naming convention

Conventions do not stop mistakes.

C VPC Service Controls

Controls movement, not where data is stored.

D Location plus policy

Frankfurt, and new resources blocked elsewhere.

THE SIGNAL

Where data may be created: dataset location plus the resource locations constraint.

> The live readings table in Bigtable is queried for one meter's readings over the last day.
>
> The first design used the reading timestamp as the row key, and one node now runs hot.

Which row key should replace it?

A meter_id, followed by a reversed timestamp

B The reading timestamp, followed by meter_id

C A hash of meter_id and timestamp, for an even spread

D A sequential number assigned to each reading

◆ PAUSE AND DECIDE

Which row key should replace it?

A **Meter, reversed time**
Writes spread, and each meter's newest rows sit together.

B **Timestamp first**
Still leads with time, so still hot.

C **Hashed key**
Even spread, but per-meter range scans are lost.

D **Sequential number**
Every new write lands on one node.

THE SIGNAL

Queries by meter over time: meter first, then a reversed timestamp.

> Regional managers query one BigQuery table of bills. Each manager must see only the rows for their own region.
>
> The table must not be copied, and managers use their own SQL as well as dashboards.

What should the team set up?

A One table per region, copied each night from the main table

B Policy tags on the region column, with Fine-Grained Reader per group

C Row access policies on region, one per manager group

D A dashboard filter in Data Studio, set to each manager's region

◆ PAUSE AND DECIDE

What should the team set up?

A Table per region

Copies, which the brief forbids.

B Policy tags

Column security, not row filtering.

C Row access policies

Each group sees only its own region's rows.

D Dashboard filter

Their own SQL sees everything.

THE SIGNAL

Same table, different rows per person: row-level security.

> Westgrid must receive regional demand within five minutes of each half hour. The readings pipeline runs on Dataflow.
>
> Operations want to be paged as soon as the pipeline's output starts falling behind event time.

What should they alert on?

A The job's data freshness, above five minutes

B Worker CPU utilisation staying above eighty percent

C A daily scheduled query counting the late rows

D The number of workers reaching the autoscaling maximum

◆ PAUSE AND DECIDE

What should they alert on?

A **Data freshness**
Measures exactly how far output trails event time.

B **Worker CPU**
High CPU can be healthy. Low CPU can still be stuck.

C **Daily query**
A day too late to page anyone.

D **Worker count**
Capacity, not staleness.

THE SIGNAL

Output falling behind event time: alert on data freshness.

> Security worries that a stolen analyst credential could be used from outside the company to copy BigQuery data into a personal project.
>
> IAM roles are already least privilege.

What adds the missing protection?

A Remove BigQuery Data Viewer from every analyst in the company

B A VPC Service Controls perimeter around the data projects

C Customer-managed encryption keys on every BigQuery dataset

D Rotate every analyst's password once a week

◆ PAUSE AND DECIDE

What adds the missing protection?

A Remove access

Stops the work, not the theft.

B Perimeter

Copying out is blocked, even with valid credentials.

C CMEK

Authorised reads still decrypt the data.

D Password rotation

A stolen token works until it expires.

THE SIGNAL

Exfiltration with valid credentials: a VPC Service Controls perimeter.

> A billing subscriber processes meter events from Pub/Sub. Events for the same meter must be processed in the order they were published.
>
> Different meters can be processed in any order, and the subscriber must scale out.

What should the team do?

A. Add a sequence number, and sort each batch in the subscriber

B. Use a single subscriber with one thread, so nothing runs in parallel

C. Create a separate topic for every meter

D. Publish with the meter ID as the ordering key, and enable ordering

◆ PAUSE AND DECIDE

What should the team do?

A Sort each batch

Batches can split one meter's events.

B One thread

Ordered, but it cannot scale.

C Topic per meter

Millions of topics to manage.

D Ordering key

Per-meter order, parallel across meters.

THE SIGNAL

Order per entity, scale across entities: an ordering key.

> Settlement files in Cloud Storage are read often for 30 days, then about once a quarter for audits.
>
> They must be deleted after seven years. The team wants the lowest cost with no manual work.

What should the team set up?

A Keep everything in Standard, and delete old files by hand each year

B Archive storage from day one, with a seven-year retention policy

C Lifecycle rules: Coldline at 30 days, delete at seven years

D Nearline from day one, with object versioning turned on

◆ PAUSE AND DECIDE

What should the team set up?

A All Standard

Pays Standard prices for years of cold data.

B Archive at once

Retrieval fees all through the busy first month.

C Lifecycle rules

Cheap for quarterly reads, deleted automatically.

D Nearline and versions

Versions add cost, and nothing ages out.

THE SIGNAL

Access falls with age: lifecycle rules. Quarterly reads: Coldline.

> The customer accounts database runs on Cloud SQL for MySQL. Analysts want its inserts, updates and deletes in BigQuery within minutes.
>
> The team wants no custom code to maintain.

What should the team use?

A. A nightly export to Cloud Storage, loaded into BigQuery

B. Federated queries from BigQuery against Cloud SQL

C. Database Migration Service, replicating into BigQuery

D. Datastream, with BigQuery as the destination

◆ PAUSE AND DECIDE

What should the team use?

A Nightly export
A day stale.

B Federated queries
Live, but every query loads production.

C DMS
Migrates to Cloud SQL or AlloyDB, not BigQuery.

D Datastream
Change data capture, merged in within minutes.

THE SIGNAL

Changes into BigQuery continuously, with no code: Datastream.

A new Dataflow job reads one Pub/Sub subscription and writes one BigQuery dataset.

Its workers currently run as the Compute Engine default service account.

What is the Google-recommended identity for its workers?

A Its own service account: Dataflow Worker plus source and sink roles

B The Compute Engine default service account, with Editor on the project

C The personal user account of the developer who wrote the pipeline

D A service account key file stored in the pipeline's Git repository

◆ PAUSE AND DECIDE

What is the Google-recommended identity for its workers?

A Dedicated account
Least privilege, one identity per job.

B Default with Editor
Far too broad, shared by everything.

C Personal account
Grants their access, and breaks when they leave.

D Key in Git
A leaked secret waiting to happen.

THE SIGNAL

Google-recommended worker identity: dedicated and least privilege.

> Analysts want a daily demand forecast for each region, 30 days ahead, from three years of history already in BigQuery.
>
> They know SQL, but not Python.

What should they use?

A Export the history, and train a model in a Python notebook first

B A linear regression model on the date column, in BigQuery ML

C An ARIMA_PLUS model in BigQuery ML, with ML.FORECAST per region

D A k-means model in BigQuery ML, clustering days by demand

◆ PAUSE AND DECIDE

What should they use?

A Python notebook

Not SQL, and the data leaves BigQuery.

B Linear regression

Ignores seasonality and trend.

C ARIMA_PLUS

A time series per region, in SQL, with seasonality.

D k-means

Clustering, not forecasting.

THE SIGNAL

Forecast a time series in SQL: ARIMA_PLUS with ML.FORECAST.

> Larkspur's analysts use on-demand pricing. One analyst's mistaken query loop scanned 300 TB in a single day.
>
> Finance wants a hard daily cap for each user, without slowing anyone else down.

What should the team set?

A. A budget alert at 80 percent of the monthly BigQuery spend

B. A custom quota on query usage per user per day

C. Move the whole project to an Enterprise edition reservation

D. Require a partition filter on every table in the project

◆ PAUSE AND DECIDE

What should the team set?

A Budget alert

Alerts, but stops nothing.

B Per-user quota

A daily limit per user, enforced.

C Reservation

Changes billing to cap one mistake.

D Partition filter

Helps some queries, caps nothing.

THE SIGNAL

A hard cap per user per day, on-demand: a custom quota.

> The raw readings table is partitioned by reading date. Regulations require only 400 days of raw data.
>
> Older partitions should disappear automatically, with no code to run.

What should the team configure?

A. A scheduled query that deletes rows older than 400 days each night

B. A table expiration of 400 days on the raw readings table

C. A partition expiration of 400 days on the table

D. Time travel set to its maximum, so old data ages out

◆ PAUSE AND DECIDE

What should the team configure?

A Nightly delete
Code to maintain, and DML to pay for.

B Table expiration
Deletes the whole table at 400 days.

C Partition expiration
Each day's partition drops at 400 days.

D Time travel
Recovery, seven days at most.

THE SIGNAL

Drop old data by age, automatically: partition expiration.

> Larkspur must move 400 TB of archived settlement history from its data centre to Cloud Storage.
>
> The only link is 200 Mbps, already half used. The move must finish within two months.

How should the data move?

A gcloud storage cp, running in parallel over the existing link

B Storage Transfer Service agents, scheduled to run overnight

C A new VPN tunnel, dedicated to the transfer

D Transfer Appliance, shipped and loaded at the data centre

◆ PAUSE AND DECIDE

How should the data move?

A gcloud storage

About a terabyte a day: over a year.

B Transfer agents

The same thin link, the same year.

C VPN tunnel

A tunnel adds no bandwidth.

D Transfer Appliance

Weeks, whatever the link.

THE SIGNAL

Hundreds of terabytes over a thin link: do the arithmetic, then ship it.

> Larkspur's on-premises Hadoop cluster runs 300 Spark settlement jobs that read from HDFS.
>
> The team wants them running on Google Cloud quickly, with minimal code changes.

What should the team do first?

A Rewrite every job as an Apache Beam pipeline on Dataflow

B Run the jobs on Managed Spark, with the data in Cloud Storage

C Convert every job into BigQuery SQL, scheduled by Dataform

D Rebuild the same Hadoop cluster on Compute Engine virtual machines

◆ PAUSE AND DECIDE

What should the team do first?

A Rewrite in Beam

Three hundred rewrites.

B Managed Spark

Same Spark code, Cloud Storage paths instead of HDFS.

C Rewrite in SQL

Another three hundred rewrites.

D Hadoop on VMs

Moved, but the team still runs the cluster.

THE SIGNAL

Existing Spark, quickly, few changes: Managed Spark with Cloud Storage.

> Westgrid, a separate company, wants daily read access to Larkspur's regional demand mart. Larkspur wants no copies of the data, and Westgrid should pay for its own queries.

How should Larkspur share it?

A Publish the mart as a listing in a BigQuery sharing exchange

B Export the mart to Cloud Storage each night, and share the bucket

C Grant Westgrid's users Data Viewer on Larkspur's whole project

D Copy the mart into a project that Westgrid owns, every night

◆ PAUSE AND DECIDE

How should Larkspur share it?

A Sharing listing
Linked, read-only, current, and the subscriber pays.

B Nightly export
A copy, and stale by morning.

C Project-wide access
Far too broad, across the whole project.

D Nightly copy
A copy, which Larkspur wants to avoid.

THE SIGNAL

Another company, no copies, they pay: a sharing listing.

> Larkspur has hundreds of BigQuery tables built up over years. The privacy team suspects some hold unlabelled personal data, such as phone numbers and bank details.
>
> They want to find it with the least effort, and keep finding it as tables change.

What should they do?

A Ask every data owner to review all of their tables by hand

B Write SQL with regular expressions, and run it across every table

C Apply dynamic data masking to every column in every table

D Sensitive Data Protection discovery, across the organisation

◆ PAUSE AND DECIDE

What should they do?

A Manual review
Slow, and owners miss what they never knew.

B Regex SQL
Brittle, and must be maintained.

C Mask everything
Masks blindly, and finds nothing.

D SDP discovery
Profiles every table, and keeps watching.

THE SIGNAL

Find unknown personal data at scale: Sensitive Data Protection discovery.

> Westgrid wants total demand per region for each half hour, published once that half hour closes.
>
> Readings stream in from Pub/Sub, with event timestamps from the meters.

Which pipeline design fits?

A Dataflow with sliding 30-minute windows, starting every minute

B Dataflow with session windows by region, and a 30-minute gap

C Dataflow with fixed 30-minute windows, keyed by region

D A BigQuery scheduled query on the raw table, every 30 minutes

◆ PAUSE AND DECIDE

Which pipeline design fits?

A Sliding windows
Thirty overlapping totals per half hour.

B Session windows
Sessions end on gaps, not on the clock.

C Fixed windows
One total per region per half hour.

D Scheduled query
Clock time, not event time.

THE SIGNAL

Per half hour, back to back, by event time: fixed windows.

A new field engineer app stores job notes for site visits. Engineers often work in basements with no signal.

Their changes must be saved on the phone, and sync when they reconnect.

Which database fits best?

A Firestore, using its client SDKs with offline persistence

B Cloud SQL for PostgreSQL, called through the app's REST API

C Bigtable, with one row for each job note

D BigQuery, using the Storage Write API from each phone

◆ PAUSE AND DECIDE

Which database fits best?

A Firestore
Offline writes, and sync on reconnect, built in.

B Cloud SQL
Every write needs a connection.

C Bigtable
No offline sync, and built for huge scale.

D BigQuery
A warehouse, not an app database.

THE SIGNAL

Mobile app, offline, sync later: Firestore.

> A Managed Airflow DAG waits up to six hours each night for a supplier file to land in Cloud Storage.
>
> The waiting sensors occupy worker slots, and other tasks are delayed.

What should the team change?

A. Add more workers, so waiting sensors no longer block other tasks

B. Make the sensor deferrable, with a triggerer running

C. Replace the sensor with a six-hour sleep in a Python task

D. Move each task into its own Cloud Scheduler job

◆ PAUSE AND DECIDE

What should the team change?

A More workers

Pays for idle workers to wait.

B Deferrable sensor

The wait moves to the triggerer, freeing slots.

C Sleep

Still holds a slot, for longer.

D Cloud Scheduler

Loses dependencies and retries.

THE SIGNAL

Long waits in Airflow: deferrable operators and a triggerer.

Paper 1 answers

QUESTIONS	ANSWERS
1 to 5	B, D, A, C, A
6 to 10	B, D, C, D, A
11 to 15	C, B, C, D, B
16 to 20	A, D, C, A, B

REMEMBER

Check each miss against its worked answer before you move on.

Where your marks came from

SECTION	QUESTIONS	REVISIT
1 Design, 22%	2, 6, 10, 14, 17	Modules 02 to 09
2 Ingest and process, 25%	1, 7, 9, 15, 18	Modules 10 to 18
3 Store, 20%	3, 8, 13, 19	Modules 19 to 26
4 Analyse, 15%	4, 11, 16	Modules 27 to 32
5 Maintain and automate, 18%	5, 12, 20	Modules 33 to 39

THE PATTERN

One weak section costs more than a scattered few. Fix the section, not the single question.

What your total suggests

SCORE	WHAT IT SUGGESTS
17 to 20	Strong. Keep sharp with papers 2 and 3.
14 to 16	Close. Revisit the weakest section, then take paper 2.
Under 14	Rework the weak sections first. Retake this paper in a week.

HONEST LIMIT

Google does not publish a pass mark. These bands guide study. They do not predict a result.

Module 43 in five lines

- Late data that must count: **allowed lateness** and late triggers.
- Stored only in one country: **dataset location** plus the resource locations policy.
- Order per meter: an **ordering key**. Changes into BigQuery: **Datastream**.
- Data falls in value with age: **lifecycle rules**, and **partition expiration**.
- Thin link, hundreds of terabytes: do the **arithmetic**, then ship it.

COMING UP

Next: module 44, practice paper 2. Twenty more questions, some pairing with today's.

MODULE 44 · EXAM STRATEGY - ALL SECTIONS

Practice Paper 2

Twenty more new questions across all five sections. Several pair with paper 1: the setting is similar, one constraint changes, and so does the answer.

SECTIONS All five MODULE 44 of 45 QUESTIONS 20

admin@sthithapragna.com

How to use this paper

- Nine seconds per question. **Pause the video** whenever you need longer.
- Read the **ask** first, then hunt for the **qualifier**.
- Feels like a paper 1 question? Find the **one constraint** that changed.
- Write your letter down. The **score slide** maps each question to its section.

> THE TRAP
>
> Feeling familiar is dangerous. The paired questions punish answering from memory.

> The bills table is partitioned by bill date. Most queries filter on one account_id, out of four million, within a date range.
>
> Those queries still scan whole partitions, and cost far more than expected.

What should the team change?

A Re-partition the whole table by account_id instead of bill date

B Create a separate table for every account

C Cluster the table by account_id, keeping date partitioning

D Add a materialized view that holds every account's bills

◆ PAUSE AND DECIDE

What should the team change?

A Partition by account

Four million values break the partition limit.

B Table per account

Four million tables to manage.

C Cluster by account

Blocks pruned inside each date partition.

D Materialized view

A copy of everything, scanned the same way.

THE SIGNAL

High-cardinality filter inside a date range: cluster, and keep the partitions.

A Dataflow streaming job writes billing events into BigQuery. Finance reports found occasional duplicate rows after worker restarts.

Duplicates are unacceptable, and the team wants the BigQuery connector itself to prevent them.

Which write method should the job use?

A The Storage Write API method, STORAGE_WRITE_API

B The at-least-once method, STORAGE_API_AT_LEAST_ONCE

C Legacy streaming inserts, with retries on every error

D File loads every ten seconds, with WRITE_APPEND

◆ PAUSE AND DECIDE

Which write method should the job use?

A Storage Write API
Exactly-once delivery into BigQuery.

B At least once
Lower latency, but duplicates are possible.

C Legacy inserts
Retries can create duplicates.

D Frequent file loads
Far too frequent, and append repeats on retry.

THE SIGNAL

Duplicates unacceptable from Dataflow: the exactly-once Storage Write API method.

> Larkspur's parent group is building a pricing service that takes writes from customers in Europe, North America and Asia.
>
> It needs relational schemas, strong consistency everywhere, and 99.999% availability.

Which database fits best?

A Cloud SQL, with cross-region read replicas on each continent

B Bigtable, with clusters on three continents

C AlloyDB, with read pools in each region

D Spanner, in a multi-region configuration

◆ PAUSE AND DECIDE

Which database fits best?

A Cloud SQL replicas
One primary for writes, replicas lag behind.

B Bigtable clusters
Eventually consistent, not relational.

C AlloyDB read pools
Read scale, one region for writes.

D Spanner multi-region
Relational, externally consistent, five nines.

THE SIGNAL

Global writes, strong consistency, five nines: Spanner multi-region.

> The customers table holds email, phone and postcode alongside non-sensitive columns. Analysts may query the table, but only the support team may read email and phone.
>
> When analysts touch those columns, they must get an error, not masked values.

What should the team set up?

A Row access policies that hide rows holding an email address

B Policy tags on email and phone, with Fine-Grained Reader for support

C Dynamic data masking on email and phone, with Masked Reader for analysts

D An authorised view without those columns, used by everyone including support

◆ PAUSE AND DECIDE

What should the team set up?

A Row policies

Hides rows, not columns.

B Policy tags

Without the role, those columns return an error.

C Dynamic masking

Masked values, which the brief rules out.

D View for everyone

Support loses access too.

THE SIGNAL

Columns restricted, error rather than mask: policy tags without masking.

> At 10:14, a faulty script deleted thousands of rows from the tariff database on Cloud SQL for PostgreSQL.
>
> Point-in-time recovery is enabled, and the last nightly backup ran at 02:00. The team must recover with the least data loss.

What should the team do?

A Restore last night's 02:00 backup over the instance

B Promote the cross-region read replica in Belgium

C Ask Customer Care to recover the rows from fail-safe

D Clone the instance to a point just before 10:14

◆ PAUSE AND DECIDE

What should the team do?

A Nightly backup

Loses eight hours of writes.

B Promote replica

The replica has already applied the delete.

C Fail-safe

A BigQuery feature, not Cloud SQL.

D Point-in-time clone

The database as it was at 10:13.

THE SIGNAL

Undo a recent mistake in Cloud SQL: point-in-time recovery to a new instance.

A supplier drops daily CSV files into an Amazon S3 bucket.

Larkspur needs them loaded into BigQuery each morning, with the least code and no servers to run.

What should the team use?

A. A Compute Engine VM running a nightly script with the AWS CLI

B. A Dataflow batch job, written in Java, reading from S3

C. A BigQuery Data Transfer Service transfer from Amazon S3

D. Datastream, with the S3 bucket configured as its source

◆ PAUSE AND DECIDE

What should the team use?

A VM and script
A server and a script to maintain.

B Dataflow job
Code to write and maintain.

C Data Transfer Service
Scheduled and managed, with no code.

D Datastream
Replicates databases, not files.

THE SIGNAL

Scheduled loads from another cloud's storage, no code: Data Transfer Service.

A small team in the German business runs a few hundred queries a week, mostly in short bursts on Monday mornings.

Monthly scans total a few terabytes. They want the lowest cost and no capacity to manage.

Which pricing model fits?

A On-demand pricing, paying for the bytes each query scans

B An Enterprise edition reservation, with a one-year commitment

C An Enterprise Plus reservation, with autoscaling

D A Standard edition reservation, with 500 baseline slots

◆ PAUSE AND DECIDE

Which pricing model fits?

A **On-demand**
Pay per query, nothing to manage.

B **Committed Enterprise**
Paying all year for a few busy hours.

C **Enterprise Plus**
Premium features nobody asked for.

D **Standard baseline**
Slots paid for around the clock.

THE SIGNAL

Small, bursty, few terabytes, nothing to manage: on-demand.

> Meter firmware images must be kept for ten years in case of a legal dispute.
>
> They are almost never read, perhaps once in several years. Retrieval fees on that rare read are acceptable.

Which storage class should the team use?

A Coldline, with a lifecycle rule to delete after ten years

B Archive, with a lifecycle rule to delete after ten years

C Nearline, because its retrieval fee is the lowest

D Standard, with Autoclass managing the class later

◆ PAUSE AND DECIDE

Which storage class should the team use?

A Coldline

Fine for quarterly reads, dearer to store.

B Archive

Lowest storage price, still read in milliseconds.

C Nearline

Built for monthly access.

D Standard and Autoclass

Starts at the highest price.

THE SIGNAL

Read less than once a year: Archive. Once a quarter would mean Coldline.

> The meter events topic carries every event type. A new alerts service only needs events whose type attribute is fault, about one percent of traffic.
>
> The team wants the alerts service to receive only those events.

What should the team do?

A Pull every message, and discard non-fault events in the service

B Create a second topic, and republish fault events into it

C Create a subscription with a filter on the type attribute

D Use an ordering key of fault, so those events arrive first

◆ PAUSE AND DECIDE

What should the team do?

A Discard in code

Receives the other ninety nine percent too.

B Republish

A republisher to build and run.

C Subscription filter

Only matching messages are delivered.

D Ordering key

Ordering, not filtering.

THE SIGNAL

Receive only some messages by attribute: a subscription filter.

> Finance wants monthly Google Cloud costs split by team, for BigQuery, Dataflow and Cloud Storage.
>
> Several teams share some projects, so project totals are not enough.

What should the team set up?

A. A separate billing account for each team, moving projects

B. Budget alerts on each project, sent to each team lead

C. A spreadsheet where every team records what it ran

D. Labels on resources and jobs, with billing export

◆ PAUSE AND DECIDE

What should the team set up?

A Billing accounts

Shared projects cannot be split this way.

B Budget alerts

Alerts on spend, not attribution.

C Spreadsheet

Manual, and never complete.

D Labels and export

Costs grouped by team label, in SQL.

THE SIGNAL

Costs by team across shared projects: labels plus billing export.

> Larkspur's marketing team needs churn scores and tariff names from the curated dataset, but must not see personal columns or any other tables.
>
> Both teams are in the same organisation, and nothing should be copied.

How should access be granted?

A An authorised view in their own dataset, with only those columns

B A BigQuery sharing listing that publishes the whole curated dataset

C Data Viewer on the curated dataset for the marketing group

D A nightly scheduled query copying the columns into a marketing table

◆ PAUSE AND DECIDE

How should access be granted?

A Authorised view

Only the chosen columns, no access to the source.

B Sharing listing

The whole dataset, built for other organisations.

C Data Viewer

Every table and column in the dataset.

D Nightly copy

A copy, which the brief rules out.

THE SIGNAL

Another team, a subset, no copies: an authorised view.

A Bigtable instance stores 400 TB of historical readings, used only by nightly batch analysis jobs.

No user-facing application reads it. The team wants to cut cost, and latency does not matter.

Which storage type fits?

A SSD storage, for its higher read throughput

B HDD storage, for latency-insensitive batch work

C Memorystore in front of Bigtable, to speed up batch reads

D SSD storage with twice as many nodes, for headroom

◆ PAUSE AND DECIDE

Which storage type fits?

A SSD

Pays for speed nobody uses.

B HDD

More terabytes per node, fine for batch.

C Memorystore

An in-memory cache for 400 TB of batch data.

D More SSD nodes

More cost, not less.

THE SIGNAL

Large, batch-only, latency irrelevant: Bigtable HDD storage.

> Auditors require that encryption keys for settlement data in BigQuery and Cloud Storage are controlled by Larkspur, rotated every 90 days, and can be disabled at once in an emergency.
>
> The keys may be held inside Google Cloud.

What should the team use?

A Google default encryption, which already rotates its keys

B Customer-supplied keys, sent with every request

C Cloud EKM, with keys held by an external partner

D CMEK in Cloud KMS, with a 90-day rotation schedule

◆ PAUSE AND DECIDE

What should the team use?

A Default encryption

Google controls it. Larkspur cannot disable it.

B Customer-supplied

Not supported by BigQuery.

C Cloud EKM

Works, but adds a partner nobody asked for.

D CMEK

Larkspur controls, rotates and can disable.

THE SIGNAL

Your control, rotation and disable, inside Google: CMEK.

> The operations team launches the same Dataflow pipeline for different regions and dates.
>
> They do not write code, and want a parameterised job they can start from the console or the command line.

How should engineering package the pipeline?

A As a Flex Template, with region and date as parameters

B As a Python script, with instructions for editing it

C As an Airflow DAG that operations rewrite for each region

D As a container image that operations run on their own laptops

◆ PAUSE AND DECIDE

How should engineering package the pipeline?

A Flex Template
Launched with parameters, no code.

B Editable script
Editing code is what they cannot do.

C Rewritten DAGs
Rewriting DAGs is still code.

D Laptop container
Laptops are not a production platform.

THE SIGNAL

Non-developers launching a pipeline with parameters: a Flex Template.

> On the etl reservation, nightly jobs have started finishing late.
>
> The team wants slot usage per job and per hour across the last two weeks, to find what changed.

Where should they look first?

A Cloud Logging, searching job logs for the word slow

B The billing export, grouped by day

C The jobs timeline views, or the resource charts

D Each query's execution plan, opened one job at a time

◆ PAUSE AND DECIDE

Where should they look first?

A Cloud Logging
Logs, not slot usage over time.

B Billing export
Cost per day, not slots per job.

C Jobs timeline
Slots per job and per period, over weeks.

D Execution plans
One job's detail, not the trend.

THE SIGNAL

Slot use over time, per job: the jobs timeline views and resource charts.

> The customer app checks a user's session and tariff on every page view, adding 40 milliseconds of database time.
>
> The data can be rebuilt from the database if lost. The team wants sub-millisecond reads.

What should the team add?

A. A Cloud SQL read replica in the same region as the app

B. A Memorystore cache in front of the database

C. A BigQuery BI Engine reservation

D. A Firestore database holding a copy of every session

◆ PAUSE AND DECIDE

What should the team add?

A Read replica
Still a database read, in milliseconds.

B Memorystore
In memory, sub-millisecond, and rebuildable.

C BI Engine
For dashboards, not app lookups.

D Firestore copy
A second database, not a cache.

THE SIGNAL

Sub-millisecond reads of rebuildable data: Memorystore.

> Engineers want to type a fault description and find the most similar past repair notes, by meaning rather than keywords.
>
> Two million repair notes are stored in BigQuery.

What should the team build?

A Note embeddings, a vector index, and VECTOR_SEARCH

B A LIKE query with wildcards around every typed keyword

C A k-means model clustering the notes

D Full-text search with search indexes on the notes

◆ PAUSE AND DECIDE

What should the team build?

A **Embeddings and vector search**
Nearest neighbours by meaning, at speed.

B **LIKE query**
Exact words only, and slow.

C **k-means**
Groups notes, but does not rank them for a query.

D **Full-text search**
Keywords, not meaning.

THE SIGNAL

Similar by meaning: embeddings, a vector index, and vector search.

> The readings pipeline must attach each meter's tariff to every reading. The tariff table has 50,000 rows, and changes about once a month.
>
> Calling the tariff database for every reading has overloaded it.

What should the pipeline do instead?

A Call the database for every reading, with more retries

B Join readings and tariffs with CoGroupByKey in each window

C Load the tariffs as a side input for the enrichment step

D Write readings to BigQuery, then join tariffs in a nightly query

◆ PAUSE AND DECIDE

What should the pipeline do instead?

A More retries

More load on an overloaded database.

B CoGroupByKey

Treats a small, slow table like a stream.

C Side input

Small and slow-changing, held beside the step.

D Nightly join

Enrichment a day late.

THE SIGNAL

Small, slow-changing lookup data: a side input.

> Larkspur's billing, metering and customer teams each want to own and publish their own data products.
>
> A central team must still enforce common quality and access rules, and let everyone discover what exists.

Which design fits best?

A One central data team building every table in a single dataset

B Each team in its own project, with no shared catalogue at all

C Copy every team's data into one central lake each night

D A Knowledge Catalog lake per domain, with central policies

◆ PAUSE AND DECIDE

Which design fits best?

A Central team

Central ownership, the opposite of the brief.

B No catalogue

Ownership, but nothing to discover or govern.

C Central copies

Copies, and central ownership again.

D Lake per domain

Domains own data, rules and search are shared.

THE SIGNAL

Domain ownership with central governance: a data mesh in Knowledge Catalog.

> In a Managed Airflow environment, DAGs sometimes vanish from the UI, and the logs show DAG import timeouts.
>
> Several DAG files query BigQuery at the top level, to build their task lists.

What should the team fix first?

A Add more workers and a larger environment size

B Move the BigQuery calls out of top-level code

C Raise the import timeout until the DAGs load

D Split each DAG file into ten smaller files

◆ PAUSE AND DECIDE

What should the team fix first?

A More workers

Workers do not parse DAG files.

B Remove top-level calls

Every parse stops running slow queries.

C Raise the timeout

Hides the cause, and slows every parse.

D Split the files

More files, the same slow calls.

THE SIGNAL

DAG import timeouts: slow top-level code. Move the work into tasks.

Paper 2 answers

QUESTIONS	ANSWERS
1 to 5	C, A, D, B, D
6 to 10	C, A, B, C, D
11 to 15	A, B, D, A, C
16 to 20	B, A, C, D, B

REMEMBER

For every miss, name the phrase you overlooked. That habit is worth more than the score.

Where your marks came from

SECTION	QUESTIONS	REVISIT
1 Design, 22%	3, 7, 13, 19	Modules 02 to 09
2 Ingest and process, 25%	2, 6, 9, 14, 18	Modules 10 to 18
3 Store, 20%	1, 8, 12, 16	Modules 19 to 26
4 Analyse, 15%	4, 11, 17	Modules 27 to 32
5 Maintain and automate, 18%	5, 10, 15, 20	Modules 33 to 39

THE PATTERN

Compare with paper 1. A section weak twice is the one to study next.

Same setting, one constraint changed

PAPER 1	PAPER 2	WHAT CHANGED
Coldline, Q8	Archive, Q8	Read quarterly, or once in years
Row access policy, Q4	Policy tags, Q4	Rows per person, or columns
Sharing listing, Q16	Authorised view, Q11	Another company, or another team
Partition expiration, Q13	Clustering, Q1	Drop by age, or filter by key

THE TIEBREAKER

If you answered a pair the same way twice, reread the constraint that changed.

Module 44 in five lines

- High-cardinality filter in a date range: **cluster**, keep the partitions.
- Duplicates unacceptable from Dataflow: the **Storage Write API** method.
- Columns denied, not masked: **policy tags**. Another team, a subset: **authorised view**.
- Recent Cloud SQL mistake: **point-in-time recovery**. A replica copies the mistake.
- Familiar setting? Find the **one constraint** that changed.

> COMING UP
>
> Next: module 45, practice paper 3, the final twenty, and what your three scores say.

MODULE 45 · EXAM STRATEGY - ALL SECTIONS

Practice Paper 3

The final twenty new questions, across all five sections, then a way to read your three scores together and decide what to study next.

SECTIONS All five MODULE 45 of 45 QUESTIONS 20

admin@sthithapragna.com

How to use this paper

- Nine seconds per question. **Pause the video** if you need longer.
- Read the **ask**, find the **qualifier**, strike what breaks a fixed fact.
- One question pairs with paper 2. Spot the **constraint** that changed.
- Keep your scores from papers 1 and 2 to hand for the **final slide**.

> REMEMBER
>
> Treat this paper like the real exam: no notes, and no rewinding the question.

> Spark jobs from the data science team write Larkspur's settlement tables to Cloud Storage. Analysts must query the same tables in BigQuery.
>
> The tables must stay in an open format, with one copy and one catalog.

Which design fits best?

A Parquet files in Cloud Storage, plus separate copies loaded into BigQuery

B BigQuery native tables, exported to Parquet each night for Spark

C Hive tables on a long-running Spark cluster, queried by federation

D Iceberg tables in the Lakehouse runtime catalog, read by both engines

◆ PAUSE AND DECIDE

Which design fits best?

A Two copies

Two copies to keep in step.

B Nightly export

A copy, a day stale for Spark.

C Hive on a cluster

A cluster to keep running for the catalog.

D Iceberg and the runtime catalog

One open format, one catalog, two engines.

THE SIGNAL

One open copy for Spark and BigQuery: Iceberg in the Lakehouse runtime catalog.

> The app team wants to measure how long customers spend in the Larkspur app per visit. A visit ends after 20 minutes with no activity.
>
> Click events stream in through Pub/Sub.

Which windowing should the Dataflow pipeline use?

A Fixed 20-minute windows, keyed by customer and app screen

B Session windows keyed by customer, with a 20-minute gap

C Sliding windows of 20 minutes, starting every minute

D A single global window, triggered every 20 minutes

◆ PAUSE AND DECIDE

Which windowing should the Dataflow pipeline use?

A Fixed windows
Clock slices cut visits in half.

B Session windows
One window per visit, closed by inactivity.

C Sliding windows
Overlapping slices, not visits.

D Global window
One window for all time.

THE SIGNAL

A visit that ends after inactivity: session windows with a gap.

A new Spanner table records meter readings at a very high write rate.

The first design uses an auto-incrementing reading number as the primary key, and writes queue on one server.

What should the team change?

A Use the reading timestamp as the primary key instead

B Keep the sequence, but sort the key in descending order

C Use a version 4 UUID as the primary key

D Add more nodes, so the hot split gets more capacity

◆ PAUSE AND DECIDE

What should the team change?

A Timestamp key

Still increasing, still one hot split.

B Descending sequence

Descending is still monotonic.

C UUID version 4

Random, so writes spread across splits.

D More nodes

A hotspot stays on one split.

THE SIGNAL

Monotonic keys make hotspots: a random UUID, or a bit-reversed sequence.

> Finance analysts work in Google Sheets. They want pivot tables over a 2-billion-row BigQuery bills table.
>
> They do not write SQL, and nothing should be exported.

What should they use?

A Connected Sheets, with pivots computed in BigQuery

B Export the table to CSV, and import it into Sheets

C A scheduled query writing 2 billion rows into a sheet

D Data Studio, embedded as an image in each sheet

◆ PAUSE AND DECIDE

What should they use?

A Connected Sheets
Sheets interface, BigQuery does the work.

B Export to CSV
Sheets cannot hold billions of rows.

C Scheduled query
Far beyond any sheet's limit.

D Embedded image
A picture, not a pivot.

THE SIGNAL

Sheets users, warehouse-sized data, no SQL: Connected Sheets.

> On one Enterprise reservation, heavy nightly ETL now overruns into the morning, and executive dashboards slow down until it finishes.
>
> Dashboards must stay fast, without starving ETL of capacity at night.

What should the team do?

A Move every dashboard project to on-demand pricing

B Separate etl and bi reservations, each with its assignments

C Raise the reservation's maximum, and hope the queries share well

D Run all the ETL jobs as batch priority queries

◆ PAUSE AND DECIDE

What should the team do?

A On-demand

Gives up predictable cost for one clash.

B Two reservations

Each workload has its own slots, idle ones shared.

C Raise the maximum

More cost, the same contention.

D Batch priority

Delays ETL, but guarantees nothing.

THE SIGNAL

Workloads competing for slots: separate reservations and assignments.

> Smart thermostats publish JSON events to Pub/Sub. The message schema already matches a BigQuery table, and no transformation is needed.
>
> The team wants the events in BigQuery, with nothing to run or maintain.

What should the team set up?

A. A BigQuery subscription that writes into the table

B. A Dataflow streaming job from a Google-provided template

C. A Cloud Run service that inserts each event through the API

D. A Pub/Sub Lite reservation feeding BigQuery

◆ PAUSE AND DECIDE

What should the team set up?

A BigQuery subscription
Pub/Sub writes directly, no subscriber to run.

B Dataflow template
Works, but a pipeline to run and pay for.

C Cloud Run service
Code to write and scale.

D Pub/Sub Lite
Turned down in March 2026.

THE SIGNAL

No transformation, nothing to run: a BigQuery subscription.

> Larkspur is retiring an on-premises Teradata warehouse with 4,000 SQL scripts written in Teradata's dialect.
>
> The scripts must run on BigQuery, and the team wants to avoid rewriting them by hand.

What should the team use?

A. Database Migration Service, pointed at the Teradata system itself

B. Datastream, replicating Teradata into BigQuery

C. A team of contractors rewriting each script by hand

D. BigQuery Migration Service and its batch SQL translator

◆ PAUSE AND DECIDE

What should the team use?

A DMS

Moves operational databases to Cloud SQL or AlloyDB.

B Datastream

Not a Teradata source, and moves data, not SQL.

C Manual rewrite

Slow, costly and error-prone.

D Batch SQL translator

Translates the dialect in bulk.

THE SIGNAL

Warehouse SQL in another dialect: BigQuery Migration Service translation.

> Larkspur's readings dataset holds 800 TB of logical bytes, but the data compresses about ten to one. Tables rarely change after loading.
>
> The team wants to cut storage cost without changing any queries.

What should the team change?

A Move the tables to Cloud Storage, as compressed CSV files

B Shorten the time travel window, on the logical billing model

C Switch the dataset to the physical storage billing model

D Delete partitions older than 90 days to save space

◆ PAUSE AND DECIDE

What should the team change?

A CSV in Cloud Storage

Changes every query, and loses native tables.

B Shorter time travel

Under logical billing, it is already included.

C Physical billing

Billed on compressed bytes, roughly a tenth.

D Delete partitions

Deletes data still needed.

THE SIGNAL

Highly compressible data: the physical storage billing model.

> Several curated tables feed regulatory reports. The team needs rules, such as non-null meter IDs and kWh within a range, checked on a schedule.
>
> Results must be recorded, and failures must raise alerts.

What should the team use?

A Knowledge Catalog data quality scans with those rules

B Analysts spot-checking a sample of each report every Monday

C Raising max_bad_records on the load jobs

D Pub/Sub schemas on the topics feeding the tables

◆ PAUSE AND DECIDE

What should the team use?

A Data quality scans
Rules, schedules, scorecards and alerts, managed.

B Spot checks
Manual, and misses most errors.

C max_bad_records
Drops bad rows instead of reporting them.

D Pub/Sub schemas
Checks message shape, not values in tables.

THE SIGNAL

Scheduled rules on tables, with results and alerts: data quality scans.

> A custom settlement job writes the line SETTLEMENT_MISMATCH to Cloud Logging whenever totals disagree.
>
> On-call staff must be paged if it appears more than five times in ten minutes.

What should the team set up?

A. A daily export of logs to BigQuery, checked each morning

B. An uptime check on the settlement job's endpoint

C. A log-based metric for that line, plus an alert

D. A budget alert on the settlement project

◆ PAUSE AND DECIDE

What should the team set up?

A Daily export

A day too late to page anyone.

B Uptime check

Checks the job is up, not what it logs.

C Log-based metric

Counts matching lines, alerts on the threshold.

D Budget alert

Spending, not errors.

THE SIGNAL

Page on a log line pattern: a log-based metric and an alerting policy.

> Analysts must join customer records across three tables on email address, but must never see real email addresses.
>
> The support team must still see emails in full.

What should the team set up?

A Policy tags on email, with no access granted to analysts

B Dynamic data masking with the SHA-256 rule for analysts

C Dynamic data masking with the nullify rule for analysts

D A Sensitive Data Protection job that redacts email in place

◆ PAUSE AND DECIDE

What should the team set up?

A Policy tags only

Queries on email fail, so no joins.

B SHA-256 masking

Same email, same hash, so joins still work.

C Nullify masking

Every email becomes null, so joins break.

D Redact in place

Support loses the real values too.

THE SIGNAL

Hidden but joinable: dynamic masking with SHA-256.

> Security found a BigQuery dataset shared with a personal Gmail address.
>
> Larkspur wants to make it impossible to grant any access to identities outside its own domain, across the whole organisation.

What should the team do?

A Review every dataset's permissions once a month

B Remove the personal account, and email staff a reminder

C Put the dataset inside a VPC Service Controls perimeter

D Set the domain restricted sharing organisation policy

◆ PAUSE AND DECIDE

What should the team do?

A Monthly review

Finds problems after the fact.

B Remove and remind

Fixes one case, prevents nothing.

C Perimeter

Blocks data movement, not IAM grants.

D Domain restricted sharing

Grants to outside identities are refused.

THE SIGNAL

No access for outside identities, ever: domain restricted sharing.

> Yesterday, an engineer deleted a folder of settlement files from a Cloud Storage bucket by mistake.
>
> The bucket was created recently with default settings, and object versioning is off.

How can the team recover the files?

A They cannot, because without versioning deleted objects are gone

B Ask Customer Care to restore them from fail-safe

C Restore them from soft delete, within the retention window

D Use BigQuery time travel on the bucket

◆ PAUSE AND DECIDE

How can the team recover the files?

A Gone for good

Soft delete keeps them, even without versioning.

B Fail-safe

A BigQuery feature, not Cloud Storage.

C Soft delete

Deleted objects restorable for 7 days by default.

D Time travel

For BigQuery tables, not buckets.

THE SIGNAL

Recently deleted objects in a new bucket: restore from soft delete.

> A nightly join between bills and accounts suddenly takes four times longer.
>
> In the execution graph, one join stage shows a maximum compute time far above its average.

What is the most likely cause?

A. The query cache has been disabled for the project

B. The reservation has too few slots for any query

C. A missing partition filter on the accounts table

D. Data skew: a few join keys carry most of the rows

◆ PAUSE AND DECIDE

What is the most likely cause?

A Cache disabled

Affects repeats, not one stage's balance.

B Too few slots

Slows every stage evenly.

C Missing filter

Shows as more bytes read, not one skewed stage.

D Data skew

A few workers do most of the work.

THE SIGNAL

Maximum far above average in one stage: data skew.

> Larkspur wants an alert message published whenever a newly arriving reading row in BigQuery shows over 50 kWh in a half hour.
>
> Analysts want to write the logic in SQL. The project already runs on an Enterprise reservation.

What should the team build?

A. A continuous query that writes matching rows to Pub/Sub

B. A scheduled query every hour that emails a CSV file

C. An on-demand query, run by hand each morning

D. A Dataflow pipeline, with the SQL logic rewritten in Java

◆ PAUSE AND DECIDE

What should the team build?

A Continuous query
SQL, running continuously, output to Pub/Sub.

B Hourly query
Up to an hour late.

C Run by hand
Manual, and a day late.

D Dataflow in Java
Works, but not SQL.

THE SIGNAL

Act on new rows as they arrive, in SQL: a continuous query.

> Developers need a writable copy of the 5 TB production bills table, to test a schema change. They will modify only a few thousand rows, and storage cost should stay low.

What should they create?

A. A full table copy, into a development dataset

B. A table clone, in a development dataset

C. A table snapshot, then run the tests on it

D. A view over the production table

◆ PAUSE AND DECIDE

What should they create?

A Full copy

Pays for all 5 TB again.

B Table clone

Writable, billed only for what changes.

C Table snapshot

Read-only, so tests cannot write.

D View

Reads production, and cannot be written.

THE SIGNAL

Writable test copy, low cost: a table clone.

> Larkspur receives 30,000 free-text complaints a month, stored in BigQuery.
>
> The team wants each one labelled billing, metering, supply or service, using SQL, without training a model.

What should the team use?

A. A k-means model on the complaint text, in BigQuery ML

B. A logistic regression model, trained on labelled examples

C. A Gemini model called from SQL with AI.GENERATE_TEXT

D. Regular expressions searching each complaint for keywords

◆ PAUSE AND DECIDE

What should the team use?

A k-means
Clusters, but not your four labels.

B Logistic regression
Needs labelled training data.

C Gemini from SQL
A prompt with the four labels, no training.

D Regular expressions
Brittle, and misses paraphrases.

THE SIGNAL

Label free text into set categories, no training: a Gemini model from SQL.

> A Dataform table rebuilds three years of readings every hour, although only the last hour's rows are new.
>
> Costs and run times keep rising as the data grows.

What should the team change?

A Make it an incremental table, adding only new rows each run

B Keep the full rebuild, but run it once a day instead of hourly

C Move the transformation into a Python script on a VM

D Turn on the query cache for the Dataform project

◆ PAUSE AND DECIDE

What should the team change?

A Incremental table

Processes only the new hour.

B Daily rebuild

Cheaper, but up to a day stale.

C Python on a VM

Leaves BigQuery, and adds a server.

D Query cache

Changed data defeats the cache.

THE SIGNAL

Only new rows each run: a Dataform incremental table.

> Larkspur runs a long-lived Spark cluster sized for the nightly peak, with its data in HDFS on the cluster's disks.
>
> The cluster sits idle 20 hours a day, and the team wants to cut cost.

What should the team change?

A. Keep the cluster, and switch every worker to a larger machine type

B. Keep the cluster, but turn off autoscaling to keep costs fixed

C. Move the data to Bigtable, and keep the cluster for compute

D. Keep data in Cloud Storage, and create a cluster per nightly run

◆ PAUSE AND DECIDE

What should the team change?

A Larger machines

A bigger idle cluster.

B No autoscaling

Fixed cost, still idle.

C Bigtable

The wrong store for Spark files.

D Cluster per run

Pay only while jobs run. Data outlives clusters.

THE SIGNAL

Idle most of the day: data in Cloud Storage, clusters per job.

> Larkspur needs dev, test and prod environments with identical datasets, buckets, topics and IAM.
>
> Every change must be reviewed before it reaches prod.

What should the team use?

A Click the resources together in the console for each environment

B Terraform in Git, applied per environment through a pipeline

C A shell script of gcloud commands, run by whoever is on call

D Copy the prod project once, then edit each copy by hand

◆ PAUSE AND DECIDE

What should the team use?

A Console clicks

Environments drift, and nothing is reviewed.

B Terraform in Git

Same code, reviewed, applied per environment.

C Shell script

Not declarative, and depends on who runs it.

D Hand edits

Copies drift at once.

THE SIGNAL

Identical, reviewed environments: infrastructure as code.

Paper 3 answers

QUESTIONS	ANSWERS
1 to 5	D, B, C, A, B
6 to 10	A, D, C, A, C
11 to 15	B, D, C, D, A
16 to 20	B, C, A, D, B

> **REMEMBER**
>
> Note your total and your section scores. The next slides read all three papers together.

Where your marks came from

SECTION	QUESTIONS	REVISIT
1 Design, 22%	1, 7, 12, 19	Modules 02 to 09
2 Ingest and process, 25%	2, 6, 9, 15, 18	Modules 10 to 18
3 Store, 20%	3, 8, 13, 16	Modules 19 to 26
4 Analyse, 15%	4, 11, 17	Modules 27 to 32
5 Maintain and automate, 18%	5, 10, 14, 20	Modules 33 to 39

THE NUMBERS

Add each section across all three papers. Sixty questions give a far steadier picture than twenty.

Reading your three scores together

IF YOU SEE	THEN
One section low in two or three papers	That is your gap. Rewatch its modules, then retake.
Misses scattered across every section	Slow down. Find the qualifier before reading options.
Paired questions answered the same way	You are matching settings. Rewatch module 42.
Scores rising from paper 1 to paper 3	It is sticking. Keep revising the weakest section.
17 or more on every paper	Strong. Plan your exam date, and keep sharp.

HONEST LIMIT

Google publishes no pass mark. These papers measure readiness for study, not a guaranteed result.

Module 45 in five lines

- One open copy for Spark and BigQuery: **Iceberg** in the Lakehouse runtime catalog.
- New rows acted on in SQL: **continuous queries**. No transformation: a **BigQuery subscription**.
- Hidden but joinable: **SHA-256 masking**. Deleted objects: **soft delete**.
- Writable cheap test copy: a **clone**. One stage far above average: **skew**.
- Read your **three scores together**, fix the weakest section, then book the exam.

> COMING UP
>
> That completes the forty-five modules. Module 00 introduces the course and is built last.